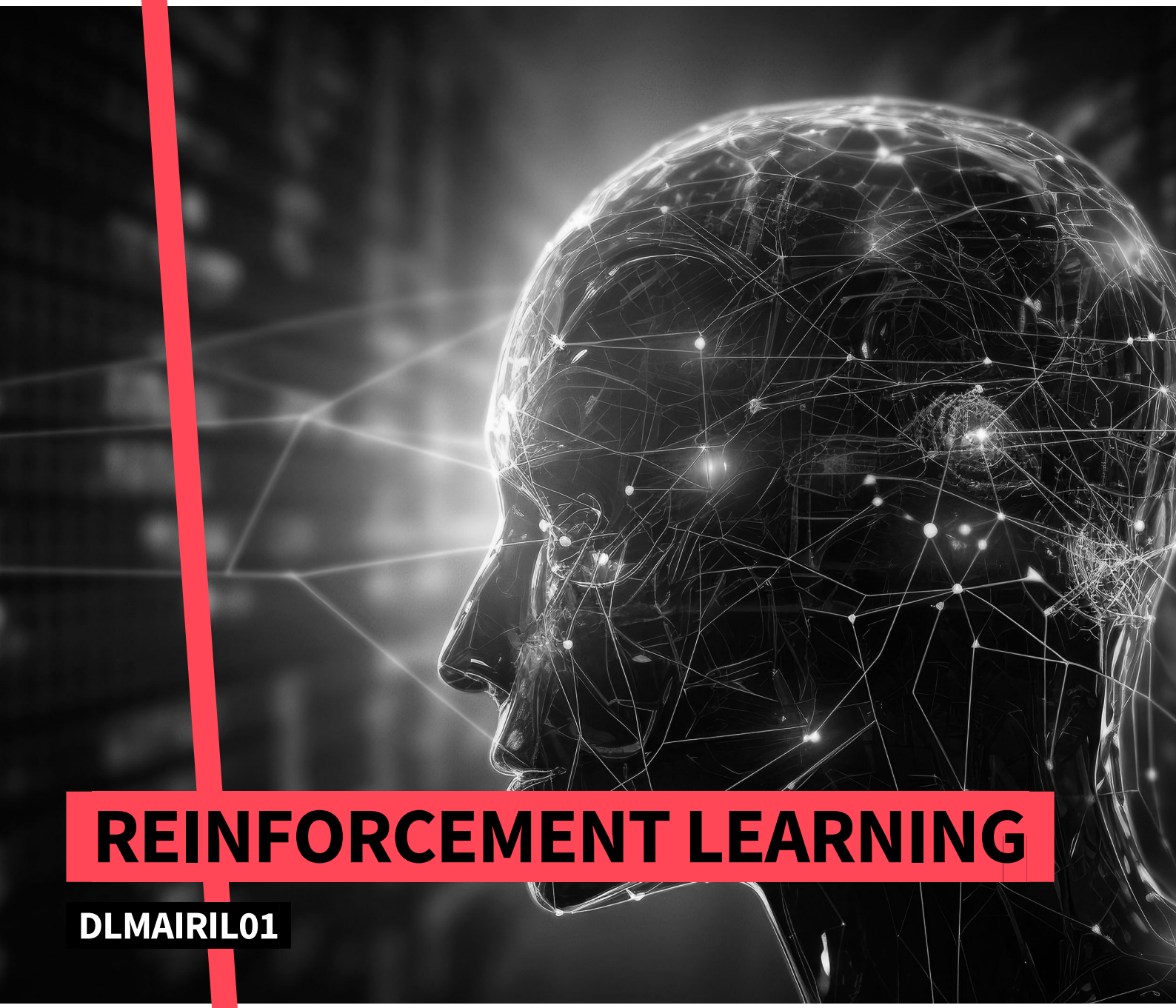
Course Book



REINFORCEMENT LEARNING

DLMAIRIL01

iu
INTERNATIONAL
UNIVERSITY OF
APPLIED SCIENCES

REINFORCEMENT LEARNING

MASTHEAD

Publisher:
IU Internationale Hochschule GmbH
IU International University of Applied Sciences
Juri-Gagarin-Ring 152
D-99084 Erfurt

Mailing address:
Albert-Proeller-Straße 15-19
D-86675 Buchdorf
media@iu.org
www.iu.de

DLMAIRIL01
Version No.: 001-2024-0328

N.N.

© 2024 IU Internationale Hochschule GmbH
This course book is protected by copyright. All rights reserved.
This course book may not be reproduced and/or electronically edited, duplicated, or distributed in any kind of form without written permission by the IU Internationale Hochschule GmbH (hereinafter referred to as IU).
The authors/publishers have identified the authors and sources of all graphics to the best of their abilities. However, if any erroneous information has been provided, please notify us accordingly.

TABLE OF CONTENTS

REINFORCEMENT LEARNING

Introduction
Signposts Throughout the Course Book . . . 6
Suggested Readings . . . 7
Required Reading . . . 9
Learning Objectives . . . 10

Unit 1
Introduction to Reinforcement Learning 11

1.1 Understanding Reinforcement Learning . . . 12
1.2 Components of Reinforcement Learning Systems . . . 32

Unit 2
Dynamic Programming and Markov Decision Processes 51

2.1 Introduction to Sequential Decision Making . . . 52
2.2 Dynamic Programming . . . 59
2.3 Bellman's Equation . . . 79
2.4 General Utility Functions . . . 100

Unit 3
Bandit Problems 109

3.1 The One-Armed Bandit Problem . . . 111
3.2 The Multiple Arm Bandit Problem . . . 118

Unit 4
Q-Learning 135

4.1 Q-Learning and Time-difference Learning . . . 140
4.2 Reinforcement Learning with Neural Networks and Deep Q-Learning . . . 150
4.3 Experience Replay . . . 153
4.4 Double Q-Learning . . . 155
4.5 Delayed Sparse Rewards and Hierarchical Learning . . . 156
4.6 Value- vs. Policy- Based Learning . . . 158
4.7 Actor Critic Learning . . . 162

Unit 5
Reinforcement Learning Approaches 169

5.1 Model-Free Learning . . . 171
5.2 Model-Based Learning . . . 172
5.3 Exploration vs. Exploitation . . . 177

1.1 A Python Implementation . . . 184
1.2 A Output . . . 186

2.1 B Rules for The Towers of Hanoi . . . 188
2.2 B Two Approaches for Solving The Towers of Hanoi . . . 188
2.3 B Program Implementation . . . 191

3.1 C Introduction . . . 201
3.2 C The Switchover Queue Control Problem . . . 202
3.3 C Program Implementation . . . 207

4.1 D Dynamic Programming Formulation of the Cake-Eating Problem . . . 215

5.1 E - Reinforcement Learning Implementation - The Agent-Environment Interaction Model . . . 219
5.2 E Reinforcement Learning Implementation - Q-learning Updates . . . 220
5.3 E Computer Implementation in Python . . . 221

Appendix
List of References . . . 225
List of Tables and Figures . . . 233

INTRODUCTION

WELCOME

SIGNPOSTS THROUGHOUT THE COURSE BOOK

This course book contains the core content for this course. Additional learning materials can be found on the learning platform, but this course book should form the basis for your learning.

The content of this course book is divided into units, which are divided further into sections. Each section contains only one new key concept to allow you to quickly and efficiently add new learning material to your existing knowledge.

At the end of each section of the digital course book, you will find self-check questions. These questions are designed to help you check whether you have understood the concepts in each section.

For all modules with a final exam, you must complete the knowledge tests on the learning platform. You will pass the knowledge test for each unit when you answer at least 80% of the questions correctly.

When you have passed the knowledge tests for all the units, the course is considered finished and you will be able to register for the final assessment. Please ensure that you complete the evaluation prior to registering for the assessment.

Good luck!

SUGGESTED READINGS

These are standard works and in-depth literature relevant for the respective course. They are not relevant for exams and do not necessarily have to be available via the IU library databases. Titles that are available are provided with a link.

GENERAL SUGGESTIONS

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT press.

Lonza, A. (2019). Reinforcement Learning Algorithms with Python. Packt Publishing, Birmingham, UK

Ravichandiran, S. (2018). Hands-On Reinforcement Learning with Python. Packt Publishing, Birmingham, UK

Bertsekas, D. P. (2019). Reinforcement learning and optimal control.

UNIT 1

Miotto, R., Wang, F., Wang, S., Jiang, X., & Dudley, J. T. (2018). Deep learning for healthcare: review, opportunities and challenges. Briefings in Bioinformatics, 19(6), 1236–1246

UNIT 2

Richard Bellman. (1954). Some Applications of the Theory of Dynamic Programming-A Review. Journal of the Operations Research Society of America, 2(3), 275.

UNIT 3

Tze-Leung Lai, & Yakowitz, S. (1995). Machine learning and nonparametric bandit theory. IEEE Transactions on Automatic Control, Automatic Control, IEEE Transactions on, IEEE Trans. Automat. Contr, 40(7), 1199–1209

UNIT 4

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. CoRR,abs/1801.01290. https://arxiv.org/abs/1801.01290

Hasselt, H. V. (2010). Double Q-learning. In: Advances in neural information processing systems(pp. 2613–2621).

UNIT 5

Kaelbling, L. P., Littman, M. L., & Moore, A. W. (1996). Reinforcement learning: A survey. Journal of artificial intelligence research, 4, 237-285.

Silver, D. et al. (2016). Mastering the game of go with deep neural networks and tree search. nature,529(7587), 484

REQUIRED READING

These are works that the students are required to read. The contents of these works can be tested in the exam. The required readings must be available via the IU library databases and are provided with a link.

UNIT 1

Knight, W. (2019). Can machines be truly creative? AlphaZero, a computer program that taught itself to be a chess grandmaster in a few hours, exhibits “the essence of creativity”, says its creator. MIT Technology Review, 2, 66.

UNIT 2

Stuart Dreyfus. (2002). Richard Bellman on the Birth of Dynamic Programming. Operations Research, 50(1), 48.

UNIT 3

Dirk Bergemann, & Juuso Valimaki. (2006). Bandit Problems. Cowles Foundation Discussion Papers.

UNIT 4

Mnih, V., et al. . (2015, February). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533. doi: 10.1038/nature14236

Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. In Thirtieth AAAI conference on artificial intelligence.

UNIT 5

Sutton, R. S. (1990). Dyna, an integrated architectures for learning, planning, and reacting based on approximating dynamic programming.In Machine learning proceedings 1990 (pp. 216–224). Elsevier.

Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. In Proceedings of the IEEE conference on computer vision and pattern recognition workshops (pp. 16–17)

LEARNING OBJECTIVES

The course “Reinforcement Learning" gives a comprehensive overview over the field of reinforcement learning.

Much of the theory of modern reinforcement learning approaches is built on a vast body of previous work on sequential decision problems and dynamic programming.

After a comprehensive introduction, these concepts are therefore discussed in detail to lay the foundation of understanding the modern approaches used in reinforcement learning.

Additionally, bandits provide further insights into decision making under uncertainty.

Equipped with this theoretical foundation, we then turn to modern reinforcement learning approaches where we focus on Q-learning with deep neural networks in particular and discuss model-free and model-based approaches to reinforcement learning.

Specific problems are discussed in detail in comprehensive appendices.

UNIT 1

INTRODUCTION TO REINFORCEMENT LEARNING

STUDY GOALS

Upon completion of this unit, you will have learned ...

- the range of problems that are solved by reinforcement learning.
- how reinforcement learning overcomes obstacles faced by more tradition methods of control theory.
- what constitutes the class of methods used by reinforcement learning.

1. INTRODUCTION TO REINFORCEMENT LEARNING

Introduction

Reinforcement learning is the branch of machine learning that is concerned with the optimization of sequential decision making processes. It has been widely and successfully applied to many diverse application in management, control and artificial intelligence. It has recently become indispensable in the control and navigation of robots, self-driving vehicles and pilotless aircraft. It has also achieved remarkable successes in game-playing, resulting in computer automated video game, backgammon and chess playing applications able to beat the best human players.

In this unit we will give a general overview of reinforcement learning. This will include the underlying methodologies and its areas of application. We will also review the relationship of reinforcement learning to classical control theory, and learn how it is able to overcome significant disadvantages of those older methods when applied to high-throughput data environments.

1.1 Understanding Reinforcement Learning

Computers are often used to automatize tasks involving decision making, prediction, classification, or pattern recognition. Machine learning involves the development of algorithms that allow a computer to learn these tasks, usually from experience gained from real or simulated data. **Learning** signifies the reliance of the underlying algorithms on data, or the ability to use data (or prior experience) to optimize the performance of these tasks, rather than on mathematical models or optimality principles, Machine learning can be broken down into several branches. Supervised learning is primarily concerned with prediction, for example a regression model for a specific quantity, and classification.

Learning
The term "learning" signifies that the algorithm is not explicitly programmed but learns relevant relationships from data.

Supervised Learning

Supervised learning
In supervised learning, the algorithms learns from labelled data where the true values are known.

The purpose of **supervised learning** is to develop classifiers and quantitative predictors. A seminal example is the classification of hand-written characters from images. The object is to construct a mapping $\widehat{f}:\boldsymbol{\mathcal{X}} \mapsto \boldsymbol{\mathcal{Y}}$, where $\boldsymbol{\mathcal{X}}$ is the universe of all machine-readable images of hand-written characters (the input), while $\mathcal{Y} = \{\mathrm{a}, \mathrm{b}, ...\}$ is the set of all characters to be recognized (the output). If $x \in \boldsymbol{\mathcal{X}}$ is, in truth, an image of a handwritten letter e, we would expect to see, with probability approaching one, $\widehat{f}\left(x\right) = \mathrm{e} \in \boldsymbol{\mathcal{Y}}$.

It is conceivable that the classifier $\widehat{f}\big(x\big)$ might consist of programmed instructions, presumably based on the known geometric properties of the characters in $\mathcal{Y}$. It should, for example, be possible to devise instructions that would allow the computer to distinguish between a circle and a line segment. However, the supervised learning approach is quite different. No attempt is made to formulate instructions. Instead, use is made of **training data** (x_1, y_1), (x_2, y_2), ..., (x_n, y_n), consisting of input-output pairs.

Training data
Training data is used during the training process, whereas test data is used for validation.

Here, x_i is a sample from the universe $\mathcal{X}$ to which the classifier will be applied, and its pair y_i is the label, or true class (this may be obtained by human judgement).

The supervised learning method will use the training data to build a classifier $\widehat{f}\big(x\big)$ for which $\widehat{f}\big(x_i\big) = y_i$ with high probability. Of course, the usual approach includes an evaluation of the classification accuracy of $\widehat{f}\big(x\big)$ using test data that has the same structure as training data (x'_1, y'_1), (x'_2, y'_2), ..., (x'_m, y'_m) but was not used to train the predictor $\widehat{f}\big(x\big)$.

One potentially crucial step in supervised learning is feature selection, which identifies the most important variables and eliminates those that are not relevant for the problem at hand. In particular, when input data for a supervised learning application is high-dimensional, it is important to identify and eliminate variables that are not informative, or which merely duplicate information contained in another variable. Suppose a multidimensional input takes the form $x = \big(x_1, ..., x_p\big) \in \boldsymbol{\mathcal{X}}$, where each component x_j, $j = 1, ..., p$ is interpretable as a separate piece of information.

It may be the case that not every component is related to response y. For example, suppose we wish to develop an application that predicts a customer's credit risk based on a number of available variables $x_1, ..., x_p$. We might expect, say,

$$x_1 = \text{current credit card balance}$$

to be useful for this application, but not

$$x_2 = \text{cityofbirth}.$$

Feature selection makes such judgements on a quantitative basis.

Feature extraction builds derived values from multi-variable input data $x = \big(x_1, ..., x_p\big) \in \boldsymbol{\mathcal{X}}$. In character recognition or other image processing applications, derived values might be measures of brightness, measures of skewness, or summaries of basic geometric features such as corners, straight lines, or circles. Derived values are then used in place of the raw data x as inputs to $\widehat{f}$.

The efficiency of supervised learning applications is often improved when inputs are first processed using various feature selection methods.

Unsupervised Learning

Unsupervised learning
Unsupervised learning uses unlabelled data to find relationships in data.

A second branch of machine leaning, **unsupervised learning**, is typically concerned with pattern recognition. If we are given data $x_1, ..., x_n \in \boldsymbol{\mathcal{X}}$ without labels $y_i \in \boldsymbol{\mathcal{Y}}$, we are likely confronting a problem in unsupervised learning. Typically, x_i is multidimensional and we are looking for previously unknown patterns. We are not, however, attempting to build a classifier mapping inputs to an output in $\boldsymbol{\mathcal{Y}}$. Therefore, unsupervised learning tends to be more in the nature of exploratory analysis, so that it can be used to inform future machine learning algorithms. An important application is clustering, which attempts to partition data into a relatively small number of groups so that the observations within each group are homogeneous, in the sense that they are similar to each other within a group but different from observations in other groups.

Consider the character classifier $\widehat{f}$ discussed above, for which the input is a machine-readable image of a single hand-written character. Of course, hand-written characters would come in groups as in a mailing address on an envelope. The classifier would then depend on a separate unsupervised learning algorithm that would isolate and extract single characters from a hand-written document. In general, unsupervised learning algorithms are useful for feature selection.

(James, Witten, Hastie, & Tibshirani, 2013) is an excellent reference for supervised and unsupervised learning.

Reinforcement learning

A sequential decision process (SDP) is any dynamic process that involves a time ordered sequence of decisions that influence some cost or reward that accrues over time. The idea is to determine the decision sequence that optimizes the defined cost or reward.

A third branch of machine learning, referred to as reinforcement learning, shares some of the essential characteristics of supervised and unsupervised learning, but is applied specifically to the problem of sequential decision making and therefore relies on a different form of data to that used by either supervised or unsupervised learning. Much of the theory of reinforcement learning is based on **dynamic programming**, which was formulated by Richard Bellman in the 1950s (Bellman, 1957). While this provides much of the mathematical foundation for reinforcement learning, as we will see below, dynamic programming has a limited computational capacity. Practically, this means that dynamic programming, at least in its exact form, is not a viable solution for applications made to exploit the type of high throughput data that is often available today.

Dynamic programming
Dynamic programming derives optimal decisions for an SDP, based on the analysis of a mathematical model.

One especially compelling early application of reinforcement learning was the development of an SDP, known as TD-Gammon, which is able to play backgammon at a competitive level (Tesauro, 1994, 1995). More recently, reinforcement learning was used by DeepMind Technologies to create a chess-playing program known as AlphaZero ("Entire human chess knowledge learned and surpassed by DeepMind's AlphaZero in four hours", The Telegraph, 6 December 2017). Reinforcement learning is particularly suited to develop game-playing applications, especially when coupled with deep neural network models (deep reinforcement learning). For example, it is currently used to create quite effective video game playing applications (Mnih et al., 2015a).

More generally, dynamic programming and reinforcement learning can be used for artificial intelligence applications involving sequential decision making. This makes them especially suitable for navigational control of autonomous vehicles. The development of such applications for automobiles (Li, Cheng, Guo, & Qiu, 2018; Ferdowsi, Challita, Saad, & Mandayam, 2018; You, Lu, Filev, & Tsiotras, 2018), drones, helicopters, and other unmanned aerial vehicles (UAV) (Loquercio & Scaramuzza, 2018; Muñoz, Barrado, Çetin, & Salami, 2019; Xue, Li, & Yang, 2019; Xu et al., 2019) is currently a thriving area of research.

Reinforcement learning is widely used in robotics (Kober, Bagnell, & Peters, 2013; Cheng, Cheng, Meng, & Zhang, 2018; Moerland, Broekens, & Jonker, 2018), expanding in to healthcare (which has brought many new problems) (Esteva et al., 2019), where computer aided diagnosis has become an especially compelling application (Kao, Tang, & Chang, 2018).

Although an SDP can be defined in continuous time, reinforcement learning is concerned with discrete time processes. Time is divided into stages $k = 1, 2, ...$ For an inventory control model, a stage might represent a fixed time interval such as a day or a week. The choice would depend on how frequently inventory decisions (reordering stock or reducing prices) will be made. Similarly, although an autonomous driving application operates in continuous time, the application of decision rules occurs at well defined decision epochs (navigational decisions, application of brakes, course corrections, change of speed). An SDP may also be a game such as chess, in which case the stage would represent a single move. A discrete time SDP can control continuous time process by defining the end points of stages using decision epochs (points in time at which decisions can be made).



EXAMPLE: A TOLL BOOTH SDP

As an example of an SDP, consider a toll system on a highway. There may be a ten booth maximum capacity, but not all booths may be operating at any given time. We can use reinforcement learning to build a control system that decides to open or close booths based on current or predicted traffic levels. A new stage may begin whenever a car entered or left the system, generating a new decision epoch.

In this way, control could be implemented essentially in real time although the SDP is formally a discrete time process.

1.1.1. Classical Control Theory

There is a rich mathematical theory for deducing the optimal control of SDPs. Much of this is based on dynamic programming, which was formulated by Richard Bellman in the 1950s (Bellman, 1957). As it happens, one relatively simple idea provides a remarkable degree of unity to this field, and a quite wide range of applications in operations research, economics, control theory, and artificial intelligence share essentially the same structure. Consider, for example, the following control problem.

EXAMPLE: INVENTORY MODEL

In preparation for the start of each business week (which we label with index $k = 1, 2, ...$), a retailer notes the current stock y_k of an item, then orders new stock w_k, which arrives soon after. There is a demand d_k for the item, so the retailer sells

$$s_k = \begin{cases} d_k & ; \ d_k \leq y_k + w_k \\ y_k + w_k & ; \ d_k > y_k + w_k \end{cases}$$

Thus, we have a decision process. The decision is the order amount w_k. When choosing w_k, the following costs and rewards must be considered:

1. A revenue of $p_s s_k$ is accrued, where p_s is the unit price. This depends on the demand d_k, which is not known in advance of the order decisions. However, failing to order sufficient stock to cover d_k will result in lost revenue. In addition, failing to meet demand on a regular basis may also cost future sales to customers preferring a dependable supply.
2. There is an inventory storage cost, which may become significant if the order amount w_k significantly over-estimates demand.
3. There is a fixed shipping cost B for each new order. This cost is not assumed if no order is made (that is, $w_k = 0$).
   Therefore, the decision maker may choose to anticipate the possibility that, for example, biweekly orders are preferable to weekly orders.

There are presumably more ways that the order decisions w_k affects the retailer costs and rewards.

The inventory problem is a classic example of the application of operations research to management science. We may, in principle, define the model mathematically. Parameters would include storage cost, shipping cost, retail price, and so on. We would also have to define a model environment, that is, exogenous factors which cannot be directly controlled, but which affect the model's dynamic behavior and costs. The demand d_k would be an example of such a factor. We may regard price as part of the environment, if it was also an externally determined factor, especially if it was subject to short-term fluctuation.

The model would also have to define the net reward precisely, as the difference between revenue and cost. Some components would probably be straightforward to define, for example, sales revenue, storage expense, or shipping expense. Other costs may be more difficult to quantify. We considered the possibility that consistently failing to meet demand (which occurs whenever $d_k > y_k + w_k$) may reduce future sales. It may be difficult to quantitatively predict this, but the very real effect would be to reduce demand d_k in the future.

Adaptive Control Systems

In such applications, model identification is usually necessary. Even in the simplest inventory control model, the amount of demand must be anticipated. This will almost certainly involve a stochastic demand model (demand being inherently random). Model identification may then involve the estimation of demand distribution parameters.

Of course, parameter estimation is normally a routine task for mathematical modelers. However, the nature of control theory presents an interesting new issue. Attainment of true optimal control usually depends on exact model identification. If the model is not exact, the control will be suboptimal and a cost will be accrued that is directly attributable to approximation error.

For this reason, there is a powerful incentive for a control system to make use of data that is observed while in operation. For example, suppose we are able to estimate a demand model for our inventory control model. We calculate the decision rules that would be optimal if our estimated demand model was exactly correct. This is referred to as the certainty-equivalence model. We then apply this decision rule to our live operations. However, because we continue to accumulate data that can be used to improve the demand model estimate, we continually recalculate our decision rules based on our increasingly accurate demand estimates. Ideally, such an adaptive control system would approach the optimal costs.

In addition, the environment itself can change. An advertising campaign may increase demand, in which case a static control system would no longer be optimal. An adaptive control system may also be able to modify its estimates of the control parameters in response to such changes.

1.1.2. The Basics of Dynamic Programming

We can think of an SDP as an ordered sequence of time indexed stages.

Within each stage, the decision maker makes a decision or takes an action and each stage is associated with a cost or reward. The process itself consists of transitions between single states belonging to a **state space**.

State space
The state space $\mathcal{X}$ is the set of all possible states.

After the SDP transitions to a new state, a decision is made in the form of an action selected from an action space $\mathcal{A}$.

An SDP is a dynamic process defined on discrete time points, or epochs, $j = 1, 2, ...$ A stage is associated with each epoch j. Each stage j contains a state x_j and an action a_j. We then refer to a state-action pair $\left(x_j, a_j\right)$.

Conceptually, within a stage, the state x_j is first observed. Then, with knowledge of the state, a decision is made, in the form of an action a_j selected from a well-defined set of possibilities. However, it is the usual practice to refine this definition, noting that not all

actions $a \in \mathcal{A}$ may be available from any given state x. We then let $\mathcal{A}_x$ be all actions available from state x. Then, (x, a) is an admissible state-action pair if action a is permitted from state x. The set of all admissible state-action pairs is the state-action space

$$\boldsymbol{\mathcal{K}} = \{(x, a) : x \in \boldsymbol{\mathcal{X}}, a \in \boldsymbol{\mathcal{A}}_x\}.$$

When action a_j is selected from state x_j, a cost (or reward) $R\big(x_j, a_j\big)$ is accrued, where $R(x, a)$ is the cost (reward) function that assigns a cost or reward to each admissible state-action pair (x, a) and is defined for every admissible state-action pair $(x, a) \in \boldsymbol{\mathcal{K}}$.

The transition between states is governed by a transition rule that determines how the SDP moves from state x_j to x_{j+1}. This rule depends on the current state-action pair $\big(x_j, a_j\big)$. Random state transactions are referred to as stochastic transitions. In these cases, a subsequent state x_{k+1} is determined as the outcome of a probability distribution, which depends on the current state-action pair (x_k, a_k). More precisely, stochastic transitions are governed by a probability kernel $P(y \mid x, a)$ defined for each possible state-action pair (x, a). Given current state-action pair $\big(x_j, a_j\big)$, the probability that the subsequent state x_{j+1} is equal to y is $P\big(y \mid x_j, a_j\big)$. Possibly, the transitions are deterministic if a subsequent state x_{k+1} is uniquely determined once the current state-action pair (x_k, a_k) is known. This means that once $\big(x_j, a_j\big)$ is known, there is only one possible value for the subsequent state x_{j+1}. In this case, the probability kernel $P\big(y \mid x_j, a_j\big)$ can be replaced by a transition function $q(x, a)$, which gives the transition rule $x_{j+1} = q\big(x_j, a_j\big)$.

A deterministic transition rule can be represented by a mapping $q(x, a) \in \boldsymbol{\mathcal{X}}$ for each admissible $(x, a) \in \boldsymbol{\mathcal{K}}$. Of course, this can also be modeled by the probability kernel by setting

$$P(q(x, a) \mid x, a) = \begin{cases} 1 \ ; \ (x, a) \in \boldsymbol{\mathcal{K}} \\ 0 \ ; \ (x, a) \notin \boldsymbol{\mathcal{K}} \end{cases}.$$

This means $P\Big\{x_{j+1} = q\big(x_j, a_j\big)\Big\} = 1$, which defines a deterministic transition rule.

The term “model” is used to refer to the components of an SDP collectively.

In particular, this includes state space $\boldsymbol{\mathcal{X}}$, action space $\boldsymbol{\mathcal{A}}$, state-action space $\boldsymbol{\mathcal{K}}$, cost function $R(x, a)$, and transition function $q(x, a)$ for deterministic transitions or probability kernel $P(y \mid x, a)$ for stochastic transitions.

Cost Accrued by a Dynamic Programming Model

We will now define some terminology that is frequently used in sequential decision problems. With respect to lifespan, there are three forms of SDP.

A finite horizon SDP has a fixed, finite number of stages K. An infinite horizon SDP continues indefinitely. In a shortest path SDP, the number of stages is finite but variable. These forms of SDP possess a terminal state $x_\Delta \in \boldsymbol{\mathcal{X}}$. When the SDP enters this state, it remains there indefinitely without accruing any further cost or reward. Essentially, the process ends when reaching $x_\Delta \in \boldsymbol{\mathcal{X}}$.

An SDP is a discounted cost/reward problem if the total accrued cost/reward is the discounted sum of the stage costs/rewards with a discount factor $\boldsymbol{\beta} \in [0, 1]$ that defines the rate at which future costs/rewards are discounted.

The total cost or reward is then given by

$$R_{discount} = \sum_{j=1}^{N} \beta^{j-1} R\big(x_j, a_j\big) \tag{1.1}$$

for a finite horizon or shortest path problem, or

$$R_{discount} = \sum_{j=1}^{\infty} \beta^{j-1} R\big(x_j, a_j\big) \tag{1.2}$$

for an infinite horizon problem. Note that for the finite horizon and shortest path SDPs, we usually set $\beta = 1$, and so it need not be included in the expression. In this case, we refer to total or undiscounted cost. In contrast, for the infinite horizon SDP, we usually have $\beta < 1$ so that R in Eqn. (1.2) will be finite.

To see how discounting works, suppose that all stage costs are non-negative, and bounded above by a finite constant $M \geq R(x, a) \geq 0$. Suppose $\beta < 1$.

Then, for any sequence of actions $a_1, a_2, ...$, we will have

$$R_{discount} = \sum_{j=1}^{\infty} \beta^{j-1} R\big(x_j, a_j\big) \leq M \sum_{j=1}^{\infty} \beta^{j-1} = \frac{M}{1-\beta},$$

Geometric series
$\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r}$ for $|k| < 1$

making use of the **geometric series**. Thus, as long as $\beta < 1$, the total discounted cost will be finite. Note that it will sometimes be convenient to include β in the expression so that both discounted and undiscounted costs can be represented by a single model.

1.1.3. Control of a SDP

The objective is to either minimize R (as cost) or maximize R (as reward) in Eqn. (1.1) or (1.2). The distinct is not important in the theory and methodology of dynamic programming, as long as the convention is consistently applied. We will generally refer to $R(x, a)$ as a cost function in either case. A maximum reward problem may be converted to a minimum cost problem simply by replacing cost function $R(x, a)$ with $-R(x, a)$. As a convention, an optimization problem can refer to either the minimum cost or maximum reward problem.

The SDP Policy

An SDP is controlled by a policy, which is a method of choosing actions at each stage. Optimization of total cost or reward is a matter of selecting the appropriate policy.

A common form of policy makes use of a policy function, which maps each state $x \in \boldsymbol{\mathcal{X}}$ to an admissible action $\boldsymbol{\mathcal{A}}_x$. When an SDP is controlled by a policy function $\phi(x)$, each state-action pair becomes $\big(x_j, a_j\big) = \big(x_j, \phi\big(x_j\big)\big)$.

This type of policy is an example of a stationary policy, which is any policy applying the same decision rule at each stage.

An optimal policy, which we denote by policy function ϕ*, is one that optimizes the expected value of R defined in Eqn. (1.1) or (1.2) for a given initial state $x = x_1$, or, ideally, all initial states.

We note that much of the theory of dynamic programming relies on the assumption that an SDP is memoryless, meaning that all subsequent behavior and costs depend only on the current state-action pair. This assumption is met when

1. The transition rule from state x_j to x_{j+1} depends only on current stage data $\big(x_j, a_j\big)$.

2. The total remaining cost accrued by the process from state x_j depends only on current stage data $\big(x_j, a_j\big)$.

In order for a controlled SDP to remain memoryless, the policy must also be memoryless. A stationary policy defined by a policy function $\phi(x)$ will be memoryless.

It is important to note that if the SDP model is memoryless, then the optimal policy will also be stationary. This simply means that if the optimal action from a given state x' is a' during stage j, we can expect a' to be the optimal action in the future whenever the SDP returns to stage x'.

Markov Decision Processes

An especially important type of SDP is the Markov decision process (MDP) that is an infinite horizon SDP with state-action space $\boldsymbol{\mathcal{K}}$, cost function $R(x, a)$, and probability kernel $P(y \mid x, a)$. It uses the discounted cost/reward accrual method with discount factor β. It can therefore be defined by the list of components $(\boldsymbol{\mathcal{K}}, R, P, \beta)$. This is simply an infinite horizon dynamic programming model, which uses discounted cost or reward and stochastic transitions.

By construction, an MDP is a memoryless SDP and the term "Markovian" is often used to denote the memoryless property. If the model components $(\mathcal{K}, R, P)$ can be identified, no data or simulations are required to determine the optimal decision rule. However, this ideal is often not practical.

We will see that much of the mathematical theory of reinforcement learning is based on models such as the MDP. However, it is also true that reinforcement learning is able to overcome many of the practical obstacles to their implementation.

Policy Evaluation

The theory of dynamic programming is largely concerned with the derivation of optimal policies. The method associated with dynamic programming is based on a simple recursive formulation of the SDP. Suppose we are given stationary policy ϕ and dynamic programming model $(\mathcal{K}, R, P)$.

We will assume that the model is stochastic and discounted because all other models can be interpreted as special cases.

The policy value function will be $V^{\phi}\big(x\big)$, $x \in \boldsymbol{\mathcal{X}}$ and gives the expected total cost when the initial state is $x_1 = x$. We can decompose $V^{\phi}\big(x\big)$ by separately considering the first stage. First, note that we have not specified the lifespan of the SDP, since our argument is the same for all cases. Then, based on either Eqn. (1.1) or (1.2), the contribution to R from the first stage will be $R(x_1, \phi(x_1))$. The process then moves to state $x_2 = y$ according to the

distribution $P(y \mid x_1, \phi(x_1))$. Then, conditional on x_2, the remaining expected contribution to R will be $\beta V^{\phi}\big(x_2\big)$. It will be more convenient to condition on the initial state $x_1 = x$, so that

$$\begin{aligned} V^{\phi}(x) &= E\Big[R \mid x_1 = x\Big] \\ &= E\Big[R(x_1, \phi(x_1)) + \beta V^{\phi}(x_2) \mid x_1 = x\Big] \\ &= R(x, \phi(x)) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^{\phi}(y) P(y \mid x, \phi(x)) \quad . \end{aligned} \tag{1.3}$$

We conclude that the value function $V^{\phi}\big(x\big)$ must satisfy $V^{\phi}(x) = R(x, \phi(x)) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^{\phi}(y) P(y \mid x, \phi(x))$ .given in Eqn. (1.3). In addition, if we know that no other function satisfies this equation, then $V^{\phi}\big(x\big)$ can be determined by finding its unique solution (if the solution were not unique, we would not know whether or not any solution that we found was the one we were interested in). For now, we simply point out that an important result in functional analysis, the Banach Fixed-Point Theorem, guarantees that Eqn. (1.3) possesses a unique solution under assumptions generally satisfied by dynamic programming problems (Puterman, 1994; Almudevar, 2014).

The problem of determining the total reward or cost of a specific policy is referred to as policy evaluation. For a dynamic programming model with stationary policy ϕ, this is equivalent to evaluating policy value function $V^{\phi}\big(x\big)$. For the dynamic programming model with stationary policy ϕ, this reduces to evaluating $V^{\phi}\big(x\big)$, which is the solution to Eqn. (1.3).

Operator
An operator maps one function to another function.

To explain how to solve this equation, we need to introduce the concept of an **operator**. Suppose $\mathcal{V}$ is a suitable set of functions defined on state space $\mathcal{X}$. An operator on $\mathcal{V}$ then modifies a function $V' \in \mathcal{V}$ to produce another function $V'' = TV' \in \mathcal{V}$. We can construct such an operator, which we denote T^{ϕ}, by using the evaluation method of the right side of Eqn. (1.3). In particular, starting with $V' \in \mathcal{V}$, $V'' = TV'$ is evaluated for each $x \in \mathcal{X}$ using the equation

$$V''(x) = R(x, \phi(x)) + \beta \sum_{y \in \boldsymbol{x}} V'(y) P(y \mid x, \phi(x)),\ x \in \boldsymbol{x} \tag{1.4}$$

This permits a very useful reformulation of Eqn. (1.3):

$$V^{\phi}(x) = T^{\phi} V^{\phi}(x)\,.$$

We can therefore view the problem as that of determining the fixed point of a fixed point equation. A fixed point equation is any equation of the form $V = TV$ (where V is a function) or $x = f(x)$ (where x is a real number).

Any solution V^* of a fixed point equation is a fixed point $V^* = TV^*$.

We can hence write the earlier equation as

$$V = T^{\phi} V$$

for some $V \in \boldsymbol{\mathcal{V}}$. The solution to the fixed point equation is then the fixed point of the policy operator and called the policy value function V^{ϕ}. We have already argued that, under general conditions, the solution to Eqn. (1.3) (which is also a fixed point of operator T^{ϕ}) exists, and is unique. This fixed point is the policy value function V^{ϕ}.

A significant advantage of this formulation is that, under general conditions, fixed points can be evaluated using the fixed point iteration method that calculates a sequence $x_1, x_2, \ldots$ using fixed point iterations $x_{n+1} = f(x_n)$. An initial value x_1 must be specified. If f possesses fixed point $x^* = f(x^*)$, then, under well known conditions, the sequence of fixed point iterates converges to x^*.

The idea extends to operators. If operator T possesses fixed point $V^* = TV^*$, then, under well known conditions, the fixed point iterates $V_{n+1} = TV_n$ converge to V^*, given a suitable initial function V_1. This is another consequence of the Banach Fixed-Point Theorem. Simply stated, suppose we are given an operator T, and we evaluate fixed point iterates $V_{n+1} = TV_n$, $n \geq 1$, given a suitable initial function V_1. If the sequence converges to a single function V^*, then this will be a fixed point $V^* = TV^*$.

The application of the fixed point iteration method to the operator T^{ϕ} permits policy evaluation using policy value iteration, where we evaluate the policy value function V^{ϕ} by calculating fixed point iterates $V_{n+1} = T^{\phi}V_n$. Here, T is the dynamic programming operator, and V_1 is a suitably chosen initial function. The sequence $V_1, V_2, ...$ converges to the fixed point $V^* = TV^*$, which is the optimal value function

$$\begin{aligned} &V_1 \in \boldsymbol{\mathcal{V}}, \\ &V_{n+1} = T^{\phi}V_n, \ n = 1, 2, \dots . \end{aligned} \tag{1.5}$$

Then the sequence of functions defined in Eqn. (1.5) converges to the unique fixed point $V^{\phi} = T^{\phi}V^{\phi}$. In practice, this means that any element of the sequence V_n is an approximation of V^{ϕ}, and that the approximation becomes increasingly accurate as n increases.

Policy Improvement

Suppose we are given a policy and policy value function ϕ, V^{ϕ}. What would be a simple way of improving ϕ? The first thing we need is a simple way of altering ϕ. Suppose that from state x_k, instead of selecting action $a_k = \phi(x_k)$, we select any other action $a_k = a'$. We would then transition to state x_{k+1} according to distribution $P(y \mid x_k, a')$, and accrue stage cost $R(x_k, a')$. Then, from state x_{k+1}, we would continue using ϕ for the remainder of the SDP.

The total cost of this new policy is easy to calculate:

$$Q^{\phi}(x_k, a') = E\Big[R(x_k, a') + \beta V^{\phi}\big(x_{k+1}\big) \mid x_k\Big] .$$

This is known in reinforcement learning as a Q-factor (for policy ϕ), and can be defined for any state-action pair $(x, a) \in \boldsymbol{\mathcal{K}}$:

$$Q^{\phi}(x, a) = R(x, a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^{\phi}(y) P(y \mid x, a) . \tag{1.6}$$

Once we have the Q-factors, we can improve the policy. Fix x, then minimize $Q^{\phi}\big(x, a\big)$ over $a \in \boldsymbol{\mathcal{A}}_x$:

$$a^*(x) = \operatorname{argmin}_{a \in \mathcal{A}_x} Q^{\phi}(x, a) . \tag{1.7}$$

Remember, however, that among the actions over which $Q^{\phi}(x,a)$ is being minimized is the one given by policy ϕ itself, that is $a = \phi(x)$. This choice simply gives back policy ϕ, so that

$$Q^{\phi}(x, a^*(x)) \leq Q^{\phi}(x, \phi(x)) = V^{\phi}(x)\,. \tag{1.8}$$

If the inequality is strict, then we have improved policy ϕ.

However, this type of "hybrid policy" of first selecting action $a^*(x)$for one stage then using policy ϕ seems, at the very least, inconvenient. Let's examine how it would work. If x_1 is the initial state, we select action $a^*(x_1)$, then transition to state x_2. From this state, we resume policy ϕ and select action $a_2 = \phi(x_2)$. However, if we have the Q-factors $Q(x,a)$ for all state-action pairs $(x,a) \in \mathcal{K}$, we can also use Eqn. (1.7) to determine $a^*(x_2)$ and use this action in place of $\phi(x_2)$. Then, by exactly the same argument we have just used, total cost accrued from x_2 will be either the same, or lower, than would be the case if we had used $a_2 = \phi(x_2)$. Clearly, we can make the same argument for all subsequent stages. This means that if we simply define a new policy $\phi'(x) = a^*(x)$ for all $x \in \mathcal{X}$, then ϕ' improves policy ϕ, in the sense that

$$V^{\phi'}(x) \leq V^{\phi}(x), \text{ forall } \quad x \in \boldsymbol{\mathcal{X}}\,. \tag{1.9}$$

The Optimal Policy

We may always define the optimal value function $V^*(x)$ that gives the optimal total accrued cost/reward when x is the initial state

$$V^*(x) = \inf_{\phi \in \boldsymbol{\Phi}} V^{\phi}(x),\ x \in \boldsymbol{\mathcal{X}}\,. \tag{1.10}$$

ϕ^* is then one which satisfies

$$V^*(x) = V^{\phi^*}(x),\ x \in \boldsymbol{\mathcal{X}}\,. \tag{1.11}$$

What happens if we attempt to "improve" ϕ^*? Necessarily, inequality (1.9) would be an equality because if we cannot improve ϕ^*, then any attempt to improve the policy would yield the equality

$$Q^{\phi^*}(x, \phi^*(x)) = \min_{a \in \boldsymbol{\mathcal{A}}_x} Q^{\phi^*}(x,a), \tag{1.12}$$

otherwise ϕ^* would not be optimal. Of course, by definition of the Q-factor, and the optimality of ϕ^*, we must have $Q^{\phi^*}(x, \phi^*(x)) = V^{\phi^*}(x) = V^*(x)$.

Then, using this identity and Eqn. (1.6), we may rewrite Eqn. (1.12) as

$$V^*(x) = \min_{a \in \mathcal{A}_x} R(x,a) + \beta \sum_{y \in \mathcal{X}} V^*(y) P(y \mid x, a)\,. \tag{1.13}$$

This is known as Bellman's Equation. It is a functional equation and its solution gives optimal value function $V^*(x)$ for memoryless SDPs. It should be compared to Eqn. (1.3), of which the policy value function V^ϕ is the unique solution. Eqn. (1.13) is a similar type of equation, the important difference being that a fixed policy is replaced by minimization over actions a $a \in \mathcal{A}_x$. However, it still defines an operator T^* on a function space such as $\mathcal{V}$ similar to that defined by Eqn. (1.4) for policy evaluation. In this case, for any $V' \in \mathcal{V}$, $V'' = T^*V'$ is evaluated for each $x \in \mathcal{X}$ using the equation

$$V''(x) = \min_{a \in \mathcal{A}_x} R(x,a) + \beta \sum_{y \in \mathcal{X}} V'(y) P(y \mid x, a),\ x \in \mathcal{X}\,. \tag{1.14}$$

Under general conditions, a unique solution to the equation

$$V = T^*V$$

exists, and the algorithm

$$\begin{aligned} &V_1 \in \mathcal{V}, \\ &V_{n+1} = T^*V_n,\ n = 1, 2, \ldots \end{aligned} \tag{1.15}$$

will converge to the optimal value function V^*. This is known as value iteration, which evaluates the optimal value function V^* by calculating fixed point iterates $V_{n+1} = TV_n$, where T is the dynamic programming operator and V_1 is a suitably chosen initial function. The sequence $V_1, V_2, \ldots$ converges to the fixed point $V^* = TV^*$, which is the optimal value function.

We may also derive an optimal policy by policy iteration or the actor-critic method. Policy iteration is a method of solving Bellman's Equation. Like value iteration, the algorithm is iterative, but alternates between two steps and produces a sequence of policy functions $\phi_1, \phi_2, ...$ rather than value functions. The first step begins with the current policy ϕ_n and evaluates the policy value function V^{ϕ_n}, possibly by applying the fixed point iteration method to the policy operator T^{ϕ_n}. Then, the policy ϕ_n is improved, yielding the next policy in the sequence ϕ_{n+1}. The algorithm terminates when strict improvement is not possible, in which case the final policy function in the sequence is the optimal policy function ϕ*. The policy evaluation step functions as the "critic", while the policy improvement step functions as the "actor".

The general setup is the following: We begin with an initial policy ϕ_1. A sequence of policies $\phi_1, \phi_2, ...$ is produced iteratively. A single iteration consists of the following two steps, given current policy ϕ_i:

- Step 1. Policy evaluation (critic) Evaluate: V^{ϕ_i} using, for example, value iteration.
- Step 2. Policy improvement (actor) Improve ϕ_i by evaluating

$$\phi_{i+1}(x) = \operatorname{argmin}_{a \in \mathcal{A}_x} R(x,a) + \beta \sum_{y \in \mathcal{X}} V^{\phi_i}(y) P(y \mid x, a), \text{ foreach } \quad x \in \mathcal{X}\,.$$

At the end of a single iteration of these two steps, we will then have $V^{\phi_{i+1}} \leq V^{\phi_i}$. We continue iterating these steps and terminate when $V^{\phi_{i+1}} = V^{\phi_i}$.

Then $\phi_i = \phi$* is an optimal policy. Note that the optimal policy itself may not be unique, even though V^{ϕ} will be. Therefore, we cannot use $\phi_{i+1} = \phi_i$ as a stopping rule.



EXAMPLE OF THE POLICY IMPROVEMENT STEP

The following is an example of the policy improvement (actor) step of policy iteration: Suppose a minimum discounted cost MDP has a state space $\mathcal{X} = \{1, 2, 3, ..., 50\}$. From state $x = 3$, two actions are available. If action $a = 1$ is chosen, the process transitions to state $x = 4$ or $x = 5$ with probability 1/2 each and a stage cost $R(3, 1) = 53$ is accrued. If action $a = 2$ is chosen, the process transitions to state $x = 2$ with a probability of one, and a stage cost of $R(3, 2) = 35$ is accrued. The discount factor is $\beta = 0.9$. Then suppose that policy iteration is used to evaluate the optimal policy. The policy value function for

the nth policy iterate ϕ_n has a policy value function with values $V'(2) = 23.5$, $V'(4) = 46.4$, and $V'(5) = 14.7$. What action will the next iterate ϕ_{n+1} choose from state $x = 3$?

The Q-factors that we need for policy ϕ_n are $Q(x, a) = R(x, a) + \beta E[V'(y)]$, where the expectation is taken over the state transition distribution for state-action pair (x, a). We then have

$$\begin{aligned} Q(3,1) &= R(3,1) + \beta(V'(4) + V'(5))/2 \\ &= 53 + 0.9 \times (46.4 + 14.7)/2 \\ &= 80.495. \end{aligned}$$

In addition,

$$\begin{aligned} Q(3,2) &= R(3,2) + \beta V'(2) \\ &= 35 + 0.9 \times 23.5 \\ &= 56.15. \end{aligned}$$

So, $Q(3, a)$ is minimized by $a^* = 2$, which will be the action chosen by $\phi_{n+1}(3) = 2$, from state $x = 3$.

1.1.4. Reinforcement Learning and Dynamic Programming

Following the inventory model that was discussed earlier, we will introduce a new SDP and consider some of the similarities and differences.

A Model for Automated Poker

Suppose that after some careful consideration, we decide to develop a program which, once interfaced with a suitable online poker website, will play the game automatically. This includes decisions regarding which cards to discard during each round, and what bets to make. This is clearly a SDP, and there is some hope that the principle of dynamic programming can be used to attain optimal play with respect to total winnings.

There are, however, several obstacles in the way of reaching this ideal.

First, the state space $\mathcal{X}$ for any dynamic programming would be very large. A player starts with a hand of five cards from a 52 card deck. There are $\binom{52}{5} = 2,598,960$ such distinct hands. Recall that $\binom{n}{k}$, or n choose k , is the binomial coefficient, equal to the number of unordered selections of k from n distinct objects. Suppose there are K rounds of betting. During each round, some number of cards may be discarded and replaced, and bets are made. The number of rounds left would have to be included as part of the definition of the state, since this information would be crucial to a decision maker. This is also true of the current amounts bet. This means that a lower bound for the size of the state space would be $K \times 2,598,960 \leq |\mathcal{X}|$.

Then, for this dynamic programming model, an action a would have to specify, at a minimum, which cards to discard. Suppose that in each round up to three cards may be discarded. There are $\binom{5}{3} + \binom{5}{2} + \binom{5}{1} + \binom{5}{0} = 10 + 10 + 5 + 1 = 26$ ways to select at most 3 cards from 5. So a lower bound for the set of all admissible actions for each state x would be $26 \leq |\mathcal{A}_x|$.

Then, if the cost function $R(x, a)$ was stored in memory in tabular form, the table would require no less than

$$K \times 2,598,960 \times 26 \leq |\mathcal{X}|\,|\mathcal{A}_x|$$

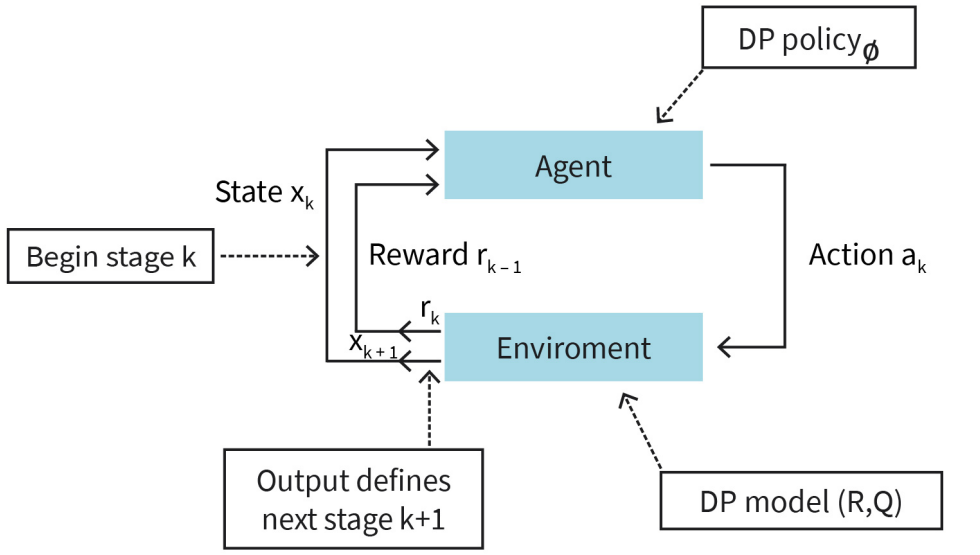
entries. Even worse, if the probability transition kernel $P(y \mid x, a)$ was stored in memory in tabular form, the table would require no less than

$$(K \times 2,598,960)^2 \times 26 \leq |\mathcal{X}|^2\,|\mathcal{A}_x|$$

entries. Of course, there may be some specialized structure in the problem that permits a more compact representation, but, absent this opportunity, these are the relevant bounds.

The second obstacle is more fundamental to the difference between reinforcement learning and dynamic programming. Suppose we were confident in our ability to efficiently store and access the model components $R(x, a)$ and $P(y \mid x, a)$. We would be still be left with the problem of deciding what these components should be. Clearly, discernible rules of probability would be involved, but to fully develop a relationship between action and reward would be difficult using mathematical theory alone.

Figure 1: Agent-environment interaction model of reinforcement learning, adapted from figure 3.1 of (Sutton & Barto, 2018a).



1.1.5. The Reinforcement Learning Platform

The principles of dynamic programming suggest that the notion of an automated poker playing application is sound, at least conceptually. If formulated correctly, such an application based on the solution to Bellman's Equation of optimality would be provably optimal and would play better than any human. Therefore, there should be some interest in overcoming the obstacles to such a project described in the poker application.

In an important sense, the theory of reinforcement learning assumes the existence of the dynamic programming model, especially its memoryless properties (Bertsekas, 2019). However, the decision maker only sees the output of that model. To make this clear, consider the reinforcement learning platform illustrated in figure above (adapted from figure 3.1 of (Sutton & Barto, 2018a). The diagram represents stage k in the SDP, and the transition to the next stage. We begin with current state x_k. We also have a record of the previous stage cost r_{k-1}. The decision maker is the agent that selects actions, either for the purpose of optimizing a reward or learning the optimal actions by a controlled process of trial and error. In the context of machine learning, the agent is an artificial intelligence application capable of learning and decision making. The agent receives the current data (x_k, r_{k-1}). We may assume that the agent is able to store process history, although the ability to do this is an important design consideration. The agent then makes decision a_k, which is input to the environment. Based on current data, the **environment** outputs stage reward r_k and subsequent state x_{k+1} with which the next stage begins.

Environment
The environment returns rewards based on the current process state, the subsequent state, and the action selected by the agent.

If the SDP is memoryless, then the environment is defined by dynamic programming model (R, Q) and is able to generate (x_{k+1}, r_k) based on input (x_k, a_k), according to the rules defined by the SDP model (R, Q).

However, from the point of view of reinforcement learning, (R, Q) is never explicitly represented. In a simulation platform, the environment is a separate program which outputs (x_{k+1}, r_k) based on standard pseudo-random simulation methods (see (S. Ross, 2014) and (Chambers, 1977) for excellent chapters on this subject). Of course, in a live application, the environment is simply the control domain (that is, an actual retail business or toll-booth installation).

This leads to the common characterization of reinforcement learning as model free in the sense that the rules used by the environment to generate rewards and state transitions are not known. A model exists, at least implicitly. But we only observe the reward accrued from any given stateaction pair (x, a). This means that in principle, we can learn an optimal sequence of actions using a controlled process of trial and error. This goes a long way to overcoming the second obstacle described in our automated poker application. If we interface an application with an on-line poker service, we have the type of environment represented in figure above and we can program an agent. In addition, this structure allows us to at least partially overcome the first obstacle of our automated poker application.

Reinforcement learning generally does not attempt to estimate dynamic programming components $R(x, a)$ or $P(y \mid x, a)$. Rather, it attempts, depending on the objective, to estimate a policy value function $V^{\phi}(x)$, $x \in \mathcal{X}$, or a related quantities known as Q-factors

$Q(x,a)$, $(x,a) \in \boldsymbol{\mathcal{K}}$ (this will be defined later in this Unit). Thus, at worst we must store objects of complexity $|\boldsymbol{\mathcal{X}}|\,|\boldsymbol{\mathcal{A}}_x|$, at best, we need only store objects of complexity $|\mathcal{X}|$. This compares very favorably to the storage requirement of $|\boldsymbol{\mathcal{X}}|^2\,|\boldsymbol{\mathcal{A}}_x|$ for the transition probability kernel required for the dynamic programming model.

1.1.6. On- and Off-Policy Reinforcement Learning

Before we examine some of the methods used in reinforcement learning in more detail, there is an important distinction that can be made. Suppose, for example, that a reinforcement learning application is designed to provide a simulated opponent for a video game player. Referring to figure above, this (human) player would be the environment and the application would be the agent. Of course, the agent would never have encountered this particular environment when first used, but would still be expected to function as an opponent. Thus, the agent has two very different goals: To learn how the environment plays in order to develop an effective playing strategy and to be a worthy opponent. In other words, the agent must both learn optimal behavior and apply optimal behavior.

These two goals conflict. The agent cannot learn optimal behavior except by trial-and-error, so that all actions need to be evaluated. Behavior on the part of the agent that attempts to learn in this way is referred to in reinforcement learning as exploratory behavior. In contrast, when the agent, after sufficient learning, is able to apply optimal decisions (is able, for example, to play the game effectively), this is referred to as exploitative behavior. We will discuss below strategies for balancing these two types of behavior. For now, we note that the usual approach is to place greater weight on exploratory behavior at first, gradually shifting to exploitative behavior as the agent learns optimal strategies (simulated game players can appear to play “randomly” at first, before developing effective strategies).

A reinforcement learning application intended to exploit, as well as learn, optimal behavior is an on-policy algorithm. This means that the goal is both to learn an optimal policy, and also to attain optimal costs/rewards during the learning process. However, this is not always required. Suppose we wish to create an optimal control policy for an inventory system. We may be able to create a computer simulation to serve as the environment.

An optimal policy would first be determined using reinforcement learning, then applied to a real-world inventory system. In this case, there is no need for to the reinforcement learning algorithm to use exploitative behavior (since no real costs are involved), and so it can use exploratory behavior for learning exclusively. This is referred to as an off-policy algorithm which means that the goal is to learn an optimal policy, but there is no attempt to attain optimal costs/rewards during the learning process

1.2 Components of Reinforcement Learning Systems

1.2.1. Policy Evaluation

The role played by a policy value function V^{ϕ} is common to both reinforcement learning and dynamic programming, and the reinforcement learning platform of Figure above gives a good idea of how it can be estimated. In the terminology of reinforcement learning, a shortest path SDP is referred to as episodic. Thus, it consists of sequence of state transitions that end with a terminal state x_{Δ} from which no more costs are accrued. Solving a puzzle such as the Towers of Hanoi would be an example. Then $V^{\phi}(x_0)$ is interpretable as the total cost accrued from initial state x_0 up to terminal state x_{Δ}. It is important to note that the algorithm designer (the reader) may freely restart the agent/environment interactions using any initial state and history.

So, we may set initial state x_0 in our reinforcement learning platform. We may also have the agent select actions $a_k = \phi(x_k)$ (as part of our design of an reinforcement learning algorithm). We then continue the agentenvironment interactions until terminal state x_{Δ} is reached, say, after N transitions. The total cost accrued is

$$R = r_1 + r_2 + ... + r_{N-1} + r_N \,. \tag{1.16}$$

but R is a random variable of mean $E\big[R\big] = V^{\phi}\big(x_0\big)$ so we may repeat these simulations, restarting the SDP at state x_0 after state x_{Δ}. Suppose we do so n times. This yields n simulated replications of trajectories from x_0 to x_{Δ}, and n total cost replications $R_1, ..., R_n$ calculated as in Eqn. (1.16).

A simple sample mean gives the estimate

$$V^{\phi}(x_0) = \frac{1}{n}\sum_{i=1}^{n} R_i,$$

which can be made increasingly accurate by increasing n.

For the moment, we have something that will serve an important function in reinforcement learning, namely, the ability to estimate $V^{\phi}\big(x\big)$, after repeating the preceding task for all initial states $x_0 = x$. Of course, this is what the policy value iteration algorithm of Eqn. (1.5) does, but that algorithm is not available if we have no representation of the dynamic programming model components $R(x, a)$ and $P(y \mid x, a)$.

The conventional term for this problem in reinforcement learning is policy evaluation or policy prediction. However, we need to point out that the development of policy evaluation methods far more efficient than what we have just described is a central feature of reinforcement learning theory.

This is especially true given that many reinforcement learning algorithms may require large numbers of policy evaluations.

The first thing to notice is that each replicated trajectory contains within it not just an estimate of $V^{\phi}(x_0)$, but also of $V^{\phi}(x_1)$, where x_1 is the second state visited in the trajectory, in particular

$$R' = R - r_1 = r_2 + ... + r_{N-1} + r_N \,.$$

In principle, we can obtain estimates of $V^{\phi}(x)$ from any replicated trajectory that visits state $x = x_k$, in the form

$$R'' = r_{k+1} + ... + r_{N-1} + r_N,$$

since R'' is an observation of the total cost accrued by the SDP from initial state $x = x_k$ until x_{Δ}. Suppose we can collect n_x such total cost observations $R_1\big(x\big), ..., R_{n_x}\big(x\big)$ from initial state x. Then we can estimate the entire policy function by constructing sample means

$$V^{\phi}(x) = \bar{R}(x) = \frac{1}{n_x} \sum_{i=1}^{n_x} R_i(x) \,.$$

This is known as Monte Carlo policy evaluation where we estimate a policy value function $V^{\phi}(x)$ directly by observing total rewards output by the environment, when the agent applies policy ϕ. This method comes two variants. The ”first-visit” method uses only those

costs accrued when a state is visited for the first time within an episode, i.e. it uses only those total cost observations $R_i(x)$ obtained when state x is being observed for the first time. The ”every-visit” method uses all costs accrued within an episode from all observations. Although the latter method might seem preferable, as it makes use of more data, it is also a biased estimate. Recall that n_x is random and therefore potentially introduces bias to the sample mean. This occurs for the every-visit method but not for the first-visit method. Both methods are used, neither being preferable to the other for all models. However, for both, the approximation error approaches zero as $n_x \rightarrow \infty$. A good basis for comparison is offered in sections 5.1-5.2 of (Bertsekas & Tsitsiklis, 1996).

1.2.2. Reinforcement Learning and Iterative Updating

At this point, it is worth reviewing a simple update formula for the sample mean. Suppose we have n observations $x_1, x_2, ..., x_n$. Then consider the sample mean

$$\overline{x}_m = \frac{1}{m} \sum_{i=1}^{m} x_i$$

obtained from the first m observations in the sequence.

Then suppose that for a certain application, we receive the observations sequentially. After receiving the first m observations we calculate $\overline{x}_m$. We then receive the next observation x_{m+1} and wish to calculate $\overline{x}_{m+1}$. Of course, rather than calculate $\overline{x}_{m+1}$ using all of the first $m+1$ observations, we could use an update rule that requires only $\overline{x}_m$ and x_{m+1}:

$$
\begin{aligned}
\overline{x}_{m+1} &= \frac{1}{m+1}\sum_{i=1}^{m+1} x_i \\
&= \frac{\sum_{i=1}^{m} x_i}{m+1} + \frac{x_{m+1}}{m+1} \\
&= \frac{m}{m+1}\overline{x}_m + \frac{1}{m+1}x_{m+1} \\
&= \overline{x}_m + \gamma_{m+1}\big(x_{m+1} - \overline{x}_m\big)
\end{aligned}
\tag{1.17}
$$

where we define the sequence of constants $\gamma_m = 1/m, m \geq 1$.

In reinforcement learning, updating rules are usually conceived of in the following way (Sutton & Barto, 2018a):

$$
[\text{newestimate}] \leftarrow [\text{oldestimate}] + [\text{stepsize}] \times \big\{\, [\text{target}] - [\text{oldestimate}] \,\big\}\,.
$$

The target in an updating rule is any quantity towards which we wish the successive updates to move and represents a goal. If the old estimate is smaller than the target, it will be increased. Of course, the target used in an updating rule may be random, but if it always has the same mean μ, the updates will tend to move towards that value. The step size is commonly referred to in reinforcement learning as a learning rate and controls the magnitude of the update. Note that if this value is set to one the update rule becomes simply

$$
[\text{newestimate}] \leftarrow [\text{oldestimate}] + 1 \times \big\{\, [\text{target}] - [\text{oldestimate}] \,\big\} = [\text{target}]\,.
$$

While this might seem desirable, it must be remembered that in many reinforcement learning applications, the target is random and this updating rule would lead to highly unstable behavior. For this reason, the learning rate is usually allowed to decrease to zero slowly.

We can represent the iterative equation for the sample mean given by $\overline{x}_{m+1} = \overline{x}_m + \gamma_{m+1}\big(x_{m+1} - \overline{x}_m\big)$ (Eqn. (1.17)) in this way. The old estimate is $\overline{x}_m$, the target is x_{m+1}, the learning rate is $1/m$, and the new estimate is $\overline{x}_{m+1}$. If each target has the same mean μ, the successive updates tend to move towards that value.

For the Monte Carlo policy evaluation method, we are trying to estimate $V^{\phi}\Big(x\Big)$. Denote the estimator $\widehat{V}^{\phi}\Big(x\Big)$. The updating rule is

$$\begin{aligned}\widehat{V}^{\phi}\Big(x_k\Big) &\leftarrow \widehat{V}^{\phi}\Big(x_k\Big) + \gamma\Big(R_k - \widehat{V}^{\phi}\Big(x_k\Big)\Big),\\ \widehat{V}^{\phi}(x_k) &\leftarrow \widehat{V}^{\phi}(x_k) + \gamma\Big(R_k - \widehat{V}^{\phi}(x_k)\Big)\end{aligned} \tag{1.18}$$

where

$$R_k = r_{k+1} + ... + r_N\,.$$

Note that the expression (1.18) is an updating rule, so we can omit the subscripts for $\widehat{V}^{\phi}(x_k)$, otherwise $\widehat{V}^{\phi}_k(x_k)$ would be the old estimate and $\widehat{V}^{\phi}_{k+1}(x_k)$ would be the new estimate. Here, the target R_k is a direct observation of the total cost assumed by an episode with initial state x_k (for the first-visit method, not all such updates are performed). Note that γ is left unspecified, but often decreases with the number of updates. In practice, reinforcement learning admits multiple strategies for choosing its value, although it is usually intended to have the same effect as $\gamma = \gamma_m = 1/m$ as used for the sample mean.

However, R_k is not the only possible target. We recognize that

$$V^{\phi}(x_k) = E\Big[r_{k+1} + \beta V^{\phi}(x_{k+1})\Big],$$

so that $r_{k+1} + \beta V^{\phi}(x_{k+1})$ could serve as a target, except for the fact that $V^{\phi}(x_{k+1})$ is unknown (otherwise we would not have to estimate it).

However, we do have an estimate $\widehat{V}^{\phi}(x_{k+1})$ and therefore, we have target $r_{k+1} + \beta\widehat{V}^{\phi}(x_{k+1})$, leading to update rule

$$\widehat{V}^{\phi}(x_k) \leftarrow \widehat{V}^{\phi}(x_k) + \gamma\Big(r_{k+1} + \beta\widehat{V}^{\phi}(x_{k+1}) - \widehat{V}^{\phi}(x_k)\Big)\,. \tag{1.19}$$

Bootstrapping
In an update rule, the estimate itself is sometimes used as a component of the target. This is referred to as bootstrapping.

The use in the target of the estimate $\widehat{V}^{\phi}$ itself is referred to as **bootstrapping**.

As for the Monte Carlo method, either the first-visit or many-visit variant may be used.

Both update rules (1.18) and (1.19) are consistent, given an appropriate choice of the learning rate sequence. However, the two procedures are not equivalent and, as for the first-visit and many-visit variants, neither is uniformly better. There is usually a bias-variance tradeoff. The target of the Monte Carlo method is unbiased, but the target of the bootstrapping method is not. However, the target of the bootstrapping method will typically have lower variance.

The Bias-Variance Tradeoff

An estimator $\widehat{\theta}$ of any parameter θ is unbiased if $E\big[\widehat{\theta}\big] = \theta$ (otherwise they are biased). Despite what we might expect, not all estimators are unbiased, including many widely used ones. The bias of an estimate is defined as

$$Bias\big[\widehat{\theta}\big] = E\big[\widehat{\theta}\big] - \theta\,.$$

For the mean squared error, the bias for the estimator $\widehat{\theta}$ is then given by

$$\begin{aligned} MSE &= E\Big[\big(\widehat{\theta} - \theta\big)^2\Big] \\ &= E\Big[\big(\widehat{\theta} - E\big[\widehat{\theta}\big] + E\big[\widehat{\theta}\big] - \theta\big)^2\Big] \\ &= var\big[\widehat{\theta}\big] + Bias\big[\widehat{\theta}\big]^2\,. \end{aligned} \tag{1.20}$$

It might seem a simple matter to subtract the bias from $\widehat{\theta}$ to yield an unbiased estimated, thereby reducing risk. Of course, we cannot do this if the bias depends on θ, which is unknown, and this is often the case.

The MSE can be used to define the consistency of a sequence of updated estimators $\widehat{\theta}_n, n \geq 1$. We say that the sequence is consistent if the MSE of $\widehat{\theta}_n$ approaches zero as n increases. An important question in reinforcement learning is whether or not a sequence of updates is consistent.

The relation $MSE = var\big[\widehat{\theta}\big] + Bias\big[\widehat{\theta}\big]^2$ given in Eqn. (1.20) results in biasvariance tradeoff, which is any attempt to minimizing the MSE by finding the best balance between variance and bias. The best estimator is not necessarily the one with the smallest variance, or the smallest bias, but the one that achieves the best tradeoff between the two, with best meaning the smallest MSE. We have, in fact, already encountered this. Recall that the every-visit variant of Monte Carlo policy evaluation has smaller variance than the first-visit variant since it uses more data. However, the every-visit variant is also biased, while the first-visit variant is not. Which variant has the the smallest MSE will depend on the application (sections 5.1-5.2 of (Bertsekas & Tsitsiklis, 1996)), however, both are consistent.

1.2.3. TD(λ)

In many reinforcement learning applications, policy evaluation must be carried out multiple times. Much of the success of reinforcement learning can be attributed to a method of policy evaluation known as temporal difference learning. In fact, this is really a family of methods, commonly denoted TD(λ), where $\lambda \in [0, 1]$ is the trace decay parameter that defines a specific TD(λ) algorithm. Higher values of λ allow the updates to be increasingly affected by future costs/rewards.

In understanding TD(λ), it helps to carefully compare update rules (1.18) and (1.19). We can, in particular, write Eqn. (1.19) as

$$\widehat{V}^{\phi}(x_k) \leftarrow \widehat{V}^{\phi}(x_k) + \gamma d_k \tag{1.21}$$

where d_k is the temporal difference of a given policy value function policy $V^{\phi}\big(x\big)$. The temporal difference is the difference in total cost/reward accrued when (a) action a is selected, then policy ϕ used afterwords; and (b) policy ϕ is always used. We can write this

$$d_k = r_{k+1} + \beta \widehat{V}^{\phi}(x_{k+1}) - \widehat{V}^{\phi}(x_k)\,. \tag{1.22}$$

Suppose we then define a general updating rule based on a weighted sum of the temporal differences $d_k, d_{k+1}, \ldots$

$$\widehat{V}^{\phi}(x_k) \leftarrow \widehat{V}^{\phi}(x_k) + \gamma \sum_{i=k}^{N-1} \lambda^{i-k} d_i, \tag{1.23}$$

for some parameter $\lambda \in [0, 1]$. After selecting, say, the first-visit or everyvisit variant, this update rule defines the TD(λ) policy evaluation method.

In fact, after some algebra, we can see that TD(0) is the bootstrap method defined by update rule (1.19), while TD(1) is the Monte Carlo method defined by update rule (1.18). Any value of λ between 0 and 1 defines a hybrid method, with λ closer to 0 emphasizing the bootstrap, and λ closer to 1 emphasizing Monte Carlo sampling.

The best choice of λ is, in practice, usually a matter of trial and error.

Favored values reported tend to be strictly between 0 and 1. See section 5.3 (Bertsekas & Tsitsiklis, 1996) for a discussion of this issue. (Singh & Dayan, 1998) give analytical derivations of the mean squared error for different choices.

1.2.4. Experience Replay and Sparse Rewards

The distinguishing feature of reinforcement learning is that it learns policy through experience (the environment), whether simulated or real. This means that the data defining this experience, used in, for example, a TD(λ) algorithm, is potentially quite a valuable commodity. Experience replay denotes a range of techniques in which data is stored and reused with this in mind. Randomized resampling is used to create essentially new experiences. Experience replay and related methods have been used to improve the performance of reinforcement learning applications for some time (Cichosz, 1999; Bertsekas, 2019).

Variants of experience replay are also targeted to a significant problem that arises in many reinforcement learning applications, that of sparse rewards (rewards that occur infrequently in an episode, possibly only at the very end). Suppose we wish to design a game-playing application. In principle, this can be based on a reinforcement learning or dynamic programming model. The concept of reward is similar in each method, and in the simplest

form of the model, there may be only one stage in which a nonzero reward can be attained. In particular, $R(x, a) = 0$, unless the state-action pair (x, a) leads immediately to a win, in which case $R(x, a) = 1$. If, more generally, most rewards r_k emerging from the environment of the reinforcement learning platform are zero, it will be difficult for the application to make progress towards an optimal policy.

One solution to this problem is the use of prioritized (or selective) experience replay (Isele & Cosgun, 2018). It is possible to weight the resampling of experience data to increase representation of sequences with positive rewards, or, more generally, sequences resulting in higher rewards. This renders learning more effective.

Another variant is hindsight experience replay. Suppose a certain episode of data is available for resampling. Perhaps in this episode a goal was not achieved, and it can therefore be classified as a failure. However, it may be the case that a similar goal was achieved. For example, a vehicle may not have been guided to a target location, but rather to a location five feet to the right. Thus, if the target is changed to the location actually reached, the episode was a success, and can be valuable learning data (either the "successful policy" can be modified, or used to perform a similar task). See (Andrychowicz et al., 2017).

It is also possible to define a more expansive notion of reward. When reward is sparse, there is little basis on which to develop a policy that selects actions progressing towards a goal in a purposeful way. In this case, it is possible to introduce intrinsic rewards for behavior which, while not resulting in a successful completion of a task, is likely to facilitate completion of the task (Chentanez, Barto, & Singh, 2005). For example, curiosity-driven behavior seeks greater diversity of experience and can be rewarded in the context of the reinforcement learning platform independently of the completion of a given task (Wesselmann, Wu, & Gašić, 2019).

1.2.5. Policy Improvement

We have learned how to evaluate a policy ϕ by approximating a value function V^{ϕ} using the TD(λ) method. However, we must still consider the control problem of determining the optimal policy ϕ* that results in the minimum total cost or maximum total reward. Suppose Φ is the family of all admissible policies $\phi \in \Phi$. If we are given two policies $\phi_1, \phi_2 \in \Phi$, we can always use policy evaluation to estimate and compare value functions V^{ϕ_1}, V^{ϕ_2}, which would allow us to choose the best policy from the two evaluated. In principle, we could simply evaluate all policies $\phi \in \Phi$, after which we could determine the optimal $\phi^* \in \Phi$. This may be feasible if Φ is small. In fact the selection of one of several actions by direct comparison of the respective rewards is the objective of bandit process models (Berry & Fristedt, 1985; T. Lai & Yakowitz, 1995).

However, for the typical SDP application, Φ would be far too large for this approach. Consider the automated poker playing application considered in the poker application. We developed lower bounds $K \times 2,598,960 \leq |\mathcal{X}|$ for the size of the state-space and $26 \leq |\boldsymbol{\mathcal{A}}_x|$ for the size of the admissible action space. A policy ϕ must map each state $x \in \boldsymbol{\mathcal{X}}$ to an action $\phi(x) = a \in \boldsymbol{\mathcal{A}}_x$.

Using basic principles of combinatorics, we therefore have the following lower bound for total number of policies:

$$26^{K \times 2,598,960} \leq \left|\Phi\right|$$

Clearly, we cannot simply perform policy evaluation for each $\phi \in \Phi$. Yet reinforcement learning can be, and is, applied to problems of this complexity.

A common method is the "Actor-Critic" method, which is based on the policy iteration method of dynamic programming. The "Critic" performs policy evaluation using, for example, a TD(λ) algorithm. Then the "Actor" performs the policy improvement step. The ability of the "Actor" to vary actions, and observe the rewards, permits estimation of the Q-factors.

These are then used to improve the policy.

1.2.6. Q-factors and the Control Problem

Q-learning
Q-learning is a TD(0) algorithm for which the target is the Q-factor $Q(x, a)$.

One of the most important methods in reinforcement learning is known as **Q-learning**, and was developed relatively recently, in the 1989 thesis of Chris Watkins (C. J. C. H. Watkins, 1989), and was then more precisely characterized in (C. J. Watkins & Dayan, 1992). It is actually based on something like a decomposition of the Bellman's equation of the form

$$Q^{\phi}(x,a) = R(x,a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^{\phi}(y) P\big(y \mid x, a\big), \ (x,a) \in \boldsymbol{\mathcal{K}} \,. \tag{1.24}$$

For the control problem, we are interested in the the optimal policy ϕ*, which can be substituted into Eqn. (1.24). However, because ϕ* is yet to be determined, it can only be applied indirectly. We then denote the resulting Q-factors, those to be used for the control problem, as simply $Q(x, a)$. Then Eqn. (1.24) becomes

$$Q(x,a) = R(x,a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^{*}(y) P\big(y \mid x, a\big), \ (x,a) \in \boldsymbol{\mathcal{K}} \,. \tag{1.25}$$

Next, Eqn. (1.25) leads to two more forms of reinforcement learning control.

Q-learning

Recall that we can write

$$V^*(y) = \min_{a \in \mathcal{A}_x} Q(x,a)\,.$$

This can be substituted into Eqn. (1.25) to get

$$Q(x,a) = R(x,a) + \beta \sum_{y \in \mathcal{X}} \left[\min_{a \in \mathcal{A}_y} Q(y,a) \right] P(y \mid x,a),\ \ (x,a) \in \mathcal{K} \tag{1.26}$$

This then leads to a form of TD(0) algorithm designed to estimate the Q-factors $Q(x,a)$ in place of a specific value function V^{ϕ}. This uses update rule

$$Q(x_k,a_k) \leftarrow Q(x_k,a_k) + \gamma \left(r_{k+1} + \beta \left[\min_{a \in \mathcal{A}_y} Q(x_{k+1},a) \right] - Q(x_k,a_k) \right). \tag{1.27}$$

Essentially, Q-learning is an iterative estimate of the Q-factors based on an approximation of Bellman's equation, which incorporates the estimate

$$\begin{aligned} &\sum_{y \in \mathcal{X}} V^*(y) P(y \mid x_k,a_k) = E\big[V^*(x_{k+1}) \mid (x_k,a_k)\big] \approx V^*(x_{k+1}) \\ &= \min_{a \in \mathcal{A}_y} Q(x_{k+1},a) \end{aligned} \tag{1.28}$$

Note that the action a_k selected from state x_k need not represent any attempt to actually achieve optimal control, approximate or otherwise.

For this reason, Q-learning is referred to as an off-policy algorithm. It is intended to estimate the Q-factors, but can do so without ever observing an SDP governed by optimal control. Therefore, it is not necessary to make any attempt to attain optimal cost performance.

SARSA

SARSA is a reinforcement learning control algorithm originally proposed in (Rummery & Niranjan, 1994). The name is an acronym for "state-actionreward- state-action". Like Q-learning, it is a TD(0) method for estimating the Q-factors $Q(x, a)$, and it relies on an estimate similar to that given in Eqn. (1.28). However, since it intends to implement an optimal policy in order to optimize accrued cost, it is an on-policy algorithm.

Accordingly, it selects actions using the rule

$$a_k = \operatorname{argmin}_{a \in \boldsymbol{\mathcal{A}}_{x_k}} Q(x_k, a),$$

using the latest estimates of the Q-factors. This results in estimate

$$\sum_{y \in \boldsymbol{\mathcal{X}}} V^*(y) P(y \mid x_k, a_k) = E\big[V^*\big(x_{k+1}\big) \mid (x_k, a_k)\big] \approx V^*\big(x_{k+1}\big) = Q$$

$$\big(x_{k+1}, a_{k+1}\big) \tag{1.29}$$

The update rule is then

$$Q(x_k, a_k) \leftarrow Q(x_k, a_k) + \gamma\big(r_{k+1} + \beta Q\big(x_{k+1}, a_{k+1}\big) - Q(x_k, a_k)\big)\,. \tag{1.30}$$

1.2.7. Exploration versus Exploitation

Control methods in reinforcement learning are usually based on the estimation of the Q-factors, as is clear from the update rules for Q-learning (Eqn. (1.27)) or SARSA (Eqn. (1.30)).

In TD(λ) algorithms, the estimate of a Q-factor $Q(x, a)$for a specific stateaction pair $(x, a) \in \boldsymbol{\mathcal{K}}$ is updated whenever a sample trajectory visits that pair (that is, $(x_k, a_k) = (x, a)$ for some Stage k). Recall from Sec. 1.2.2 that for most statistical estimates

we have $MSE \approx C/\sqrt{n}$, where C is some constant, and n is the number of observations contributing to the estimate. This holds also for TD(λ) (Bertsekas, 2019), where n becomes the number of visits to (x, a).

Recall that the optimal action $\phi^*(x) = a^*$ for state x is the one minimizing $Q(x, a)$, according to the identity:

$$Q(x, a^*) = \min_{a \in \boldsymbol{\mathcal{A}}_x} Q(x, a)\,. \tag{1.31}$$

It is therefore important to understand that correct identification of the optimal action a^* cannot be guaranteed unless $Q(x, a)$ is known exactly for all $a \in \boldsymbol{\mathcal{A}}_x$. It is also true, however, that if the approximation error of all Q-factors approaches zero, the probability of correctly identifying the optimal actions for each state using Eqn. (1.31) approaches one.

We have just argued that for the estimates of $Q(x, a)$ we will have $MSE \approx C/\sqrt{n}$, where n is the number of times (x, a) is visited during the learning process of the reinforcement learning algorithm. Thus, as a general rule, in order for a reinforcement learning algorithm to converge to the optimal policy, it is necessary that each state-action pair $(x, a) \in \boldsymbol{\mathcal{K}}$ is visited infinitely often. In practice, this means that a reinforcement learning model must continue to explore the entire state-action space in a balanced manner.

Recall the distinction between on-policy and off-policy reinforcement learning algorithms made in Sec. 1.1.6. This is based on two different forms of behavior on the part of the agent component of the reinforcement learning platform (see Figure above). Exploratory behavior is concerned with learning optimal policies rather than applying them. As we have just argued, the specific goal of this behavior should be to ensure that each state-action pair (x, a) is continuously visited. A reinforcement learning algorithm concerned solely with learning an optimal policy is an off-policy algorithm, and its agent need only exhibit exploratory behavior. This does not pose a problem for an off-policy algorithm such as Q-learning since selection of actions is under no particular constraint.

In contrast, an on-policy algorithm seeks to optimize cost or reward. If the Q-factors $Q(x, a)$ were known exactly, this could be done simply by using policy

$$\phi(x) = \min_{a \in \boldsymbol{\mathcal{A}}_X} Q(x, a), \tag{1.32}$$

which is optimal, as we have seen. Of course, in a reinforcement learning application, only estimates of $Q(x, a)$ will be available for use in Eqn. (1.32).

When actions are selected by a policy function $\phi(x)$ constructed in this way, this is referred to as a greedy policy. Since the goal of an on-policy algorithm is to optimize reward, then it is natural for the agent to use this type of policy. SARSA is an on-policy algorithm and makes use of a greedy policy.

Of course, if only estimates of $Q(x, a)$ are available, there is no guarantee that a greedy policy will be optimal. This can only be guaranteed if each Q-factor estimate becomes increasingly accurate, which can only happen with sufficient exploratory behavior on the part of the agent.

This defines the exploitation versus exploration trade-off. It arises from the two inherently conflicting goals of on-policy reinforcement learning. First, we would would like to learn the optimal policy (exploration). Second, we would like to use the optimal policy (exploitation), assuming the SDP is online. We first recognize that, in principle, we can only approximate the optimal policy. This means the more accurate the approximation, the lower the actual accrued cost we intend to minimize. But in order to learn, the SDP must explore, which is inherently sub-optimal. Therefore, the objective is to balance the two behaviors. In short, Exploitation refers to the attempt to maximize reward, exploration refers to the learning oriented behavior

A simple expedient is the ϵ-greedy algorithm. First, we refer to a policy of Eqn. (1.32) as greedy, since it attempts to reduce cost. An ϵ-greedy policy is a randomized policy that selects the greedy policy with probability 1−ϵ (exploitation) and an exploratory policy with probability ϵ. This may simply be a random selection from among all admissible policies. Then ϵ is usually allowed to converge to zero so that the greedy policy becomes increasingly dominant. However, ϵ must decrease slowly enough to ensure that exploratory behavior is selected infinitely often (the choice $\epsilon_k = 1/k$ will achieve this).

A number of well known reinforcement learning algorithms rely on balanced sampling of state-action spaces, for example, prioritized sweeping (Moore & Atkeson, 1993a). Furthermore, a number of algorithms are specifically designed to achieve a balance of the exploitation/exploration trade-off.

The E^3 algorithm (Kearns, Mansour, & Ng, 2002) mixes exploratory and certainty equivalence controls (or, the best we can do given the amount of learning we are allowed). The R-max algorithm (Brafman & Tennenholtz, 2002) adopts the simpler expedient of temporarily lowering the cost $R(x, a)$ for underrepresented state-action pairs (x, a), which more efficiently targets exploratory behavior.

1.2.8. Artificial Neural Networks and Reinforcement Learning

One especially compelling application of reinforcement learning was the development of a SDP, known as TD-Gammon, which is able to play backgammon at a competitive level (Tesauro, 1994, 1995). If this SDP was formulated as a dynamic programming model, then the state space $\boldsymbol{\mathcal{X}}$ would have to contain all possible board configurations, and would therefore clearly be too large to store in exact form. To make such applications feasible, one possibility would be to use compression techniques to store at least an approximation of dynamic programming model components R and Q, value functions $V(x)$, or Q-factors $Q(x, a)$ as efficiently as possible.

This can be accomplished by the use of artificial neural networks (ANN) for dynamic programming (Bertsekas & Tsitsiklis, 1996) or deep reinforcement learning (Mnih et al., 2015a).

We note that ANNs were first introduced in the 1940s (McCulloch & Pitts, 1943), although they have achieved much greater prominence more recently with the advent of deep learning (Aggarwal, 2018).

Multilayer Perceptron

An artificial neural network (ANN) is a mapping from an input to an output. As such, it can be trained for a supervised learning application.

A wide range of networks are based on an architecture in the form of a directed graph (V, E) with n_t nodes V labelled $i = 1, ..., n_t$. For example, the multilayer perceptron (Hassoun Mohamad, 1995) has the following architecture. Edge $(i, j) \in E$ exists if node i connects to node j. Node i is labelled by a real number x_i and edge (i, j) is labelled with real-valued weight w_{ij}.

Table 1

Output Node		
/	|	\
H1	H2	H3
\	|	/
Input Node n		

Nodes are arranged in three disjoint layers, an input layer of n_i nodes, a hidden layer of n_h nodes, and an output layer consisting of a single node.

Networks are usually fully connected, that is, each input node is connected to each hidden node, and each hidden node is connected to the output node.

Optionally, each input node is connected to the output node (skip layer connections), which are implemented here. No other type of connection is permitted.

Denote the input and hidden nodes I and H, and input-to-hidden, hiddento- output, and input-to-output edges IH, HO, and IO respectively. Labels for the hidden layer nodes are given by the formula

$$x_{j'} = \phi\left(\sum_{(\mathrm{i},\mathrm{j}) \in \mathrm{IH}:\, \mathrm{j} = \mathrm{j}'} \mathrm{w_{ij'}x_i}\right),\ j' \in H$$

where $\phi(u)$ is the threshold function (often $\phi(u) = \exp(u)/(1+\exp(u))$).

This generates a relationship between the input vector and the output node label

$$\eta(x) = \sum_{(i,j) \in IO} w_{ij'} x_i + \sum_{(i,j) \in HO} w_{ij'} \phi\left(\sum_{(\mathrm{i},\mathrm{j}) \in IH} \mathrm{w_{ij'}x_i}\right).$$

One input node (the bias node, labelled 0) is added, which accepts a constant input of 1. Note that the transformation ϕ is not applied to the output node.

As in a supervised learning application, the ANN uses training data to determine the weights, which essentially function as the parameters of a nonlinear regression model.

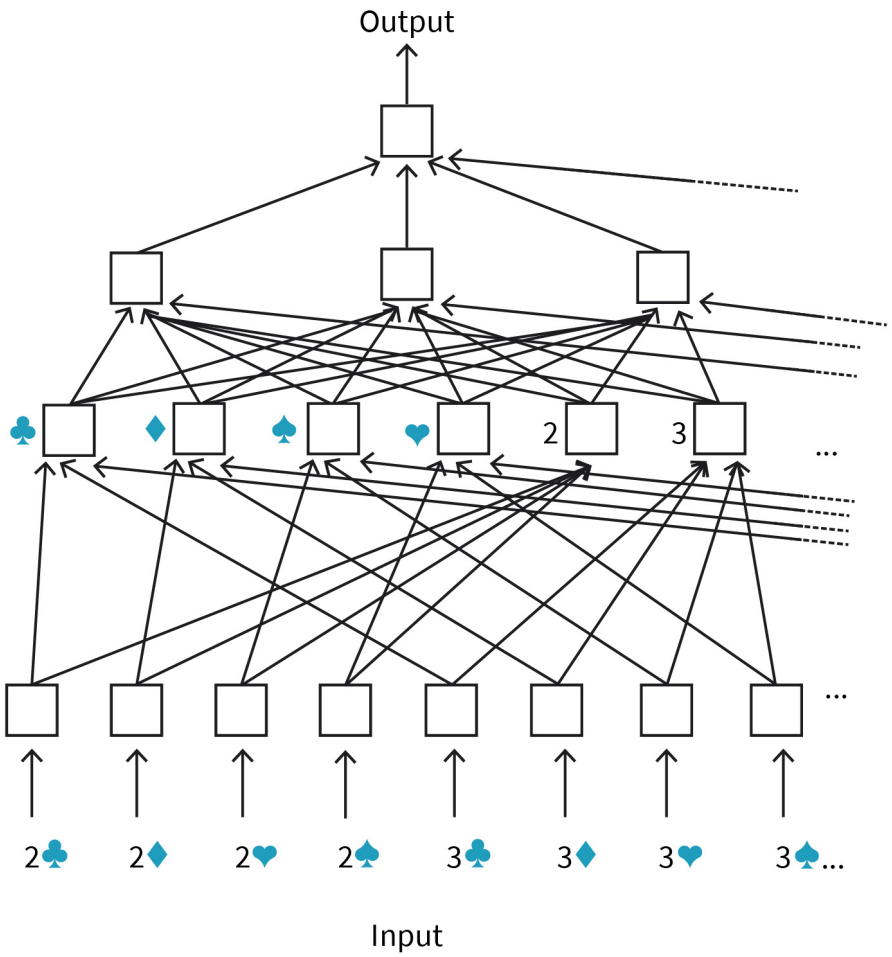
Deep Neural Networks

The multilayer perceptron captures all the essential features of many ANNs.

A typical feature of a deep neural network is that its architecture supports more than one hidden layer. The value of this can be illustrated by considering a partial view of an ANN architecture that might be used in the automated poker application of Sec. u1.box.poker (figure below). There is one output node, and two hidden layers. The input layer contains one node for each of the 52 playing cards. The input therefore can represent a 5 card poker hand. Each input node is set to 1 if the hand contains the card labelled (only the first 8 nodes of the input layer are shown). The intention in deep learning is that feature selection is incorporated into the first several hidden layers. Notice in figure below that there are nodes in the first hidden layer for each suit (clubs, diamonds, hearts, and spades), and the first two nodes associated with each of the ranks (2, 3,..., 10, J, Q, K, A). Notice also that the hidden layer node labeled clubs is connected only to input nodes representing cards of that suit. It is therefore able to store specifically the number of cards of suit clubs in the input hand. Similarly, the hidden layer node labeled with rank 2, clubs is connected only to input nodes representing cards of that rank, and so stores the number of cards of rank 2. Therefore, the earlier hidden layer perform the function of feature selection (Sec. 1.1), capturing the essential characteristics of the hand in a compact form. This information is then used in the additional hidden layers to approximate the value function.

A considerable advantage of this approach for reinforcement learning is that a good approximation of the value function can be attained without the need to visit every state.

Figure 2: ANN architecture for automated poker application of Sec. u1.box.poker.



SUMMARY

In this unit we were introduced to reinforcement learning, which, along with supervised learning and unsupervised learning, constitutes one of the principle branches of machine learning. The range of applications of reinforcement learning was described, as well as its mathematical foundations in dynamic programming, which has long been used to solve many classical control problems in management, operations research and artificial intelligence. The goal of both methods is the identification of optimal decision rules (or policies) for sequential decision processes (SDP).

Reinforcement learning and dynamic programming were then compared, emphasizing a number of shortcomings of classical methods that reinforcement learning and other forms of machine learning are able to overcome.

The reinforcement learning platform was introduced, consisting of the agent and the environment. The agent is responsible for learning through a controlled process of trial-and-error, optimal decision rules for a given environment, which outputs rewards that depend on the agent's actions. The environment is either a computer simulation or a real-life system. An on-policy algorithm will eventually apply the optimal decision rules after sufficient learning.

Various techniques associated with reinforcement learning were reviewed.

TD(λ) denotes a class of algorithms that may be used by the reinforcement learning platform to evaluate the reward obtained by a given policy (policy evaluation) or to estimate the Q-factors used to determine an optimal control.

Two methods for determining optimal control in this manner were defined: Q-learning (off-policy) and SARSA (on-policy). An alternative method for determining the optimal policy known as policy iteration (or the actor-critic method) was also introduced.

The exploration versus exploitation trade-off for on-policy reinforcement learning algorithms was then discussed. This occurs when the imperative of optimizing rewards (using greedy or exploitative control) conflicts with the need of the agent to first learn the optimal policy. The type of exploratory control required for learning is almost always suboptimal from the point of view of optimizing rewards.

However, rewards cannot be optimized until a sufficient amount of learning has permitted identification of the optimal policy. A number of algorithms permitting a gradual transition from exploratory to exploitative control were introduced (ϵ-greedy policies, the E^3 algorithm and the "R-max" algorithm being examples).

Finally, the use of supervised learning techniques, in particular artificial neural networks (ANN), in reinforcement learning was discussed.

Many SDPs generate state spaces $\mathcal{X}$ which are too large to be represented in tabular form in computer memory (for example, the game of backgammon has approximately 10^{20} distinct states).

Furthermore, because a reinforcement learning model learns by experience, the need to sufficiently explore a large state space x can pose practical difficulties.

ANNs and other approximation methods, create compact representations of tables indexed by large state spaces, making such data structures feasible. In addition, newer methods such as deep neural networks automate feature selection, reducing the dimension of the state-space to allow more efficient learning.

UNIT 2

DYNAMIC PROGRAMMING AND MARKOV DECISION PROCESSES

STUDY GOALS

After completing this unit you will have learned ...

- how to precisely define and formulate sequential decision processes.
- how to determine and apply optimal decision rules for these processes.
- how these optimization principles can be implemented by computer programs
- the role played by utility theory in applications from economics.

2. DYNAMIC PROGRAMMING AND MARKOV DECISION PROCESSES

Introduction/Case study

The purpose of this unit is to introduce the concept of the sequential decision process (SDP), which is any dynamic process involving a sequence of decisions which influence some cost or reward that accrues over time.

Ideally, the decisions which optimize the cost or reward can be determined.

In this unit, we will cover how such models can be precisely defined, how cost (reward) can be calculated, and how those decisions which optimize performance can be determined.

The mathematical methods we will use are based on dynamic programming, which was largely formulated by Richard Bellman in the 1950s (Bellman, 1957). As it happens, a single, relatively simple idea provides a remarkable degree of unity to this field, and quite a wide range of applications in operations research, economics, control theory, and artificial intelligence will be seen to share essentially the same structure. Versions of the method were introduced by a number of distinguished mathematicians and economists earlier, in the 1940's (Von Neumann, Morgenstern, & Kuhn, 1944; Arrow, Blackwell, & Girshick, 1949).

2.1 Introduction to Sequential Decision Making

2.1.1. Applications of Dynamic Programming

Dynamic programming is the basis for an astonishingly wide variety of optimization problems that can be formulated as SDPs (Puterman, 1994; Bertsekas, 1995a, 1995b; Bertsekas & Tsitsiklis, 1996; Sutton & Barto, 2018a; Powell, 2011).

Operations Research

This field is concerned with the use of mathematical theory to inform decision making within complex processes, usually sequential in time. A quantitative performance metric can usually be defined (for example, total revenue, probability of success), so that the objective is to determine a decision rule which optimizes performance with respect to this metric.

Operation research is widely used to solve problems in management, engineering, transportation and military or rescue operations. Its methods are especially suitable for determining optimal strategies for resource allocation, search problems and scheduling. Especially prominent examples include supply-chain management, inventory systems, queuing systems, utility expansion and project planning. Applications of this type (for example, the problem of inventory control in (Arrow, Harris, & Marschak, 1951)) appeared quite early, predating the formal development of the theory of dynamic programming by Richard Bellman.

Economics

Apart from its use in improving the efficiency of a wide range of management and engineering applications, dynamic programming is used to solve problems in economics of various kinds. A common application is the design of optimal investment and savings strategies, often within a decision theoretic framework (Adda, Cooper, & Cooper, 2003). Dynamic programming is also used to develop models of human decision making (for example, a dynamic programming model of retirement behavior is proposed in (Rust, 1989)). (Wolpin, 1996) gives a review of the application of dynamic programming to public-policy assessment (for example, to predict the effect of a minimum wage on teenage unemployment). Dynamic programming applications in economics are often qualitative in nature, intended to lead to general principles rather than specific decision rules. This is especially true of the application of dynamic programming to game theory (Von Neumann et al., 1944; Shubik, 1959; Scarf & Hansen, 1973).

Artificial Intelligence

Dynamic programming has been widely used in many artificial intelligence applications. In addition, it provides the mathematical basis of a branch of machine learning known as reinforcement learning, which focuses on determining optimal decisions for a sequential decision process based on the observation of rewards and affecting their outcome by applying varying actions (Powell, 2011; Sutton & Barto, 2018a; Bertsekas, 2019). In a sense reinforcement learning can be regarded as an approximate form of dynamic programming.

One especially compelling early application of reinforcement learning was the development of an SDP, known as TD-Gammon, which is able to play backgammon at a competitive level (Tesauro, 1994, 1995). More recently, reinforcement learning was used by DeepMind Technologies to create a chess-playing program known as AlphaZero ("Entire human chess knowledge learned and surpassed by DeepMind's AlphaZero in four hours", The Telegraph, 6 December 2017). This method is especially suited to develop game-playing applications, especially when coupled with deep neural network models (deep reinforcement learning), and is currently used, for example, to create quite effective video game playing applications (Mnih et al., 2015a).

More generally, dynamic programming and reinforcement learning can be used for artificial intelligence applications involving sequential decision making. This makes them especially suitable for navigational control of autonomous vehicles, and the development of such applications for automobiles (You et al., 2018) or helicopters (Xue et al., 2019) is

currently a thriving area of research. Health care is another prominent area in this field (Esteva et al., 2019), computer aided diagnosis being an especially compelling application (Kao et al., 2018).

In this unit, we will first define the SDP models we consider. This includes the dynamic rules governing the process, the method by which cost is accrued, the rules under which the process terminates (if it does), and the exact form of the decision process itself.

After this, we will introduce Bellman's equation, which formulates the optimization principle underlying most of the methods we will be considering.

Once this is available, we can develop algorithms with which to determine optimal policies. These algorithms express the decision rules used to attain optimal cost or reward.

We will then discuss utility theory that replaces objective measures of cost or reward with the more general concept of value, especially to decision makers with varying preferences regarding risk or wealth accumulation.

This is of importance for applications of dynamic programming in the field of economics and finance.

However, we will first introduce the basic ideas of dynamic programming using the classic stagecoach problem.

2.1.2. Introduction to Dynamic Programming: The Stagecoach Problem

Figure below shows a map of stagecoach routes in the form of a directed graph.

Towns are represented by 11 nodes labeled A to K. A traveller starts in A, and wishes to journey to K. Available routes between towns are represented by arrows, or directed edges. Each edge is labeled by the distance of the route. The problem is to find the route with the minimum total distance.

Clearly, the traveller has a choice to make. A journey from A to K could pass through D, F, G and I before reaching K. For convenience, we'll denote this route A-D-F-G-I-K. Fortunately, the map has enough information to calculate the total distance of this route. Write the distance of, say, edge A-D as $D\{A-D\}=2$. To calculate the distance of route A-D-F-G-I-K, we'll need the following distances:

$$\begin{aligned} D\{A-D\}&=2\\ D\{D-F\}&=4\\ D\{F-G\}&=6\\ D\{G-I\}&=4\\ D\{I-K\}&=3\,. \end{aligned}$$

Clearly, to calculate the total distance for the journey, we simply add the distances that label the edges defining the journey:

$$D\{A-D-F-G-I-K\} = 2+4+6+4+3 = 19\,.$$

The total distance of this route is 19.

But this hardly settles the matter, since A-D-F-G-I-K is not the only route available and, presumably, we wish to find the route from A to K with the minimum total distance. What is the best strategy for finding the optimal route?

Finding the minimum distance route using enumeration

Enumeration may refer either to the problem of evaluating the number of elements of a **set**, or of constructing a readable list of these elements.

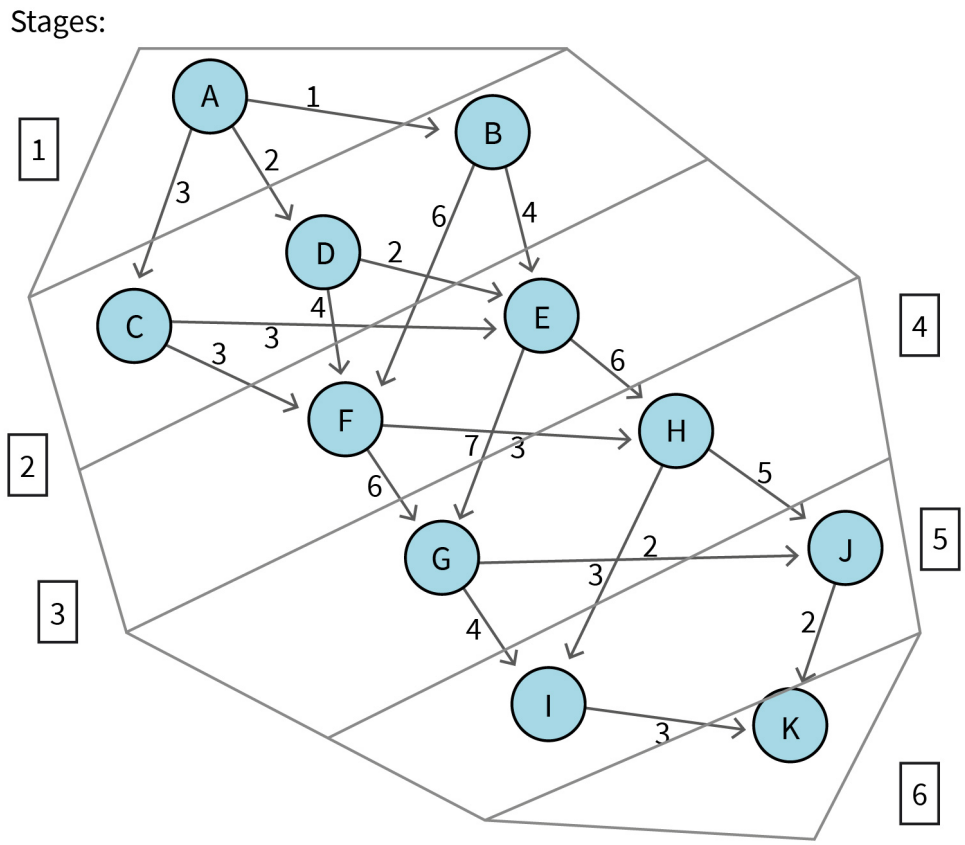
Set
a set is an unordered collection of unique elements

The latter problem is referred to as constructive enumeration. When we refer to an enumeration, we will assume that a constructive enumeration is available.

The enumeration strategy (also known as the brute force approach) is an obvious choice. We count and list all possible routes, and calculate the total distance for each. We then simply pick the shortest route. As it happens, the map shown in figure below possesses structure that would simplify this approach. Upon close examination, it can be seen that the nodes divide naturally into clusters $\{A\}$, $\{B,C,D\}$, $\{E,F\}$, $\{G,H\}$, $\{I,J\}$, $\{K\}$. This allows any route from A to K to be divided into well-defined Stages between clusters:

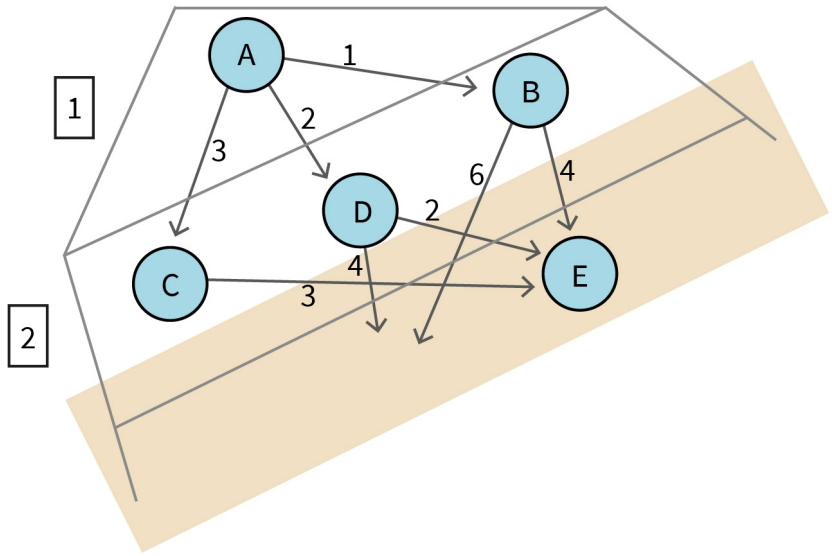
$$\begin{aligned}
&\text{Stage1}: \{A\} \text{to} \{B,C,D\}\\
&\text{Stage2}: \{B,C,D\} \text{to} \{E,F\}\\
&\text{Stage3}: \{E,F\} \text{to} \{G,H\}\\
&\text{Stage4}: \{G,H\} \text{to} \{I,J\}\\
&\text{Stage5}: \{I,J\} \text{to} \{K\}\,.
\end{aligned}$$

Figure 3: Stagecoach problem map.



In figure below, the original map of figure above is shown partitioned into stages (node K becomes the unique member of stage 6). This means that, for example, stage 3 of the route consists of that portion of the journey in which the traveller journeys from either E or F to either G or H (exactly one of these transitions must be part of the journey). A route from A to K then consists of a sequence of stages 1 to 5. This makes it easy to enumerate the routes. At stage 1 we have $n_1 = 3$ choices (travel to B, C or D, starting from A). At stage 2, note that we may travel to E or F no matter what choice was made at stage 1 (this need not be true of all conceivable maps, but it happens to be true of the map shown in figure below).

Figure 4: Stagecoach problem. Map of Fig. 2.1 is shown partitioned into Stages $k = 1, ..., 6$.



So at stage 2 we have $n_2 = 2$ choices. Continuing in this way the number of choices available at stages 3, 4, and 5 are $n_3 = 2$, $n_4 = 1$ and $n_5 = 1$, respectively.

We may now rely on a principle of combinatorics known as the **rule of product**. Suppose a procedure can be broken down into K tasks, and there are always n_i distinct ways to perform the ith task, for $i = 1, ..., K$.

Rule of product
The rule of product is a combinatorial method of enumerating the number of ways to perform a sequence of K tasks

Then there are $n_1 \times n_2 \times ... \times n_K$ distinct ways to perform the entire procedure.

Note that the set of choices available for each task may depend on the choices made for other tasks. It is important that the number of choices for a task is always the same. Clearly, this condition holds for the stagecoach problem, so that by the rule of product, the total number of routes from A to K will be

$$N_{route} = n_1 \times n_2 \times n_3 \times n_4 \times n_5 = 3 \times 2 \times 2 \times 2 \times 1 = 24\,. \tag{2.1}$$

This does not seem to be a prohibitively large number, and an enumerative approach is feasible using a computer, or even pencil and paper, given enough patience.

Then, it is a simple matter to decide whether one route, say A-D-F-G-IK, is preferable to another route, for example, A-D-F-G-J-K. We simply calculate then compare the two total distances:

$$D\{A-D-F-G-I-K\} = 2+4+6+4+3 = 19$$
$$D\{A-D-F-G-J-K\} = 2+4+6+2+2 = 16\,.$$

Clearly, A-D-F-G-J-K is preferable.

However, suppose the number of stages K, and/or the number of choices $n_1, ..., n_K$ were increased by several orders of magnitude. The computation time required to implement the enumerative approach would increase exponentially, and would eventually become infeasible. In a game of backgammon, each of two players has 15 markers, which are arranged on a board with 24 positions. The number of unique marker configurations, which determines the state space, is approximately 10^{20} (Sutton & Barto, 2018a).

Now, the crucial question is: Can we improve on the computational complexity of the enumerative strategy?

2.1.3. Divide and Conquer Algorithms

At this point, we will draw attention to a feature of this calculation that seems obvious. In particular, the problem of calculating the total distance can be decomposed into independent sub-problems:

$$D\{A-D-F-G-I-K\} = D\{A-D-F-G\} + D\{G-I-K\} = 12 + 7 = 19$$
$$D\{A-D-F-G-J-K\} = D\{A-D-F-G\} + D\{G-J-K\} = 12 + 4 = 16\,.$$

More directly, for each route: [Total distance from A to K] + [Total distance from A to G] = [Total distance from G to K].

In other words, in order to calculate the distance from G to K, we don't need to know anything about the route prior to G. We only need to know that we are at node G in order to determine the minimum remaining route to K.

As was suggested earlier, with respect to the problem of calculating route distances, this fact seems obvious. However, it has very important consequences with respect to the problem of minimizing route distances.

Thus, it is clear how a simple modification of the original route A-D-F-G-I-K can decrease the total distance, suggesting that we can improve on the brute-force method. We therefore consider a different approach.

A divide-and-conquer algorithm is one which decomposes a complex problem into multiple, simpler, sub-problems. This is usually accomplished by some recursion method.

The divide-and-conquer method is naturally applied to this problem. If the distance of route segments can be calculated independently of each other, they can also be minimized independently of each other.

At this point, we will need to introduce some new notational conventions and also consider not just the single problem of determining the minimum distance route from A to K, but the entire class of problems of determining the minimum distance route between any two nodes from different clusters.

We will use the symbol D^* to denote such a minimum distance so that, for example, $D^*(B, I)$ is the minimum distance from B to I.

Suppose we next wish to determine the minimum distance route from A to K that passes through G. In optimization theory, such a condition is usually referred to as a constraint. To determine the answer, we can independently determine the minimum distance routes from A to G and G to K. We can modify our notation to introduce constraints, so that $D^*(A, K \mid G)$ denotes the minimum distance of all routes from A to K which pass through G (borrowing notation normally reserved for the conditional probability $P(A \mid B)$. We would then have

$$D^*(A, K \mid G) = D^*(A, G) + D^*(G, K)\,.$$

Of course, the goal of problem 1 is to determine $D^*(A, K)$, and any route with this distance. At this point, we note that any route must pass through either G or H. Therefore, we must have the identity

$$\begin{aligned} D^*(A, K) &= \min\{D^*(A, K \mid G), D^*(A, K \mid H)\} \\ &= \min\{D^*(A, G) + D^*(G, K), D^*(A, H) + D^*(H, K)\} \end{aligned} \tag{2.2}$$

We now have something approaching a divide-and-conquer algorithm, since we can determine $D^*(A, K)$ once we determine $D^*(A, G)$, $D^*(G, K)$, $D^*(A, H)$ and $D^*(H, K)$ (and the associated routes). But does this improve the brute-force enumeration method? We can use the product rule method of Eqn. (2.1) to calculate the number of routes needed to be enumerated in order to calculate each of the four quantities:

$$\begin{aligned} &D^*(A, G) \rightarrow 3 \times 2 \times 1 = 6 \\ &D^*(A, H) \rightarrow 3 \times 2 \times 1 = 6 \\ &D^*(G, K) \rightarrow 2 \times 1 = 2 \\ &D^*(H, K) \rightarrow 2 \times 1 = 2\,. \end{aligned}$$

Therefore, the total number of routes that need to be enumerated is $6 + 6 + 2 + 2 = 16$, less than the 24 routes between A and K calculated in Equation (2.1). We also note that the routes enumerated by the method of Eqn. (2.2) contain fewer nodes, and therefore fewer arithmetic operations are required to calculate their total route distance. Thus, if anything, this comparison of route enumerations understates the advantage of method Eqn. (2.2).

2.2 Dynamic Programming

Dynamic programming is an example of a divide-and-conquer algorithm, and this structure permits a significant reduction in computation time.

Further improvements in computational efficiency can be attained using various approximation techniques. Overall, this makes feasible the solution of optimization problems which could not be contemplated using brute-force enumeration alone. However, whatever efficiencies and approximations are introduced, most general solution methods for a large class of SDPs rely on a single type of model, which we will introduce next.

2.2.1. Dynamic Programming Models

SDPs are often stochastic models, meaning that at least some of its components are randomly determined. In this section, we will introduce some basic ideas of stochastic models.

Random Variables and Stochastic Models

In order to calculate probabilities, we begin with a sample space S This is the set of all possible outcomes. A random experiment occurs when an element of S is selected at random. Suppose E is a subset of S (that is, all elements of E are also in S). Then E is called an event and $P(E)$ is the probability that E occurs, or alternatively, $P(E)$ is the probability that the outcome of the random experiment is in E. We must always have $0 \leq P(E) \leq P(S) = 1$.

A random variable is a numerical outcome $X \in \mathbb{R}$ associated with a random experiment. Associated sets of the form

$$A = \{X \in E\}$$

for subsets $E \subset \mathbb{R}$ are assumed to conform to the rules of probability.

The random variable may also be considered a random experiment with a sample space of outcomes $S_X \subset \mathbb{R}$, in which case X must be a value in S_X. The set S_X is referred to as the support of X. If S_X is discrete then X is a discrete random variable, typically integer variables. A discrete set is either finite, or can be represented as an indexed sequence $a_1, a_2, ...$

If S_X is continuous, then X is a continuous random variable, e.g. real numbers. A continuous set is infinite and cannot be represented as an indexed sequence $a_1, a_2, ...$

Distribution
A distribution of a random variable X defines any probability of the form $P(E)$.

The **distribution** of a random variable X is expressible as probabilities of the form $P(X \in E)$. We may adopt the shorthand

$$P_X(E) = P(X \in E)\,.$$

If S_X consists of integers, we use the shorthand

$$p_i = P(X = i)\,.$$

In this case, the distribution of X consists of the values of p_i for all $i \in S_X$.

EXAMPLE: COIN FLIP

Suppose we toss a coin three times. Assume that all possible outcomes have equal probability. The outcomes are

$$\text{HHH HHT HTH HTT THH THT TTH TTT.}$$

Let X be the number of heads. Then $S_X = \{0, 1, 2, 3\}$. The distribution of X is then

$$\begin{aligned} p_0 &= P(X = 0) = P(TTT) &&= 1/8 \\ p_1 &= P(X = 1) = P(TTH, THT, HTT) &&= 3/8 \\ p_2 &= P(X = 2) = P(HHT, HTH, THH) &&= 3/8 \\ p_3 &= P(X = 3) = P(HHH) &&= 1/8\,. \end{aligned}$$

Note that

$$p_0 + p_1 + p_2 + p_3 = 1$$

as expected.

The **conditional probability** of A given B is defined as

$$P\Big(A \mid B\Big) = \frac{P(A \cap B)}{P(B)},$$

provided $P(B) > 0$. Given the previous example, we have

$$\begin{aligned} P\Big(X = 3 \mid \Big\{X\text{isodd}\Big\}\Big) = \frac{P(\{X = 3\}\text{and}\{X\text{isodd}\})}{P(\{X\text{isodd}\})} \\ &= \frac{P(\{X = 3\})}{P(\{X\text{isodd}\})} \\ &= \frac{1/8}{3/8 + 1/8} \\ &= 1/4\,. \end{aligned}$$

Conditional probability
The conditional probability of an event A given B is the probability that A occurs if we know B has also occurred.

Components of the Dynamic Programming Model

We can think of an SDP as a sequence of time indexed stages. Each stage is associated with a cost or reward. In addition, a decision or action must be taken within each stage. These affect the future behavior and cost of the SDP. The process itself consists of transitions between states belonging to a state space $\boldsymbol{\mathcal{X}}$. An SPD assumes a single state within each stage. The state space $\boldsymbol{\mathcal{X}}$ is the set of all possible states. After the SDP transitions to a new state, a decision in the form of an action selected from an action space $\boldsymbol{\mathcal{A}}$ is made.

The components of a dynamic programming model are as follows:

1. An SDP is a dynamic process defined on discrete time points, or epochs, $j = 1, 2, ...$ A stage is associated with each epoch j. Each stage j contains a state x_j and action a_j. We then refer to a stateaction pair $\big(x_j, a_j\big)$. Conceptually, within a stage, the state x_j is first observed. Then, with knowledge of the state, a decision is made, in the form of an action a_j selected from a well-defined set of possibilities.
   However, it is the usual practice to refine this definition, noting that not all actions $a \in \mathcal{A}$ may be available from any given state x.
   We then let $\mathcal{A}_x$ be all actions available from state x. Then, (x, a) is an admissible state-action pair if action a is permitted from state x.
   The set of all admissible state-action pairs is the state-action space.

   $$\boldsymbol{\mathcal{K}} = \big\{(x, a) : x \in \boldsymbol{\mathcal{X}}, a \in \boldsymbol{\mathcal{A}}_x\big\}.$$

2. When action a_j is selected from state x_j, a cost (or reward) $R\big(x_j, a_j\big)$ is accrued, where $R(x, a)$ is the cost (reward) function defined for every admissible state-action pair $(x, a) \in \boldsymbol{\mathcal{K}}$.
3. There is a transition rule that determines how the SDP moves from state x_j to x_{j+1}. This rule depends on the current state-action pair $\big(x_j, a_j\big)$. State transitions that are random are referred to as stochastic transitions, meaning that a subsequent state x_{k+1} is determined as the outcome of a probability distribution, which depends on current state-action pair (x_k, a_k). Stochastic transitions are governed by a probability kernel $P(y \mid x, a)$ defined for each possible state-action pair (x, a). Given current state-action pair $\big(x_j, a_j\big)$, the probability that the subsequent state x_{j+1} is equal to y is $P\big(y \mid x_j, a_j\big)$. Possibly, the transitions are deterministic, meaning that once $\big(x_j, a_j\big)$ is known, there is only one possible value for the subsequent state x_{j+1}. In this case, the probability kernel $P\big(y \mid x_j, a_j\big)$ can be replaced by a transition function $q(x, a)$, which gives the transition rule $x_{j+1} = q\big(x_j, a_j\big)$.
   A deterministic transition rule can be represented by a mapping $q(x, a) \in \boldsymbol{\mathcal{X}}$ for each admissible $(x, a) \in \boldsymbol{\mathcal{K}}$. Of course, this can also be modeled by the probability kernel by setting

   $$P(q(x, a) \mid x, a) = \begin{cases} 1 \ ; \ (x, a) \in \boldsymbol{\mathcal{K}} \\ 0 \ ; \ (x, a) \notin \boldsymbol{\mathcal{K}} \end{cases}$$

   This means $P\big\{x_{j+1} = q\big(x_j, a_j\big)\big\} = 1$, which defines a deterministic transition rule.

The components of the SDP model describe the rules governing how the process behaves. The components of an SDP are collectively referred to as a model. In particular, this includes state space $\boldsymbol{\mathcal{X}}$, action space $\boldsymbol{\mathcal{A}}$, stateaction space $\boldsymbol{\mathcal{K}}$, cost function $R(x, a)$, and transition function $q(x, a)$ (for deterministic transitions) or probability kernel $P(y \mid x, a)$ (for stochastic transitions). We will assume, at least at first, that the model is completely known to the decision maker (in fact, reinforcement learning is motivated in part by the possibility that this is not the case). However, we have not yet formulated a decision problem and there will be more than one way to do this.

Formulating the Stagecoach Problem Using a Dynamic Programming Model

In practice, the type of analysis associated with dynamic programming rewards mathematical clarity (and also penalizes the lack thereof). It is therefore essential to make sure that all objects making up the SDP are carefully defined, starting with $\boldsymbol{\mathcal{X}}$, $\boldsymbol{\mathcal{A}}$ and $\boldsymbol{\mathcal{K}}$. For the stagecoach problem we have

$$
\begin{aligned}
\mathcal{X}&=\{A,B,C,D,E,F,G,H,I,J\}\\
\mathcal{A}&=\{B,C,D,E,F,G,H,I,J,K\}\\
\mathcal{A}_A&=\{B,C,D\}\\
\mathcal{A}_B&=\{E,F\}\\
&\vdots\\
\mathcal{A}_J&=\{K\}\\
\mathcal{K}&=\{(A,B),(A,C),(A,D),(B,E),(B,F),(C,E),(C,F),(D,E),(D,F),\\
&\quad (E,G),(E,H),(F,G),(F,H),(G,I),(G,J),\\
&\quad (H,I),(H,J),(I,K),(J,K))\}
\end{aligned}
$$

Note that state space $\boldsymbol{\mathcal{X}}$ does not include K. This is not needed, since, being the terminal node, no decision is ever made from it. Similarly, A is not included in $\boldsymbol{\mathcal{A}}$, since, being the initial node, it never defines an action.

However, as one becomes more experienced with dynamic programming, a certain flexibility becomes essential. We are possibly defining a sequential decision process which may eventually become embedded within a larger one, and, looking ahead, we would like to include K in the state-space. The problem is that the set $\boldsymbol{\mathcal{A}}_K$ would be empty, which has the potential to create problems (that $\boldsymbol{\mathcal{A}}_K$ is nonempty is a common regularity assumption in the theory of dynamic programming). An easy solution is to define a "dummy action" Δ. We could even assign to this action the function of terminating stopping the process. We then set

$$\mathcal{A}_K = \{\Delta\},$$

so that the only action available from state K is Δ. If we then extend the cost function to $R(K,\Delta) = 0$, we have not changed the decision process in any important way, other than to facilitate future modifications (we define this idea more rigorously below).

Otherwise, we can see that the state-action space $\boldsymbol{\mathcal{K}}$ is in one-to-one correspondence with the map of the figure entitled as "Stagecoach problem map", since an action from a state is nothing more than a choice of edge along which to continue the journey.

Similarly, the cost function $R(x,a)$ is simply the distance associated with the edge $x-a$. For example, from the figure entitled as "Stagecoach problem map" we have

$$
\begin{aligned}
&R(A, B) = 1, \\
&R(A, C) = 3, \\
&\vdots \\
&R(I, K) = 3, \\
&R(J, K) = 2\,.
\end{aligned}
$$

Finally, to complete the definition of the environment, if action a_j is taken from state x_j, the transition rule is fully defined by the assignment $x_{j+1} = q\big(x_j, a_j\big) = a_j$. We then have

$$
\begin{aligned}
&q(A, B) = B, \\
&q(A, C) = C, \\
&\vdots \\
&q(I, K) = K, \\
&q(J, K) = K\,.
\end{aligned}
$$

Memoryless SDPs

At this point, we will introduce a crucial concept. The decision process may or may not be memoryless. Intuitively, this means that to make an optimal decision, we only need to know the current state of the process, and not its history. Of course, this has a formal mathematical definition, which we give next.

An SDP is memoryless, meaning that all subsequent behavior and costs depend only on the current state-action pair if the following rules apply:

1. The transition rule from state x_j to x_{j+1} depends only on current stage data $\big(x_j, a_j\big)$.
2. The total remaining cost accrued by the process from state x_j depends only on current stage data $\big(x_j, a_j\big)$.

Transition rules defined by either a transition function $q(x, a)$ or probability kernel $P(y \mid x, a)$ are memoryless, but we also need to ensure that cost accrual and the decision rules themselves are memoryless.

SDP Lifetime

The lifetime of an SDP must be precisely defined. An SDP has a finite horizon if the number of stages is a finite number $K < \infty$ that is fixed in advance. In contrast, an SDP has an infinite horizon if the number of stages is infinite, meaning that the process continues indefinitely. Finally, for a shortest path SDP, the number of stages is finite but possibly variable.

An SDP may have one state $x_\Delta \in \boldsymbol{\mathcal{X}}$, designated the terminal state. The process ends once this state is reached. Note that when we say "process ends" we might mean that the process never leaves state x_Δ, nor accrues any further cost. In this way, an infinite horizon process can "end" at a terminal state. A shortest path SDP must have a terminal state.

There is some flexibility in this classification system since it is often possible to reformulate a problem of one class as a problem of another class.

For example, it is always possible to formulate a finite horizon problem as a shortest path problem. While the stagecoach problem is naturally expressed as a finite horizon problem, it can be formulated as a shortest path problem by defining terminal state $x_{\Delta} = K$.

Intuitively, the obvious distinction between a finite horizon SDP and a shortest path SDP concerns whether or not the number of stages of the process is fixed or variable. However, another important distinction concerns the exact role played by the stage. In all SDPs, the stage defines a discrete time period in which a state x_j is occupied, an action a_j is taken, and a cost $R\big(x_j, a_j\big)$ is accrued. However, in the stagecoach problem, the stage imposes further structure on the SDP. Both the states and actions partition by stage in the sense that each state and each action can be observed in only one stage (for example, state $x_3 = E$ can occur only in stage $j = 3$).

We must consider the possibility that an SDP may be memoryless, but that various components may vary by stage.

Time Homogeneity

An SDP in which some component of the model changes by stage is a time inhomogeneous SDP, otherwise it is a time homogeneous SDP process. This means that, for a time inhomogeneous SDP, the cost function or transition rule does not remain constant as the stages progress, whereas they remain constant for time homogeneous SDPs.

A finite horizon process is usually time inhomogeneous, while a shortest path process is usually time homogeneous. For this reason, when the state space of a finite horizon problem does not naturally partition by stage, it may be necessary to expand the definition of a state to include a stage index. For example, suppose that an inventory control model we define a stage as a single day. At stage j, the state x_j might be the quantity of stock at the beginning of the day and the action a_j may be the amount of stock reordered on that day after considering current stock x_j. We could then expand the state, written as $x_j = \big(w_j, j\big)$, where w_j is now the stock at the opening of day j, and the day index j is now incorporated into the state.

One thing to keep in mind when designing dynamic programming models is that there are usually multiple ways to formulate an SDP.

SDP Cost Accrual Method

The SDP has a cost accrual method that defines the manner in which stage costs or rewards and consolidated into a single value. Suppose an SDP starts at initial state x_1 and continues according to its horizon or termination. The process accrues a total cost, which can be defined recursively.

Suppose from x_1 action a_1 is selected. The immediate stage cost is $R(x_1, a_1)$ according to the model's cost function, and the process transfers to state x_2, according to the model's transition rule. We can imagine the process restarting from what is now the initial state x_2. Suppose the total accrued cost of this SDP is R_2. Then the total accrued cost of the original

SDP with initial state x_1 is $R_1 = H(R(x_1, a_1), R_2)$, where the function $H(a, b)$ defines the accrual method. It is important to note that this form of cost accrual satisfies the memoryless property for SDPs.

A cost accrual method is defined by a function $H(a, b)$, which may assume various forms.

Total Cost Accrual Method

A finite horizon or shortest path SDP is a total cost/reward problem. In this case, the total accrued cost/reward is the sum of the stage costs/rewards

$$R_{total} = R(x_1, a_1) + R(x_2, a_2) + ... + R(x_{N-1}, a_{N-1}) + R(x_N, a_N), \tag{2.3}$$

where $N = K$ for a K-stage finite horizon problem. This corresponds to the accrual method defined by $H(a, b) = a + b$.

Discounted Cost Accrual Method

The motivation for considering the infinite horizon lifetime is to model SPDs that are expected to run indefinitely. For example, while a game usually has a natural terminal state, an inventory control system can typically be expected to continue indefinitely, so that the total accrued cost will be an infinite sum. This means that unless we expect cost to converge to zero at a fast enough rate, the total accrued cost will be infinite and the optimization problem will not be well defined (clearly, this would not be a reasonable constraint). One widely used solution is to introduce discounting. An SDP is a discounted cost/reward problem with discount factor $\beta \in [0, 1]$ if the total accrued cost is

$$R_{discount} = \sum_{j=1}^{N} \beta^{j-1} R\big(x_j, a_j\big) \tag{2.4}$$

for a finite horizon or shortest path problem, or

$$R_{discount} = \sum_{j=1}^{\infty} \beta^{j-1} R\big(x_j, a_j\big) \tag{2.5}$$

for an infinite horizon problem. This corresponds to the accrual method defined by $H(a, b) = a + \beta b$. As before, the total accrued cost/reward is the sum of the stage costs/rewards, however, each term is now modified with a discount factor $\beta \leq 1$ that determines the rate at which future costs/rewards are discounted.

To see how discounting works, suppose all stage costs are non-negative, and bounded above by a finite constant $M \geq R(x, a) \geq 0$. Suppose $\beta < 1$.

Then for any sequence of actions $a_1, a_2, ...$ we will have

$$R_{discount} = \sum_{j=1}^{\infty} \beta^{j-1} R\big(x_j, a_j\big) \leq M \sum_{j=1}^{\infty} \beta^{j-1} = \frac{M}{1-\beta},$$

making use of the geometric series. Thus, as long as $\beta < 1$, the total discounted cost will be finite.

$\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r}$ for $|k| < 1$

We may refer to the total and discounted cost accrual methods as additive (in particular, $H(a, b)$ is a linear combination of a and b). We will see that this property is important when considering stochastic SDPs. However, cost accrual methods need not be additive, as we see next.

Minimax Cost Accrual

An SDP is a **minimax cost problem** if the accrued cost is

$$R_{max} = \max\big\{R(x_1, a_1), R(x_2, a_2), ..., R(x_N, a_N)\big\} \tag{2.6}$$

Minimax cost problem
For a minimax cost problem the total accrued cost is the maximum stage cost over all stages.

for a finite horizon or shortest path problem, or

$$R_{max} = \max\{R(x_1, a_1), R(x_2, a_2), ...\} \tag{2.7}$$

for an infinite horizon problem. This corresponds to the accrual method defined by $H(a, b) = \max\{a, b\}$.

Maximin Reward Accrual

Maximin reward problem
For a maximin reward problem the total accrued reward is the minimum stage reward over all stages.

A finite horizon or shortest path SDP is a **maximin reward problem** if the accrued reward is

$$R_{min} = \min\{R(x_1, a_1), R(x_2, a_2), ..., R(x_N, a_N)\} \tag{2.8}$$

for a finite horizon or shortest path problem, or

$$R_{min} = \min\{R(x_1, a_1), R(x_2, a_2), ...\} \tag{2.9}$$

for an infinite horizon problem. This corresponds to the accrual method defined by $H(a, b) = \min\{a, b\}$.

We present two applications in which the minimax and maximin criteria would define a natural performance goal.

Controlling Course Deviation: The Minimax Criterion

Suppose that dynamic programming will be used to control the navigation of a vehicle. Such a system can likely be broken down into a collection of tasks. One task may be for the vehicle to travel in a straight line between two points. At regular decision epochs, deviation from the planned trajectory will be measured by sensors and a course correction will be applied.

Suppose the stage k deviation is given by ϵ_k and that it is imperative that course deviation never exceeds some safety threshold τ_S. In such systems, we will inevitably face scenarios under which any single deviation beyond the tolerance $|\epsilon_k| > \tau_S$ will result in a system failure (for example, veering off a highway or hitting a wall). Thus, the objective is to construct a decision rule which will govern each course correction. The performance goal of this decision rule will be to minimize course deviation in some aggregate sense.

We can assume that the sign of ϵ_k signifies the left/right orientation of the deviation. The stage cost can then be taken as absolute deviation $|\epsilon_k|$. The total additive cost can then be written as follows:

$$R_{tot} = \sum_{k=1}^{N} |\epsilon_k|,$$

where N defines the end of the trajectory. Is this an appropriate optimization goal? The problem with using R_{tot} as the performance criterion is that its optimization does not guarantee that each single deviation will be within some bound. We could not rule out the possibility that, from the point of view of minimizing R_{tot}, the best strategy is to accept a small number of large course deviations at the beginning of the trajectory in order to attain a smaller total additive absolute deviation in later stages.

Of course, this would result in a system failure.

Clearly, a more appropriate accrual method is the minimax method, which attempts to minimize

$$R_{max} = \max_{k \in \{1, ..., N\}} |\epsilon_k|$$

A decision rule which achieves $R_{max} < \tau_S$ by a large enough margin would attain the performance objective of preventing a catastrophic course deviation $|\epsilon_k| > \tau_k$ at any point on the trajectory.

Maintaining Minimum Fuel Requirements: The Maximin Criterion

We next give an example of an SDP for which the maximin reward criterion would be appropriate.

Suppose a dynamic programming model for a transportation network dispatching system defines the current location of a vehicle and a decision regarding its next destination as a stage. The process state at stage k consists of the current vehicle location x_k as well as the amount of fuel f_k currently contained in the vehicle. Then, a decision is defined as the pair $a_k = (x_{k+1}, g_k)$, where x_{k+1} is the next destination and g_k is the amount of fuel to add to the vehicle. Then, let d_k be the distance between x_k and x_{k+1}. The total available fuel per kilometer for the mission is then

$$r_k = \frac{f_k + g_k}{d_k}.$$

Clearly, the essential objective is to ensure that the vehicle contains enough fuel for the mission, and any decision rule must meet this constraint. To fix ideas, we will assume that there is a safety threshold, say τ_F, which specifies the minimum fuel requirement per kilometer, so that we must have

$$r_k \geq \tau_F$$

for every stage k. A suitable performance criterion can be defined by taking r_k as the stage cost, then using the maximin reward accrual method. The optimization problem will then be to maximize the minimum reward:

$$R_{min} = \min_{k \geq 1} r_k .$$

This will ensure that minimum fuel requirements are met for each mission.

Note that the SPD may have an infinite horizon and needs to rely on discounting to ensure that R_{min} remains bounded.

2.2.2. Policies and Actions

An SDP is controlled by a policy, which is a method of choosing actions at each stage. The goal of a policy is usually to optimize total accrued cost or reward. In order to achieve this, a policy must usually depend at least on the current state. For this reason, it is important to characterize exactly what information may be available to inform the choice of action.

Decision Theory and Information

Suppose information X is available to make a decision, and that the optimal decision rule is $\delta_X(X)$, in the sense that it minimizes some cost. Then suppose new information Y becomes available, and that the best decision rule using combined information (X, Y) is $\delta_{XY}(X, Y)$. Then δ_{XY} is at least as good a decision rule as δ_X, since we can always set $\delta_{XY}(X, Y) = \delta_X(X)$ (that is, we can always ignore the new information Y).

Conversely, suppose we are given an optimal decision rule $\delta_{XY}(X, Y)$ based on information (X, Y). We discover, by mathematical or empirical investigation, that the optimal decision rule based on the reduced information X, say $\delta_X(X)$, is, in effect, equal to δ_{XY}. Then decision rule $\delta_X(X)$ is preferable, especially from the point of view of computational efficiency.

The relationship between information and decision rules presents a tradeoff.

Additional information can never impair an optimal decision rule, but it can increase the computational complexity of any algorithm used to compute it. In dynamic programming this issue can be decisive. As we will see, a crucial step in the design of any SDP is the identification of the information that is available to make a decision, and how much of that information will be used.

In an SDP, a Stage j decision a_j is made after state x_j is observed. Also (ideally) the **history** up to that stage would be known, which would consist of all state-action pairs (x_i, a_i) for all previous stages $i = 1, ..., j-1$, which we denote

History
For an SDP, process history is a record of all stage data available at a point of time just before an action is to be selected.

$$H_j = \left(x_1, a_2, ..., x_{j-1}, a_{j-1}\right).$$

For example, in the stagecoach problem, suppose we are at stage $j = 3$. If the traveller is at node E, then the state is $x_3 = E$. The decision to be taken at this stage is the decision of the next city, either G or H. Mathematically, we represent this as $a_3 \in \{G, H\}$. Then, the process history consists of the two state-action pairs (x_1, a_1), (x_2, a_2). If the route taken so far is A - C - E - H, then the current state-action pair and history at stage $j = 3$ are

$$\begin{aligned}(x_3, a_3)&=(E, H),\\ H_3&=(A, C, C, E)\end{aligned}$$

he most general form of a decision rule will use both the current state and the history, and will therefore take the form

$$a_j = \phi_j\left(x_1, a_1, ..., x_{j-1}, a_{j-1}, x_j\right) = \phi_j\left(H_j, x_j\right)$$

for stage j. At this point, the decision rule makes use of the information $\left(H_j, x_j\right)$. How much of this do we need? We usually expect that an optimal decision could not be made without knowledge of the current state x_j. But do we need H_j? If the SDP is memoryless, then all information that may be used to predict future behavior is contained in x_j alone. Therefore, an optimal decision can depend on x_j alone and can take the simpler form of a policy function $\phi j(x)$ that defines a decision rule for stage j by assigning a single action to each state:

$$a_j = \phi_j\left(x_j\right).$$

Note that we have allowed the policy function to depend on the stage j.

Our preference from this point on will be to remove this dependence. If necessary, identification of the stage can be incorporated into the state. In this case, we can make use of a stationary policy function, which does not depend on the stage, and can therefore be written $\phi_j\left(x\right) = \phi\left(x\right)$, that is, we may remove the subscript j. Here, the decision rules are the same for all stages. It is important to note that this form of decision rule satisfies the memoryless property.

2.2.3. Value Functions

Once we have a model and a cost accrual method, we can assign a cost to a policy, and on this basis select an optimal policy, which we denote ϕ^* when considering stationary policies. To fix ideas, suppose we define the policy value function $V^{\phi}\big(x\big)$ to be the total cost/reward accrued under policy ϕ, under any method, from initial state x. Then an optimal policy $\phi^*(x)$ achieves the optimal total accrued cost/reward. In other words $V^{\phi^*} = V^*$.

Deterministic SDP models

A deterministic SDP will use deterministic state transitions function $q(x, a)$.

Suppose we are given a stationary policy $\phi(x)$ for a shortest path SDP using the total cost accrual method. Then from Equation (2.3) we have:

$$V^{\phi}(x_1) = R(x_1, \phi(x_1)) + R(x_2, \phi(x_2)) + ... + R(x_{N-1}, \phi(x_{N-1})) + R(x_N, \phi(x_N)), \tag{2.10}$$

fixing any $x = x_1$. The definition of the policy value function is the same for initial state $x = x_2$:

$$V^{\phi}(x_2) = R(x_2, \phi(x_2)) + R(x_3, \phi(x_3)) + ... + R(x_{N-1}, \phi(x_{N-1})) + R(x_N, \phi(x_N)), \tag{2.11}$$

which can be substituted back into Eqn. (2.10) to give

$$V^{\phi}(x_1) = R(x_1, \phi(x_1)) + V^{\phi}(x_2), \tag{2.12}$$

which conforms to the cost accrual method $H(a, b) = a + b$. In general, we then have:

$$V^{\phi}(x) = R(x, \phi(x)) + V^{\phi}(q(x, \phi(x)))\,. \tag{2.13}$$

What if we use discounted cost? The argument is much the same, due to the manner in which discounted costs conveniently factor. Eqn. (2.10) becomes

$$\begin{aligned} V^{\phi}(x_1) &= R(x_1, \phi(x_1)) + \beta R(x_2, \phi(x_2)) + ... + \beta^{N-1} R(x_N, \phi(x_N)) \\ &= R(x_1, \phi(x_1)) + \beta\big[R(x_2, \phi(x_2)) + ... + \beta^{N-2} R(x_N, \phi(x_N))\big] \\ &= R(x_1, \phi(x_1)) + \beta V^{\phi}(x_2)\,. \end{aligned}$$

It is important to note that when we evaluate $V^{\phi}\big(x_2\big)$ in the preceding equation the discounting is restarted as though x_2 is the initial state of stage 1. Using more general notation gives

$$V^{\phi}(x) = R(x, \phi(x)) + \beta V^{\phi}(q(x, \phi(x)))\,. \tag{2.14}$$

The only difference between the recursion formulae of Eqn. (2.13) and (2.14) is the appearance of the discount factor β in (2.14). We could therefore regard (2.13) as a special case of (2.14) obtained by setting $\beta = 1$ (which our definition allows).

For a minimax SDP, the type of recursive relationship given in Eqn. (2.13) or (2.14) would now become

$$V^{\phi}(x) = \max\Big\{R(x, \phi(x)), V^{\phi}(q(x, \phi(x)))\Big\}\,. \tag{2.15}$$

We will see an example of a minimax SDP in Sec. 2.4.3.

Stochastic SDP models

For a stochastic SDP, the stage cost will be a random variable. Since the value function measures the accrual of costs, it is the usual practice to use expected cost in place of realized cost. Therefore, to define the cost of a stochastic SDP model, we need to use the notion of the expected value of a random variable X.

The Expected Value of a Random Variable

The quantity $E[X]$ is called the expected value of a random variable X and represents a theoretical average of an infinitely large sample. The notation

$$\mu = \mu_X = E[X]$$

is frequently used. For discrete random variables the expected value is calculated by

$$E[X] = \sum_{x \in S_X} xP(X = x), \tag{2.16}$$

that is, for every possible outcome in the support S_X we contribute $xP(X = x)$ to the sum.

We may condition on an event A and use conditional probabilities $P(X = x \mid A)$ in place of $P(X = x)$ in Eqn. (2.16). This gives the conditional expected value

$$E[X \mid A] = \sum_{x \in S_X} xP(X = x \mid A)\,.$$

Accrual of Random Costs/Rewards

This principles behind the deterministic SDP models are much the same when state transitions are stochastic, but we must then rely on the calculus of probability (both the cost $R(x, a)$ and the state transition rules may be stochastic). We have already introduced the stochastic kernel $P(y \mid x, a)$ to model stochastic state transitions. We can also replace cost function $R(x, a)$ with a random variable $\widehat{R}\big(x, a\big)$, possessing a distribution that depends on (x, a). However, in this case something like the original cost function will reappear defined as

$$R(x, a) = E\big[\widehat{R}(x, a)\big]\,.$$

We would then consider the expected total cost of a stochastic SDP with initial state x_1.

$$E[R_{total}] = E\big[\widehat{R}(x_1, \phi(x_1)) + \widehat{R}(x_2, \phi(x_2)) + ... + \widehat{R}(x_N, \phi(x_N))\big]\,. \tag{2.17}$$

Then $V^{\phi}\big(x\big)$ is the expected total cost conditional on initial state $x_1 = 1$.

So we define

$$
\begin{aligned}
V^{\phi}(x) &= E[R_{total} \mid x_1 = x] \\
&= E\big[\widehat{R}(x_1, \phi(x_1)) \mid x_1 = x\big] + E\big[\widehat{R}(x_2, \phi(x_2)) + ... + \widehat{R}(x_N, \phi(x_N)) \mid x_1 = x\big] \\
&= E\big[\widehat{R}(x_1, \phi(x_1)) \mid x_1 = x\big] + E\big[V^{\phi}(x_2) \mid x_1 = x\big] \\
&= R(x, \phi(x)) + \sum_{y \in \boldsymbol{\mathcal{X}}} V^{\phi}(y) P(y \mid x, \phi(x)),
\end{aligned}
\tag{2.18}
$$

(note the use of conditional expectations). Discounting can be introduced in the same way that leads from Eqn. (2.13) to (2.14) to yield

$$
V^{\phi}(x) = R(x, \phi(x)) + \beta \sum_{y \in \mathcal{X}} V^{\phi}(y) P(y \mid x, \phi(x)) \,.
\tag{2.19}
$$

However, the stochastic properties of the SDP introduce an important technical consideration. In general, expectation is a linear operation. This means that if X, Y are any two random variables, and a, b are two constants, we have

$$
E[aX + bY] = aE[X] + bE[Y] \,.
$$

This allows a cost accrual method $H(a, b)$ to be applied to expected costs, in addition to the actual observed costs, when the accrual method is additive.

This ensures that Eqn. (2.19) will hold. Otherwise, this convenient structure may not be available. This will be the case for minimax cost, since, in general,

$$
maxE[X], E[Y] \leq E[maxX, Y],
$$

and the inequality is usually strict. The subtleties of this type of calculation may be better appreciated after reviewing the excellent introduction to conditional expectations given in (S. Ross, 2014).

2.2.4. Markov Decision Processes

A stochastic process may be defined as an indexed collection of random variables $\{X_t\}$, $t \in \boldsymbol{\mathcal{T}}$, assuming a value in some state space $\boldsymbol{\mathcal{X}}$. The index t usually represents time, although stochastic processes may also be used to describe random processes defined on some space.

Most stochastic processes are either discrete time, and take the from of a sequence $X_1, X_2, ...$, or continuous time, and may be represented as a process $X[t]$ on a subset $t \in [0,\ \infty)$, with $X[t]$ being the value of the process at time t.

The Markov chain is a discrete time stochastic process. The defining property is the memoryless property or Markovian property, essentially meaning that the distribution of future states depends on the current state X_j, but not on previous states.

Definition of a Markov chain

Suppose we are given a discrete time stochastic process $X_n \in \mathcal{X}$, $i = 0, 1, 2, ...$, which assumes values in a discrete state space $\mathcal{X}$. Without loss of generality we have either a finite state space $\mathcal{X} = \{0, 1, ..., n\}$ or countable state space $\mathcal{X} = \{0, 1, ...\}$. Then X_i is a Markov chain if the following memoryless property holds:

$$\begin{aligned} P\big(X_{n+1}{=}j \mid X_n = i, X_{n-1} = i_{n-1}, ..., X_1 = i_1, X_0 = i_0\big) \\ {=}P\big(X_{n+1} = j \mid X_n = i\big) \\ {=}P_{ij}\,. \end{aligned}$$

The quantity P_{ij} is called the transition probability from state i to state j. We also have transition probability matrix (or transition matrix)

$$P = \begin{bmatrix} P_{00} & P_{01} & P_{02} & \cdots \\ P_{10} & P_{11} & P_{12} & \cdots \\ \vdots & \vdots & \vdots & \\ P_{i0} & P_{i1} & P_{i2} & \cdots \\ \vdots & \vdots & \vdots & \end{bmatrix}$$

Row i of transition matrix P is equivalent to the conditional probability .

$$P\big(X_{n+1} = j \mid X_n = i\big) = P_{ij},\ j \in \boldsymbol{\mathcal{X}}\,.$$

Also note that P will be a matrix of infinite dimension when X is countable.

We also have no difficulty conceiving of P as 'doubly infinite' when the state space is the set of positive and negative integers $\{..., -2, -1, 0, 1, 2, ...\}$, which requires no important change of our definition.

Example of a Markov chain

We consider an example of a two-state Markov chain, which, despite its simplicity, demonstrates a number of important features of Markov chains.

Formally, we have state space $\boldsymbol{\mathcal{X}} = \{0, 1\}$.

For example, the time index $i = 0, 1, 2, ...$ may represent a sequence of days, and we may wish to define a simple infection model, in which state $i = 0$ represents a healthy state H and $i = 1$ represents a sick state I (for example, due to an infection). The transition matrix is therefore the 2×2 matrix

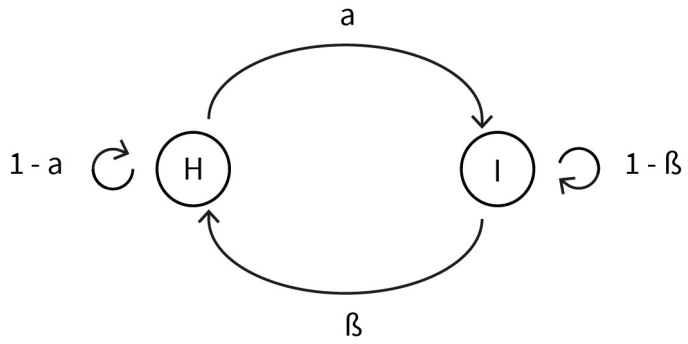
$$P = \begin{bmatrix} P_{00} P_{01} \\ P_{10} P_{11} \end{bmatrix}.$$

However, the true degrees of freedom of P is 2, since each row is constrained, as a probability distribution, to sum to 1. We can therefore write, without loss of generality,

$$P = \begin{bmatrix} 1 - \alpha & \alpha \\ \beta & 1 - \beta \end{bmatrix}. \tag{2.20}$$

for two numbers $\alpha, \beta \in [0, 1]$. This means that if a subject is healthy on day i, they are sick on day $i + 1$ with probability α, and if the subject is sick on day i, they are sick or healthy in day $i + 1$ with probability β. The state transition diagram for an infection model is shown in figure below.

Figure 5: State transition diagram for infection model.



Definition of a Markov Decision Process

A particularly important class of SDP is known as the Markov decision process (MDP). which is an infinite horizon SDP with state-action space $\boldsymbol{\mathcal{K}}$, cost function $R(x, a)$ and probability kernel $P(y \mid x, a)$. It uses the discounted cost/reward accrual method with discount factor β. We have, in fact, already introduced it as a time homogenous infinite horizon SDP with stochastic transitions and discounted cost accrual method (although the theory also extends to the average cost model). The model is defined by the n-tuple

$(\boldsymbol{\mathcal{K}}, R, P, \beta)$, where $\boldsymbol{\mathcal{K}}$ is the state-action space $\boldsymbol{\mathcal{K}} \subset \boldsymbol{\mathcal{X}} \times \boldsymbol{\mathcal{A}}$, and we are given cost function $R(x, a)$, $(x, a) \in \boldsymbol{\mathcal{K}}$, probability kernel $P(y \mid (x, a))$ and discount factor $\beta \in [0, 1]$ (we usually assume that $\beta < 1$, but the mathematical theory is more flexible if $\beta = 1$ is an admissible choice).

The mathematical theory of MDPs is quite rich, and this has much to do with the MDP's alternative identity as a controlled Markov process.

Suppose we fix a policy ϕ. We can think of this as eliminating the role of the action in the SDP, which then becomes a Markov chain with transition probabilities .

$$p_{x,\,y} = P(y \mid x, \phi(x)) \,.$$

We can therefore think of an MDP as a class of Markov chains, which can be selected (and altered) with the purpose of satisfying a well-defined decision problem.

If the reader is new to this topic, a good starting point would be (S. Ross, 2014). The text (S. M. Ross, 1996) then covers Markov chains at the next level. This second volume will cover theory sufficient for most issues arising from the study of MDPs. A more advanced treatment of Markov chains, especially suitable for the study of dynamic systems, can be found in (Brémaud, 1999).

We will usually assume that K is a discrete set, although not necessarily finite. We have further assumed that the SDP is a discrete-time process, defined on time indices $i = 1, 2, \ldots$. If the value of i has any quantitative interpretation (or time units), this usually plays no important role (although see the case study of Appendix C).

Of course, it is easy to think of SDP models which violate these assumptions, but which seem to otherwise possess all the characteristics of the SDPs considered in this course book. We will, in fact, consider an SDP for which both the state x and action a are real numbers (Appendix C).

In such cases, it may be possible to rely on methods associated with realvalued functions, primarily the identification of necessary (or stationary) conditions for optimality using first order derivative constraints. It is also possible to approximate an SDP with real state-action spaces using grid partitions to generate approximate SDPs with discrete state-action spaces.

This method has attained considerable refinement in (Chow & Tsitsiklis, 1989) and (Chow & Tsitsiklis, 1991).

A typical MDP model is demonstrated in Appendix C.

2.3 Bellman's Equation

There are several ways to view dynamic programming as the optimization problem associated with the minimization of cost. The simplest is to view it as the problem of minimizing $V^{\phi}\big(x\big)$ for some fixed x by considering all possible policy functions $\phi \in \Phi$. We may only be interested in carrying this out for a single state $x = x_1 \in \boldsymbol{\mathcal{X}}$, however, we have already seen that evaluation of $V^{\phi}\big(x_1\big)$ specifically will require evaluation of $V^{\phi}\big(x\big)$ at other states. So the theory of dynamic programming considers the evaluation of V^{ϕ} as a function defined in function space $V^{\phi} \in \boldsymbol{\mathcal{V}}$, where $\boldsymbol{\mathcal{V}}$ includes, at the very least, all possible policy value functions $V^{\phi}\big(x\big)$.

How can we make this a well-defined optimization problem? We have already seen that we can fix a policy function $\phi \in \Phi$, where Φ is the class of all admissible policies, and evaluate the policy value function $V^{\phi}\big(x\big)$, which gives the total accrued cost from initial state x under that policy.

Then, what would it mean to say that a policy $\phi^* \in \Phi$ is optimal? If the memoryless property holds, it would have to mean that the same policy ϕ^* was optimal for each initial state $x \in \boldsymbol{\mathcal{X}}$. We would then have

$$V^{\phi^*}(x) \leq V^{\phi}(x),\ x \in \boldsymbol{\mathcal{X}},$$

for all $\phi \in \Phi$. That is, a policy is optimal if it minimizes accrued cost from any initial state x. Working in the opposite direction, we can define the optimal value function $V^*(x)$ that gives the optimal total accrued cost/reward when x is the initial state.

$$V^*(x) = \inf_{\phi \in \Phi} V^{\phi}(x),\ x \in \boldsymbol{\mathcal{X}}\,. \tag{2.21}$$

For any initial state x, $V^*(x)$ is the largest lower bound for the accrued cost among all admissible policies. It is important to understand the order in which the operations defining Eqn. (2.21) are carried out. We first fix state x, then minimize over $\phi \in \Phi$. As we have written it, the minimizing ϕ can vary with x, and so there is no guarantee that $V^*\big(x\big) = V^{\phi^*}\big(x\big)$ for some policy ϕ^*. However, we can guarantee that

$$V^*(x) \leq V^{\phi}(x),\ x \in \boldsymbol{\mathcal{X}} \tag{2.22}$$

for any $\phi \in \Phi$, so if we can find some $\phi^* \in \Phi$ for which

$$V^* \equiv V^{\phi^*} \tag{2.23}$$

then that would be an optimal policy.

Then, suppose such an optimal policy ϕ^* exists for a minimum cost stochastic SDP with discounted cost accrual. The policy value function $V^{\phi^*} = V^*$ will satisfy Eqn. (2.19):

$$V^{\phi^*}(x) = R(x, \phi^*(x)) + \beta \sum_{y \in X} V^{\phi^*}(y) P(y \mid x, \phi^*(x)) \; x \in X$$

Next, we consider a slight modification of the optimal policy ϕ^*. From state x we select action $a \in \mathcal{A}_x$, then resume optimal policy ϕ^* from all subsequent states. We denote the expected accrued cost of this modified policy by

$$V^*(x, a) = R(x, a) + \beta \sum_{y \in \mathcal{X}} V^*(y) P(y \mid x, a) \; x \in \boldsymbol{\mathcal{X}}, \tag{2.24}$$

noting that $V^{\phi^*} = V^*$. However, by the memoryless property, the policy that is optimal from any state x_j in an SDP is the same policy that would be optimal if x_j was the initial state. Therefore, the optimal policy from any state x can be deduced from Eqn. (2.24) simply by minimizing $V^*(x, a)$ over a for each $x \in \mathcal{X}$:

$$V^*(x) = \min_{a \in \boldsymbol{\mathcal{A}}_x} V^*(x, a)$$

$$= \min_{a \in \boldsymbol{\mathcal{A}}_x} R(x, a) + \beta \sum_{y \in X} V^*(y) P(y \mid x, a) \; x \in X .$$

In reinforcement learning, the quantity $V^*(x, a)$ is also known as a Q-factor, denoted $Q(x, a)$ where $Q(x, a)$ is the total cost/reward accrued when action a is selected, then the optimal policy is used afterwards.

2.3.1. Bellman's Principle of Optimality

We can now state the fundamental principle of dynamic programming. We are given a memoryless SDP. Let $V^*(x)$ be the optimal accrued cost from initial state x. Let $V^*(x,a)$ be the cost accrued when action a is selected from initial state x, and the optimal policy is applied from the subsequent state onwards. Then Bellman's equation follows from the identity

$$V^*(x) = \min_{a \in \boldsymbol{\mathcal{X}}} V^*(x,a),\ x \in \boldsymbol{\mathcal{X}}\,. \tag{2.25}$$

Bellman's equation is a functional equation, the solution of which gives optimal value function $V^*(x)$ for memoryless SDPs.

Various forms of Bellman's equation follow from Eqn. (2.25). If the SDP is deterministic and employs cost accrual method $H(a,b)$, then

$$\begin{aligned} V^*(x,a) &= H(R(x,a), V^*(q(x,a))), \\ V^*(x) &= \min_{a \in \boldsymbol{\mathcal{A}}_x} H(R(x,a), V^*(q(x,a)))\,. \end{aligned}$$

If the SDP is stochastic and employs discounted cost accrual, then

$$V^*(x,a) = R(x,a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^*(y) P(y \mid x, a)\ \ ,$$

$$V^*(x) = \min_{a \in \boldsymbol{\mathcal{A}}_x} R(x,a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^*(y) P(y \mid x, a)\ \ .$$

For the deterministic model with discounted cost, this becomes

$$\begin{aligned} V^*(x,a) &= R(x,a) + \beta V^*(q(x,a)), \\ V^*(x) &= \min_{a \in \boldsymbol{\mathcal{A}}_x} R(x,a) + \beta V^*(q(x,a)), \end{aligned}$$

and for minimax cost we have

$$\begin{aligned} V^*(x,a) &= \max\{R(x,a), V^*(q(x,a))\}, \\ V^*(x) &= \min_{a \in \boldsymbol{\mathcal{A}}_x} \max\{R(x,a), V^*(q(x,a))\}\,. \end{aligned}$$

We may then determine the optimal policy by setting $\phi(x)$ equal to the minimizing action in Eqn. (2.25), whatever form it takes. It is also a necessary condition since any example of the process must be a solution to it. All of the examples we will explore below will make use of this operation.

However, significant mathematical issues must be considered. The mere formulation of an equation does not guarantee that it possesses a solution.

For example, consider the following case: Find a real number x which satisfies $x^2 = -1$. Alternatively, an equation may possess multiple solutions, leading to the need to characterize any one solution we might obtain (this issue typically arises when attempting to maximize or minimize a function using first order stationary conditions). An additional issue arises in optimization, in particular, the existence, or otherwise, of an actual optimal solution (place this rock as close to that wall as possible, without touching it).

Our arguments presented here will be as detailed as needed, but we refer the reader to the literature for an in-depth mathematical proof of the techniques used here. A significant concern is that despite the apparent simplicity of Bellman's equation, its solution can represent a considerable computational challenge, particularly when state spaces are very large (recall that for backgammon, we would have $\left|\mathcal{X}\right| \approx 10^{20}$). This was referred to by Richard Bellman as the curse of dimensionality and it is what motivates much of the research in this field.

2.3.2. Backward Induction

One method of solving Eqn. (2.25), which is suitable for finite horizon problems, is known as backward induction. It proceeds by determining the optimal policy for the final stage, which usually has a straightforward solution. Once this is done, optimal policies can be determined for stages in a backwards sequence, starting from the last stage. It relies on the existence of a well-defined terminal stage. In Eqn. (2.25), we can discern an interesting feature that yields a simple solution method in this case.

If for a given state x we know all the quantities on the right-side of the equation then we can calculate $V^*(x)$. We will know $R(x,a)$, so the issue is whether or not we known the values of $V^*(q(x,a))$ (for the deterministic SDP), which represent the values of the value function at all states to which x could transition. The stage structure of the finite horizon problem permits this approach. The calculation of $V^*(x)$ for the states associated with the final stage of a finite horizon SDP will usually be trivial. We can then evaluate $V^*(x)$ for all states in stage $K-1$ using Eqn. (2.25), since we know $V^*(x)$ for all states in stage K. We repeat the process for stages $K-2, K-3, ..., 1$ so that $V^*(x)$ is calculated for all states, which solves the problem. Next, we summarize the method.

We are given a finite horizon K-stage SDP with state space $\boldsymbol{\mathcal{X}}$ and terminal state $x_\Delta \in \boldsymbol{\mathcal{X}}$. We assume that the state-space can be partitioned by stage, $\boldsymbol{\mathcal{X}} = \boldsymbol{\mathcal{X}}_1 \cup ... \cup \boldsymbol{\mathcal{X}}_K \cup \{x_\Delta\}$ so that any state at stage j must be an element of $\boldsymbol{\mathcal{X}}_j$. We assume that $V^*(x_\Delta)$ is known.

Then for any state $x \in \boldsymbol{\mathcal{X}}_K$ we have, for deterministic models with cost accrual method $H(a,b)$,

$$V^*(x) = \min_{a \in \mathcal{A}_x} H(R(x,a), V^*(x_\Delta)), \text{ forall } \quad x \in \mathcal{X}_K .$$

We then set stage $j = K-1$, and solve

$$V^*(x) = \min_{a \in \mathcal{A}_x} H(R(x,a), V^*(q(x,a))), \text{ forall } \quad x \in \mathcal{X}_{K-1} .$$

We then iteratively decrease $j = K-1, K-2, ..., 1$.

For the stochastic additive cost model, the algorithm is essentially the same. We assume that stage K always transitions to terminal state x_Δ.

Then, calculate

$$V^*(x) = \min_{a \in \mathcal{A}_x} R(x,a) + \beta V^*(x_\Delta), \text{ forall } \quad x \in \mathcal{X}_K .$$

Set stage $j = K-1$, and solve

$$V^*(x) = \min_{a \in \mathcal{A}_x} R(x,a) + \beta \sum_{y \in \mathcal{X}_{j+1}}^{n} (y) P(y \mid x, a), \text{ forall } \quad x \in \mathcal{X}_j \quad .$$

We then iteratively decrease $j = K-1, K-2, ..., 1$. For total cost accrual, set $\beta = 1$.

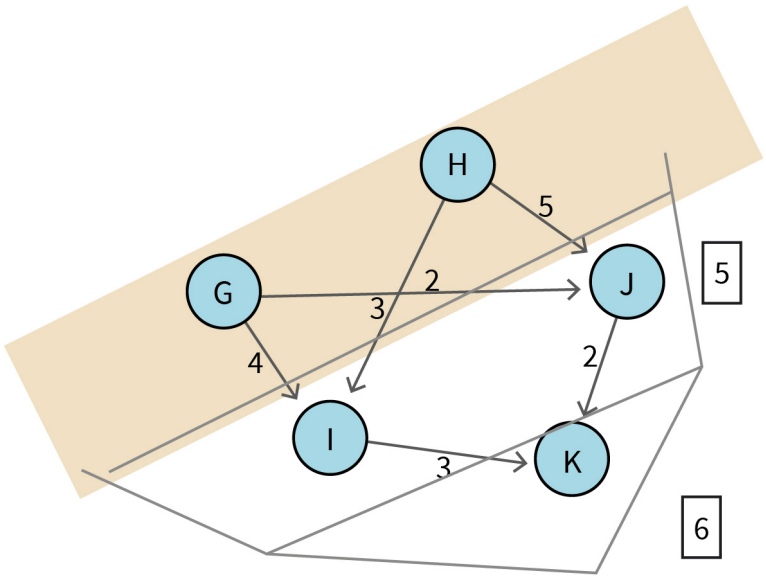
Solving the Stagecoach Problem Using Backward Induction

At this point, we have defined a dynamic programming model (Sec. 2.2.1) and a cost accrual method. So, beginning with the value function at the terminal state $V^*(K) = 0$, we now solve the stagecoach problem by applying backward induction.

Stage 5:

The section of the original map in the figure entitles as "Stagecoach problem map" representing the transition from stage 5 to stage 6 is highlighted in figure below. Trivially, $V^*(I) = 3$ and $V^*(J) = 2$. This is because only one action, K, is available from each state I and J, corresponding to stage costs 3 and 2 respectively. So $\phi^*(I) = K$ and $\phi^*(J) = K$.

Figure 6: Stagecoach problem. Section of the map of Fig. 2.1 representing the transition from Stage 5 to Stage 6.



Stage 4:

Eqn. (2.25) may be applied directly. We first evaluate $V^*(x,a)$, then minimize over a for each x:

$$\begin{aligned} V^*(G,I) &= R(G,I) + V^*(I) = 4+3 = 7, \\ V^*(G,J) &= R(G,J) + V^*(J) = 2+2 = 4, \\ V^*(G) &= \min\{V^*(G,I), V^*(G,J)\} = \min\{7,4\} = 4\,. \end{aligned}$$

For example, $R(G,I) = 4$, since the stage cost of a transition from G to I is 4 (figure entitled as "Stagecoach problem map"). So, from G it is optimal to travel to $\phi^*(G) = J$, which yields an optimal remaining distance from G of $V^*(G) = 4$. We do the same for state H:

$$\begin{aligned} V^*(H,I) &= R(H,I) + V^*(I) = 3+3 = 6, \\ V^*(H,J) &= R(H,J) + V^*(J) = 5+2 = 7, \\ V^*(H) &= \min\{V^*(H,I), V^*(H,J)\} = \min\{6,7\} = 6\,. \end{aligned}$$

So, from H it is optimal to travel to $\phi^*(H) = I$, which yields an optimal remaining distance from H of $V^*(H) = 6$.

Stage 3:

Again, Eqn. (2.25) may be applied directly. We first evaluate $V^*(x,a)$, then minimize over a for each x:

$$
\begin{aligned}
V^*(E,G) &= R(E,G) + V^*(G) = 7 + 4 = 11, \\
V^*(E,H) &= R(E,H) + V^*(H) = 6 + 6 = 12, \\
V^*(E) &= \min\{V^*(E,G), V^*(H,J)\} = \min\{11, 12\} = 11\,.
\end{aligned}
$$

So, from E it is optimal to travel to $\phi^*(E) = G$, which yields an optimal remaining distance from E of $V^*(E) = 11$. We do the same for state F:

$$
\begin{aligned}
V^*(F,G) &= R(F,G) + V^*(G) = 6 + 4 = 10, \\
V^*(F,H) &= R(F,H) + V^*(H) = 3 + 6 = 9, \\
V^*(F) &= \min\{V^*(F,G), V^*(F,H)\} = \min\{10, 9\} = 9\,.
\end{aligned}
$$

So, from F it is optimal to travel to $\phi^*(F) = H$, which yields an optimal remaining distance from F of $V^*(F) = 9$.

Stage 2:

As before, we first evaluate $V^*(x, a)$, then minimize over a for each x:

$$
\begin{aligned}
V^*(B,E) &= R(B,E) + V^*(E) = 4 + 11 = 15, \\
V^*(B,F) &= R(B,F) + V^*(F) = 6 + 9 = 15, \\
V^*(B) &= \min\{V^*(B,E), V^*(B,F)\} = \min\{15, 15\} = 15\,.
\end{aligned}
$$

So, from B there are two optimal actions, which we will express as $\phi^*(B) = \{E, F\}$, which yields an optimal remaining distance from B of $V^*(B) = 15$.

$$
\begin{aligned}
V^*(C,E) &= R(C,E) + V^*(E) = 3 + 11 = 14, \\
V^*(C,F) &= R(C,F) + V^*(F) = 3 + 9 = 12, \\
V^*(C) &= \min\{V^*(C,E), V^*(C,F)\} = \min\{14, 12\} = 12\,.
\end{aligned}
$$

So, from C it is optimal to travel to $\phi^*(C) = F$, which yields an optimal remaining distance from C of $V^*(C) = 12$.

$$
\begin{aligned}
V^*(D,E) &= R(D,E) + V^*(E) = 2 + 11 = 13, \\
V^*(D,F) &= R(D,F) + V^*(F) = 4 + 9 = 13, \\
V^*(D) &= \min\{V^*(D,E), V^*(D,F)\} = \min\{13, 13\} = 13\,.
\end{aligned}
$$

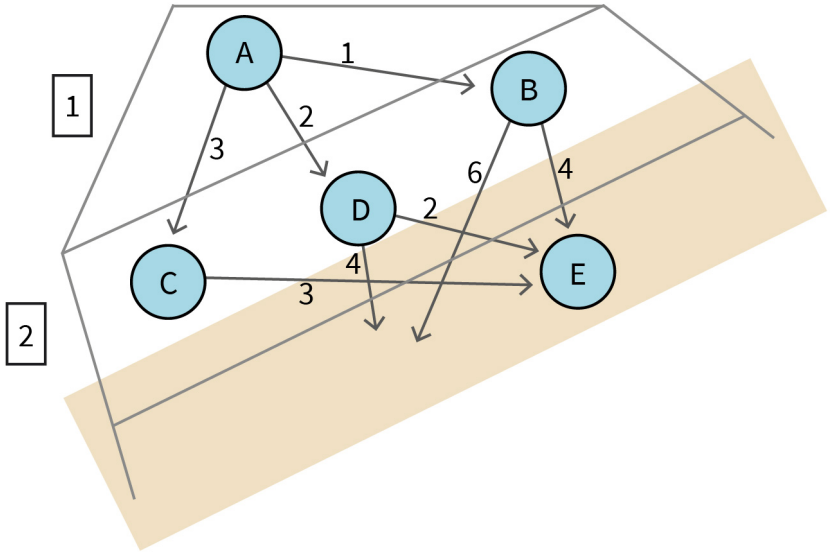
So, from D there are two optimal actions $\phi^*(D) = \{E, F\}$, either of which yield an optimal remaining distance from $DofV^*(D) = 13$.

Stage 1:

The section of the original map in the figure entitles as "Stagecoach problem map" representing the transition from stage 1 to stage 2 is highlighted in figure below. We have now reached the initial state, from which we use the same method:

$$
\begin{aligned}
V^*(A,B) &= R(A,B) + V^*(B) = 1 + 15 = 16, \\
V^*(A,C) &= R(A,C) + V^*(C) = 3 + 12 = 15, \\
V^*(A,D) &= R(A,D) + V^*(D) = 2 + 13 = 15, \\
V^*(A) &= \min\{V^*(A,B), V^*(A,C), V^*(A,D)\} = \min\{16, 15, 15\} = 15\,.
\end{aligned}
$$

Figure 7: Stagecoach problem. Section of the map of Fig. 2.1 representing the transition from Stage 1 to Stage 2.



So, from A there are two optimal actions $\phi^*(A) = \{C, D\}$, either of which yield an optimal remaining distance from A of $V^*(A) = 15$.

We now have a solution to the stagecoach problem. The minimum possible distance from A to K is 15. Note, however, that this is achieved by multiple routes. From A it is optimal to travel either to C or D. From C the sole optimal route to K is C-F-H-I-K. From D it is equally optimal to travel to either E or F. From E the sole optimal route to K is E-G-J-K, and from F the sole optimal route to K is F-H-I-K. This gives a total of three routes which achieve the minimum possible distance of 15:

$$
\begin{aligned}
&A - C - F - H - I - K, \\
&A - D - E - G - J - K, \text{ and} \\
&A - D - F - H - I - K\,.
\end{aligned}
\tag{2.26}
$$

The Stagecoach Problem Revisited: Reward versus Cost

Next, we consider a modification of the stagecoach problem which highlights some issues regarding the memoryless property. As in the original problem, there are 11 towns labeled A to K, and a traveller must select a route starting from town A and ending in town K according to the map shown in the figure entitles as "Stagecoach problem map". Suppose the traveller has an opportunity to sell a commodity along the route. This can be done

only once, and must be done either at node C or H. This sale will earn the traveller a \$100 profit. The journey itself costs the traveller \$2 $\times$ D, where D is the total distance of the route taken. Therefore, the object is to find the route which maximizes the reward

$$R = 100 \times I_{C,\,H} - 2 \times D,$$

where $I_{C,\,H} = 1$ if the route includes C or H (possibly both), and $I_{C,\,H} = 0$ otherwise.

The first thing to note is that this is a maximization problem, while the original stagecoach problem is a minimization problem. Fortunately, this does not change the mathematics of dynamic programming in any important way, since all we need to do is multiply the reward by -1 to turn it into a cost. We can implement this by replacing reward $R(x, a)$ with cost $-R(x, a)$ in Eqn. (2.25):

$$V^*(x) = \min_{a \,\in\, \boldsymbol{\mathcal{A}}_x} \left(-R(x, a) + V^*(q(x, a))\right).$$

If we wish, we may rewrite this as

$$V^*(x) = \; - \max_{a \,\in\, \boldsymbol{\mathcal{A}}_x} \left(R(x, a) - V^*(q(x, a))\right),$$

or alternatively

$$[-V^*(x)] = \max_{a \,\in\, \boldsymbol{\mathcal{A}}_x} \left(R(x, a) + [-V^*(q(x, a))]\right). \tag{2.27}$$

Recall that in the original Eqn. (2.25), $V^*(x)$ plays the role of an unknown function to be determined. This role is played in Eqn. (2.27) by $[\,-V^*(x)]$.

Of course, this symbol is just a placeholder for a function to be solved, so the equation retains its interpretation if $[\,-V^*(x)]$ is replaced by, say $V'(x)$, or even the original $V^*(x)$. In other words, if $R(x, a)$ is a reward, and we wish to maximize the accrued reward, we need only replace Eqn. (2.25) with

$$V^*(x) = \max_{a \,\in\, \boldsymbol{\mathcal{A}}_x} \left(R(x, a) + V^*(q(x, a))\right). \tag{2.28}$$

The solution method remains the same, except that we maximize instead of minimize.

There is, however, another issue to consider for this problem. It does not satisfy the memoryless property of SDPs. To see this, return to the figure entitles as "Stagecoach problem map", and suppose we are at node E and need to choose between G and H for the next stage. Recall that we earn a reward of \$100 if we visit either node C or H. This is where the memoryless property is violated. If we have already visited C we would not accrue the \$100 reward by visiting H, otherwise we do. Thus, the future reward depends not only on the current stage state and action, but on the process history.

However, dynamic programming can still play a crucial role in the solution to this problem. The total reward is $R = 100 \times I_{C,H} - 2 \times D$. If there exists a route which optimizes both terms, this will be the optimal solution. In other words, we are looking for a route that achieves the minimum distance of 15 while passing through C or H.

As it happens, two of the three optimal routes listed in Eqn. (2.26) satisfy this constraint. If this was not the case, we could then use dynamic programming to find the optimal route which passes though C or H, by finding the shortest routes from A to C; A to H; C to K; and H to K.

Don't forget that we would also need to compare this route to the optimal route which does not pass through C or H.

Stochastic Dynamic Programming: Optimal Treasure Seeking

We next consider a stochastic dynamic programming problem. Imagine that while vacationing in the Bahamas, you hear about some sunken treasure worth about $13,000. For $700/hr you can hire a boat, equipment and crew, for up to 5 hours a day. You estimate that the time it takes to find the treasure is an exponential random variable with mean $\mu = 13$ hours.

You have only three days left on your vacation. How long should you hire the boat each day? It is useful when solving optimization problems to try to anticipate the general form of the solution. Suppose on a certain day we decide to rent the boat for 5 hours. It is understood that this decision must be taken before the day's search begins. Thus, if after the first hour's search the treasure is found, we must still pay for the four remaining hours that were planned (dynamic programming is often used to solve this type of planning problem).

We might then do a "back of the envelope" calculation. Suppose we simply decided in advance to rent the boat for 13 hours (the expected search time).

The probability of finding the treasure would be $1 - \exp(-13/13) \approx 0.632$.

The total expected reward would therefore be

$$R = \$13,000 \times 0.632 - \$700 \times 13 = -\$884 \quad ,$$

yielding an expected net loss. Clearly, we would be better off planning the search as an SDP.

We then have a finite horizon SDP with $K = 3$ stages (each of the remaining three days will define a stage). We must first decide how to define the state x. We want to formulate a memoryless SDP and also include all information needed to make a decision. Clearly, we need to know whether or not the treasure has already been found (we would not spend money on the boat if it has). So, set $s = 1$ if the treasure has been found, and $s = 0$ otherwise. However, our discussion suggests that the number of search days remaining would also be important information. Denote this quantity w. So, on the morning of our third

and final day the state would be $x_3 = (w, s) = (1, 0)$ if the treasure has not yet been found, and $x_3 = (1, 1)$ if it has. The action space is $\boldsymbol{\mathcal{A}} = \{0, 1, 2, 3, 4, 5\}$ (the possible number of hours in the boat), and the stage reward is

$$R\big(w, 0, a\big) = 13000 \times \big(1 - e^{-a/13}\big) - 700 \times a, \;\; a \in \big\{0, 1, 2, 3, 4, 5\big\},$$

with $R(w, 1, a) \;\; = \;\; 0$. Bellman's equation then becomes

$$\begin{aligned} V^*(w, 1) &= 0, \;\; w \geq 0, \\ V^*(w, 0, a) &= R\big(w, 0, a\big) + V^*\big(w - 1, 0\big) \times e^{-a/13} + V^*\big(w - 1, 1\big)\big(1 - e^{-a/13}\big), \\ &= R\big(w, 0, a\big) + V^*\big(w - 1, 0\big) \times e^{-a/13}, \\ V^*(w, 0) &= \max_{a \,\in\, \boldsymbol{\mathcal{A}}} V^*(w, 0, a) \end{aligned} \tag{2.29}$$

with terminal value function $V^*(0, s) = 0$ for $s = \;\; 0, 1$. This means that if we know the value of $V^*(w - 1, 0)$, we can substitute it into the expression for $V^*(w, 0, a)$ in Eqn. (2.29), then maximize $V^*(w, 0, a)$ over $a \in \mathcal{A}$. This gives $V^*(w, 0)$, and the maximizing action a^* determines the optimal policy $\phi^*(w, 0) = a^*$, while $\phi^*(w, 1) = 0$, since there would be no benefit to renting the boat if we have already found the treasure.

Starting with $V^*(0, 0) = 0$ at the final stage 3, we make the following calculation:

Stage 3:

Substitute $V^*(0, 0) = 0$ into Eqn. (2.29).

$$\begin{aligned} V^*(1, 0, a) &= 13000 \times \big(1 - e^{-a/13}\big) - 700 \times a + V^*\big(0, 0\big) \times e^{-a/13} \\ &= 13000 \times \big(1 - e^{-a/13}\big) - 700 \times a, \end{aligned}$$

then maximize over $a \in \boldsymbol{\mathcal{A}}$. We can find the maximum of $V^*(1, 0, a)$ by calculating its value for each $a = 0, 1, ..., 5$. After doing so, we find that the optimal action is $\phi^*(1, 0) = a^* = 5$, yielding $V^*(1, 0) = 650 \,.\, 7$.

Stage 2:

Substitute $V^*(1, 0) = 650 \,.\, 7$ into Eqn. (2.29).

$$\begin{aligned} V^*(2, 0, a) &= 13000 \times \big(1 - e^{-a/13}\big) - 700 \times a + V^*\big(1, 0\big) \times e^{-a/13} \\ &= 13000 \times \big(1 - e^{-a/13}\big) - 700 \times a + 650 \,.\, 7 \times e^{-a/13}, \end{aligned}$$

then maximize over $a \in \boldsymbol{\mathcal{A}}$. Now, the optimal action will be $\phi^*(2, 0) = a^* = 4$, yielding $V^*(2, 0) = 1121 \,.\, 5$.

Stage 1:

Finally, substitute $V^*(2,0) = 1121.5$ into Eqn. (2.29).

$$\begin{aligned} V^*(3,0,a) &= 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a + V^*\big(2,0\big) \times e^{-a/13} \\ &= 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a + 1121.5 \times e^{-a/13}. \end{aligned}$$

As before, maximizing over $a \in \mathcal{A}$ yields optimal action $\phi^*(3,0) = a^* = 3$ and value function $V^*(3,0) = 1469.4$.

The optimal strategy is to rent the boat for 3,4, or 5 hours, if needed, on days 1, 2, or 3, respectively. This yields an expected reward of $1469.40. As anticipated, the optimal strategy is to budget search time more sparingly at the beginning of the schedule when there is greater flexibility in planning.

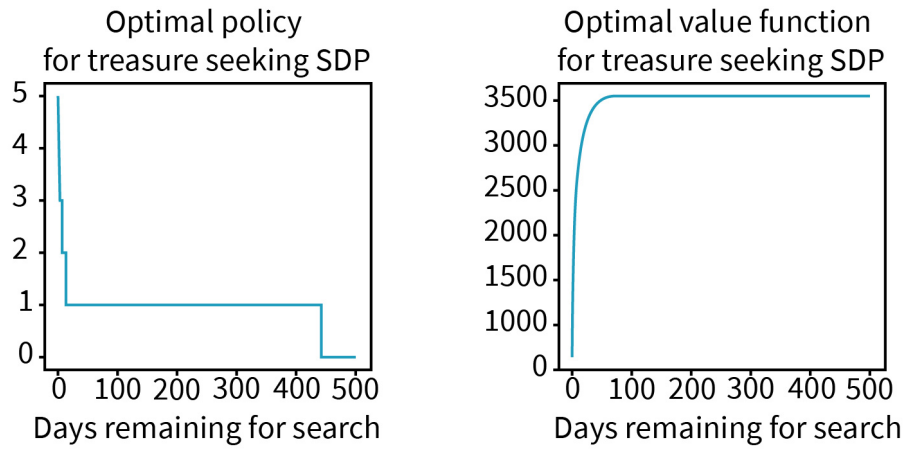
Extending Backward Recursion Indefinitely

Up to this point, we have conceived of the treasure seeking problem as a finite horizon SDP with $K = 3$ stages. Suppose we manage to arrange things so that we have an extra day of treasure seeking, if needed. We decide, accordingly, to recalculate the optimal control for a $K = 4$ stage finite horizon SDP. However, it should be clear that the optimal policy for the final 3 days of the $K = 4$ stage SDP will be the same as that just calculated for the 3 days of the $K = 3$ Stage SDP. In fact, $V^*(w,s)$ and $\phi^*(w,s)$, for $w = 0,1,2,3$ and $s = 0,1$, will be the same for any $K \geq 3$ stage SDP. This follows from the form of Eqn. (2.29), and the fact that we must have $V^*(0,s) = 0$ for any number of stages K.

This leads to an interesting feature of dynamic programming. To determine a general solution to a fixed horizon SDP, we often do not need to specify a fixed number of stages K. Given that we know $V^*(0,0) = 0$, we can evaluate $V^*(w,0)$ for any positive integer w by evaluating Eqn. (2.29) recursively (which requires w iterations).

A computer program was used to produce the two graphs shown in figure below, which plot the optimal policy $\phi^*(w)$ (number of hours to rent the boat when w search days remain), and optimal value function $V^*(w)$ (optimal expected reward when w search days remain). In our program, iterations up to $w = 500$ were performed. We originally noted that the optimal action $\phi^*(w)$ tended to decrease as w increased. This is because when we have more flexibility in planning, it makes sense to be more cautious in our expenditures. This avoids the situation in which we are committed to pay for the boat after we have discovered the treasure.

Figure 8: Output from sample python program for the Treasure Seeking problem Sec. u2.sec.treasure.example.



In fact, the optimal policy is $\phi^*(w) = 1$ for most of the range considered, which represents the most cautious expenditure of resources possible while still seeking the treasure (left plot of the figure above). Another interesting feature of the solution is that for all large enough w the optimal policy is $\phi^*(w) = 0$, in which case no search is done on that day. In other words, there is a strict upper bound on optimality, in the sense that increasing the total search time indefinitely, even when allocated in an optimal manner, will eventually force a decrease in the expected reward. Accordingly, that expected reward can never exceed a specific value, somewhere in the vicinity of $3,500 (right plot of figure above).

See Appendix A for additional details.

2.3.3. Value Iteration

An understanding of value iteration, and much of the more advanced theory of optimization in general, will become more natural from the point of view of functional analysis. We usually think of a function $y = f(x)$ as the mapping of one real number x to another real number y. Functional analysis typically concerns a mapping $g = Tf$ from one function $f(x)$ to another function $g(x)$. In this case, T is often referred to as an **operator**.

Operator
An operator maps one function to another function.

The reader has undoubtedly encountered many examples, whether or not this terminology was used. For example, consider function $f(x) = x^2 + 1$.

The derivative operator is well known:

$$f'(x) = 2x = \frac{d}{dx} f(x)\,.$$

In functional analysis, in order to emphasize the generality of an operator, we might write it this way:

$$f'(x) = Tf(x)\,.$$

Then consider the following form of Bellman's Equation:

$$V^*(x) = \min_{a \in \boldsymbol{\mathcal{A}}_x} R(x,a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^*(y) P(y \mid x, a), \; x \in \boldsymbol{\mathcal{X}}\,. \tag{2.30}$$

We can use Eqn. (2.30) to define the dynamic programming or Bellman operator T.

Suppose we are given any function $V' : \mathcal{X} \to \mathbb{R}$. We apply the operator to V' to produce another function $V'' : \mathcal{X} \to \mathbb{R}$ by calculating

$$V''(x) = \min_{a \in \boldsymbol{\mathcal{A}}_x} R(x,a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V'(y) P(y \mid x, a)$$

for each $x \in \mathcal{X}$. This can then be written more compactly as

$$V'' = TV',$$

where it is understood that the evaluation of TV' involves a separate computation for each $x \in \boldsymbol{\mathcal{X}}$. Then Bellman's equation (2.30) becomes

$$V^* = TV^*\,. \tag{2.31}$$

Fixed Point Algorithms

Eqn. (2.31) is an example of a fixed point equation, the solution of which is a fixed point. Any equation of the form $V = TV$ (where V is a function) or $x = f(x)$ (where x is a real number) is referred to as a fixed point equation.

Any solution V^* of a fixed point equation is a fixed point $V^* = TV^*$

Such equations are ubiquitous in applied mathematics and numerical analysis, so it is worth understanding how fixed points can be evaluated using the fixed point iteration method: Here we calculate a sequence $x_1, x_2, ...$ using fixed point iterations $x_{n+1} = f(x_n)$. An initial value x_1 must be specified. If f possesses fixed point $x^* = f(x^*)$, then under well known conditions, the sequence of fixed point iterates converges to x^*. The idea extends to operators. If operator T possesses fixed point $V^* = TV^*$, then under well known conditions the fixed points iterates $V_{n+1} = TV_n$ converge to V^*, given a suitable initial function V_1.



EXAMPLE OF A FIXED POINT ALGORITHM

Suppose we are given function

$$f(x) = 1 - 0.95e^{-x},$$

and we wish to solve the fixed point equation

$$x = f(x).$$

The solution to such an equation is a fixed point. Under certain conditions a fixed point equation can be solved using a fixed point algorithm:

$$\begin{aligned} x_1 &= \text{suitably chosen initial solution,} \\ x_{i+1} &= f(x_i),\ i = 1, 2, ... \end{aligned}$$

Then $\lim_{i \to \infty} x_i = x^*$, where $x^* = f(x^*)$ is a fixed point. In an example like this, we would not get an exact solution, in the sense that no element of the sequence will equal x^* exactly. However, the elements of the sequence are increasingly accurate approximations of a fixed point, and the theory of fixed point algorithms would allow us to determine precise approximation error bounds.

For our example, if we set the initial solution to be $x_1 = 1$, the first few iterations would be

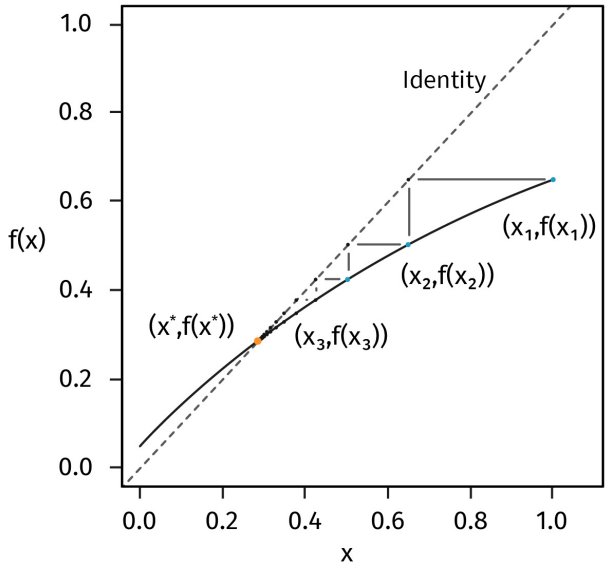
$$
\begin{aligned}
x_1 &= 1, \\
x_2 &= f(x_1) = 0\,.6505 \\
x_3 &= f(x_2) = 0\,.5043 \\
&\vdots
\end{aligned}
$$

leading to an approximate fixed point $x^* \approx 0\,.2870483$,

$$
f(0\,.2870483) = 1 - 0\,.95e^{-0\,.2870483} \approx 0\,.2870483\,.
$$

The progress of the fixed point algorithm is shown in the figure below.

Note the role played by the identity $f_I(x) = x$. The fixed point must be the intersection of the functions $f(x)$ and $f_I(x)$, and the points $(x_i, f(x_i))$, $i = 1, 2, 3, \ldots$ make predictable progress towards that point.



The fixed point x^* is indicated by the red point, while the first three iterations are indicated by blue points. The identity is indicated by the dashed grey line.

Value Iteration as a Fixed Point Algorithm

As Eqn. (2.30) and (2.31) make clear, the optimal value function V^* can be interpreted as the solution to a fixed point equation. Value iteration is then a fixed point algorithm applied to the dynamic programming operator T: In value iteration we evaluate the optimal value function V^* by calculating fixed point iterates $V_{n+1} = TV_n$, where T is the dynamic programming operator, and V_1 is a suitably chosen initial function. The sequence $V_1, V_2, \ldots$ converges to the fixed point $V^* = TV^*$, which is the optimal value function.

We determine fixed point $V^* = TV^*$ by using fixed point iterations

$$\begin{aligned} V_1 &= \text{suitably chosen initial solution,} \\ V_{i+1} &= TV_i,\ i = 1, 2, \ldots . \end{aligned}$$

Then the iterations V_i are increasingly accurate approximations of V^*.

The fixed point of Bellman's operator then allows us to determine an optimal policy. This is any policy ϕ^* satisfying .

$$V^{\phi^*} = V^* .$$

2.3.4. Policy Iteration: The Actor-Critic Method

The problem of determining the total reward or cost of a specific policy as opposed to determining an optimal policy is referred to as policy evaluation.

For the dynamic programming model with stationary policy ϕ, this reduces to evaluating policy value function $V^{\phi}\big(x\big)$.

As we have seen, we can use Bellman's equation to determine an optimal policy ϕ^*. On the other hand, if we are interested not in optimization, but in evaluating the cost $V^{\phi}\big(x\big)$ accrued from any initial state $x \in \boldsymbol{\mathcal{X}}$ for any fixed policy ϕ, then we can make use of Eqn. (2.19):

$$V^{\phi}(x) = R(x, \phi(x)) + \beta \sum_{i=1} V^{\phi}(y) P(y \mid x, \phi(x)) \,.$$

(2.32)

This equation has the same structure as Eqn. (2.30), given here again for reference:

$$V^*(x) = \min_{a \in \boldsymbol{\mathcal{A}}_x} R(x, a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V^*(y) P(y \mid x, a), \; x \in \boldsymbol{\mathcal{X}} \,.$$

In particular, we can recognize the policy operator T^{ϕ}, that defines a fixed point equation $V = T^{\phi} V$ and is associated with the policy function V^{ϕ}.

The policy operator maps a function V' to V'' using the evaluation method

$$V''(x) = R(x, \phi(x)) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V'(y) P(y \mid x, \phi(x))$$

for each $x \in \boldsymbol{\mathcal{X}}$. This can then be written more compactly as

$$V'' = T^{\phi} V' \,.$$

Then Eqn. (2.19) is another fixed point equation

$$V^{\phi} = T^{\phi} V^{\phi},$$

and, under well defined conditions, V^{ϕ} can be approximated with arbitrarily small error by a fixed point iteration method known as policy value iteration:

$$
\begin{aligned}
V_1 &= \text{suitably chosen initial solution,} \\
V_{i+1} &= T^{\phi} V_i, \ i = 1, 2, \ldots .
\end{aligned}
\tag{2.33}
$$

The algorithm generates a sequence $V_1, V_2, \ldots$ of increasingly accurate approximations of V^{ϕ}.

This method is clearly appropriate when we are interested not in optimization, but in the performance of a specific policy. However, algorithm (2.33) can also be used as a component in a method of solving Bellman's equation known as policy iteration or the actor-critic method introduced in (Howard, 1960). Like value iteration, the algorithm is iterative, but alternatives between two steps, and produces a sequence of policy functions $\phi_1, \phi_2, \ldots$ rather than value functions. The first step begins with the current policy ϕ_n, and evaluates the policy value function V^{ϕ_n}, possibly by applying the fixed point iteration method to the policy operator T^{ϕ_n}.

Then the policy ϕ_n is improved, yielding the next policy in the sequence ϕ_{n+1}. The algorithm terminates when strict improvement is not possible, in which case the final policy function in the sequence is the optimal policy function ϕ^*. The policy evaluation step functions as the critic, while the policy improvement step functions as the actor.

An important difference between value iteration and policy iteration is that value iteration generates a sequence of value functions $V_1, V_2, \ldots$, while policy iteration generates a sequence of policies $\phi_1, \phi_2, \ldots$. However, the goal is the same. We expect the sequence $V_1, V_2, \ldots$ to converge to the optimal value function V^*, while we expect the sequence $\phi_1, \phi_2, \ldots$, to converge to some optimal policy ϕ^*, in the sense that $V^{\phi_1}, V^{\phi_2}, \ldots$ converges to $V^{\phi^*} = V^*$. The technical problem is that while the optimal value function V^* will always be unique, the optimal policy ϕ^*, that is, one for which $V^{\phi^*} = V^*$, might not be. Thus, this possibility must be considered when designing a policy iteration algorithm.

The steps of a policy iteration algorithm have already been described above in various sections, so we need only assemble them into a coherent algorithm: We begin with an initial policy ϕ_1. A sequence of policies $\phi_1, \phi_2, \ldots$ is produced iteratively. A single iteration consists of the following two steps.

Given current policy ϕ_i:

- Step 1. Policy Evaluation (Critic): Evaluate V^{ϕ_i} using, for example, algorithm (2.33)
- Step 2. Policy Improvement (Actor): Improve ϕ_i by evaluating

$$
\phi_{i+1}(x) = \operatorname{argmin}_{a \in \boldsymbol{\mathcal{A}}_x} R(x, a) + \beta \sum_{y \in \mathcal{X}} V^{\phi_i}(y) P(y \mid x, a), \ \ \forall x \in \boldsymbol{\mathcal{X}} .
$$

We will then have $V^{\phi_i+1} \leq V^{\phi_i}$.

- Terminate the iterations when $V^{\phi_i+1} = V^{\phi_i}$.

Then $\phi_i = \phi^*$ is an optimal policy. The terminology signifies that the critic evaluates a policy, while the actor improves a policy.

It must be noted that policy iteration and value iteration have the same objective, that is, the determination of the optimal policy for a dynamic programming model. They are, therefore, exchangeable in general. One advantage of policy iteration is that it converges to an exact solution in a finite number of steps, provided the action space $\boldsymbol{\mathcal{A}}$ is finite. In addition, according to the authors of (Sutton & Barto, 2018a), "[p]olicy iteration often converges in surprisingly few iterations". On the other hand, while value iteration is suitable for finite horizon, shortest path or infinite horizon problems (see the example in Appendix B), policy iteration is reported to be relatively inefficient for finite horizon SPDs (see Section 6.4 of (Puterman, 1994)). The best choice will often depend on the structure of the particular problem.

2.3.5. Algorithms for the Optimal Control of MDPs

We have introduced Markov decision processes (MDP) in Sec. 2.2.4 as an especially important special case of the dynamic programming model.

They are able to model a quite large variety of SDPs using a relatively compact model definition $(\boldsymbol{\mathcal{K}}, R, P, \beta)$. Furthermore, the supporting mathematical theory is very rich, and MDPs are known to possess stable and predictable long-term behavior.

However, they possess another property that can represent a considerable disadvantage, popularly known as the curse of dimensionality. This term was used by Richard Bellman in the 1950's to describe the manner in which the size of the state-action space $\boldsymbol{\mathcal{K}}$ can increase rapidly as an SDP model is made more complex (Bellman, 1957). To be sure, we have seen the computational advantage of dynamic programming over brute force enumeration when considering the stagecoach problem. However, the remaining problem can be seen when we consider the computational complexity of a single iteration of the value iteration algorithm. First, consider a model with deterministic transitions:

$$V_{j+1}(x) = TV_j = \min_{a \in \boldsymbol{\mathcal{A}}_x} R(x,a) + \beta V(q(x,a)),\ x \in \boldsymbol{\mathcal{X}}\,. \tag{2.34}$$

Suppose the number of actions admissible at each state is bounded by N_a, so that $|\boldsymbol{\mathcal{A}}_x| \leq N_a$ for all $x \in \boldsymbol{\mathcal{X}}$. It is possible, depending on the manner in which the model is formulated, that N_a is much smaller than $|\boldsymbol{\mathcal{A}}|$, in which case N_a is the important number. For example, in a game of backgammon the action space $\mathcal{A}$ might represent all possible moves a player might make.

However, for each given state x (board configuration), only a small subset $\boldsymbol{\mathcal{A}}_x \subset \boldsymbol{\mathcal{A}}$ would be available.

Then for each $x \in \mathcal{X}$, a quantity of the form $R(x,a) + \beta V(q(x,a))$ must be calculated as many as N_a times. This must be done for each state, so the total number of such calculations is

$$\text{Iteration complexity (deterministic model)} = N_x \times N_a, \tag{2.35}$$

where $N_x = |\boldsymbol{\mathcal{X}}|$ is the number of states.

The complexity is greater for the stochastic model. A single iteration of the value iteration algorithm takes the form:

$$V_{j+1}(x) = TV_j = \min_{a \in \boldsymbol{\mathcal{A}}_x} R(x,a) + \beta \sum_{y \in \boldsymbol{\mathcal{X}}} V_j(y) P(y \mid x,a), \; x \in \boldsymbol{\mathcal{X}}\,. \tag{2.36}$$

As in the deterministic case, this equation must be calculated as many as $N_x \times N_a$ times. However, in contrast with Eqn. (2.34), the calculation also includes a summation over $\boldsymbol{\mathcal{X}}$, used to evaluate an expected value (recall Eqn. (2.15)). The complexity is now

$$\text{Iteration complexity (stochastic model)} = N_x^2 \times N_a\,. \tag{2.37}$$

We also need to consider storage cost. Unless $V(x)$, $\phi(x)$, $V(x,a)$, $R(x,a)$ and $P(y \mid x,a)$ have analytic forms, they must be stored in computer memory in tabular form. Then $V(x)$ and $\phi(x)$ would need to store N_x entries; $V(x,a)$ and $R(x,a)$ $N_x N_a$ entires; and $P(y \mid x,a)$ $N_x^2 N_a$entries.

Possibly, specific features of a model can mitigate computational and storage complexity. Suppose, for example, that the stochastic kernel $P(y \mid x,a)$ permits transitions to only two other states from any state-action pair (x,a). Then the summation over $y \in \mathcal{X}$ in Eqn. (2.36) has only two nonzero terms, and the iteration complexity $N_x^2 N_a$ can be replaced by $N_x N_a$. Sparse matrix methods can play an important role in cases such as this (Van Loan & Golub, 1983).

The complexity analysis presents quite formidable consequences. Suppose we wish to model a multiple queuing system (a supermarket, an airport security checkpoint, to give two examples). The object might be to develop an SDP to control, for example, the number of queues, based on the current queue lengths. The cost function would include the expense of maintaining an open queue (so that opening more queues would incur greater expense), but may also include a term that penalizes longer queues (a customer who encounters too long a queue may choose to shop elsewhere, representing a quantifiable cost).

What information would be included in the definition of a state? We might assume that each queue has a finite capacity K. In practice, evaluation of current system cost, or of the required distributional properties, would require knowledge of the number of customers in each queue (the queue configuration), which we could write $(W_1, W_2, ..., W_M)$ for an M queue system. Each quantity W_i is an integer between 0 and K, so the number of such configurations is $(K+1)^M$. A large, but feasible, queuing system might have capacity $K = 20$, $M = 10$, giving $\left(K+1\right)^M \approx 1.67 \times 10^{13}$. This number would represent a lower bound on the iteration complexity, rendering value iteration, at least in its exact form, computationally infeasible. But such systems exist, and hence also the need for optimal control systems.

Backward induction is not generally applicable to MDPs, but both value iteration and policy iteration are commonly used.

2.4 General Utility Functions

In the decision problems we have considered so far, we have attempted to optimize a cost or reward, possibly as an expected value, expressible in what we might call objective units (currency or distance, for example).

There are two possible shortcomings to this type of model:

1. Optimizing a cost or reward as an expected value does not take risk into account. For example, consider the following payoff table:

   Table 2

	Probability	Action 1	Action 2
Event 1	1/2	500	-1,000
Event 2	1/2	500	3,000
Expected Payoff		$500	$1,000

   Two actions are available, and the payoff received depends on which of two events occurs. To maximize the expected payoff, Action 2 would be selected. However, the significant probability of losing $1,000 may be unacceptably high, and the decision maker may prefer the certainty of a $500 payoff following either event.
2. Different decision makers may have different attitudes towards the various levels of payoffs available, and stochastic variation may inform the relative preferences. Someone who makes decisions of the type shown in the above table on a regular basis may choose Action 2 repeatedly, since uncertainty due to random variation would diminish in the long run. Someone who makes the decision only once may prefer Action 1.

2.4.1. Utility Functions

The types of problems introduced above can be solved by the theory of utility, which quantifies a specific decision maker's attitude to cost, reward and risk. This is important when multiple decision makers respond differently to the same set of economic conditions.

As an example, consider the problem of how much money a decision maker would be willing to pay for x bottles of a soft drink. A thirsty customer on a hot day may be willing to pay \$1.00 for a first soft drink (say, the nominal price at which the soft drink would be available to a customer).

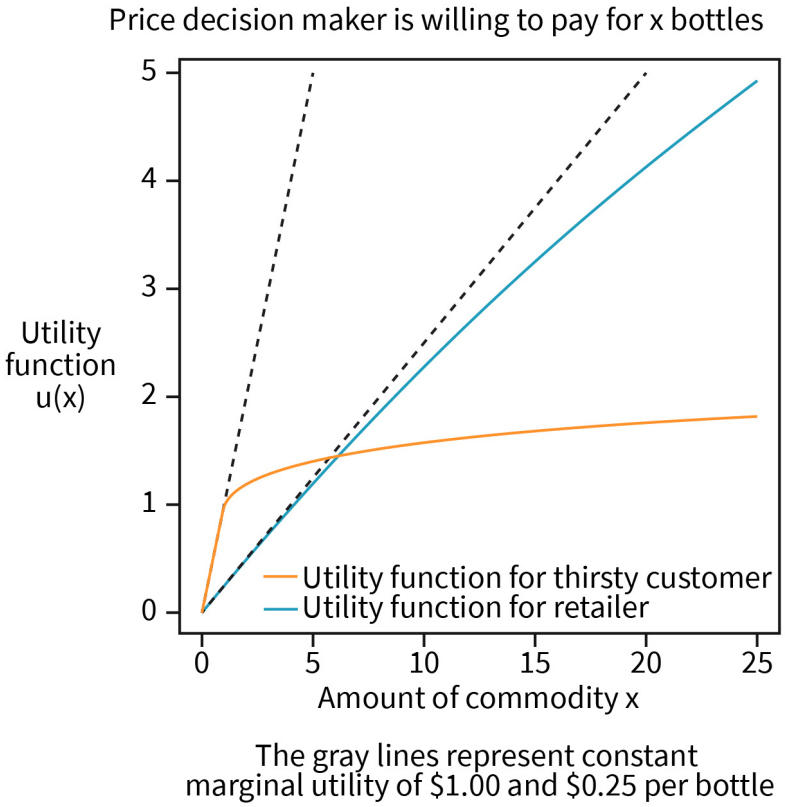
However, the customer might not be willing to pay this price for additional drinks. Perhaps the maximum acceptable price would be only \$0.50 for a second drink, \$0.20 for a third drink, and so on, the customer no longer being thirsty.

Next, consider a retailer. We have assumed that soft drinks are generally sold for \$1.00 a bottle. So, if a profit is to be made, the price paid by the retailer would have to be less, say \$0.25 per bottle. Furthermore, the marginal advantage of stocking an additional bottle will eventually decrease as the amount of stock x increases.

The customer and the retailer each have their own utility function $u(x)$, which represents the value of an amount of commodity x to a decision maker. In this scenario, the commodity may have a nominal price, but utility is the price a decision maker would be willing to pay. If the utility is below the nominal price, then the purchase is unlikely to be made, unless negotiations follow.

Figure below shows two possible utilities functions that might capture the decision makers preferences. The thirsty customer is willing to pay the nominal price of \$1.00 for one drink. The customer's utility function (the red function in figure below) therefore increases at a constant rate of \$1.00 per unit until $x = 1$ is reached (use the gray lines for reference). However, above $x = 1$ the slope of the utility function decreases quickly, indicating that the customer is not willing to pay the nominal price \$1.00 for a second drink (and would therefore probably be content with a single drink). Of course, the utility function is always increasing, assuming that any number of units x would be regarded as an asset rather than a liability.

Figure 9: Utility functions for thirsty customer and retailer for the soft drink example of Sec. 2.4.1. The gray lines represent constant marginal utility of $1.00 and $0.25 per bottle.



The utility function for the retailer is also shown (the blue function in figure above). The retailer is interested in making a profit, so there would be no point in purchasing the commodity at the nominal price. The retailer may estimate that continued operation would require a profit margin of 75%. Therefore, at the current nominal price, the retailer would be unwilling to pay more than $0.25 per unit, which therefore gives the slope of the retailer's utility function near $x = 0$ (use the gray lines for reference).

However, depending on anticipated demand, the marginal advantage of purchasing an additional new unit will likely decrease as x increases.

Correspondingly, the slope of the retailers utility function decreases as x increases.

2.4.2. Classification of Utility Function by Risk

If we look at the utility functions of figure above can see that, while they are increasing, the rate at which they increase falls. This tells us that less value is placed on each newly acquired unit as the number of units increases. It turns out that this type of qualitative property is important in the theory of utility, so we will need the following definitions.

Concave and Convex Functions

A function $u(x)$ is said to be concave if for any two points on the curve the line segment joining these points lies below or on the curve. If the line segment lies below the curve except at the endpoints, then the function is said to be strictly concave.

Similarly, a function $u(x)$ is said to be convex if for any two points on the curve the line segment joining these points lies above or on the curve. If the line segment lies above the curve except at the endpoints, then the function is said to be strictly convex. See figure below.

Note that if $u(x)$ is a linear function, it is both concave and convex, but neither strictly concave nor strictly convex. Also, if $u(x)$ is twice differentiable then $u(x)$ is strictly concave (convex) if and only if the second derivative $u''(x)$ is negative (positive) for all x. We also note that a utility function is almost always an increasing function. This leads to an important classification system for utility functions.

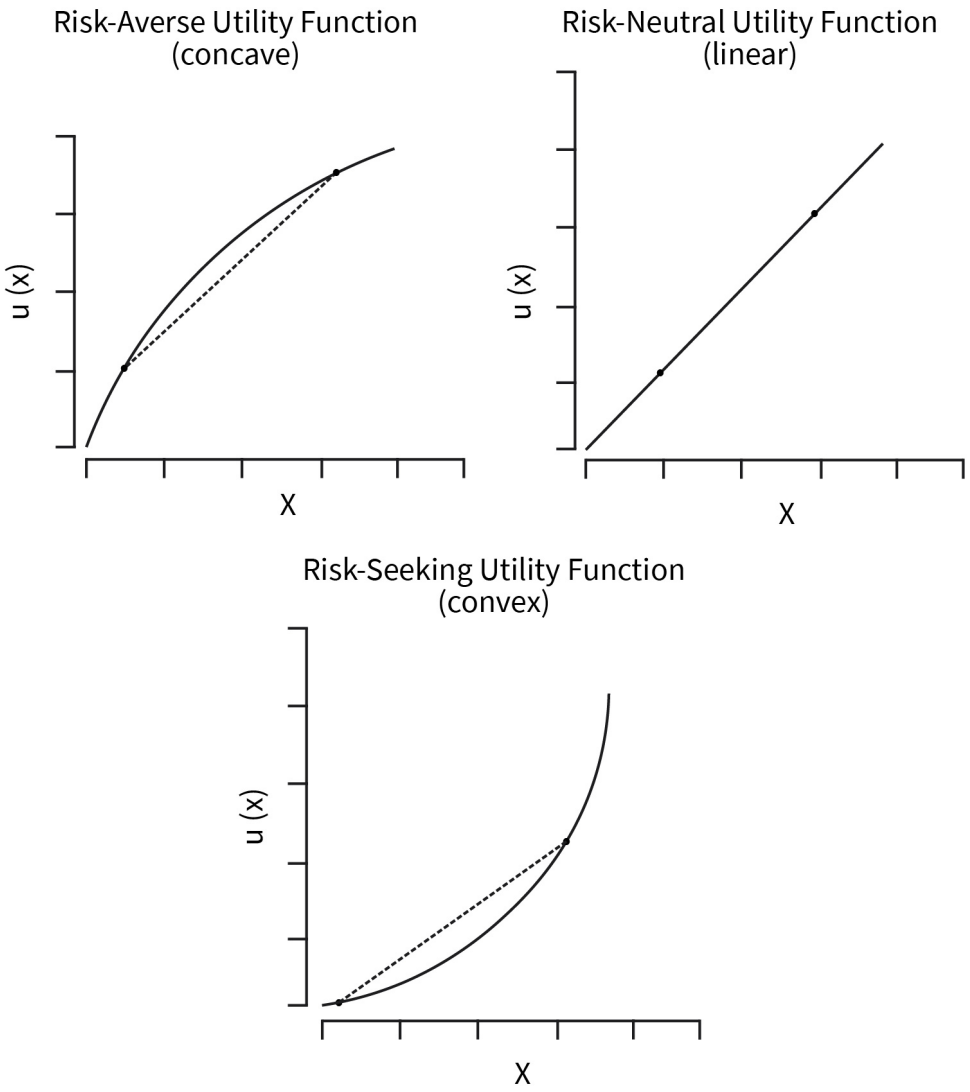
Classes of Utility Functions by Risk

We can characterize a decision maker's attitude towards risk by noting whether the utility function is convex, concave or linear:

- A utility function is risk-averse if it is strictly concave.
- A utility function is risk-neutral if it is linear.
- A utility function is risk-seeking if it is strictly convex.

Examples are shown in Figure below.

Figure 10: Examples of risk-averse, risk-neutral and risk-seeking utility functions.



The relationship between risk and convexity can be expressed analytically by considering changes in wealth w when $u(w)$ measures the utility of that wealth to a decision maker. Suppose wealth is increased by a fixed amount Δ_w. Then utility increases by an amount

$$\Delta_u^+ = u\big(w + \Delta_w\big) - u\big(w\big)\,.$$

Then suppose wealth is decreased by the same amount Δ_w. Then utility decreases by an amount

$$\Delta_u^- = u(w) - u(w - \Delta_w)\,.$$

If $\Delta_u^- > \Delta_u^+$ then the decision maker is more concerned with avoiding losses than in increasing wealth. This behavior is appropriately characterized as risk-averse. Furthermore, the constraint $\Delta_u^- > \Delta_u^+$ is simply an alternative definition of concavity.

Similarly, if $\Delta_u^+ > \Delta_u^-$ then the decision maker is more concerned with increasing wealth, and is willing to tolerate some risk of loss in order to do so. This behavior is appropriately characterized as risk-seeking. This condition is simply an alternative definition of convexity.

An example of a dynamic programming problem which makes use of utility theory is given in Appendix D.

2.4.3. Minimax Criterion: The Stagecoach Problem Revisited

As in the original stagecoach problem, there are 11 towns labeled A to K, and a traveller must select a route starting from town A, ending in town K, according to the map shown in figure entitled as "Stagecoach problem map".

So far, our applications have used additive cost accrual methods. This seems natural, since in our examples, cost or reward has been expressible in units that themselves are additive (money, distance, and so on). However, particularly in the field of economics, more general forms of cost are defined which allow optimal policies to capture decision theoretic preferences for risk-averse, risk-neutral or risk-seeking behavior. We have already seen how utility can be used for this purpose.

This greater level of generality can be extended to the cost accrual method.

In fact, dynamic programming can be applied when cost accrual is not additive, provided the SDP retains its memoryless structure, which will be the case for the minimax criterion introduced in Eqn. (2.6).

Suppose we modify our objectives of the stagecoach problem. We recognize that the terrain in between towns can be quite harsh (especially for our horses). So we wish to avoid routes with large distances between towns.

Once we reach a town along our route, we can rest as long as we need before the next segment of our journey. Therefore, the goal is not to choose a route that minimizes the total travel distance, but to choose a route that minimizes the maximum distance between towns. This is an example of the minimax criterion (in contrast, the maximin criterion attempts to maximize a minimum reward). The minimax criterion applied to cost is an example of risk-averse preference.

Previously, the policy value function associated with the minimum total distance for a deterministic K-stage finite horizon SDP was:

$$V^{\phi}(x_1) = R(x_1, \phi(x_1)) + R(x_2, \phi(x_2)) + ... + R(x_K, \phi(x_K)),$$

where $R(x,a)$ is the cost function representing the stage distances, and ϕ is the policy. For the minimax criterion problem the total cost becomes

$$V^{\phi}(x_1) = \max\{R(x_1, \phi(x_1)), R(x_2, \phi(x_2)), ..., R(x_K, \phi(x_K))\}\,.$$

Note that in this representation, state x_1 is not necessarily state A, since we need to define V^{ϕ} for all states. However, we will always have the same terminal state $\phi(x_K) = K$. So $V^{\phi}(x)$ represents the minimax distance cost from state x to K under policy ϕ.

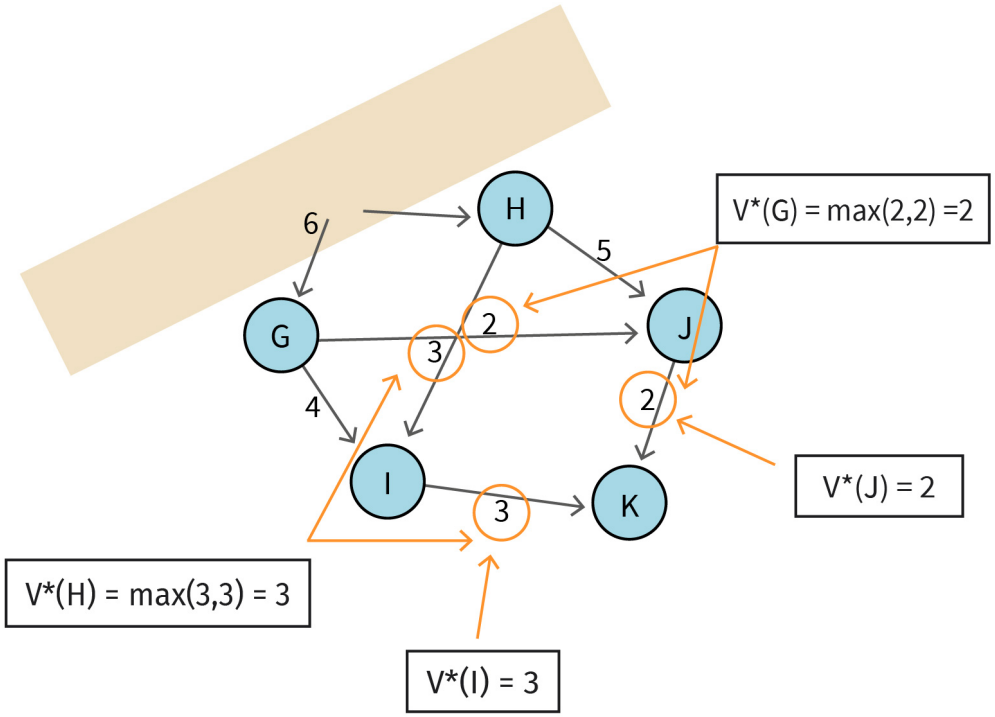
We can then construct a recursive expression for the optimal value function $V^*(x)$ based on Bellman's equation (2.25):

$$\begin{aligned} V^*(x,a) &= \max\{R(x,a), V^*(q(x,a))\}, \\ V^*(x) &= \min_{a \in \mathcal{A}_x} \max\{R(x,a), V^*(q(x,a))\} \\ &= \min_{a \in \mathcal{A}_x} \max\{R(x,a), V^*(a)\}\,. \end{aligned} \tag{2.38}$$

At this point we can determine V ∗(x) using a backward induction algorithm.

Figure below illustrates the calculates for Stages 4 and 5.

Figure 11: Stagecoach problem with minimax cost criterion. Section of the map of Fig. 2.1 representing the calculations for Stages 4 and 5.



Stage 5:

Trivially, $V^*(I) = 3$ and $V^*(J) = 2$ (only one action, $\phi^*(I) = \phi^*(J) = K$, is available from each state I and J).

Stage 4:

Eqn. (2.38) may be applied directly.

$$\begin{aligned} V^*(G,I) &= \max\{R(G,I), V^*(I)\} = \max\{4,3\} = 4, \\ V^*(G,J) &= \max\{R(G,J), V^*(J)\} = \max\{2,2\} = 2, \\ V^*(G) &= \min\{V^*(G,I), V^*(G,J)\} = \min\{4,2\} = 2\,. \end{aligned}$$

So, from G it is optimal to travel to $\phi^*(G) = J$, which yields an optimal remaining minimax distance from G of $V^*(G) = 2$.

We do the same for state H:

$$\begin{aligned} V^*(H,I) &= \max\{R(H,I), V^*(I)\} = \max\{3,3\} = 3, \\ V^*(H,J) &= \max\{R(H,J), V^*(J)\} = \max\{5,2\} = 5, \\ V^*(H) &= \min\{V^*(H,I), V^*(H,J)\} = \min\{3,5\} = 3\,. \end{aligned}$$

So, from H it is optimal to travel to $\phi^*(H) = I$, which yields an optimal remaining minimax distance from H of $V^*(H) = 3$.

The reader can complete the remaining stages, verifying that the unique optimal minimax route from A to K is A-C-F-H-I-K, and the optimal minimax distance is $V^*(A) = 3$. Recall that in the original minimum total distance problem (Sec. 2.3.1) there were three optimal paths with equal minimum total distance of 15. These were, from Eqn. (2.26),

$$\begin{aligned} &A - C - F - H - I - K, \\ &A - D - E - G - J - K, \text{ and} \\ &A - D - F - H - I - K\,. \end{aligned}$$

The first of these (but only the first) is also optimal with respect to the minimax criterion.



SUMMARY

In this unit, we were introduced to the sequential decision process (SDP), reviewing its history, as well as the unifying role played by dynamic programming. We then defined formally the dynamic programming model, discussing its advantages and disadvantages. In particular, we described how the "curse of dimensionality" motivates the research and development into ways to render the model more computationally efficient.

We then reviewed the three principle algorithms of dynamic programming, backward induction, value iteration and policy iteration, used to derive optimal control policies for SDPs. We also described how these optimization principles can be implemented by computer programs.

Following this, a special class of SDP problems from the field of economics was introduced , which made use of utility theory, in order to allow more flexible definitions of cost and reward. We saw how dynamic programming methods can be adapted to these problems.

UNIT 3

BANDIT PROBLEMS

STUDY GOALS

After completing this unit you will have learned ...

- the definition of a bandit problem, and its role in optimal sequential decision processes.
- the various approaches to designing optimal solutions to the bandit problem, including ϵ-greedy policies, softmax policies and index policies.

3. BANDIT PROBLEMS

Introduction/Case study

In medical research, there may be a fundamental conflict between the imperative of providing the best possible treatment to any patient, and the imperative of discovering what the best treatment is. The randomized clinical trial (RCT) is the primary means of comparing two or more medical treatments. Suppose we have a conventional treatment T_C, and

experimental treatment T_E. We have n subjects selected for the trial. Each subject is randomly assigned one of the two treatments. Each treatment being compared is referred to as an "arm" of the trial. Following treatment, from each subject a measure X of the efficacy (or therapeutic merit) of the treatment is collected (for example, blood pressure, viral count or tumor size).

A comparison of this measure is used to determine the superior treatment.

When the trial is complete, the average efficacy measures $\overline{X}_C$ and $\overline{X}_E$ are calculated for the groups receiving treatments T_C and T_E, respectively.

The concept of "clinical equipoise" was introduced in (Freedman, 1987) as "... a state of genuine uncertainty on the part of the clinical investigator regarding the comparative therapeutic merits of each arm in a trial". Suppose we can estimate the probability π that experimental treatment T_E is superior to conventional treatment T_C. Furthermore, sup-

pose we can reasonably assert that $\pi \approx 0.75$. If in a clinical practice a decision had to be made to assign one of the two treatments to a single subject, then the ethical decision would be to assign T_E, as long as the risks for this treatment are no greater than for treatment T_C. Similarly, if $\pi < 1/2$, the ethical choice would be treatment T_C.

This poses a dilemma for a RCT. If there was evidence that $\pi < 1/2$ or $\pi > 1/2$, then it could be regarded as unethical to assign a treatment, other than the choice that would be made in a clinical practice, to any subject in the study. This is because that choice would only be made after considering the patient's best interest. However, in a RCT, treatments are assigned to each subject at random. Therefore, a RCT should ideally be carried out in a state of clinical equipoise, mathematically defined as $\pi = 0.5$, which represents the maximum state of uncertainty regarding the efficacy of the candidate treatments T_C and T_E.

However, another point of view can be taken. While it is true that it is ethical to assign the treatment that is superior with probability greater than 1/2, there is also an ethical imperative to remove any uncertainty in this regard. That is, we would like to be able to estimate either $\pi = 0$ or $\pi = 1$.

The theory of bandit processes is intended to balance the imperative of assigning the superior treatment with the imperative of discovering what that treatment is. Like the RCT, a bandit process assigns treatments to subjects, then observes the outcome. Unlike the RCT, as evidence emerges that one treatment is superior, subjects are increasingly likely to be assigned that treatment. In other words, a bandit process compares two or more treatments in a sequential manner. Each is sampled enough times to allow identification of the best treatment, while gradually weighting the sampling in favor of that treatment.

Bandit processes are a class of sequential decision processes (SDP) which are dynamic processes involving a sequence of decisions which influence some cost or reward that accrues over time. Recently, they have become an important chapter of reinforcement learning, which defines a class of algorithms which attempt to determine decision rules for an SDP which yield optimal rewards through a controlled process of trial and error (Sutton & Barto, 2018a).

We can think of a one-armed bandit as a type of conceptual slot machine.

If we pull the arm (the slot machine's mechanical lever) we get a reward r.

The reward, however, is usually a random variable, possessing distribution $f(r)$ with mean μ. We can think of μ as the mean value of pulling the arm, and the problem interests us because it is important to quantify this value. If we know this density (or at least μ), nothing more needs to be done. Otherwise, in order to estimate μ, we must pull the arm n times, collect the sample of rewards $r_1, ..., r_n$, then calculate the estimate, say,

$$\widehat{\mu} = \overline{r} = \frac{1}{n} \sum_{i=1}^{n} r_i$$

3.1 The One-Armed Bandit Problem

The problem becomes much more than an estimation problem if the objective is to maximize some definition of **total reward**, and the one-armed bandit just described is only one source of reward.

Total reward
For an SDP, total reward represents the accumulation of rewards.

In the simplest form of the problem, there is a sequence of stages $j = 1, 2, ...$ where a stage defines the sequential unit of an SDP. Within each stage, a decision is made, and a reward is accrued. Each stage has a discount factor ρ_j, where a sequence of discount factors ρ_j assigns differential weights to the to the successive rewards of a bandit process.

Suppose, one way or another, we earn reward r_j at stage j. The (expected) total reward is then

$$R = \sum_{j=1}^{\infty} \rho_j E\big[r_j\big] .$$

(3.1)

The purpose of the discount factors is to ensure that the total reward R is well defined. For a finite horizon SDP, where the number of stages is finite and fixed in advance, we would have $\rho_j = 1$, $j = 1, ..., M$; and $\rho_j = 0$, $j > M$. Thus, we only accrue rewards for M decision epochs, or stages.

For an infinite horizon SDP, where the number of stages is infinite, we would have $\rho_j = \beta^{j-1}$ $j \geq 1$ for some $\beta \in (0, 1)$. Thus, we accrue rewards over an unbounded sequence of stages. However, as long as the reward can be bounded, R will be well defined. Suppose we always have $r_j \geq 0$, and $E\big[r_j\big] \leq B < \infty$ for all $j \geq 1$, then

$$R = \sum_{j=1}^{\infty} \beta^{j-1} E\big[r_j\big] \leq B \sum_{j=1}^{\infty} \beta^{j-1} = B \times \left(1 + \beta^1 + \beta^2 + ...\right) \tag{3.2}$$

$$\leq \frac{B}{1-\beta} < \infty ,$$

$\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r}$ for $|k| < 1$

using the geometric series. Then suppose at each stage j we can either pull the arm, and receive reward r_j from the bandit, or accept a fixed constant reward δ, setting $r_j = \delta$. Our stage j decision is then to take one of two actions $aj \in b, f$ (i.e. "bandit" or "fixed" reward δ). The total reward is therefore

$$
\begin{aligned}
R &= \sum_{j=1}^{\infty} \rho_j E\big[r_j\big] \\
&= \sum_{j=1}^{\infty} \rho_j \big(E\big[r_j\big] I\big\{a_j = \boldsymbol{b}\big\} + \delta I\big\{a_j = \boldsymbol{f}\big\} \big) \\
&= \sum_{j=1}^{\infty} \rho_j \big(\mu I\big\{a_j = \boldsymbol{b}\big\} + \delta I\big\{a_j = \boldsymbol{f}\big\} \big) .
\end{aligned}
$$

where μ is the expected value of r_j. Note that, in general, $I\{A\}$ is an indicator function which equals 1 of condition A is satisfied, and equals 0 otherwise. It is commonly assumed that bandit rewards are independent and identically distributed, although more complex models can be used when this is not the case. It is also possible that mean rewards change over time, in which case adaptive procedures result in decision rules for SDPs that change over time. This would be appropriate in a bandit process if the mean reward of a one-armed bandit does not remain constant over time.

See, for example, (Berry & Fristedt, 1985).

If we ask which sequence of actions maximizes R, the answer is one sense trivial. If $\mu > \delta$, always choose the bandit, and if $\delta \geq \mu$, never choose the bandit (note that $\delta = \mu$either action is equally optimal, but we do need to make a decision). The problem arises, of course, when we don't know μ, and it is this case which motivates the theory of bandit problems.

3.1.1. Iterative Update rules For Reward Estimates

At this point, it is worth reviewing a simple update formula for the sample mean. Suppose we have n observations $x_1, x_2, ..., x_n$. Then consider the sample mean

$$\overline{x}_m = \frac{1}{m}\sum_{i=1}^{m} x_i,$$

obtained from the first m observations in the sequence.

Then suppose for a certain application we receive the observations sequentially.

After receiving the first m observations we calculate $\overline{x}_m$. We then receive the next observation x_{m+1}, and wish to calculate $\overline{x}_{m+1}$. Of course, rather than calculating $\overline{x}_{m+1}$ using all of the first $m+1$ observations, we could use an update rule that requires only $\overline{x}_m$ and x_{m+1}:

$$\begin{aligned}
\overline{x}_{m+1} &= \frac{1}{m+1}\sum_{i=1}^{m+1} x_i \\
&= \frac{\sum_{i=1}^{m} x_i}{m+1} + \frac{x_{m+1}}{m+1} \\
&= \frac{m}{m+1}\overline{x}_m + \frac{1}{m+1}x_{m+1} \\
&= \overline{x}_m + \gamma_{m+1}\big(x_{m+1} - \overline{x}_m\big),
\end{aligned} \tag{3.3}$$

where we define the sequence of constants $\gamma_m = 1/m$, $m \geq 1$. Note that under the conventions of reinforcement learning, γ_m is generally referred to as the learning rate that controls the rate of change of the successive estimates. Sometimes the subscript of γ_m is omitted, so that the learning rate is simply γ. In this case, it is simply understood that the rules for determining the learning rate, and whether it is constant or changing, is left unspecified for moment.

We can represent the iterative equation for the sample mean $\overline{x}_{m+1} = \overline{x}_m + \gamma_{m+1}(x_{m+1} - \overline{x}_m)$, see Eqn. (3.3)) as a more general updating rule:

$$\widehat{Q} \leftarrow \widehat{Q} + \gamma\big(r - \widehat{Q}\big)\,. \tag{3.4}$$

Here $\widehat{Q}$ is an estimate of average reward μ, and r is one observation from a sequence used to construct the estimator $\widehat{Q}$. If we apply the update rule of Eqn. (3.3) then $x_1, x_2, ...$ represent the successive rewards from a single one-armed bandit, assumed to be a sample from a distribution with mean μ. On the right side of the assignment, $\widehat{Q} = \overline{x}_m$ and $r = x_{m+1}$. On the left side of the assignment, $\overline{x}_m$ is updated to $\overline{x}_{m+1}$. The use of the assignment operator $\leftarrow$ implies that an estimator is being updated, resulting in a sequence. This implies the appropriate indexing system, although this is often omitted in the literature. In addition, in Eqn. (3.3)

$\gamma = \gamma_{m+1} = 1/(m+1)$. In update rule (3.4) γ is left unspecified, and usually decreases with the number of updates. In practice, reinforcement learning admits multiple strategies for choosing its value, although it is usually intended to have the same effect as $\gamma = \gamma_m = 1/m$ as used for the sample mean.

3.1.2. Policies for Bandit Problems

The bandit problem is a type of SDP. At any given stage j we take action a_j (ie. pull arm a_j), representing a choice from among sources of reward. The problem we are currently considering in this unit has two reward sources, but we will ultimately consider problems with K sources, in which case an action is a choice $a_j = k \in \{1, ..., K\}$. Accordingly, reward r_j is accrued from that source. We assume that all available information gained from previous actions is available to help guide the choice of action a_j. We may refer to this information collectively as the the process history which is the record of all stage data available at a point of time just before an action is to be selected. For each previous stage $i = 1, ..., j-1$, we have the observed pair (a_i, r_i). Thus, we have access to all accrued rewards and their sources.

Data from the process history can be summarized by updating rules. Many algorithms require the action frequencies which is are calculated from the process history, and represent the number of times a specific action was previously chosen:

$$n_{k,\,j} = \sum_{i\,=\,1}^{j} I\big\{a_j = k\big\} = \text{Number of times action } k \quad \text{was chosen}\,.$$

(3.5)

We may also maintain estimates of the mean rewards μ_k for each reward source using an update rule such as Eqn. (3.4). An obvious choice is

$$\widehat{Q}_{k,\,j} = \tfrac{1}{n_{k,\,j}} \sum_{i\,=\,1}^{j} r_i I\{a_i = k\}, \tag{3.6}$$

although we will study other methods.

The final element of the decision problem is the policy Φ, which is a sequence $\Phi = (\phi_1, \phi_2, ...)$ of rules that map all available information to an action for each stage, i.e. the policy is a decision rule that allows us to select an action based on the available history. This policy can be written as:

$$\phi_j\big(a_1, r_1, a_2, r_2, ..., a_j, r_j\big) = k \in \big\{1, ..., K\big\},\ j \geq 1\,.$$

The history can be reduced to the current reward estimates and action frequencies, which are defined by Eqn. (3.5) and (3.6), and the policy is more naturally expressed

$$\phi_j\big(n_{1,\,j}, ..., n_{K,\,j}; \widehat{Q}_{1,\,j}, ..., \widehat{Q}_{K,\,j}\big) = k \in \{1, ..., K\},\ j \geq 1\,.$$

The problem, in summary, is to determine the policy Φ which maximizes total reward (3.1), given a specific sequence of discount factors $\rho_1, \rho_2, ...$

We finally note that the optimization problem is often characterized in terms of regret, which is defined as the difference between the maximum total reward attainable by any policy and the total reward attained by Φ.

The largest possible value of total reward defined in Eqn. (3.1) is easy to deduce. If $\mu^* = \max\limits_k \mu_k$ is the largest possible average reward, accrued by selecting arm k^*, then the largest possible total reward is clearly attained by selecting only action $a_j = k^*$ (note that we need not assume arm k^* is uniquely optimal), giving optimal total reward

$$R^* = \sum_{j=1}^{\infty} \rho_j \mu^* . \tag{3.7}$$

An optimal policy (any policy achieving the maximum possible total reward) is then $\phi_j = k^*$, no matter what information is available. Suppose a policy Φ results in a sequence of actions $a_1, a_2, ...$ Then the realized reward is

$$R^{\Phi} = \sum_{j=1}^{\infty} \rho_j E\Big[\mu_{a_j}\Big] . \tag{3.8}$$

Then regret can be formally defined as

$$\Delta^{\Phi} = R^* - R^{\Phi} . \tag{3.9}$$

Clearly, $R^* \geq R^{\Phi}$, so $\Delta^{\Phi} \geq 0$ is always non-negative, and can be interpreted as the amount by which the expected attained reward of policy Φ is smaller that the optimal achievable (essentially, the cost of using a sub-optimal policy). This means that $\Delta^{\Phi} = 0$ if and only if Φ is an optimal policy, and that the problem of maximizing total reward given in Eqn. (3.1) is exactly the same as minimizing regret.

The theory of bandit problems anticipates that we do not know the mean rewards μ_j, so it is inevitable that any attempt to estimate these quantities will introduce regret. But before discussing the current state of the art of this problem, we will introduce a number of possible solutions which may seem, at first, reasonable, but which are significantly flawed.

Naive Solution 1

If the problem is that we don't know μ, we can always estimate it. We might regard the problem as a statistical one, and accept that we can only make the correct decision with a certain confidence level $1 - \alpha$. Then a sample size N can be selected in advance. For the

first N stages the bandit is selected. This yields estimate $\widehat{Q}_N$. Then for the remaining stages select the bandit if $\widehat{Q}_N > \delta$, and select the fixed reward if $\delta \geq \widehat{Q}_N$. We can write the policy specifically as

$$a_j = \begin{cases} b \ ; \ j \leq N \\ b \ ; \ j > N, \widehat{Q}_N > \delta \\ f \ ; \ j > N, \delta \geq \widehat{Q}_N \, . \end{cases}$$

How good is the policy described as naïve solution 1? We can enumerate several of its properties.

1. It gives the optimal total reward if $\mu > \delta$ and $\widehat{Q}_N > \delta$, but not if $\mu < \delta$, which cannot be known in advance. This is because the first N rewards are sub-optimal.
2. The long run behavior of the policy depends entirely on whether or not the ranks of μ, δ are estimated correctly (i.e. if $\widehat{Q}_N > \delta$ whenever $\mu > \delta$).

Naive Solution 2

There is a simple way to improve naïve solution 1. Suppose after the first N stages we observe $\widehat{Q}_N > \delta$. This means that we will choose the bandit at all future stages. But this will only be optimal if $\mu > \delta$. Suppose this is not the case. Because we are continuing to collect data from the bandit, we can refine our estimate of μ. By stage $j > N$, we can calculate $\widehat{Q}_j$. If $\mu < \delta$, the successive values of $\widehat{Q}_j$ will be below δ with increasing frequency, so that we can eventually make the (correct) ranking judgement, and take actions $a_j = f$ from that point on.

Of course, this doesn't solve the problem of "getting stuck" with a suboptimal solution entirely. If after the initial sample sample of N stages we incorrectly decide that $\mu < \delta$, we would select action $a_j = f$ for all $j > N$, and never collect more data with which to refine our ranking estimate.

3.2 The Multiple Arm Bandit Problem

The transition from the one to multiple arm bandit problem illuminates the problem in an interesting way, and much of the structure of the latter problem is captured by the simpler one.

Suppose we replace the fixed reward δ option with additional arms. To fix ideas, suppose we have $K = 3$ arms, each of which give a reward with a normal distribution $r_k \sim N(\mu_k, \sigma^2)$, $k = 1, 2, 3$, with common standard deviation $\sigma = 20$ and means $\mu_1 = 25$, $\mu 2 = 30$, $\mu 3 = 35$. Again, if we know all of the parameters, we would always select arm $k = 3$, since it offers the largest average reward. But how do we proceed if the means μ_k are unknown?

The issues here are much the same as for the one arm vs fixed reward problem. We could first obtain a fixed size sample from each arm, estimate the means, then remain with the arm estimated to yield the largest average reward. Of course, as noted in our discussion of the two naive solutions presented in Sec. 3.1, if we continue to collect data (as would always be the case for this application), it would obviously make sense to use it, given its potential to correct an error in our ranking estimate.

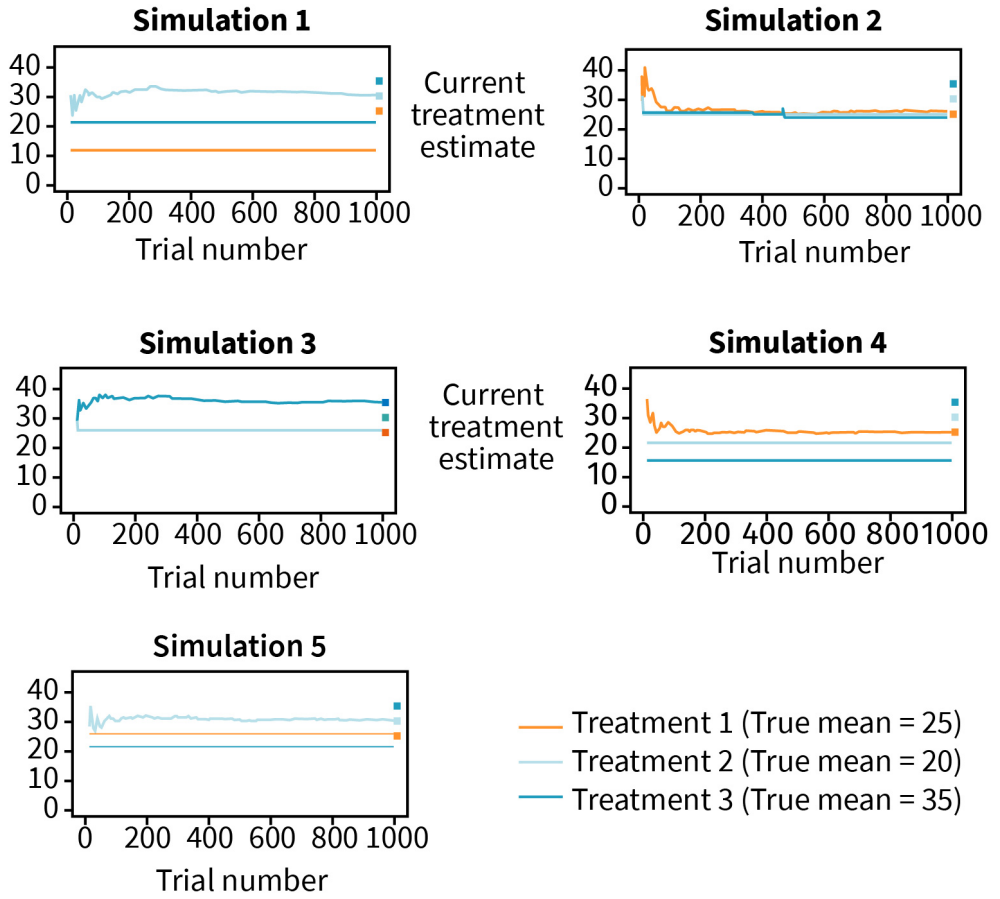
So we propose a compromise. We start by collecting an initial sample of size N from each arm (this requires $3N$ stages). Noting that our goal is to maximize reward, even when we are collecting information, we might not make N too large. We then define our SDP precisely. At the end of any stage j, we have estimates $\widehat{Q}_{k,\,j}$ of the respective means μ_k for $k = 1, 2, 3$.

Then for $j > 3N$, we choose the arm currently estimated to have the highest average reward, in particular,

$$a_j = \text{argmax}_{k\,\in\,\{1,\,2,\,3\}}\widehat{Q}_{k,\,j}\,.$$

Figure below shows five replications of a computer simulation of this bandit process. An initial sample of size $N = 5$ is collected, which is used as the initial values of each estimate. The plots then show the values of $\widehat{Q}_{k,\,j}$ for each arm (the initial sample is collected during stages $j = 1, ..., 15$, which are not plotted).

Figure 12: Maximum reward selection policy discussed in Sec. 3.2 for a three-armed bandit process.



If the method works, then the estimate of the highest mean $\mu_3 = 35$ should eventually become larger than the other two, and remain so. However, this generally does not happen. The problem can best be seen in simulation 1. In each plot the true mean is represented by a color-coded box at the right end of the plot. We can therefore see that after about $j > 300$ trials the estimate of the $k = 2$ arm reward $\widehat{Q}_{2,j}$ is very close to the true value μ_2. Furthermore, because $\widehat{Q}_{2,j} > \widehat{Q}_{1,j}$ and $\widehat{Q}_{2,j} > \widehat{Q}_{3,j}$, arm $k = 2$ is estimated to have the largest average reward, so the next action will be $a_j = 2$. Meanwhile, since the remaining arms $k = 1, 3$ are not selected, the estimates $\widehat{Q}_{1,j}, \widehat{Q}_{3,j}$ will remain unchanged, since there is no new data from those arms.

At this point, the only way an arm other than $k = 2$ can be selected is if the estimates $\widehat{Q}_{2,j}$ eventually move below the constant level $\widehat{Q}_{3,j}$ shown in the plot of simulation 1. While this is possible, it would be an event of very small (and shrinking) probability. Recall that the law of large numbers predicts that a sequence of sample means such as $\widehat{Q}_{2,j}$ will converge to the true mean μ_2 as the sample size grows indefinitely. Thus, with probability close to 1, if simulation 1 were allowed to continue, it would select arm $k = 2$ indefinitely.

In general, the conclusions vary considerably among the five simulations, so this particular decision rule is too unreliable, and the problems with naïve solutions described in Sec. 3.1 remain.

3.2.1. Exploitation vs. Exploration Trade-off

The simulation study reported in figure above demonstrates the exploitation versus exploration trade-off. It arises from the two inherently conflicting goals of the bandit problem. First, we would would like to estimate the average reward of each arm, so we can make the optimal choice (exploration).

Second, we would like select the optimal arm (exploitation). This dilemma occurs in many sequential decision problems where we need to learn an optimal policy (exploration) which conflicts with the imperative of applying the current best estimate of the optimal policy exploitation.

Figure above (especially Simulation 1) shows what happens when there is insufficient exploration, in particular, we may end up making sub-optimal decisions. We refer to a policy as greedy if it selects actions which yield the optimal reward assuming our estimates are accurate. Figure above demonstrates a greedy policy. In contrast, an exploratory policy ensures that each arm has been selected enough times to permit identification of the arm giving the maximum reward. The problem with a greedy policy is that our estimates may not be sufficiently accurate to identify the optimal arm. A simple expedient is to force more exploration by using a ϵ-greedy policy. This is a randomized policy that selects the greedy policy with frequency $1 - \epsilon$ (exploitation) and an exploratory policy with frequency ϵ. The exploratory policy may simply be a decision to pull any lever at random.

Of course, because of the imperative to exploit, we do not allow exploratory behavior to dominate. Therefore ϵ is usually chosen to be relatively small.

Suppose we select $\epsilon = 1/5$. We could simply enforce an exploratory policy at every fifth stage. Alternatively, at each stage j we could simulate a binary random variable U_j satisfying $P\big(U_j = 1\big) = 1 - P\big(U_j = 0\big) = 1/5$, and enforce an exploratory policy if $U_j = 1$.

The ϵ-greedy policy is an example of a **forced exploration policy**.

Forced exploration policy
A forced exploration policy seeks to ensure that sufficient exploration of all available actions of a SDP occurs.

Refining the Exploitation/Exploration Balance

Defining a constant exploration rate for an ϵ-greedy policy is enough to ensure (by the law of large numbers) that the optimal arm will eventually be identified. However, sub-optimal rewards will continue to be accrued at a constant rate, long after the optimal arm can be identified with near certainty. Therefore, forced exploration policies often adjust the exploitation/ exploration balance over time, with exploration weighted more heavily in the early stages, and exploitation increasingly favored in later stages (assuming the rewards do not change in time).

The ϵ-greedy policy can be easily modified to achieve this goal, by allowing ϵ to converge to zero, so that the greedy policy becomes increasingly dominant. Suppose an exploratory policy is selected with probability ϵ_j at stage j (we could easily simulate binary random variables U_j satisfying $P\big(U_j = 1\big) = 1 - P\big(U_j = 0\big) = \epsilon_j$, as described earlier). We are free to choose whichever sequence ϵ_j, $j \geq 1$ we wish. It can be allowed to converge to zero, so that exploitative decisions increasingly dominate. But, if we wish to ensure that the optimal arm is eventually identified, ϵ_j must decrease slowly enough to ensure that exploratory behavior is selected infinitely often. A simple rule that ensures this is

$$\sum_{j=1}^{\infty} \epsilon_j = \infty \ . \tag{3.10}$$

Suppose, to fix ideas, we set $\epsilon_j = 1/j^r$, where $r > 0$. Then condition (3.10) is satisfied if $r \leq 1$, but not if $r > 1$. We can refer to this strategy as an ϵ-greedy policy with vanishing exploration, where the rate at which an exploratory policy is applied is allowed to converge to zero, so that optimal decisions eventually dominate.

Chapter 2 of (Sutton & Barto, 2018a) can be highly recommended for an in-depth discussion and evaluation of these methods.

Softmax Action Selection

Another forced exploration policy that is commonly used is similar to the ϵ-greedy policy with vanishing exploration, except that at each stage j arm k is selected with probabilities defined by the softmax function that is used to convert a set of real numbers into a probability distribution by first applying the exponential transformation to each, then normalizing.

$$P\big(a_j = k\big) = \frac{e^{\widehat{Q}_{k,\,j-1}/\tau}}{\sum_{k=1}^{K} e^{\widehat{Q}_{k,\,j-1}/\tau}} = \pi_k(k), \tag{3.11}$$

where there are K arms, $\widehat{Q}_{k,\,j-1}$ is the estimate of the arm k reward available immediately preceding stage j, and $\tau > 0$ is referred to as the temperature parameter. This is referred to as the softmax policy. For a bandit problem, softmax policy selects an arm according to a distribution obtained by applying the softmax function to the current reward estimates for each arm, scaled by a temperature parameter τ. Note that the softmax selection rule does not explicitly construct distinct exploitative and exploratory policies.

Rather, it implements the exploitation/exploration balance by assigning higher selection probabilities to arms with higher reward estimates.

The properties of the policy are highly dependent on the temperature parameter.

If τ is very large, then the exponential quantity $e^{\widehat{Q}_{k,\,j-1}/\tau}$ is very close to 1. In this case, each arm will have approximately the same selection probability, and the policy will be close to purely exploratory. On the other hand, as τ approaches zero, the selection probability of whatever arm k has the largest estimate $\widehat{Q}_{k,\,j-1}$ will approach 1, yielding essentially a greedy policy. See Section 2.2 of (Sutton & Barto, 2018a).

We will repeat our simulation of the bandit problem using the softmax policy. Figures below show summaries similar to figure above. The two implementations differ only by the temperature parameter $\tau = 1$ or $\tau = 100$ respectively. For the lower temperature $\tau = 1$ (figure below), the policy barely differs from a purely greedy policy, similar to figure above, and exhibits the same negative properties.

On the other hand, increasing the temperature to $\tau = 100$ induces sufficient exploration in the early stages to allow the optimal arm $k = 3$ to be consistently identified (figure below).

Figure 13: Softmax policy for a three-armed bandit process, for temperature parameter $\tau = 1$.

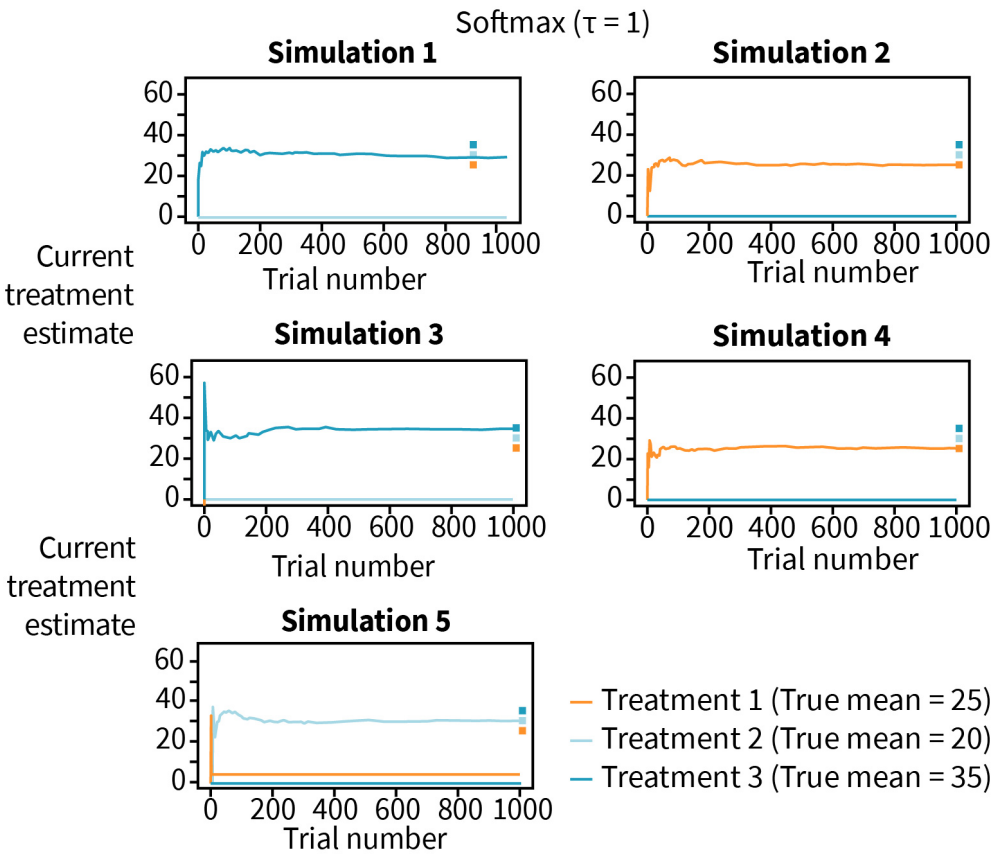
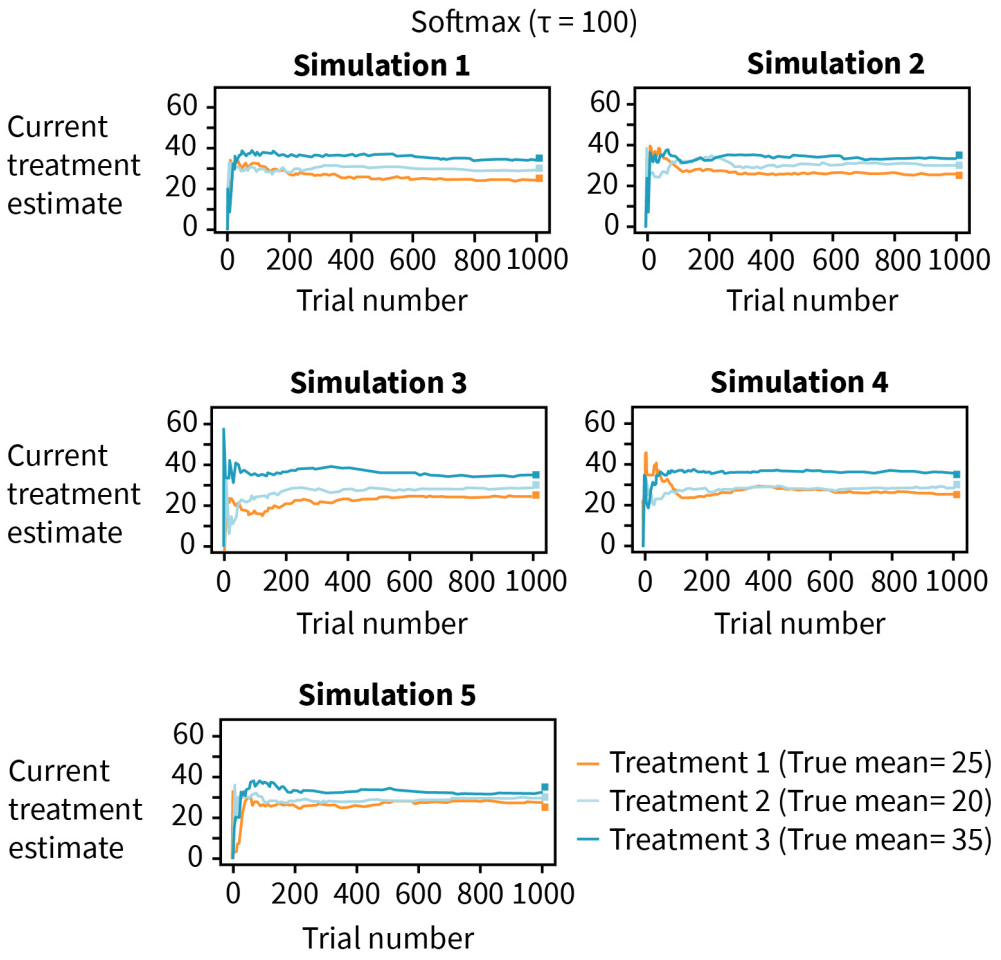


Figure 14: Softmax policy for a three-armed bandit process, using temperature parameter $\tau = 100$.



Gradient Policy

Suppose we modify the softmax action selection distribution (3.11) by replacing the reward estimates $\widehat{Q}_{k,\,j-1}$ with a more general numerical preference $H_{k,\,j}$, giving

$$P\big(a_j = k\big) = \frac{e^{H_{k,\,j}}}{\sum_{k=1}^{K} e^{H_{k,\,j}}} = \pi_j(k)\,. \tag{3.12}$$

Note that $H_{k,\,j}$ can be of arbitrary units, its essential property being that larger values tend to favor selection of arm k at stage j. Several options for $H_{k,\,j}$ have been proposed by the theory of bandit problems, and we will introduce a few below. Therefore, we can omit the temperature parameter τ without loss of generality (simply set $\tau = 1$). Note that when we do this, we are essentially incorporating the parameter τ into the definition of $H_{k,\,j}$.

Next, suppose in Eqn. (3.12) we retain the original interpretation of $H_{k,j}$ as an updated estimate of reward μ_k:

$$H_{k,j+1} \leftarrow H_{k,j} + \gamma\big(r_j - H_{k,j}\big)\,. \tag{3.13}$$

If we set initial value $H_{k,1} = 0$, then by Eqn. (3.12) each action is equally likely to be selected at the beginning of the algorithm.

It sometimes helps in bandit problems to regard the collection of arm rewards $\mu_1, ..., \mu_K$ as themselves having some type of distributional properties.

Suppose, as an example, we have $K = 3$ arms with rewards $\mu_1 = -1.0$, $\mu 2 = -0.5$, $\mu 3 = +1.5$. We can say that the average reward is

$$\bar{\mu} = \frac{\mu_1 + \mu_2 + \mu_3}{3} = \frac{-1.0 - 0.5 + 1.5}{3} = 0\,.$$

Next, suppose we add a constant amount c to each mean μk, transforming them to $\mu_k + c$. This leaves the bandit problem unchanged in most ways.

The optimal arm remains the same. In addition, it may be shown that the regret Δ^{Φ} of any policy will be unchanged, since the constant c will cancel in the expression of Eqn. (3.9). Thus, the optimization problem itself remains unchanged, thus so will the optimal policy.

What about the softmax policy itself? A method of improving this method is suggested by examining what happens when we add an arbitrary number c to each arm reward μ_k. This will be equivalent to adding c to each estimate $H_{k,j}$ in Eqn. (3.12). If we denote the new probabilities $\pi_j^c\big(k\big)$, these become

$$\begin{aligned}
\pi_j^c\big(k\big) &= \frac{e^{H_{k,j}+c}}{\sum_{k=1}^{K} e^{H_{k,j}+c}} \\
&= \frac{e^c e^{H_{k,j}}}{\sum_{k=1}^{K} e^c e^{H_{k,j}}} \\
&= \frac{e^{H_{k,j}}}{\sum_{k=1}^{K} e^{H_{k,j}}} \\
&= \pi_j\big(k\big)\,.
\end{aligned}$$

Thus, the softmax selection rule does not change.

It may seem surprising, therefore, that the constant c can have a significant effect on the softmax algorithm. If we look at update rule (3.13), it can be seen that as c becomes larger, the increment $\gamma\big(r_j - H_{k,j}\big)$ is a smaller proportion of the target value of the estimate μ_k, so that progress of the estimate as a function of sample size may be much slower.

The gradient policy is a modification of the softmax policy which removes this effect. The difference is that the reward estimates $\widehat{Q}_{k,\,j-1}$ are replaced by a preference score $H_{k,\,j}$ for which updates are based on the relative values of the arm rewards, rather than their absolute value. Hence the update rule (3.13) is replaced by

$$H_{k,\,j+1} \leftarrow H_{k,\,j} + \gamma\big(r_j - \overline{r}_j\big)\big(1 - \pi_j\big(k\big)\big) \tag{3.14}$$

when arm $a_j = k$ is selected at stage j, r_j is the accrued reward, and $\overline{r}_j$ is the average of all rewards, irrespective of reward source. In addition, for any arm k' not selected at stage j, the following update is performed:

$$H_{k',\,j+1} \leftarrow H_{k',\,j} - \gamma\big(r_j - \overline{r}_j\big)\pi_j\big(k\big)\,. \tag{3.15}$$

As can be seen, if the arm k reward is relatively large, the increment $\gamma\big(r_j - \overline{r}_j\big)\big(1 - \pi_j\big(k\big)\big)$ of Eqn. (3.14) will be positive. This has the effect of accelerating progress towards identification of the correct reward ranking.

See Section 2.8 of (Sutton & Barto, 2018a).

3.2.2. Index Methods

The various methods described in Sec. 3.2.1 can be described as actionvalue methods, in the sense that a policy is based on a current best estimate of the reward rankings, supplemented by forced exploration to improve these estimates. For a bandit problem, this means that the action-value method combines arm selection based on the current arm reward rankings with forced exploration. If forced exploration is vanishing, then a welldesigned policy will converge to optimal performance (in this case, optimal performance is nothing more than repeatedly selecting the best arm).

A second approach can be denoted index methods. Here, at the beginning of each stage j, an index is calculated for each arm, based on any data available for that arm (i.e. all previously observed rewards). The arm with the largest index is selected, then the observed reward is added to the pool of data. This idea can simplify both the analysis and implementation of bandit algorithms, since the index of each arm is calculated independently of the other arms. However, at the same time provably optimal algorithms exist which take this form (Gittins, Glazebrook, & Weber, 2011).

In fact, the policy of figure entitled as "Maximum reward selection policy discussed in Sec. 3.2 for a three-armed bandit process" is an example of an index method, since samples means are calculated independently for each arm, and the largest of these is used to select the next arm. Of course, it is not a very effective index method. Fortunately, there is a simple way of modifying it. Suppose we think of an index as having two components:

$$\mathrm{Index} = [\mathrm{GreedyComponent}] + [\mathrm{UncertaintyComponent}]\,. \tag{3.16}$$

Remember that we select the arm with the largest index. Since we wish to favor arms with higher rewards, we might simply set the "Greedy Component" for arm k to the current reward estimate $\widehat{Q}_{k,j-1}$. However, we also wish to select arms that may be underrepresented in the total sample. If such an arm has a relatively small estimated reward, this may be because the true average reward really is small, but it may also be due to large estimation error associated with the small sample size. We would like such an arm to be selected, so its makes sense to set "Uncertainty Component" to be some measure of statistical uncertainty.

A Review of Confidence Intervals and Confidence Bounds

A natural method of constructing an index structured as in Eqn. (3.16) is suggested by considering the standard confidence interval (CI), upper confidence bound (UCB) and lower confidence bound (LCB) with confidence level $1-\alpha$.

We take a random sample of size n, $X_1, ..., X_n$ from distribution with density function $f_\theta(x)$, where θ is an unknown parameter to be estimated.

We may construct an estimate $\widehat{\theta}$ using the data, in the hopes that $\widehat{\theta} \approx \theta$.

A more precise form of inference is the confidence interval (with confidence level $1-\alpha$):

$$CI_{1-\alpha} = [L, U] \tag{3.17}$$

which is constructed so as to satisfy the property

$$P(\theta \in CI_{1-\alpha}) \geq 1-\alpha\,. \tag{3.18}$$

If possible, we design a confidence interval so that Eqn. (3.18) is an equality.

However, this is not always possible, and in this case we say the confidence interval is conservative. Usually, α is small, and the usual convention is to use $\alpha = 0.05$. In this case, a confidence interval contains the true value of θ with probability $1-\alpha = 0.95$ ("nineteen times out of twenty", as is commonly stated).

However, sometimes it makes more sense to give only a UCB or LCB, again, with confidence level $1-\alpha$. A UCB $U_{1-\alpha}$ satisfies

$$P(\theta \leq U_{1-\alpha}) \geq 1-\alpha, \tag{3.19}$$

while a LCB $L_{1-\alpha}$ satisfies

$$P(\theta \geq L_{1-\alpha}) \geq 1-\alpha\,. \tag{3.20}$$

If the random sample is from a normal distribution $N(\mu, \sigma^2)$, and σ^2 is unknown (which is almost always the case), then we have

$$
\begin{aligned}
CI_{1-\alpha} &= \overline{X}_n \pm t_{n-1;\alpha/2}\frac{S_n}{\sqrt{n}} = \left[\overline{X}_n - t_{n-1;\alpha/2}\frac{S_n}{\sqrt{n}}, \overline{X}_n + t_{n-1;\alpha/2}\frac{S_n}{\sqrt{n}}\right], \\
U_{1-\alpha} &= \overline{X}_n + t_{\alpha}\frac{S_n}{\sqrt{n}}, \\
L_{1-\alpha} &= \overline{X}_n - t_{\alpha}\frac{S_n}{\sqrt{n}},
\end{aligned}
\tag{3.21}
$$

where

$$
\begin{aligned}
\overline{X}_n &= \frac{\sum_{i=1}^{n} X_i}{n}, \\
S_n^2 &= \frac{\sum_{i=1}^{n} (X_i - \overline{X}_n)^2}{n-1}.
\end{aligned}
$$

are sample mean and variance, and $t_{\nu;\alpha}$ is the α critical value for a t-distribution of ν degrees of freedom (in particular $P\left(T > t_{\nu;\alpha}\right) = \alpha$, where T has a t-distribution of ν degrees of freedom).

The advantage of, say, the UCB is in applications in which it is not strictly necessary to estimate θ, but only to claim that θ is no larger than a certain number (the UCB). While it is not strictly incorrect to use the upper bound of a level $1-\alpha$ confidence interval for this purpose, it will be more efficient to use a level $1-\alpha$ UCB.

EXAMPLE

The distinction between the confidence interval and the confidence bound is demonstrated in the following example.

If we collect a sample of size $n = 5$ from a population with mean μ, and obtain

$$
\begin{aligned}
\overline{X}_5 &= 34.6 \\
S_5 &= 5.7
\end{aligned}
$$

then to calculate a 95% confidence interval we read the critical value

$$
\begin{aligned}
t_{n-1,\alpha/2} &= t_{4,0.025} \\
&= 2.7764
\end{aligned}
$$

from a suitable table or computer program. The confidence interval is then

$$
\begin{aligned}
CI_{.95} &= \overline{X}_n \pm t_{n-1,\alpha/2}\frac{S_n}{\sqrt{n}} \\
&= 34.6 \pm 2.7764\frac{5.7}{\sqrt{5}} \\
&= 34.6 \pm 7.08 = (27.52, 41.68)
\end{aligned}
$$

To calculate the UCB we need critical value

$$
\begin{aligned}
t_{n-1,\alpha} &= t_{4,0.05} \\
&= 2.1318,
\end{aligned}
$$

giving

$$
\begin{aligned}
U_{.95} &= \overline{X}_n + t_{n-1,\alpha}\frac{S_n}{\sqrt{n}} \\
&= 34.6 + 2.1318\frac{5.7}{\sqrt{5}} \\
&= 34.6 + 5.4 = 40.0.
\end{aligned}
$$

Index Methods Based on Upper Confidence Bounds

There is an interesting relationship between confidence intervals and confidence bounds and the index defined in Eqn. (3.16). If we define a bandit process index to be a UCB, rather than an estimate of μ_j, then it is interpretable as the highest plausible value of μ_j for a given confidence level $1-\alpha$ interval. Since both the estimate of μ_j , and the uncertainty of that estimate, are incorporated into the index, both exploration and exploitation are well balanced. Furthermore, because the margin of error approaches zero as the sample size increases, the balance places increasing weight on exploitation.

An obvious strategy is therefore to set

$$
\begin{aligned}
[\text{GreedyComponent}] &= \widehat{Q}_{k,j-1}, \\
[\text{UncertaintyComponent}] &= t_\alpha \frac{S_{n_{k,j-1}}}{\sqrt{n_{k,j-1}}}
\end{aligned}
$$

leading to:

$$
\text{Index} = \widehat{Q}_{k,j-1} + t_\alpha \frac{S_{n_{k,j-1}}}{\sqrt{n_{k,j-1}}}, \tag{3.22}
$$

when the data is approximately normally distributed. Some confidence level $1-\alpha$ close to one would then be chosen.

While the motivation for the index defined in Eqn. (3.22) is quite intuitive, it must be understood that the problem of designing an efficient confidence interval is not the same as minimizing regret, and a mathematical theory exists which gives the smallest possible regret attainable, and policies which obtain this (T. L. Lai & Robbins, 1985). In (Auer, Cesa-Bianchi, & Fischer, 2002) a simple index policy attaining optimal regret was presented. It resembles the UCB index of Eqn. (3.22), but also has some notable differences. It is given by

$$[\text{GreedyComponen}] = \widehat{Q}_{k,\,j-1},$$
$$[\text{UncertaintyComponent}] = \sqrt{\frac{2\log(j-1)}{n_{k,\,j-1}}}$$

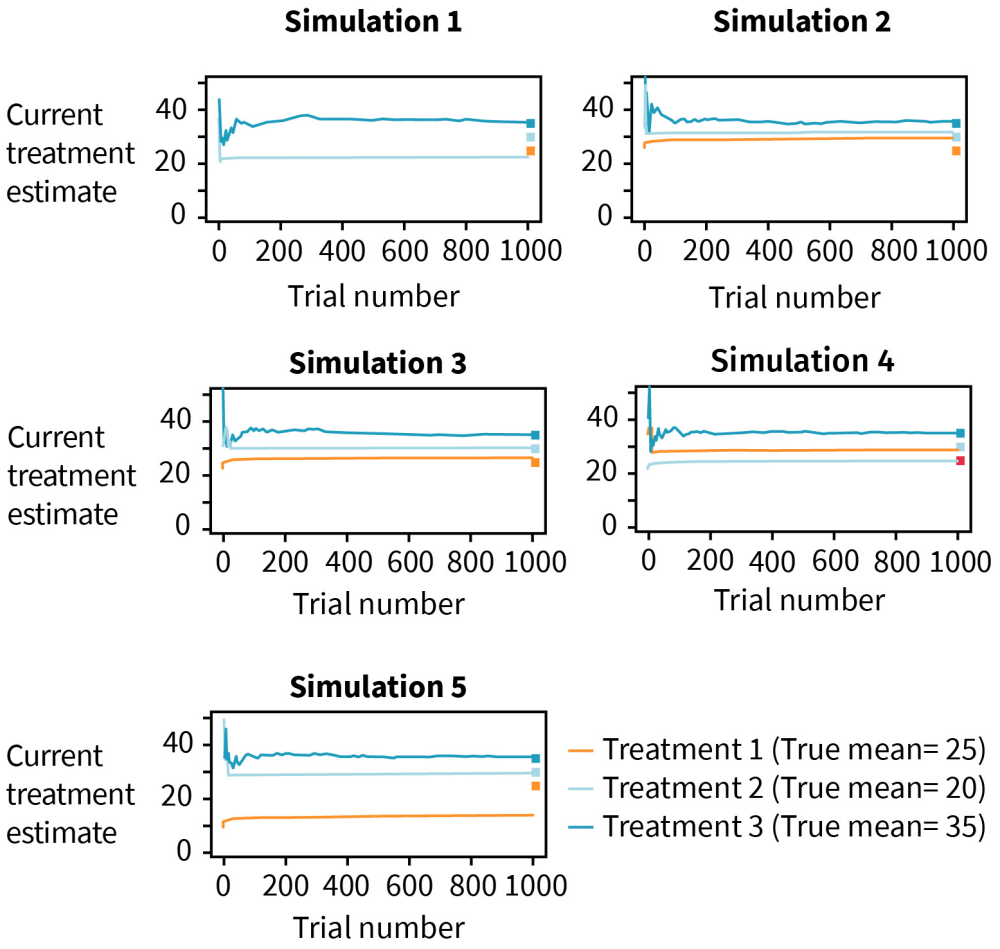
leading to:

$$\text{Index} = \widehat{Q}_{k,\,j-1} + \sqrt{\frac{2\log(j-1)}{n_{k,\,j-1}}}\,. \tag{3.23}$$

The uncertainty component of index (3.23) shares the proportionality to $1/\sqrt{n}$ with the corresponding component of index (3.22) , where we take n to be whatever current sample size is being used to construct the estimate defining the greedy component. However, rather surprisingly, neither the sample variance nor the critical value used in Eqn. (3.22) appear anywhere in Eqn. (3.23). Furthermore, the uncertainty component of index (3.23) contains a factor $\sqrt{2\log(j-1)}$ not included in index (3.22). Note that while $n_{k,\,j-1}$ is the total number of actions selecting arm k immediately prior to the jth stage, $j-1$ is the total number of all actions. See, for example, Section 12.4.4 of (Powell, 2011) for further discussion of these differences.

Despite these differences, index (3.23) is usually referred to as an upper confidence bound method, and is conventionally denoted by the acronym `UCB1` where the index method is a modification of an UCB for the reward mean.

Figure 15: UCB1 policy, discussed in Sec. 3.2.2.



SUMMARY

In this unit, we are introduced to special class of sequential decision processes (SDP), referred to as bandit processes. They are important in medical research, and have more recently been identified as an important model in reinforcement learning.

In a bandit process a decision maker is present with a finite set of one-armed bandits. These are components of an SDP which yields a random reward when a decision to "pull its arm" is made. Each one-armed bandit has a characteristic reward mean μj, where j is the label of a specific arm. The decision process consists of a sequence of stages, in each of which the decision maker selects an arm, then accrues the resulting reward.

The objective is to maximize the expected total reward R, a weighted sum of the stage rewards accrued. If the reward means were known, the decision maker could simply pull the arm with the largest mean reward indefinitely. The problem arises when the means are not known. Thus, the sequence of arm selections has two goals: first, to identify the largest reward mean; second, to maximize the total reward.

These two goals conflict, because in order to identify the largest mean, each arm must be selected with a high enough frequency. This leads to the exploitation versus exploration trade-off. Any successful decision rule (or policy) for a bandit process must balance the need to explore the arms, to identify the optimal arm, and the need to maximize reward (exploitation).

Most bandit process policies are one of two forms. An action-value method combines a preference for larger rewards based on the current arm reward estimates with forced exploration (this is decision behavior intended to ensure sufficient exploration of all arms). The ϵ-greedy policy, softmax policy and gradient policy, described in this unit, fall into this category.

On the other hand, a policy for a bandit process is an index method if it calculates an index separately for each arm, then selects the arm with the highest index at any given stage. Such methods are often related to the theory of confidence intervals and confidence bounds.

We introduced one such policy, known as the `UCB1` policy.

UNIT 4

Q-LEARNING

STUDY GOALS

On completion of this unit, you will have learned ...

- How the Q-function is defined.
- How neural networks can be used in reinforcement learning.
- How to improve Q-learning with neural networks.
- How policy gradient methods work.
- How to combine value based learning and policy based learning in actor-critic approaches.

4. Q-LEARNING

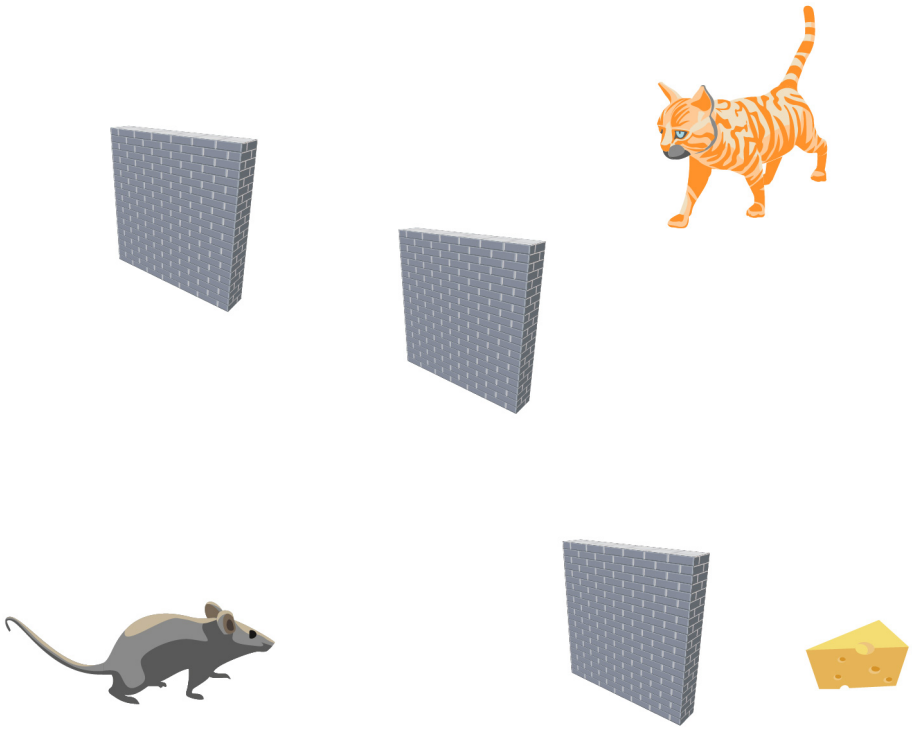
Introduction/Case study

Reinforcement learning is concerned with how to use experiences by interacting with an environment and therefore learning which actions are beneficial.

This means that we need to make a series of decisions about how to respond to our environment. We have previously encountered Sequential Decision Problems (SDP) and the formalism how to determine the best course of action. This lead us to the discussion of dynamic programming for SDP and Bellman's equation (of optimality).

Before we revise the core elements, let us build an intuitive picture of what we want to achieve. Consider a simple game between a cat and a mouse as shown in figure below:

Figure 16: Cat and Mouse Game



The mouse tries to get a piece of cheese while the cat tries to catch the mouse, but a number of obstacles are in the way of both cat and mouse. In particular:

- Both cat and mouse have eight degrees of freedom - they can move up, down, left, right, or in any diagonal. At any moment, they can take any of these actions.

- The mouse tries to avoid getting caught by the cat. If the cat catches the mouse, the game is over. A new piece of cheese appears and the position of cat and mouse remains unchanged.
- If the mouse gets the cheese by moving to the same position as the piece of cheese, the mouse scores a point.
- If the cat catches the mouse, the cat scores a point and the current game ends. The scores for cat and mouse are updated and the game starts again.

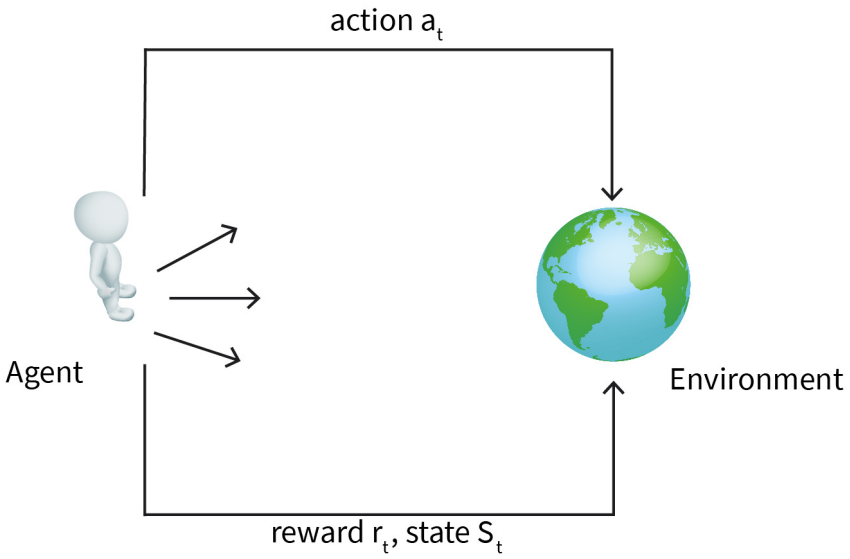
In this simple game the mouse learns how to get to the cheese without getting caught by the cat. During each term of the game, the mouse knows the layout of the game, i.e. the location of the cheese, the cat and the obstacles and based on this knowledge the mouse has to decide where to move next. During a series of games the mouse then needs to learn the best sequence of actions to take, i.e. in which direction to move at any given step. If the movements of the cat followed a distinct and fixed pattern and the mouse knew this, the mouse could determine how best to move without trying it. However, if the movements of the cat are not fully known in advance, the mouse has to gain experience by playing the game multiple times.

Sequential and Markov Decision Problems

We now formalise this description in the language of reinforcement learning and Sequential Decision Problems (SDP). Looking at figure below, an **agent** such as the mouse interacts with an environment, which, in the case of the above game, is the board on which cat, mouse and cheese are placed.

Agent
The agent is the entity that executes actions in the environment.

Figure 17: Elements of the reinforcement environment



The agent observes a **state** s_t such as the layout of the game board and the position of cat, mouse and cheese at any moment.

State
The state is a specific configuration of the environment at this moment during the interactions.

The set of all possible states is called the state space $\mathcal{S}$. In each turn of the decision problem, the agent needs to choose an action a_t and the set of all possible actions is denoted $\mathcal{A}$. Note that not all actions may be accessible from all states.

Upon carrying out an action a, the state of the environment changes from s to s' according to a transition probability $P(s' \mid s, a)$. This can include a deterministic change where the system always enters the same new state s' following action a, in this case P is a transition function. However, in general $P(s' \mid s, a)$ is a probability distribution or kernel that determines the probability that the environment is in state s' if action a has been taken in state s.

Markov property
The Markov property states that, given the present state, the future states are independent of the past states.

Markov decision problems (MDP) are a special case of sequential decision problems as they fulfill the **Markov property**. This means that the SDP is memoryless, i.e. the current state contains all relevant information and we do not need to keep a history of all previous states and actions to understand how future states develop.

In the following explanation, we will mostly assume that we focus on a Markov decision problem. Upon completing an action, the agent receives a reward r_t, where the reward is a function of the current state, the action, and the new state the environment takes once the action is taken: $r = r(s, a, s')$. Notably, the reward only depends on the current action, not on possible future actions. After action a is taken we can calculate the expected reward

$$r(s, a) = \sum_{s'} r(s, a, s') P(s' \mid s, a) \tag{4.1}$$

where we have to sum over all possible future states s' in which we can end up with probability $P(s' \mid s, a)$ after taking action a in state s. This is because the transition between states is stochastic, i.e. we can end up in different states even if we start from the same state and take the same action. Hence the process can be described as follows:

1. Start with some initial state s_0
2. For any time t starting with $t = 0$ to ∞
   a) Observe state s_t
   b) Take action a_t, the environment will change to the new state s_{t+1} with probability $P(s_{t+1} \mid s_t, a_t)$
   c) Receive reward r_t

Solving SDP and MDP

This leads to a sequence of states, actions and rewards as we interact with the environment: $s_0, a_0, r_1, s_1, a_1, r_2, \ldots$. The total (discounted) reward is then given by $R(s_0) = r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots$. This quantity is also called the "return". In finite decision problems, i.e. decision problems that run for a fixed number of steps, we can simply add all rewards we obtain at each step and the sum will always be a finite number. However, in an infinite decision problem that can run indefinitely, we need to make sure that the sum of all rewards remains a finite number. This can be achieved by introducing the discount factor γ. If $\gamma \in (0, 1)$, the sum is always finite.

We can extend this and allow $\gamma = 1$ to recover the undiscounted sum.

The agent then needs to learn a **policy** $\pi(s): \mathcal{S} \rightarrow \mathcal{A}$, which determines what to do, i.e. which possible action to take when a given state s is observed. This means that the above sequence of events is modified:

Policy
The policy is a set of rules that determine how to choose an action.

1. Start with some initial state s_0
2. For any time t starting with $t = 0$ to ∞
   a) Observe state s_t
   b) Take action according to the current policy $a_t = \pi(s_t)$, the environment will change to the new state s_{t+1} with probability $P(s_{t+1} \mid s_t, a_t)$
   c) Receive reward r_t

Hence, the total reward also now depends on the policy $R^\pi(s_0)$. Since the environment is stochastic as expressed by the transition probabilities $P(s' \mid s, a)$, $R^\pi(s)$ is generally different between different interactions with the environment. We therefore define the value function of policy π as the total expected reward:

$$V^\pi(s) = E_\pi[R^\pi(s)] \tag{4.2}$$

where $E_\pi[\,\cdot\,]$ is the expectation value over the sequence of actions executed in one run of interactions with the environment under a given policy π.

Different policies will lead to different values of the value function. The one with the highest value is called the optimal value function with the optimal policy $V^* = V^{\pi^*}(s) \geq V^\pi(s)$ as all other policies lead to the same or lower values.

Assumptions

Previously, we have discussed how to find the optimal policy using dynamic programming, which lead us to the discussion of Bellman's equation (of optimality). However, we have so far always made a very important assumption: We know how the environment works, or more specifically, we know the transition probabilities $P(s' \mid s, a)$. These transition probabilities define our knowledge of the environment and represent a model of the environment since we encode all responses to our actions in these numbers.

In this sense, “solving” the sequential decision problem is a planning or optimization problem as we know how our environment responds stochastically to our actions. We can use methods such as value or policy iteration to find the optimal policy.

We have also made another, perhaps less obvious assumption, namely that we can actually handle and perform the required calculations to find the optimal policy. In the simple case of the cat-and-mouse game, this is not too complex: The game board (the environment) has only a limited number of squares, say, a 9×9 grid. The mouse and the cat can only move in one of eight directions, i.e. the mouse can only take one of eight actions per state to interact with the environment. However, handling the resulting computations can quickly become very complex if we have many possible actions or the action space $\mathcal{A}$ becomes continuous. For example, if we want to fly a plane or helicopter or control an autonomous vehicle, the state-space $\mathcal{S}$ also becomes very large and complex: In the case of a car, we observe a given traffic situation and can take a wide range of actions such as accelerating, steering to braking. Within each of these actions, we have a broad spectrum if we e.g. apply the brakes gently, use engine- or recuperative- braking, or need to perform a hard or emergency brake with the friction based brakes.

Additionally, the objectives we have imagined so far were rather straightforward.

For example, the mouse wants to get to the cheese while also avoiding the cat. We can easily imagine more complex scenarios, for example, in order to get the cheese, the mouse first has to go through a maze, retrieve a key that, in turn, unlocks a box that contains the cheese. Going straight to the cheese will not help the mouse. Such hierarchical problems are very difficult to model using the methods we have encountered so far.

In this unit, we will focus on how we can approach sequential decision problems when we don’t have a model of the environment, i.e. the transition probabilities $P(s' \mid s, a)$ are not known and the state-action space becomes too large to handle when using the dynamic programming approaches discussed earlier.

4.1 Q-Learning and Time-difference Learning

Q-Factors

Before we start to investigate how to optimize our choice of actions without having access to the model describing the environment, we should briefly revise the dynamic programming approach for sequential decision problems.

We start with the case that we have only one step in the sequence left and only need to take one final action, so we want to determine which is the optimal action. If the environment is in state s, we receive the reward $r(s, a)$ upon executing this action. We now define the Q-factor as

$$Q_1(s,a) \equiv r(s,a) \tag{4.3}$$

The Q-factor is defined as the (total) reward we receive when taking the optimal action, i.e. following an optimal policy. Hence, the optimal policy leading to the highest reward is

$$\pi_1(s) = \arg\max_a \quad Q_1(s,a), \tag{4.4}$$

which means that we choose the action a so that it maximises the reward we receive upon executing it. We can express the same in terms of the value function

$$V_1(s) = \max_a \quad Q_1(s,a)\,. \tag{4.5}$$

Let us then consider the case when we have two steps left in the SDP.

Starting from some state s, we take action a and receive a reward $r(s,a)$.

Following the discussion above, the reward expressed in the value function for the last step is $V_1\Big(s'\Big) = \max_a \quad Q_1\Big(s',a\Big)$ if we take the optimal action a.

Since we consider the last two steps, the last step does not start from s but from the state s', which we reach through performing an action from the initial state s. In the stochastic SDP, the probability of being in state s' is given by $P(s'\,|\,s,a)$ after executing action a from state s. Hence, the reward for the last step is:

$$\sum_{s'} P(s'\,|\,s,a)V_1(s') = \sum_{s'} P(s'\,|\,s,a)\max_a Q_1(s',a)\,. \tag{4.6}$$

Putting the reward from the first step and the last step together we get the total reward:

$$\begin{aligned} Q_2(s,a) &= r(s,a) + \gamma \sum_{s'} P(s'\,|\,s,a)V_1(s') \\ &= \underbrace{r(s,a)}_{\text{last step}} + \gamma \underbrace{\sum_{s'} P(s'\,|\,s,a)\max_a Q_1(s',a)}_{\text{first step}} \end{aligned}$$

Similar to the case above, the optimal policy for taking an action is the one that maximizes the total reward $Q_s(s,a)$:

$$\pi_2(s) = \arg\max_a Q_2(s,a) \tag{4.7}$$

with the optimal value function

$$\begin{aligned} V_2(s) &= \max_a Q_2(s,a) \\ &= \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V_1(s') \right]. \end{aligned}$$

We can repeat the process and recursively add more steps to take in the analysis o f the sequential decision problem. Then, $Q_k(s,a)$ is the total reward we obtain when we start from state s, take action a, and then always choose the optimal action for the remaining $k-1$ steps. The corresponding value function is $V_k(s) = \arg\max_a Q_k(s,a)$ expressing the total reward to be gained if we act optimally for k steps starting from some state s. We can express this as the iterative formulae for Q and V:

$$Q_{k+1}(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_a Q_k(s',a) \tag{4.8}$$

and

$$V_{k+1}(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_a Q_k(s',a) \right]. \tag{4.9}$$

Irrespective of where we start, the sequence of value functions V_0, V_1, V_2will converge to the optimal value function V^* and we obtain Bellman's equation of optimality:

$$V^*(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \,|\, s,a) V^*(s') \right] . \tag{4.10}$$

We can incorporate the stochastic nature of the environment by taking the expectation value $E_\pi\big[\,\cdot\,\big]$ on the right hand side. This is the value iteration algorithm we have discussed previously when considering the dynamic programming approach to solving sequential decision problems.



STATE VALUE ITERATION

1. At $k = 0$, initialize $V_0(s)$ for some state s.
2. Repeat:
   a) $V_{k+1}(s) = \max_a \; \big[r(s,a) + \gamma \sum_{s'} P(s' \,|\, s,a) V_k(s') \big]$
   b) set $k = k + 1$
3. Stop if $\big| V_{k+1} - V_k \big| < \epsilon$ for a small number $\epsilon > 0$, i.e. if the value iteration converges

We can express a similar algorithm in terms of $Q(s,a)$



ACTION VALUE ITERATION

1. At $k = 0$, initialize $Q_0(s,a)$ for some state s.
2. Repeat:
   a) $Q_{k+1}(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \,|\, s,a) \max_a Q_k(s',a)$
   b) set $k = k + 1$
3. Stop if $\big| Q_{k+1} - Q_k \big| < \epsilon$, i.e. if the value iteration converges

$V(s)$ is also called the state-value function and describes the expected return if we start from state s and then follow a policy π. $Q(s, a)$ is also called the action-value function and describes the expected return if we start from state s, take action a, and then follow a policy π.

The value of the Q-function for a specific state-action pair (s, a) quantifies the reward we will obtain from an environment in state s when we take action a. Since we want to optimize our rewards, the Q-function helps us to find the best action a by looking for the one which leads to the highest Q-value (or reward).

Handling unknown transition probabilities

The above is the same approach that we took when solving the sequential decision problem using dynamic programming. So far, we have "known" our environment in the sense that we have a model of the environment described by the transition probabilities $P(s' \mid s, a)$.

We now consider the case where we do not have access to $P(s' \mid s, a)$, but we can interact with the environment. Hence, we do not have a model and we cannot compute an optimal state-value function $V^*(s)$ or action value function $Q^*(s, a)$, but we can execute actions in the environment and receive a reward. Hence, we observe a state, execute an action, receive a reward, observe a new state, execute another action, etc. This can be expressed as the sequence $s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, ...$ The aim of reinforcement learning is to learn from these experiences so that the choice of actions we execute becomes better and better over time, where "better" means that we receive a higher return, a cumulative reward, or reduce an associated cost. Essentially, we replace the transition probabilities $P(s' \mid s, a)$ with the experiences we gain by interacting with the environment. To develop this approach, we start again from the value iteration over the Q-function:

$$Q_{k+1}(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \max_a Q_k(s')$$

which, as we recall, is the total (expected) reward if we start from state s, take action a, and then always choose the optimal action for the remaining $k - 1$ steps. This can be implemented by using two arrays to store all Q-values, one array for Q_{k+1}, and one for Q_k. However, this would lead to very large storage requirements, in particular for complex environments with many possible states and actions.

We now modify the equation to use only one array to store all values for $Q(s, a)$. The algorithm then becomes

1. Initialize $Q(s, a)$.
2. Repeat for each pair (s,a):

$$Q(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_a Q(s',a)$$

3. Stop if convergence is reached.

The above algorithm does not make any assumptions about the order in which the state-action pair (s, a) is encountered or whether some pairs are encountered multiple times or not at all. Rather than saying that we need to visit all state-action pairs (s, a) and update the Q-function for the pair (s, a), we can also say the following: The agent interacts with the environment and if the agent encounters the state-action pair (s, a), the Q-function is updated.

This shift in perspective allows us to incorporate the experiences from the environment: Rather than calculating the Q-function by making sure we visit all state-action pairs (s, a), we let the agent interact with the environment and record the resulting experiences in a running update of the Q-function.

Note that we now have only one $Q(s, a)$ function - but we still need to model the transition probabilities $P(s' \mid s, a)$. However, as we do not know these transition probabilities, we need to remove this requirement. Using the expression for the expected reward in Eqn. (4.1), we can write the update rule for the Q-function as:

$$\begin{aligned} Q(s,a) &= r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_a Q(s',a) \\ &= \sum_{s'} P(s' \mid s,a) \left[r(s,a,s') + \gamma \max_a Q(s',a) \right]. \end{aligned}$$

Whenever we encounter the state s and take action a, we will receive some reward r and observe a new state s'. Due to the stochastic nature of our environment we end up in different states s' even if we start from the same state and take the same action, i.e. for each state-action pair (s, a) we have a number of different experiences with different rewards and new states.

We can express this as: $(s, a : r_1, s'_1), (s, a : r_2, s'_2), (s, a : r_3, s'_3), ..., (s, a : r_n, s'_n)$.

Q-learning

We now make the following approximation to the update rule:

$$Q_n(s,a) = \frac{1}{n}\sum_{i=1}^{n}\left[r_i + \gamma \max_a Q(s_i', a)\right]. \tag{4.11}$$

Essentially, we drop the transition probabilities and instead approximate the Q-function after having gained n experiences (i.e. samples from interacting with the environment) with the average of the immediate rewards and the discounted rewards following the optimal actions at each step. As the agent continues to interact with the environment, we have $n+1$ experiences or samples in the next step: $\left(s, a : r_{n+1}, s'_{n+1}\right)$ which we add to the set of experiences we have recorded so far. Hence, in the next step we have

$$Q_{n+1}(s,a) = \frac{1}{n+1}\sum_{i=1}^{n+1}\left[r_i + \gamma \max_a Q(s_i', a)\right].$$

At this point, we need the iterative update rule for the sample mean which is given by

$$\mu_m = \frac{1}{m}\sum_{i=1}^{m} x_i\,.$$

If we add one more data-point x_{m+1}, we can avoid calculating the whole sum by using an iterative approach:

$$
\begin{aligned}
\mu_{m+1} &= \frac{1}{m+1}\sum_{i=1}^{m+1} x_i \\
&= \frac{1}{m+1}\sum_{i=1}^{m} x_i + \frac{1}{m+1}x_{m+1} \\
&= \frac{m}{m+1}\mu_m + \frac{1}{m+1}x_{m+1} \\
&= \frac{m+1-1}{m+1}\mu_m + \frac{1}{m+1}x_{m+1} \\
&= \Big(1 - \frac{1}{m+1}\Big)\mu_m + \frac{1}{m+1}x_{m+1}\,.
\end{aligned}
$$

If we use the above, we can express the Q-function after we have added one more experience using the iterative update rule as:

$$
Q_{n+1}(s,a) = \Big(1 - \frac{1}{n+1}\Big)Q_n(s,a) + \frac{1}{n+1}\Big[r_{n+1} + \gamma \max_a Q_n\big(s'_{n+1}, a\big)\Big]
$$

Remember that we only have one Q function which we update iteratively and the Q-learning update rule becomes:

$$
\begin{aligned}
Q(s,a) &= (1-\alpha)Q(s,a) + \alpha\Big[r + \gamma \max_a Q(s',a)\Big] \\
&= Q(s,a) + \alpha\Big[r + \gamma \max_a Q(s',a) - Q(s,a)\Big]
\end{aligned}
\tag{4.12}
$$

where we have replaced the term $\frac{1}{n+1}$ with a small learning rate α.

This is the Q-learning algorithm developed by (C. J. C. H. Watkins, 1989; C. J. Watkins & Dayan, 1992), for further details also see (Sutton & Barto, 2018b, p. 131-132).

Q-LEARNING ALGORITHM

1. Initialize Q(s,a) arbitrarily.
2. Start at some initial state s
3. Repeat until reaching a terminal state:
   a) Choose an action a for the current state s.
   b) Take the action a and observe the reward r received from the environment and the new state s' the environment is in after the action is taken.
   c) Update the Q-values according to Eqn. (4.12):

$$Q(s,a) = Q(s,a) + \alpha\Big[r + \gamma \max_a Q(s',a) - Q(s,a)\Big]$$

d) Set the current state to the new state, i.e. $s \leftarrow s'$

The terminal state is a state that ends traversing or exploring of the Markov decision problem, i.e. once we enter a terminal state we cannot leave it.

Episode
An episode is the sequence of actions and states from the first, initial state to the terminal state

Examples are "game-over" or the character in a video-game "dies". To continue learning, we need to start a new **episode**, i.e. a new initial state once we have encountered a terminal state.

The Q-learning algorithm contains all aspects we need to learn how to interact with an unknown environment: Starting from a clean slate (initializing the $Q(s,a)$ function), we perform actions and observe rewards and new states. For each state-action pair (s,a) we encounter, we update the Q-function. As we learn, we get better and better at interacting with the environment. This means that new experiences are used to improve the $Q(s,a)$-function which then quantifies which is the best action to take, i.e. which action will lead to the highest reward given the current state s.

Hence we choose the action which results in the highest value of $Q(s,a)$in a given state s. A proof of the convergence of Q-learning can be found in (Melo, 2001).

In practical applications, $Q(s,a)$ can be implemented as a (very large) table or matrix where state s and action a serve as indices to access the specific elements $Q(s,a)$. However, if we only choose the action a that leads to the highest reward or Q-value from the ones we have observed so far, we will not know if there is another action that will lead to an even higher reward. It is therefore advisable to try out some actions randomly to gain more experience. This is called the exploration-exploitation dilemma and we will discuss this later in more detail.

Time-Difference Learning and SARSA

The Q-learning algorithm is an example of a time-difference learning approach.

To understand this in a bit more detail, let us look at what incorporating one experience into Eqn. (4.12) looks like. When we go from step k to $k+1$ by adding one experience, the equation is given by

$$Q_{n+1}(s,a) = Q_n(s,a) + \alpha\Big[r + \gamma \max_a Q_n(s',a) - Q_n(s,a)\Big] \tag{4.13}$$

where we added the indices n and $n+1$ for clarity. Looking back at Eqn. (4.8), the part $r + \gamma \max_a Q_n\Big(s',a\Big)$ is the contribution to the Q-function if we add one experience. If we define

$$\Delta = r + \gamma \max_a Q_n(s',a) - Q_n(s,a), \tag{4.14}$$

we can express the update rule for the Q-function as

$$Q_{n+1}(s,a) = Q_n(s,a) + \alpha\Delta \tag{4.15}$$

where the learning rate α is typically small as we don't want to risk "ruining" our learned Q-values with one "bad" experience. Essentially, we iteratively update our knowledge of the Q-function by adding one "experience" at a time and the change in the Q-function is proportional to the Q-value between the previous and the current step. This leads to the name "time-difference learning" as we update our knowledge of the environment using subsequent interactions. Instead of taking one step, we could consider taking several steps at a time and we can generally label these kind of algorithms as $TD(\lambda)$.

In Q-learning, we have immediately updated the Q-value and used a policy to choose an action a derived from a policy based on this changing Q-value.

This is not the only choice we can make. The SARSA algorithm (Sutton & Barto, 2018b, p. 129 ff.) differs from the Q-learning algorithm such that the next action is not chosen from the (implicit) policy that is derived from the updated Q-function where we always seek to choose the action that results in the highest immediate reward but from the same policy used to choose the first action. This means that we don't use the maximum reward for the next state as in Q-learning but we obtain a reward following an action chosen according to the current policy. This leads to the sequence (s, a, r, s', a') which also gives the following algorithm it's name.

The algorithm can be summarized as:



SARSA ALGORITHM

1. Initialize $Q(s,a)$ arbitrarily.
2. Start at some initial state s
3. Repeat until reaching a terminal state:
   a) Choose an action a for the current state s (following a specific policy).
   b) Take the action a and observe the reward r received from the environment and the new state s' the environment is in after the action is taken.
   c) Choose an action a' for the new state s' (following the same policy).
   d) Update the Q-values according to Eqn. (4.12):

   $$Q(s,a) = Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$$

   e) Set the current state to the new state, i.e. $s \leftarrow s'$ and the action to the current action, i.e. $a \leftarrow a'$.

The SARSA algorithm is called "on-policy" because the action for the next state s' is chosen from the same policy as for the original state s, whereas Q-learning is called "off-policy" since we do not use the reward from the current policy when updating the values.

4.2 Reinforcement Learning with Neural Networks and Deep Q-Learning

The Q-learning or SARSA algorithm is everything we need in order to learn how to explore and navigate in an unknown environment and to learn from experiences so that the actions we choose become better with time. We do not need to know the transition probabilities that model the environment but we can iteratively build a $Q(s, a)$ function that allows us to evaluate specific choices for actions a in a given state s.

However, even though we have everything in place, using Q-learning for anything but a simple tasks such as the cat-and-mouse game or navigating a small grid world will not work in practice. The main reason is that we have to build the $Q(s, a)$ function explicitly. For example, we could implement $Q(s, a)$ as a two-dimensional array or a table in a database.

However, this will quickly become very difficult to handle if we have many states and/or actions. For example, if we think of flying a plane, driving a car, or playing a complex video game such as shown in figure below, building such a table or array becomes infeasible and computationally too expensive:

Figure 18: Videogame on a gaming console



The state-action space becomes too big as the number of states becomes bigger and bigger and, additionally, the actions are no longer discrete and finite as in the case of a small grid where we can only move up, down, left, right, diagonally, and maybe interact with

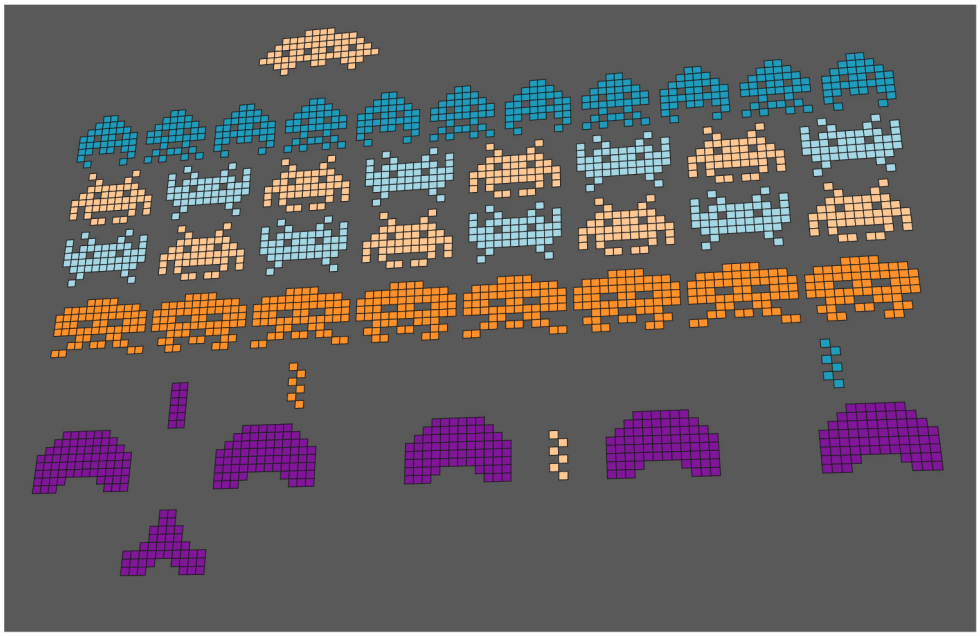
some other figures. Instead, the action space is continuous or quasi-continuous, i.e. there are so many actions that differ only a little to one another that they can be regarded as continuous. As an example, consider braking a car: There are near infinite choices of how hard to brake, whether to rely on regenerative or engine braking alone, use friction braking, etc. Additionally, many of the entries in the array or table will remain at the initialized value (e.g. zero if we initialize all values to zero to start with) as it is very unlikely to encounter a specific state-action pair (s, a) if either or both the number of states and actions is large.

We can take the following approach to apply Q-learning to more complex environments: Instead of building a big array or table, we approximate the Q-function using some method that depends on relatively few parameters θ. Hence, instead of $Q(s, a)$, we now have$Q(s, a; \theta)$ and by using some functional form or approximation we can estimate $Q(s, a)$.

Traditionally, the approximate function $Q(s, a; \theta)$ was built using a linear model (see e.g. (Sutton & Barto, 2018b, p. 204 ff.)) and the "features" or variables of the linear model were typically hand-crafted and designed by domain experts. For example, in the cat-and-mouse game, features could be the distance between cat and mouse or mouse and cheese, the number of obstacles in the way, etc.

In 2013, researchers from DeepMind demonstrated in video games such as Pong, Breakout, Space Invaders, Seaquest, or BeamRider by Atari, that deep neural networks perform much better than traditional methods in learning the approximate $Q(s, a; \theta)$-function (Mnih et al., 2015b, 2013).

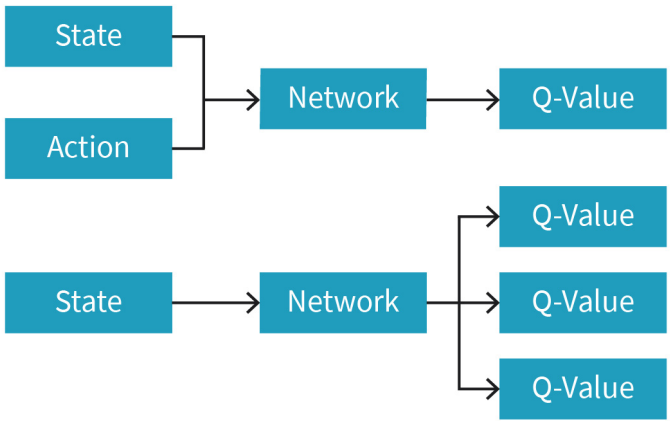
Figure 19: Atari video game: Space Invaders



Compared to real-life situations, self-driving cars, or others, these video games are still comparatively simple, but this work demonstrated for the first time that deep neural networks can be used effectively to approximate the $Q(s, a; \theta)$-function, outperforming other approaches. In this work, the network takes the raw pixel image as input, processes it using a series of convolutional neural network layers to learn the content of the image and then determine a Q-value for each possible action.

Considering the tabular approach for modelling $Q(s, a)$ earlier, the simplest approach is to replace the table with a (deep) neural network that takes a state s and action a as input and computes an approximate Q-value as output as shown in the left part of figure below. However, this again requires that we can cover all state-action pairs sufficiently well to create many training samples for the network. It is therefore more efficient in practical applications to train a network that takes a state s as input and computes an output for each possible action a (right part of figure below).

Figure 20: Schematic setup of Deep Q-Networks: The network learns the mapping of state-action pairs (s, a) to a Q-value (left) or computes a Q-value for all possible actions (right).



The network learns the mapping of state-action pairs (s, a)
to a Q-value (top) or computes a Q-value for all possible actions(bottom).

The key aspect of training a deep neural network used to approximate the Q-value function is to define the loss function that the network should optimize. In particular, we need to define a training target. Looking back at Eqn. (4.15), we saw that we could write the Q-value update rule as $Q_{k+1}(s, a) = Q_k(s, a) + \alpha\Delta$, where Δ is defined as $\Delta = r + \gamma \max_a Q_k(s', a) - Q_k(s, a)$, which we have already encountered in the discussion about timedifference learning. This means that the Q-value converges if the immediate reward from the environment plus the future discounted rewards is close or equal to the estimate of this quantity expressed by the Q-value. Remember that the Q-value quantifies how "good" an action is, i.e. if we know the Q-value function sufficiently well, we can determine the best action a leading to the highest reward by looking at all possible actions a and take the optimal one, i.e. the one which leads to the highest Q-value or reward.

Hence, we define the **training target** of the deep Q-network as

$$y = r + \gamma \max_a Q(s', a; \theta) \ . \tag{4.16}$$

Training target
The training target in supervised learning is the label of the associated feature vector.

We can approximate the future rewards for the next state s' by applying the neural network for all possible actions a and get an approximate Q-value for each of the possible actions (Geron, 2019, p. 633 ff.). As this is essentially a supervised regression task, we can take the mean squared error between the the target and $Q(s, a; \theta)$ as the loss function

$$\mathcal{L} \sim (y - Q(s, a; \theta))^2 \tag{4.17}$$

and train the neural network using gradient descent.

Again, we have everything at our disposal to explore new environments and derive optimal actions. Compared to the tabular approach where $Q(s, a)$ is stored in an array or table, using deep neural networks allows the navigation of complex environments with many states or a continuous action space. However, using this "vanilla" approach often does not give satisfactory results and the deep reinforcement learning neural network becomes unstable or "forgets" what it has learned so far: As the agent explores one part of the environment, the experiences they gain and the resulting policy updates based on the updated approximate $Q(s, a; \theta)$-function may "break" what the network has learned in other parts of the environment.

In the following section, we will discuss a few approaches to improve the Q-learning approach based on neural networks we have seen above.

4.3 Experience Replay

In Q-learning, an agent explores the environment by executing actions and observing the resulting rewards and change in the environment. More formally we can say that the agent observes a state s and uses the deep Q-network to evaluate which action a to take next. Upon performing the action, the agent receives a reward r and observes a new state s'. This sequence is referred to as the experience tuple (s, a, r, s').

Looking back at Eqn. (4.15), we saw that the Q-values are only updated using the last experience tuple or interaction with the environment. However, training neural networks, especially deep neural networks, requires a large amount of training data. We can create such a training sample over time by storing all of the experience tuples that we have encountered so far and instead of training the network on just the experience tuple gained from the last interaction with the environment, we randomly sample from this dataset.

This approach is called experience replay (McClelland, McNaughton, & O'Reilly, 1995; O'Neill, Pleydell-Bouverie, Dupret, & Csicsvari, 2010; Lin, 1993; Mnih et al., 2015b). The more the agent interacts with the environment, the larger the training sample becomes as more experience tuples are stored.

Taking a random sample from this replay buffer or replay memory at a time when processing the training data is a crucial detail. To understand its importance we consider in the figure entitled as "Videogame on a gaming console" again: In complex video-games or reallife applications, subsequent experience tuples may be highly correlated.

For example, pressing a button on the game controller may result in a slight change of posture of one of the players. Comparing the images associated with the subsequent states s and s' after taking, for example, the action “lower right foot by tiny amount” results in nearly identical images.

Hence, those two states and the associated experience tuples are highly correlated.

(I.I.D.)
i.i.d. means identically and independently distributed

However, when training neural networks we typically assume that the training data-samples are independent from each other and taken from the same generating function **(i.i.d.)**. Taking the experience tuples at random from the replay buffer before feeding them to the deep Q-network during the training process reduces the correlation between subsequent training samples seen by the network considerably, making the training process more stable.

In this approach all experiences have the same “value”: We sample uniformly from the replay memory regardless of whether or not an experience was deemed “important”. Also, experiences will be added to the replay buffer at the same rate that they occur. However, in many scenarios, some experiences may be more “valuable” than others. For example, some experiences may be rare compared to other experiences or more difficult to obtain. Furthermore, some experiences may not be helpful at the early stages of exploring the environment and training the Q-network but may be important at later stages. To address these issues, Schaul et al. (Schaul, Quan, Antonoglou, & Silver, 2015) suggest using prioritized replay instead of plain experience replay. The core idea is to favour those experiences with a high expected learning progress, i.e. those experiences from which an agent (or the Q-network) can learn the most. This can be approximated by the TD learning error given by Eqn. (4.14), which is the difference between the reward obtained by taking the best (correct) action and the one we take based on the current state of the Q-network. Essentially, if we predict the correct reward, we have already exploited all knowledge from these experience tuples, whereas if the prediction is quite far from what we observe when we perform an action, we can still learn from interacting with the environment. Using $p_i = \Delta_i + \epsilon$, the probability of sampling transition i is defined as

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \tag{4.18}$$

where ϵ is a small positive constant to prevent numerical issues if the error is small or zero, and α determines the strength of the prioritization scheme.

4.4 Double Q-Learning

Unfortunately, even with a sufficiently large replay buffer added to the training procedure, a reinforcement learning approach based on deep neural networks as discussed so far is often not stable enough to handle complex situations. We can likely solve simple systems such as the "cart-pole" exercise where we try to balance a stick on a moving cart and make it stay upright - but not more complex video games such as Space Invaders or even more complex ones.

The main reason is that that we use the deep Q-network twice in the Q-learning algorithm. Going back to Eqn. (4.16), the training target is based on the output of the Q-network, meaning that the predictions of the neural network are used in the target definition. This is a major difference to supervised learning where the targets of the labelled data are all known in advance and are independent of the learning approach. This dependency on the learning algorithm, in this case the weights of the neural network, can create a feedback loop and the network can become unstable. Mnih et al. (Mnih et al., 2013, 2015b) avoid this issue by using a copy of the deep Q-network to define the targets. Hence, there are now two versions of the same network: One network, called the online network is used by the agent to explore the environment and the other one, called the target network, is used to define the training target. In order to avoid the feedback loop, the target network is updated periodically every few iterations by copying the weights from the online network to the target network. The original Q-learning algorithm is then recovered as the special case where the weights are copied every iteration. Since the target network is updated only every few iterations, it is less prone to rapid changes in the approximated Q-value function as the agent explores different regions of the environment.

In (Mnih et al., 2015b), the target network was updated every 10,000 iterations of gaining an experience tuple.

A further approach to tackle this issue is called "Double Q-learning" (Hasselt, 2010; Van Hasselt, Guez, & Silver, 2016). This method aims to improve a specific issue in both tabular and deep Q-learning: The "max" operator in the Q-learning update rule (see Eqn. (4.12)) uses the same Q-table or the same deep Q-network both to select and evaluate an action and we choose the one that leads to the highest estimated Q-value. However, since the Q-value is an estimation, we have to accept that there are some fluctuations - the Q-value is not derived from the full known sequential decision problem but from the online learning based on experiences. Hence, even if two or more actions would have the same Q-value if we had access to the optimal solution of a fully known sequential decision problem, the estimates for this Q-value will be slightly different in general. As we always choose the one with the highest Q-value, in this case the highest estimate from either a Q-value table or neural network, we may introduce a bias.

To implement Double Q-learning in neural networks, Hasselt et al. build on the work by Mnih et al. (Mnih et al., 2015b). Specifically, they also use an online and a target network where, again, the target network is a copy of the online network and the parameters are updated every few iterations.

In the original implementation by Mnih et al. , the target network is used to select the next best action and to evaluate the Q-value, i.e. the quality of the action. In order to avoid introducing another deep Q-network, the authors suggest using the online network instead of the target network to select the best action for the next state and then using the target network to compute the Q-values for the best action once selected. The new target then becomes (Van Hasselt et al., 2016)

$$y_t^{\text{Double DQN}} \equiv r_{t+1} + \gamma \underbrace{Q\left(\mathrm{s_{t+1}}, \underbrace{\underset{\mathrm{a}}{\operatorname{argmax}}\ \mathrm{Q}\left(\mathrm{s_{t+1}}, \mathrm{a}; \theta_\mathrm{t}\right)}_{\text{online network}}, \theta_\mathrm{t}^{-}\right)}_{\text{target network}} \tag{4.19}$$

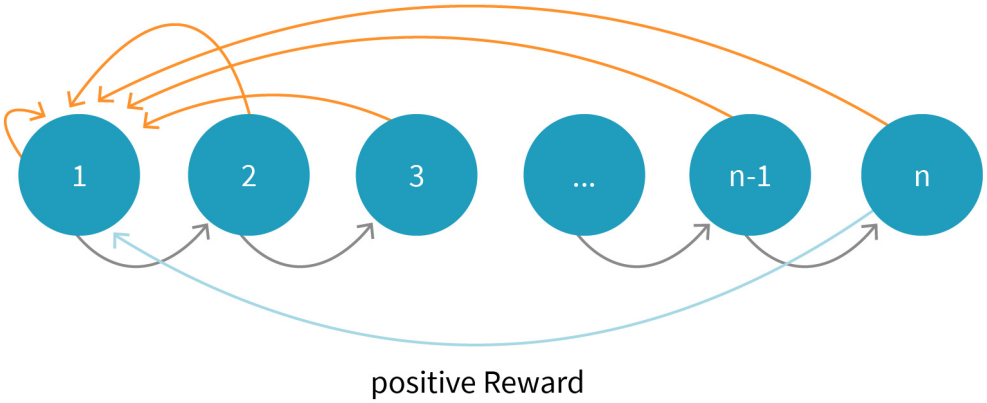
where θ_t are the parameters of the online network and θ_t^- are the parameters of the target network.

4.5 Delayed Sparse Rewards and Hierarchical Learning

In our discussions about reinforcement learning so far we have, for the most part, implicitly assumed that we can learn relatively easily which actions are beneficial or not. In the simple cat-and-mouse game, the mouse only has to go to a few paces to get close to the cheese (or get caught by the cat), steering, accelerating, or braking has an immediate effect, and in the simple cart-pole exercise, we can find out quickly if the pole falls over.

However, many other scenarios are different: It may take a long time to get a positive reward or a large number of steps must be taken before a positive reward can be obtained. These scenarios are more difficult since it is hard to keep on exploring if we don't know how long it will take until we get a positive reward. One of these examples, the blind cliff walk shown in figure below was indeed one of the core motivations behind introducing prioritized experience replay (Schaul, Quan, et al., 2015).

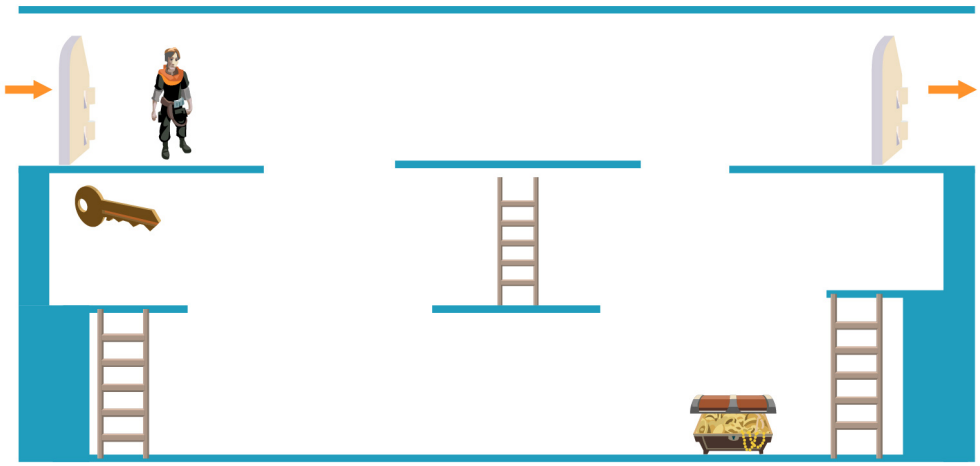
Figure 21: Blind Cliff Walk



The agent starts at the first position and the goal is to reach the end of the chain at position n. In the blind cliff walk, the agent only gets a reward if they take the "right" decision, move towards the end of the chain, and actually reach the end of the chain. In all other cases where the agent takes the "wrong" decision, they fall off the cliff, the episode ends, and it is restarted by placing the agent at the first step again. Essentially, the agent has to learn to only move forward, never backward, without knowing how long the sequence is and when a reward will be obtained.

In other situations, the environment will be highly complex and there may be several objectives to learn from the environment which may or may not be linked to an overall objective. In their work, Kulkarni et al. (Kulkarni, Narasimhan, Saeedi, & Tenenbaum, 2016) use the example of the videogame "Montezuma's Revenge" by Atari which is similar to the setup shown in figure below: A player enters a level and needs to explore the level and pick up, for example, a key to open the door to the next level. They may or may not pick up some treasures on their way - this does not necessarily improve their chances to complete the level but counts towards the overall reward.

Figure 22: Layout of a complex games with multiple sub-goals



Gaining knowledge about the environment can then be understood as reaching individual goals within the exploration process. In the standard sequential decision problems or reinforcement learning, the effective knowledge about a state s is captured by the value function $V(s)$ which is essentially a metric that measures how "good" a given state is for the overall objective of the sequential decision problem.

Sutton et al. (Sutton, Modayil, Degris, Pilarski, & White, 2011) propose to extend this to a general value function and introduce the Horde architecture where the reinforcement learning agent consists of multiple sub-agents called demons that explore the environment. Each demon addresses a single goal-oriented question about the environment and hence contributes to the overall knowledge of the environment. The corresponding general value function $V_g\big(s\big)$ is the value function for a state s while trying to achieve goal g. A set of these value functions then represents different aspects of the knowledge that we have about the environment, for example, how to reach the key or collect some treasure. Schaul et al. (Schaul, Horgan, Gregor, & Silver, 2015) extend this with a universal value function $V(s, g; \theta)$ where a single goal oriented value function is used for all possible goals and states and hence for all sub-agents pursuing them. This complex function can, for example,. be approximated using neural networks.

However, learning various goals while exploring the environment may not be sufficient to act in that environment as the goals may depend on each other. Coming back to the example of the video game shown in figure above, the agent has to learn that there is a key and how to retrieve it before finding the door leading to the next level which needs to be unlocked with the key retrieved previously. Some treasure may be picked up on the way (or not). Hence, the goals are not independent or indeed of the same value: Collecting treasure will improve the overall score - but not help to advance to the next level. Without getting the key first, finding the door to the next level does not help. Kulkarni et al. (Kulkarni et al., 2016) propose the framework "hierarchical deep Q-learning (h-DQN)" to incorporate hierarchies of value functions. The key idea is to introduce a top-level controller called a "meta-controller" that analyzes the state s and chooses new goals g, and a low-level module called controller that learns how to choose optimal actions a so that a given goal g is reached. After completing a goal, the meta-controller chooses another goal and the process is repeated with the new goal. This way, hierarchical goals, or goals that become relevant at different times of the exploration, can be reached. In the case of the game shown in figure above, the meta-controller would first focus on getting the key, then using the key to open the door to the next level.

4.6 Value- vs. Policy- Based Learning

In the discussions so far we were always guided by improving or optimizing the reward we receive from the environment by observing a state and deriving how much this state is "worth". This is the value function $V^{\pi}(s) = E_{\pi}[R^{\pi}(s)]$ from Eqn. (4.2). We also introduced the Q-value which relates the combination of observing a state s and taking action a with the expected reward from the environment and we would choose the action a so that it maximizes the value function or equivalently the Q-values. Hence, our choice of action

was always determined by taking the most "valuable" choice with the highest Q-value. This means we have an implicit policy that determines how an agent chooses an action for a state s. Our previous policies can be expressed as

$$\pi(s) = \arg\max_a Q(s, a) \tag{4.20}$$

as we have already seen in Eqn. (4.4). This is also called a **greedy policy**.

Greedy policy
A greedy policy optimizes the next immediate step and chooses the action that leads to the highest immediate reward.

However, if there are many actions or the action space is continuous, we cannot explore all possible actions.

We will now discuss a different approach where we focus on the policy itself. The policy determines how we choose our actions in a given state and "maps" a state to an action. We can differentiate between a deterministic and a stochastic policy. In a deterministic policy, we always perform the same action a in state s, i.e. $a = \pi(s)$. In a stochastic policy, we have a probability that determines which of the available actions will be taken, i.e. $\pi(a, s) = P(A_t = a, S_t = s)$. The main objective of reinforcement learning is to optimize the total reward of all our actions in the future given a state s. This implies that we don't optimize the next action to take (although we can do this in greedy policies), but focus on the longterm goal, meaning that we want to optimize over all possible actions and states. We can express the **objective function** as

Objective function
The objective function is the term we wish to optimize, e.g. a cost to minimize.

$$J(\theta) = E_{\pi_\theta}\left[\sum_k \gamma^k r_k(s, a)\right] \tag{4.21}$$

where θ are the parameters of the policy we can change to optimize our actions and $\gamma \in (0, 1]$ is a discount factor. Remember that we generally need a discount factor in infinite decision problems whereas finite decision problems do not necessarily have to include a discount factor since the sum of rewards will be finite anyway. However, we can capture both cases by using a discount factor and allowing $\gamma = 1$. The quantity

$$v_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}(s, a) \tag{4.22}$$

is also called the cumulative return for the future trajectory of the problem, starting from the current state t and projecting over all future states denoted by index k. Instead of v_t, the return is often denoted by R_t as well in the literature.

We can approximate the expectation value by taking the **sample mean**:

Sample mean
The sample mean is given by $\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$

$$J(\theta) = \frac{1}{n} \sum_{i} \sum_{k} \gamma^{k} r_{t+k,i}(s,a)$$

(4.23)

where the index i runs over all samples we have obtained by interacting with the environment following policy π_θ. If the environment is continuous, we replace the expectation value $E[\,\cdot\,]$ with the average value over a distribution function $d_{\pi_\theta}\big(s\big)$ over the states

$$J(\theta) = \sum_{s} d_{\pi_\theta}\big(s\big) V_{\pi_\theta}\big(s\big)$$

(4.24)

where $V_{\pi_\theta}\big(s\big)$ is the value function. In order to achieve our objective, we have to find the policy π_θ according to which we choose the actions that will optimize the overall reward.This means that, in order to maximize the objective function Eqn. (4.21), we must find the set of parameters θ on which our policy depends. This implies that policy based reinforcement learning is an optimization problem. The optimal parameters θ^* are those that maximise the objective function

$$\theta^* = \arg\max_a \underbrace{E_{\pi_\theta}\left[\sum_k \gamma^k r_{t+k}(s,a)\right]}_{J(\theta)} \tag{4.25}$$

and we can determine the best parameters. Again, we iteratively determine the best parameters using the gradient. The corresponding update rule is

$$\theta_{\pi'} = \theta_\pi + \alpha \nabla_\theta J(\theta) \tag{4.26}$$

where α is a small learning rate. This is called gradient ascent instead of descent (where we would use a minus sign). According to the policy gradient theorem (see (Sutton & Barto, 2018b, p. 325) for a proof), the gradient is given by

$$\nabla_\theta J(\theta) = E_{\pi_\theta}\Big[\nabla_\theta \log \pi_\theta(s,a) Q_{\pi_\theta}(s,a)\Big] . \tag{4.27}$$

As in Eqn. (4.23), we can approximate the expectation value by sampling from the policy to choose the agent's actions and interact with the environment

$$\nabla_\theta J(\theta) \approx \frac{1}{n} \sum_{i=0}^{N} \Big[\nabla_\theta \log \pi_\theta(s,a) Q_{\pi_\theta}(s,a)\Big] \tag{4.28}$$

where i once again runs over all samples N that we have obtained from interacting with the environment and we have used the sample mean to approximate the expectation value from a finite sample.

Note that, unlike Q-learning which is an off-policy learning approach, policy gradient based learning is an on-policy approach. This means that the samples are obtained using the current policy. Consequently, as soon as we change the policy, we need to discard all old samples as they belong to a different policy and we have to start sampling again from the beginning. The policy gradient approach is similar to policy iteration of sequential decision problems encountered earlier where all transition probabilities defining the model are known.

Using the actual return value v_t (sum of discounted rewards) as an estimate for the Q-value, we can turn the above into the "REINFORCE" algorithm (Sutton & Barto, 2018b, p. 328) and (Williams, 1992).



REINFORCE ALGORITHM

1. Initialize the parameters θ arbitrarily.
2. Repeat:
   a) Sample the environment for a given policy π_θ for N steps. This leads to a sequence of states, actions and rewards that depend on the policy π_θ: $\{s_0, a_0, r_1, ..., s_{N-1}, a_{N-1}, r_N\}$
   b) For each step $i \in [0, N]$: update parameters according to: $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(s_i, a_i) v_i$
   c) Return parameter set θ and choose new policy

If we compare policy and value function based approaches, we generally have:

- In value function based learning, we learn the Q-values by interacting with the environment (e.g. using a deep Q-network) and use an implicit policy $\pi(s) = \arg\max_a Q(s, a)$ taking the action that has the highest associated value.
- In policy based learning, we optimize the overall reward by changing the parameters of a policy function using gradient ascent (Eqn. (4.26)).

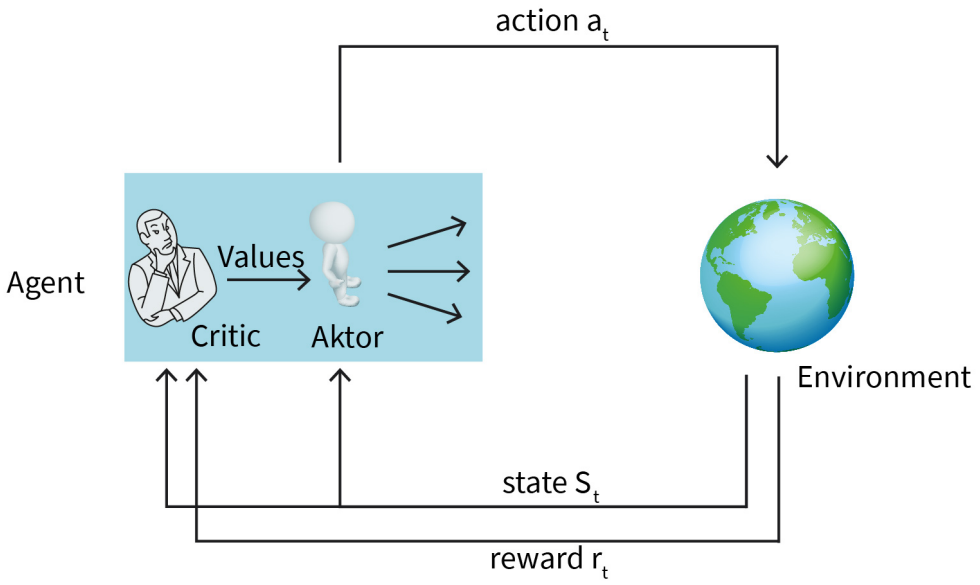
4.7 Actor Critic Learning

So far, we have discussed how to handle complex environments using neural networks by either using a value function or by looking at the policies that govern the choice of actions. In the value-function based approach we saw how we can use deep neural networks to learn about the environment and use this to determine the next action to take. This lead us to the development of deep Q-networks for off-policy learning and SARSA for onpolicy

learning, as well as improvements such as (prioritized) experience replay, double Q-learning, etc. We also investigated how we can use an explicit policy and learn the best choice of actions directly. This lead us to the discussion about policy-gradient based methods.

We now build a joint approach by combining neural networks in order to estimate the return from the environment with policy gradients. The general idea is to have two networks focusing on different aspects as illustrated in figure below: An actor starts with a given policy and then learns to improve by incorporating the "advice" of a critic. The critic focuses on evaluating the policy. The approach is reminiscent of working with a coach: An athlete works to improve their technique, while the coach gives useful tips which help the athlete to improve.

Figure 23: Actor-Critic Setup



The overall approach of Actor-Critic learning is very similar to policy iteration, also called the actor-critic method in sequential or Markov decision problems. However, in SDP we knew the setup of the problem, in particular, the transition probabilities $P(s' \mid s, a)$ and the rewards we would obtain from the environment. This allows the policy to be fully evaluated. Now, we do not know the model of the environment, i.e. we do not know the transition probabilities or the rewards and need to learn about the environment by gaining experience and "acting out" a policy choice and then improving on it. This also implies that the the next choice of policy may be (substantially) worse than the current one as we cannot fully evaluate the policies. The three approaches can be summarized in the following table:

Table 3

RL aspect	Action/Value learning	Policy learning	Actor - critic
estimate rewards	learn $V(s)$ or $Q(s,a)$	evaluate $\sum_i \gamma^i r_i(s,a)$	learn $V(s)$ or $Q(s,a)$
improve policy	$\pi(s) = \arg\max_a Q(s,a)$	$\theta_{\pi'} = \theta_\pi + \alpha \nabla_\theta J(\theta)$	$\theta_{\pi'} = \theta_\pi + \alpha \nabla_\theta J(\theta)$

We now combine two approaches: learning an action-value function, and a policy gradient. Hence, we need to maintain two sets of parameters via two neural networks, one for the critic and one for the actor. They take over the following roles:

- Critic: Learn the action-value function Q, parameters: ω.
- Actor: Update policy in the direction suggested by critic, parameters: θ

According to the policy gradient theorem (see Eqn. (4.27)), the policy gradient $\nabla_\theta J(\theta)$ depends on the $Q_{\pi_\theta}\Big(s,a\Big)$ values for a specific policy π_θ.

This assumes that we could obtain the future (discounted) rewards from the environment by acting according to the policy. Now that we do not have access to this function, we use a deep Q-learning network to estimate it, this is the role of the critic. Hence, we approximate

$$Q_\omega\Big(s,a\Big) \approx Q_{\pi_\theta}\Big(s,a\Big)\ . \tag{4.29}$$

This means that we perform an approximate policy gradient ascent as we don't have access to the full policy gradient. The policy gradient theorem then becomes

$$\nabla_\theta J\Big(\theta\Big) \approx E_{\pi_\theta}\big[\nabla_\theta \log \pi_\theta(s,a) Q_\omega(s,a)\big] \tag{4.30}$$

and the update rule for the policy gradient (see Eqn. 4.26) becomes

$$\theta' \approx \theta + \alpha \nabla_\theta \log \pi_\theta(s,a) Q_\omega(s,a)\ . \tag{4.31}$$

The general actor-critic algorithm can then be described as (see (Sutton & Barto, 2018b, p. 332)):

BASIC ACTOR-CRITIC ALGORITHM

1. Initialize parameters ω, θ. Start from some state s.
2. Perform initial action a according to some policy π_θ.
3. Repeat:
   a) Obtain reward r following action a and observe state s'.

b) Choose a new action a' following the current policy $\pi_\theta(s', a')$.
c) Evaluate the update rule for the approximate policy gradient according to Eqn. (4.31):

$$\Delta\theta = \alpha \nabla_\theta \log \pi_\theta(s, a) Q_\omega(s, a)$$

and update the neural network for the policy gradient using the learn-rate α
d) Evaluate the temporal difference error

$$\delta = r(s, a) + \gamma Q_\omega(s', a') - Q_\omega(s, a)$$

and update the neural network learning the Q-values

$$\Delta\omega = \beta \quad \delta \quad \nabla_\omega Q_\omega(s, a)$$

using the learn-rate β. Note that this step is on policy as the action as chosen according to the current policy π_θ (SARSA), rather than the one that maximises the Q-value (off-policy in Q-learning). Therefore, there is no "max" operator here.
e) set the current state to the new state, i.e. $s \leftarrow s'$ and the action to the current action, i.e. $a \leftarrow a'$.

Essentially, the actor-critic algorithm combines the approaches from SARSA and REINFORCE where separate neural networks are used to learn the value of the actions, the Q-value, and the approximate policy gradient. Note that the learn rates α and β of the two networks are not identical. As it would be quite inefficient to choose just one action according to the current policy and then update both networks, the process is often parallelized where many agents interact with the environment. In a synchronized setup, many agents interact with the environment and obtain experience tuples (s, a, s', r) and the approximate policy network is updated when a number of these tuples have been obtained. This setup is synchronous because all agents act according to the same policy and the policy is updated for all agents at the same time. Mnih et al. (Mnih et al., 2016) extend this to asynchronous agents where the change of policy parameters θ affects the agents at different times, for example, if some actions take longer to execute than others. This allows for more flexibility since the agents do not have to wait until all agents have completed their actions before the next update of the parameters of the policy network. To increase the sample efficiency, Wang et al. (Z. Wang et al., 2016) introduce experience replay into actor-critic setups in a similar spirit as experience replay reduced correlations between experiences and allowed a more effective use of the experience tuples in deep Q-learning.

One of the challenges of the basic actor-critic algorithm is that the gradients we need to compute may show a large variance: The different paths that agents use to explore the environment and gain experience tuples may vary drastically from one update to the next. One way of reducing the variance is to introduce the advantage function which is defined as

$$A(s,a) = Q(s,a) - V(s)\ . \tag{4.32}$$

We can think of the advantage in the following way: The Q-value can be decomposed into the value of the state $V(s)$ which determines how "good" it is to be in this state and the advantage is an indication of how good an action a is compared to other actions in this specific state s. Intuitively, we can compare this to a group of students in a class: the state function would represent the average performance of students in this class, e.g. their average grade. The advantage would then allow us to compare how much better (or worse) the performance of a student is compared to the overall performance in this class. Hence, learning the advantage instead of the Q-values shifts the focus from learning how "good" an action is towards how much better or worse it is compared to what we would achieve with an "average" action. Schulman et al. extend this concept and introduce the general advantage estimation for continuous control problems (Schulman, Moritz, Levine, Jordan, & Abbeel, 2015). In order to avoid having two networks in Eqn. (4.32) to learn the state-action value function $Q(s,a)$ and the state value function $V(s)$, we can use just one network trained on $V(s)$: We start from the time-difference error expressed in the value function (Sutton & Barto, 2018b, p. 120) given by

$$\delta = r(s,a) + \gamma V(s') - V(s) \tag{4.33}$$

and recall that we can use Eqn. (4.8) to express the Q-value as

$$Q(s,a) = r(s,a) + \gamma V(s')$$

where we have also used that the value of the state is given by taking the action that maximises Q (see Eqn. (4.6)). Hence, we can write the advantage as

$$A(s,a) = r(s,a) + \gamma V(s') - V(s)\,, \tag{4.34}$$

which means that the time-difference error is an estimator of the advantage function. Mnih et al. (Mnih et al., 2016) propose an asynchronous update mechanism called A3C (Asynchronous Advantage Actor Critic), the synchronous variant is called A2C (OpenAI, 2017). Haarnoja et al. (Haarnoja, Zhou, Abbeel, & Levine, 2018) propose the "Soft Actor Critic" which maximises the expected reward but also simultaneously maximises the entropy.

This approach allows a more full and efficient exploration of the environment as the agent behaves as randomly as possible, including actions that would not be easily taken by other approaches. The "Proximal Policy Optimization" algorithm (Schulman, Wolski, Dhariwal, Radford, & Klimov, 2017) further stabilizes the policy gradient estimation by introducing a surrogate function which is optimized instead of the "raw" gradient discussed above. For example, a "clipped surrogate objective" is used where the range of the objective function to be optimized is constrained. This avoids large policy updates which may even go in different directions in subsequent updates and favors smaller changes instead. Furthermore, the "adaptive Kulback-Leibler Penalty Coefficient" uses the Kulback-Leiber divergence between the old and new policy as a regularization scheme to avoid that the new proposed policy after the gradient update is too far away from the old one.



SUMMARY

The field of reinforcement learning has seen many significant advances recently since it was discovered how neural networks can be used effectively in this field. Previously, reinforcement learning was often limited by the complexity of many systems which cannot be tackled effectively with tabular approaches. This unit focuses on how to use neural networks in reinforcement learning putting emphasis on deep Q networks where the state-action function is learned by a deep neural network, as well as various improvements that make more efficient use of available data or improve the learning process. Policy gradient methods allow to learn a policy according to which agents choose their actions directly, allowing in particular the optimization for the overall expected reward in a more long-term perspective.

However, this means that we can determine the rewards over a long sequence of actions. Actor-critic approaches combine learning the state-value or action-value function from experiences with policy optimization using policy gradients. This mechanism can intuitively be compared to working with a coach or teacher who gives feedback on the current choice of actions.

UNIT 5

REINFORCEMENT LEARNING APPROACHES

STUDY GOALS

On completion of this unit, you will have learned ...

- the key concepts of model-free reinforcement learning.
- the ideas behind model - based reinforcement learning.
- the exploration-exploitation dilemma.
- how to approach the exploration-exploitation dilemma in both modelfree and model-based reinforcement learning.

5. REINFORCEMENT LEARNING APPROACHES

Introduction/Case study

Decision making in complex environments is a very challenging task. In order to optimize the cumulative expected rewards, also known as the return, we have to choose our actions appropriately.

Markov decision problem
A MDP is a decision problem that fulfill the Markov property: Given the present state, the future states are independent of the past states.

When discussing sequential or **Markov decision problems** (MDP), we assumed that we had full knowledge of the specifics of the problem, in particular, the transition probabilities $P(s' \mid s, a)$, which allow us to determine how to get from one state s to the next state s' by executing action a, as well as the expected reward $r(s, a)$ we obtain from performing the action.

If these are fully known, we can use, at least in principle, techniques such as dynamic programming to "solve" the sequential or Markov decision problem, leading to Bellman's equation of optimality. Techniques such as value or policy iteration can be used to determine how to act optimally in a given decision problem.

This approach comes to its limits if the number of states and/or actions is very large or continuous. Furthermore, in many realistic scenarios, we have no knowledge of the the transition probabilities or the reward. The field of reinforcement learning is primarily concerned with how to tackle these situations, where an agent needs to explore the environment and learn from its actions in order to act better in the future. Fundamentally, we can approach this in two different ways: We can either try to learn the parameters of the environment by interacting with it, namely the transition probabilities and rewards. Once we have these, we can apply the same methods as if we knew the full sequential or Markov decision problem.

This is known as "model-based learning". Or we can learn how to choose "good" actions directly, without learning a model. This is called "modelfree learning". Neither option is better than the other per se - either have their merits and a number of approaches have been suggested for each.

Which approach to follow also depends on the problem at hand.

In either case, since we do not know the environment, agents need to perform actions and observe the new state s' as well as the reward $r(s, a)$.

Since we also want to optimize the overall cumulative reward, executing unknown actions is risky in the sense that it might not be optimal and lead to sub-par rewards. However, if we only stick to the actions we have already performed and are known to us, we might miss out on actions that are even better than what we have experienced so far. This is called the "exploration-exploitation dilemma".

5.1 Model-Free Learning

Model-free approaches are used when we face sequential or Markov decision problems where we do not know the parameters of the decision problems, most importantly, when we do not know the transition probabilities $P(s' \mid s, a)$ that bring us from one state to the next via some action, as well as the rewards we would obtain. The model-free approach can be further distinguished into model-free prediction where we estimate the value function of the unknown decision problem and model-free control where we optimize the value function.

The general idea of model-free reinforcement learning is to learn the solution, i.e. how to choose the best action, directly without having to build a model of the environment first. Since we don't know the environment, the only way we can learn about it is via experiences: We observe a state s, choose an action a, and observe which reward r we obtain and in which new state s' we end up. This sequence is also called the "experience tuple" (s, a, r, s').

A relatively straight forward way to learn from the environment is called the **Monte Carlo** Evaluation, see (Sutton & Barto, 2018b, p. 92). In the beginning, we choose a policy π, according to which we choose our action a, and initialize a state value function V arbitrarily as well as an empty list that holds all the returns, i.e. the cumulative discounted reward from the interaction with the environment. Remember that the value of a state is the expectation value of the return that can be approximated by using the sample mean.

Monte Carlo
The Monte Carlo method obtains its name from the town Monte Carlo with the famous casinos, alluding to the random approach used in the method.



MONTE CARLO POLICY EVALUATION (EVERY VISIT)

1. Initialize:
   - Policy π to evaluate
   - Arbitrary state-value function V
   - List of returns for all states $Returns(s)$
2. Repeat:
   a) Use policy π to generate an episode
   $(s_0, a_0, r_1, s_1, a_1, r_2, ..., s_k - 1, a_k - 1, r_k)$
   b) Loop over all states s in the episode. For each state s
   i. For each occurrence of state s in the generated episode
   A. Retrieve return r_i for this occurrence of s.
   B. Append return r_i to list $Returns(s)$
   C. Compute the value function: $V\Big(s\Big) = \frac{1}{N}\sum_j Returns\Big(s\Big)_j$.

The Monte-Carlo method allows the evaluation of a policy π from the experiences directly. We do not need to determine the transition probabilities and we observe the rewards following our actions but we do not need to know the function or logic behind them. Hence the Monte-Carlo approach is a model-free learning algorithm since we do not need to

know the structure of the underlying decision problem. This approach also uses the simplest possible approach to estimating the value function by taking the average over the observed returns. However, this approach requires that we generate entire **episodes** by interacting with the environment. This makes the Monte-Carlo approach very time-consuming since we need to create long sequences for each policy π we wish to evaluate. Furthermore, it requires that the episodes terminate, i.e. that there is a state at which the system ends, such as "game over". Depending on the situation this may not be the case - or it may take a long time to reach this terminal state.

Episodes
An episode is the sequence of interactions with the environment from some initial state to the end.

In temporal difference (TD) learning, we also learn directly by interacting with the environment without knowing the underlying transition probabilities or reward mechanisms. Like the Monte-Carlo approach, TD learning is also model free. However, unlike the Monte-Carlo approach, we do not generate complete episodes but can learn online and we can improve the agent's behaviour after each interaction with the environment. Prominent approaches in temporal difference learning are Q-learning and SARSA. The general idea behind Q-learning is to start from an arbitrary action-value function $Q(s, a)$ in some initial state s_0. Then, while interacting with the environment, we choose an action a so that it maximizes the action-value function $Q(s, a)$ for that state, i.e. we take the action that results in the highest reward based on our current knowledge. Next, we observe the new state and the actual reward we obtain and then update the Q-values iteratively according to

$$Q(s,a) + \alpha\Big[r + \gamma \max_a Q(s',a) - Q(s,a)\Big]$$

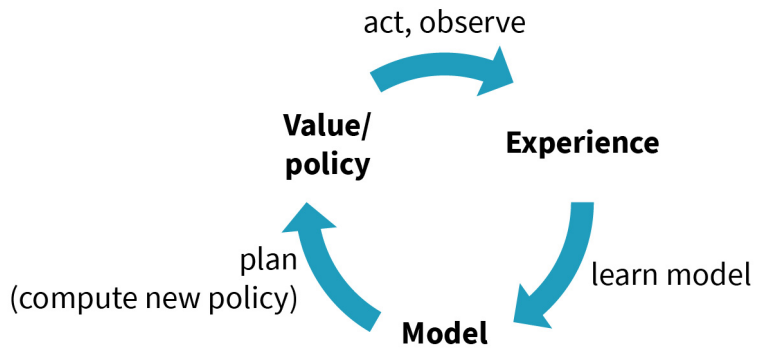
where α is a small learning rate. The Q-learning algorithm was originally developed by Watkins (C. J. C. H. Watkins, 1989; C. J. Watkins & Dayan, 1992), for further details also see (Sutton & Barto, 2018b, p. 131-132).

The main difference between Q-learning and SARSA is that Q-learning is an "off-policy" algorithm, whereas SARSA is an "on-policy" algorithm, where the actions are chosen according to a specific policy. In Q-learning, we greedily choose the action that maximises the current knowledge of the action-value function $Q(s, a)$ for a given state.

5.2 Model-Based Learning

The general intuition behind model-based reinforcement learning is that we use the experience from interacting with the environment to build a model of the environment and then use this model to employ methods such as dynamic programming to devise the best policy to guide our actions. The overall idea is shown in figure below where we iteratively interact with the environment to build a better model.

Figure 24: Model-based reinforcement learning



The model is then described by the state-**space** $\mathcal{S}$, the action-space $\mathcal{A}$, the transition probabilities $P(s' \mid s, a)$ and the rewards R. Unlike the case where these functions are all known *a priori*, they depend now on some parameter θ that reflects our current knowledge of the environment. Assuming that we know the state space and the action space, our model $\mathcal{M}_\theta$ consists of the estimates $P_\theta(s' \mid s, a)$ and R_θ for the transition probabilities and rewards. As in the case of a fully known problem, $P_\theta(s' \mid s, a)$ determines how to get from the current state s to the new state s' via some action a where the reward is determined by R_θ. However, unlike the fully known decision problem, we need to determine or learn these model parameters from episodes of experience tuples $(s_0, a_0, r_1, s_1, ..., s_k)$. The simplest way to implement this is to create a lookup table where we record all experience tuples $(s_t, a_t, r_{t+1}, s_{t+1})$ and then refer to this table when we encounter the corresponding state-action pair (s, a). The more experience tuples we add to building the model, the better the estimates become and the better the (approximate) dynamic programming or planning step will be. For example, the underlying model can then be solved with policy or value iteration - keeping in mind that the model is not fully known, i.e. we use approximate dynamic programming.

Space
A space consists of mathematical objects with a given relationship or structure.

Unsurprisingly, the simple approach with a look-up table is not very efficient in complex or continuous environments. One approach that can be used to learn the model more effectively is the Dyna algorithm (Sutton, 1990). The general idea is to start from a given state and an initial model of the decision problem. Then, choose some action, observe the resulting reward and new state and use this to update the model. Repeat the following n times: take a known action from a known state, use this to update the model based on reward and observed new state, and improve the state-value or state-action function. The overall process is then repeated indefinitely. The intuition behind this approach is that if it is unfeasible to visit all state-action pairs infinitely often to derive an optimal policy and accurate state-value or action-value function, each new experience improves the previous knowledge.

Ball et al. follow the general idea of the Dyna algorithm, but approach model-based reinforcement learning from the view of an active learner in their “Ready Policy One” (Ball, Parker-Holder, Pacchiano, Choromanski, & Roberts, 2020) algorithm. The core idea is that they seek to build an optimal **world model** since obtaining samples from the “real-world”

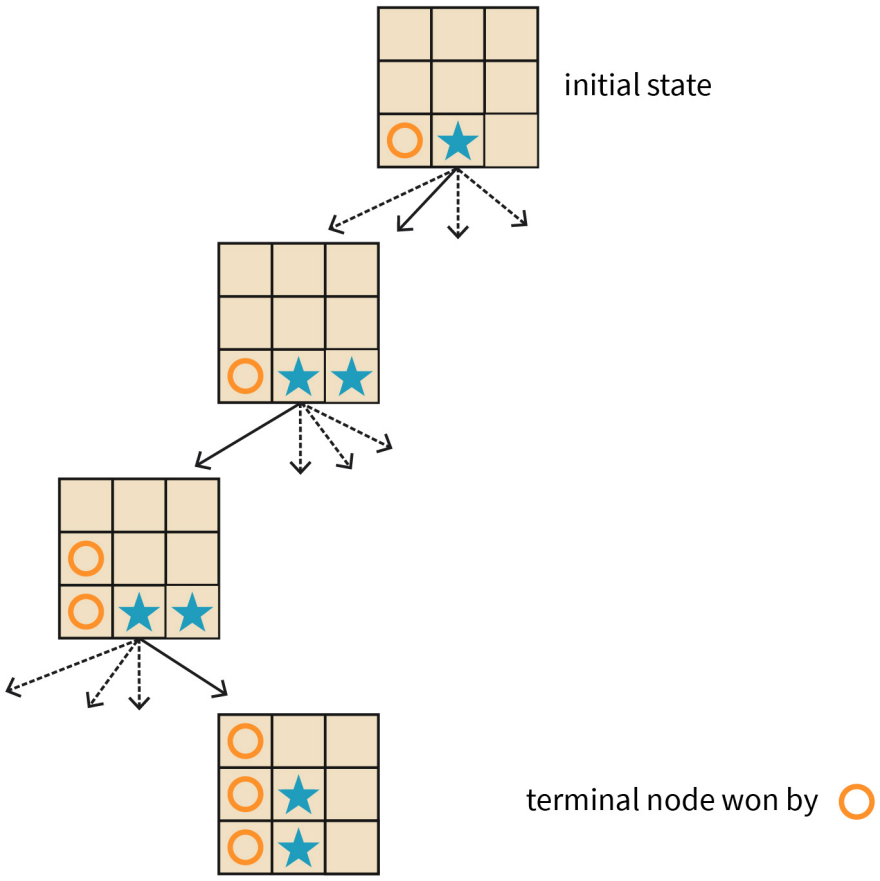
World model
The world model is a model-based description of the environment the agent acts in.

is expensive, but optimizing the agents behavior using the world model is much "cheaper". Hence, finding an optimal policy using the world-model is computationally much easier to do than interacting with the real environment.

The key point is then to add data from the environment using the Ready Policy One algorithm that is disagrees with the predictions from the world-model in regions where the disagreement is relevant to improving the policy governing the choice of actions by the agent.

In other cases, we can build a perfect model of the environment where we can determine how the state of the environment will change given a particular action. However, in some environments, it may take a long time until we get a reward, i.e. we need to follow a long "trajectory" of actions before we can decide whether the original action was promising or not. At least in principle, we can take an action using a relatively simple policy and then keep going until we hit a terminal state. For example, in a game, we can make a move, observe the new state and then take a "bruteforce" approach of making further moves until one player wins. Then we can go backward and judge whether that original move was promising or not. This is illustrated in figure below for the game of Tic-Tac-Toe. Since we do not know the opponent's moves, at each stage of the game, we play out all possible further moves to judge which is the most promising one.

Figure 25: Game Tree in Tic-Tac-Toe



Once we reach the terminal state and one player wins, we propagate the result back up and decide which is the most promising move. However, apart from small environments, this becomes quickly intractable as the number of (game) search trees we would have to follow becomes very large very quickly. As an example we can consider games such as chess or, in particular, Go (Shotwell, 1994). In Go, two players aim to claim as much territory of the game board as possible. Placing a stone, we know the new state precisely. The rules are relatively simple, however given the size of the playing board, it is estimated that there are approximately 10^{170} positions (Tromp & Farnebäck, 2006). These complex scenarios can be tackled with "Monte Carlo Tree Search" (Coulom, 2006). Essentially, instead of performing a full tree search, we only look at those parts that are most promising. The basic algorithm can be divided into the following four steps that are repeated until some constraint is met. For example, some computational resource may be exhausted or a time-limit in the game play has elapsed.

1. Selection: We start from the current (game) state and traverse the search tree until we reach a node that is not yet fully explored, meaning that below this node, not all possible further moves have been investigated. The way we move from node to node depends on some metric such as the number of times this state has been visited or the expected cumulative reward for this state.
2. Expansion: We now expand the tree and add one or more child nodes to the node we are currently in, where each new node represents a new (game) state.
3. Simulation: For each of the added child nodes, the further behavior of the environment is simulated. In the case of a game, we now simulate the game play either until we reach a terminal state (win/lose) or until some other criterion is met such as the elapsed time or maximum depth. This new part of the tree is now evaluated.
4. Backpropagation: The result of the previous simulation step is then propagated back up the tree so that the metrics involved in selecting the node in the selection steps are updated.

The success of AlphaGo (Silver et al., 2016) reaching and exceeding human level performance in the game of Go is based on a combination of Monte Carlo Tree Search and reinforcement learning approaches based on deep neural networks for both value and policy evaluation.

The performance of model-based reinforcement learning is ultimately based on the accuracy of the model determined from the environment. Since the policy of the agent is derived from the model, the accuracy directly affects the quality or the effectiveness of the policy and hence, the agents actions. Janner et al. propose a framework called "Model Based Policy Optimization" (MBPO) (Janner, Fu, Zhang, & Levine, 2019) to address this question. The intuition behind their approach is that using a model to predict long **trajectories** from a given state is very difficult, because the small errors each model contains (since it is not an exact representation of the environment) compound or aggregate over time. This means that the model becomes more unreliable the farther we look into the future.

Trajectories
Trajectories are the sequence of actions from the current state onwards.

However, in order to derive an optimal policy, we need to look far into the future. The authors approach this by starting from a random state previously seen when the agent interacted with the environment, and then using an ensemble of models to generate short trajectories of future states based on the agent's actions. Essentially, instead of using the model to look far into the future and generate a long trajectory from an initial state, MBPO uses many short trajectories probabilistically from many arbitrary states and builds up the optimal policy for a given task from small "snippets" of sequences of states.

One challenge in researching model-based reinforcement learning approaches is to compare new proposals with existing algorithms and approaches.

In order to make new research comparable, Wang et al. propose a benchmark (T. Wang et al., 2019) and provide 18 environments that are particularly suited for model-based algorithms.

5.3 Exploration vs. Exploitation

Executing actions to optimize the expected (long-term) reward is at the core of reinforcement learning problems. Choosing suitable actions is particularly challenging when we do not have a model of the environment, i.e. if the transition probabilities are not known. In this case, the only way to learn about the environment is to gain experience summarized by the experience tuples (s, a, r, s') where we observe a state, perform an action, receive a reward and observe a new state. However, the key question remains: How do we choose the action. If we know the action-value function $Q(s, a)$ for a given combination of state and action, we can determine which action is best, for example, by choosing the action that maximizes $Q(s, a)$. However, if we have never encountered such a combination so far, it is very difficult to estimate what the value of the Q-function would be.

We might be tempted to stick with the combinations we know and have encountered so far - but then we do not know if some other action might not be much better than the ones that we have encountered so far. On the other hand, unknown actions may have a disastrous impact on our (expected) overall performance. This is known as the "exploration vs. exploitation dilemma": Should we stick with our known but maybe mediocre results or risk something new? In an intuitive way this is similar to the famous Feynman restaurant problem (Gottlieb, 2017) which aims to optimize the total rating of eating m meals at a restaurant with n items on the menu. While the restaurant problem can be solved analytically, the corresponding reinforcement problem is more complex: Suppose we are at a restaurant and we do not know the number of items on the menu nor how many times we are going to eat there over the course of an "episode".

How can we determine the best way to get a good meal? On one hand, we can try a few and then stick with our favourite meal, however, this might be quite boring after a few nights out and there could be another meal we might like even more.

More formally, if we choose the action that always gives the highest reward with the current knowledge about the environment, we call this approach "greedy (Sutton & Barto, 2018b, p. 26). However, this way we do not explore any new actions but only exploit the knowledge we have gained so far. The simplest approach to incorporate exploration is to introduce a small probability ϵ which is used to determine whether we should take the currently known best action (greedy) or take a new action with unknown result. This approach is called ϵ-greedy. The easiest way is to choose the new action randomly from the available action space $a \in \mathcal{A}$. We can then summarize the ϵ-greedy policy as

$$a_t = \begin{cases} a_t^* & : \quad \text{with} \quad \text{probability} \quad 1 - \epsilon \\ a_t & : \quad \text{random} \quad \text{action} \quad \text{with} \quad \text{probability} \quad \epsilon \end{cases} \tag{5.1}$$

where a_t^* is the current optimal action leading to the highest immediate reward or (cumulative) return. We should allow a larger value of ϵ in the beginning as we start to explore the environment and then slowly decrease its value as we gain more knowledge about the environment.

Another common approach is to choose the action according to a probability defined by the Boltzmann equation:

$$P(a) = \frac{e^{Q(s,a)/T}}{\sum_{a \in \mathcal{A}} e^{Q(s,a)/T}} \tag{5.2}$$

where T is a free parameter that need to be tuned during the training process. In physical systems, it is associated with the temperature, here it controls the rate at which new actions are explored. Compared to the ϵ-greedy strategy this approach takes the current knowledge of the stateaction function $Q(s,a)$ into account, i.e. how "good" an action is. In the limit of high "temperatures", i.e. $T \to \infty$, the actual value of $Q(s,a)$ plays a small role and the probability of choosing an action approaches a uniform distribution with $P(a) \propto \frac{1}{|\mathcal{A}|}$. As the temperature is lowered, the approach becomes more greedy, favouring the ones with highest $Q(s,a)$ values. Cesa-Bianchi et al. (Cesa-Bianchi, Gentile, Lugosi, & Neu, 2017) discuss the theoretical background of this approach in more depth.

$|\mathcal{A}|$ denotes the number of items in $\mathcal{A}$.

"Thompson sampling" (Thompson, 1933, 1935) originates in bandit problems but can also be used in finite-horizon Markov decision problems, see e.g. (Osband, Russo, & Van Roy, 2013). In this approach the actions are chosen according to the posterior distribution in a Bayesian approach. Intuitively we can understand this as constructing a probability distribution describing the "success" of an action and use this to define which action to choose. In a multi-armed bandit problem where we want to maximize e.g. the reward obtained from playing at multiple slot machines, we start by assuming that all machines are equal and we assume a uniform distribution for obtaining the rewards. As we start to play, we can modify the probability distribution based on the observations of which machines yield which reward. These updated probability distributions can then be used to guide our further play. After we have played long enough, each bandit or slot-machine will be associated with a probability distribution describing its success rate and the reward we can obtain from it. An in-depth discussion can be found in (Russo, Roy, Kazerouni, & Osband, 2017). Osband et al. (Osband, Blundell, Pritzel, & Roy, 2016) build on the idea of Thompson sampling. One of the main challenges of Thompson sampling is that the posterior, i.e. the success rate, needs to be calculated. While this is possible in small problems such as a multi-armed bandit, it is intractable in complex scenarios. Instead, the authors use deep Q-networks but instead of calculating one $Q(s,a)$-estimate, they use bootstrapping to build a sample or distribution of Q-values. Hence, these k different $Q_{k=1,...,N}$ values provide an estimate of the uncertainty of the Q-value. During learning, randomly choose one of the Q_k estimates and base the action on this via $a_t = \max_a Q_k(s_t, a)$

Stadie et al. (Stadie, Levine, & Abbeel, 2015) investigate the effectiveness of ϵ-greedy, Boltzmann and Thompson sampling and propose an exploration "bonus" to further improve the exploration scheme.

The above discussion implicitly assumes that we obtain a reward from the environment which allows us to judge the quality of our action. However, in many cases, the rewards are sparse. For example, imagine a video game where the agent has to explore the environment until it finds a secret key or treasure before they can move on to a new level or explore further. In such a scenario, the agent will receive no reward for almost all actions

the agent can take unless they find the key. These situations are very difficult to model as the agent does not really gain any experience by "wandering" around and cannot determine how to explore further or how the exploration guides the understanding and learning of the environment. "Curiosity" proposed by Schmidhuber (Schmidhuber, 1991) and more recently by Pathak et al. (Pathak, Agrawal, Efros, & Darrell, 2017) is an approach to address this situation by introducing an intrinsic reward. Hence, the total reward of an agent is comprised of an intrinsic reward r^i and an external reward r^e, where the external reward is mostly zero. The general idea behind the intrinsic reward is based on how difficult it is for an agent to predict the next state s' given the current state s and a given action a. The larger the difference between predicted next state and actual next state, the more the agent can learn. One crucial insight in (Pathak et al., 2017) is to focus on those changes in the state that can be influenced by the action and ignore all changes in the states outside the control of the agent. The authors use the analogy of leaves rustling in the wind - as the agent explores the environment, all the leaves change their position and orientation due to the wind. Hence the new state may be very different from the predicted state since all leaves in the environment are different - but this has, in general, very little to do with the actions of the agent or how it should behave in the environment. Fundamentally, this is due to the fact that some parts of complex and realistic environments cannot be explored by the agent and change randomly due to external influences. Basing an intrinsic reward on these changes distracts the agent from what it can learn.

The above approaches focused on model-free reinforcement learning. If we choose a model-based approach, the following algorithms tackle the exploration-exploitation dilemma: The "Explicit Explore or Exploit" (E^3) (Kearns & Singh, 2002) algorithm approaches this by making an explicit distinction between exploration and exploitation phases when learning about the environment. Specifically, the algorithm separates the environment into a known and an unknown Markov decision problem, dividing the states into known and unknown states. Known states are those which have been visited sufficiently often so that we can estimate the associated transition probabilities and expected rewards "accurately enough". Once we have visited an unknown state a given number of times, it is moved to the set of known states. In this way, we can construct a Markov decision problem with known transition probabilities and rewards and devise an optimal policy. The outline of the algorithm is as follows:

1. There are no known states to begin with. Start at some arbitrary state s_0.
2. Take an action to reach a new state.
3. Repeat:
   a) If the state is not a known state, take an action a which has been taken least often in this state so far.
   b) If the state is a known state, compute the optimal policy maximising the expected return. If this return is close to the optimal one, execute the policy over T steps. Otherwise, choose a policy that brings us back to an unknown state as quickly as possible.

Hence, the general idea is to obtain the optimal reward from the known decision problem in the exploitation stage. Otherwise, go back to the unknown states and continue to learn and improve the model of the Markov decision problem.

Like the E^3 algorithm, the "R-max" algorithm (Brafman & Tennenholtz, 2002) also builds a model of the Markov decision problem as the agent explores the environment. The intuition behind this approach is as follows: Again we separate the MDP into known and unknown states. If we encounter an unknown state-action pair, we set a fixed reward $\widehat{R}_{\max}$. If a state-action combination has been visited sufficiently often, it becomes "known" and the actual reward will be used. Essentially, the agent builds a MDP with known rewards and transition probabilities while exploring the environment, using $\widehat{R}_{\max}$ as a proxy until the actual values are known.

Then the optimal policy can be computed from the known MDP:

$$R(s,a) = \begin{cases} \widehat{R}\big(s,a\big) & : \quad \text{for} \quad \text{known} \quad \big(s,a\big) \\ \widehat{R}_{\max} & : \quad \text{for} \quad \text{unknown} \quad \big(s,a\big) \end{cases}. \tag{5.3}$$

The Dyna algorithm (Sutton, 1990) also builds a model of the environment using experiences from the interaction of the agent and environment to update the model. However, the algorithm per se does not specify how to choose an action. We can, as Sutton suggests, use Q-learning and ϵ- greedily choose the action maximising the current $Q(s,a)$ value. However, particularly in the early stages of the model building, we have very little knowledge about the environment and hence of the model parameters.

"Prioritized sweeping" (Moore & Atkeson, 1993b) and "generalized prioritized sweeping" (Andre, Friedman, & Parr, 1998) address the question of which state-action combination should be chosen. We could choose this randomly but this would be inefficient. Instead, the choice is based on the difference $|r + \gamma \max_a Q(s',a) - Q(s,a)|$ (i.e. the absolute value of the temporal difference error). If this exceeds some threshold, we can learn more from this state-action pair than from others and we should prioritize sampling from these states to learn the most from interacting with the environment.



SUMMARY

Reinforcement learning algorithms can be broadly categorized as either model-free or model-based. In model-free approaches, we seek to optimize the choice of actions directly, based on observations of and interactions with the (real) environment. Model-based approaches, on the other hand, first seek to build a model from the interaction with the environment and then optimize a policy that guides the choice of actions. One reason to build a model is that experiences from the environment are difficult to obtain or computationally expensive whereas optimizing a policy based on a model is much easier in comparison. In either case, the reinforcement learning agent has to choose between using known actions to optimize the (long-term) reward, which is the overall goal of reinforcement learning, or explore the environment to potentially find more beneficial action sequences.

This is known as the exploitation-exploration dilemma and a number of algorithms exist to approach this.

BACKMATTER

A - Introduction/Case study

The python code in file `IUBH-RL-Unit02-Example01.py` implements the backward induction algorithm for the optimal treasure seeking problem It is a stand-alone single source file. This largely consists of repeating the following iteration steps for $w = 1, 2, ...$, where w is the number of days remaining in the search (including the day on which an action is to be selected).

Recall that $V^*(0, s) = 0$, that is, no reward is accrued when no search days remain. Also, $s = 1$ if the treasure has been found, so an action need only be considered if $s = 0$, and we set $V^*(w, 1) = 0$ for all $w \geq 0$. The Q-value is then

$$V\big(w, 0, a\big) = 13000 \times \big(1 - e^{-a/13}\big) - 700 \times a + V^*\big(w - 1, 0\big) \times e^{-a/13},$$

and the optimal value function is then

$$V^*(w, 0, a) = \max_{a \in \{0, 1, ..., 5\}} V(w, 0, a)\,.$$

1.1 A Python Implementation

In `IUBH-RL-Unit02-Example01.py` the value function $V(x)$ is represented by a one-dimensional array `valueFunction`:

Code

```
### This will be the value function V(x).
### We will allocate
### memory dynamically.

valueFunction = array('l')
valueFunction = []
```

The policy function $\phi(x)$ is represented by a one-dimensional array `policyFunction`:

Code

```
### This will be the policy function phi(x).
### We will allocate
### memory dynamically.

policyFunction = array('i')
policyFunction = []
```

It is not necessary to keep in memory $V(x, a)$ for each state-action pair $(x, a) \in \mathcal{K}$. We may therefore represent $V(x, a)$ as a single one-dimensional vector of length six, reusing this memory for each new state we consider.

This is represented by object `valueActionFunction`:

Code

```
### This will be V(x,a). We only required a fixed
### amount of memory.

valueActionFunction = array('l')
valueActionFunction = [0,1,2,3,4,5]
```

The recursion is performed within a fixed-length loop, allowing 500 iterations:

Code

```
### Specify the number of iterations (days) for our
### recursive calculation.

nDays = 500

### The outer loop controls the recursive iterations.

for day in range(nDays):
```

The Q-values are calculated in a loop which enumerates the actions $a \in \{0, 1, ..., 5\}$. This is done using the following code:

Code

```
### The inner loop ranges over the action-space
### to calculate V(x,a) for the current state.
### Here, action is the number of hours to rent
### the point (range(6) is a list of the integers
### ( 0,1,2,3,4,5).

for action in range(6):

### The heart of the dynamic programming algorithm
### lies in this  single line. All other code
### supports this line by
### (1) creating or importing the necessary objects;
### (2) controlling execution of the line;
### (3) storing the output of this line;
### (4) displaying the output of this line.
```

```
    valueActionFunction[action] =  \
    13000*(1-math.exp(-action/13)) \
    - 700*action + valueTemp*math.exp(-action/13)
```

Then, the maximum value of $V(w, 0, a)$ is appended to the value function stored in `valueFunction`, and the maximizing a is appended to the policy function stored in `policyFunction`.

Code

```
### Update the current value function V(x) = max V(x,a).

valueTemp = max(valueActionFunction)

### Append the current value of V(x) to the
### value function object valueFunction.

valueFunction.append(valueTemp)

### Capture the current optimal action, then append
### it to the policy
### function object policyFunction.

policyFunction.append(
    valueActionFunction.index(max(valueActionFunction))
    )
```

1.2 A Output

To display the dynamic programming solution, the optimal value function $V^*(w, 0)$ (stored in `valueFunction`) and the the optimal policy function $\phi^*(x)$ (stored in `policyFunction`) are plotted against w, the number of search days remaining. Standard `python` plotting functions are used.

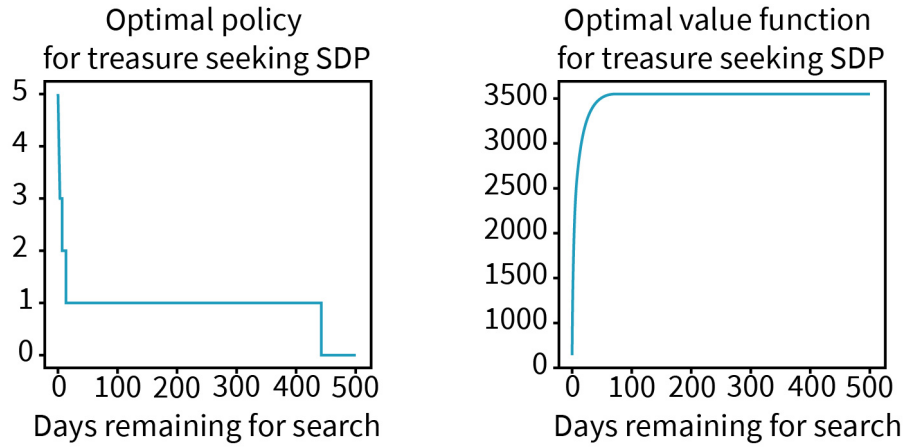
Code

```
### Plot the policy function against the number of
### remaining search days.

plt.plot(range(1,nDays+1),policyFunction,linewidth=2)
plt.xlabel('Days Remaining for Search')
plt.ylabel('Optimal Number of Hours to Rent Boat')
plt.title('Optimal Policy for Treasure Seeking SDP')
plt.show()

### Plot the value function against the number of
### remaining search days.
```

```
plt.plot(range(1,nDays+1),valueFunction,linewidth=2)
plt.xlabel('Days Remaining for Search')
plt.ylabel('Optimal Expected Reward')
plt.title('Optimal Value Function for Treasure \
Seeking SDP')
plt.show()
```



B - Introduction/Case study

The implementation is in the file `IUBH-RL-Unit02-Example02.py`.

One of the most compelling applications of artificial intelligence, and of dynamic programming and reinforcement learning in particular, is the development of game playing algorithms. An especially impressive (and relatively early) case study is TD-Gammon (Tesauro, 1994, 1995), which is able to play backgammon competitively in international tournaments (TD refers to temporal difference learning).

Subsequent success stories are reported with the advent of deep learning, as increasing computer capacity has expanded the range of this type of application to, for one example, video games (Mnih et al., 2015a). To gain some insight into this type of application, we will consider what is, at least by current standards, a relatively simple single-player game.

The Towers of Hanoi is a well-known puzzle, invented in 1883 by the French mathematician Édouard Lucas. The game platform consists of three fixed vertical pegs, which are numbered, and N round disks. The disks have holes allowing them to be stacked directly onto the pegs. The disks vary in diameter, and may be labeled 1 to N from smallest to largest. Figure below shows a schematic diagram of the platform for $N = 7$.

2.1 B Rules for The Towers of Hanoi

To begin the game, the disks are stacked on peg 1 in increasing order of size. The object is to move all disks from peg 1 to peg 3 using a sequence of moves according to the following rules.

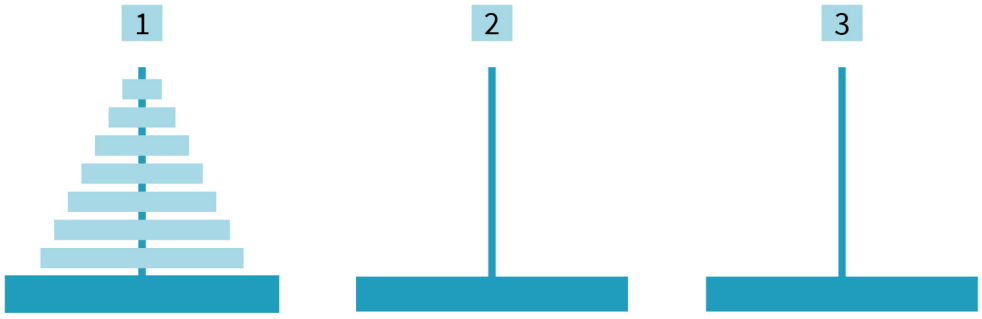
The game begins with all disks stacked on peg 1 in increasing order of size.

The game ends when all disks are stacked on peg 3. This is done using a sequence of moves obeying the following rules.

1. The smallest disk from any nonempty peg is moved to another peg.
2. A disk may never be placed on top of a smaller disk.

The rules ensure that at any stage of the game the disks are always stacked in increasing order of size, however they are distributed among the pegs.

Figure 26: Towers of Hanoi game platform



2.2 B Two Approaches for Solving The Towers of Hanoi

We will consider two approaches to solving this problem. A comparison of these approaches should yield some insight into the nature of artificial intelligence, and the role played by dynamic programming and reinforcement learning. Because the game was invented in the 19th century, at least one of our approaches should not require a computer.

We start by allowing the number of disks N to vary. If $N = 1$, the game is trivial, and can be finished in one move:

MOVE 1: Move disk 1 from peg 1 to peg 3

For $N = 2$, the game is not much more difficult:

$$\begin{array}{l} \text{MOVE 1: Move disk 1 from peg 1 to peg 2} \\ \text{MOVE 2: Move disk 2 from peg 1 to peg 3} \\ \text{MOVE 3: Move disk 1 from peg 2 to peg 3} \end{array} \tag{2.1}$$

In fact, The Towers of Hanoi is often used by mathematicians as an interesting example of recursion. We can begin to see this as we move from $N = 2$ to $N = 3$. As usual, the recursive method often relies on observations which, by themselves, seem trivial. It turns out that identification of a general recursive solution to the Towers of Hanoi problem depends on the following insights:

1. The admissibility of a move does not depend on the position of the largest disk, since any other disk may be placed on top of it.
2. If we know how to move N disks from peg 1 to peg 3, we know how to move them between any two pegs.

B.2.1. General Recursive Solution for The Towers of Hanoi

These two insights lead to a general recursive method for solving the puzzle.

To solve The Towers of Hanoi for N disks:

$$\begin{array}{l} \text{STEP 1: Move the first } N-1 \text{ disks from peg 1 to peg 2.} \\ \text{STEP 2: Move disk } N \text{ from peg 1 to peg 3.} \\ \text{STEP 3: Move the first } N-1 \text{ disks from peg 2 to peg 3.} \end{array}$$

It is worth noticing the resemblance between the general recursive solution and the particular solution for $N = 2$ given in Eqn. B.1.

The general recursive solution offers an excellent demonstration of recursive programming techniques. But we need to have first deduced the solution, which was known in the 19th century. So we have an example of human reasoning, not artificial intelligence.

B.2.2. Dynamic Programming Solution for The Towers of Hanoi

We offer as a second approach dynamic programming. However, we will use value iteration rather than backward induction. As already discussed, when designing a dynamic programming algorithm it is important to characterize precisely the form of a decision, and the information needed to make it. In fact, in an application like this, it is the definition of the state-action space K which may be most crucial.

Looking at Figure above, it seems reasonable to define the state x as a specific disk configuration over the 3 pegs. We would need to know, at least, how the disks are partitioned. However, because the moves ensure that the disks are always ordered by size, this information is enough for our needs.

If we knew, for example, that disks 1,2 and 5 were placed on peg 1, we would also know their order. We could therefore represent a state as an array of length N, in which the ith entry identified the peg on which disk i is currently placed.

A state transition then consists of a disk move following the rules of the game. Then, for example, if Figure above represents the initial state, then disk 1 is moved to peg 2, the first 2 states would be

$$x_1 = (1, 1, 1, 1, 1, 1, 1),$$
$$x_2 = (2, 1, 1, 1, 1, 1, 1)\,.$$

The game end when terminal state $x_\Delta = (3, 3, 3, 3, 3, 3, 3)$ is reached.

The action-space $\boldsymbol{\mathcal{A}}$ then consists of six possible actions:

$$\boldsymbol{\mathcal{A}} = \{\text{Move top disk from peg } i \quad \text{to peg } j{:}\, i \neq j,\ i, j \in \{1, 2, 3\}\}$$

However, not all actions $a \in \boldsymbol{\mathcal{A}}$ are admissible from each state. For example we cannot move a disk from peg i to j if either (i) peg i is empty; or (ii) the smallest disk on peg i is larger than the smallest disk on peg j. In fact, $\boldsymbol{\mathcal{A}}_x$ will always be a strict subset of $\boldsymbol{\mathcal{A}}$.

The cost function itself will be quite simple. B ut we first note that we have not yet introduced any optimization problem. So far, the question is: Is it possible to move the disks from peg 1 to peg 3, and if so, how? We could, however, just as easily ask: What is the minimum number of transitions required to move the disks from peg 1 to peg 3? If dynamic programming can solve this, we would obtain both this minimum number, and the policy (or state transition sequence) that would attain this. And it is well worth noting that as a bonus we would find out whether or not the general recursive solution given in Sec. B.2.1 was the shortest possible.

We therefore have a shortest path SDP, with terminal state $x_\Delta = (3, ..., 3)$.

Only dummy action Δ is available from x_Δ, so we set $\mathcal{A}_{x_\Delta} = \{\Delta\}$. No further cost is accrued from x_Δ, so we must have $R(x_\Delta, \Delta) = 0$. Otherwise, the cost of a transition is "one move", so $R(x, a) = 1$ for all $(x, a) \in \boldsymbol{\mathcal{K}} - \{(x_\Delta, \Delta)\}$.

Bellman's equation can now be constructed:

$$V^*(x) = \min_{a \in \mathcal{A}_x} R(a, x) + V^*(q(x, a)) \tag{2.2}$$

where $q(x, a)$ is equal to the state resulting when admissible action $a \in \mathcal{A}_x$ is selected from state x. Then Eqn. (B.2) defines operator T on a function space $\mathcal{V}$.

2.3 B Program Implementation

This algorithm is implemented as a `python` program in file `IUBH-RL-Unit02-Example02.py`, which can be run as single source code. The user can change the number of disks by modifying directly the line:

Code

```
# Set the number of disks.

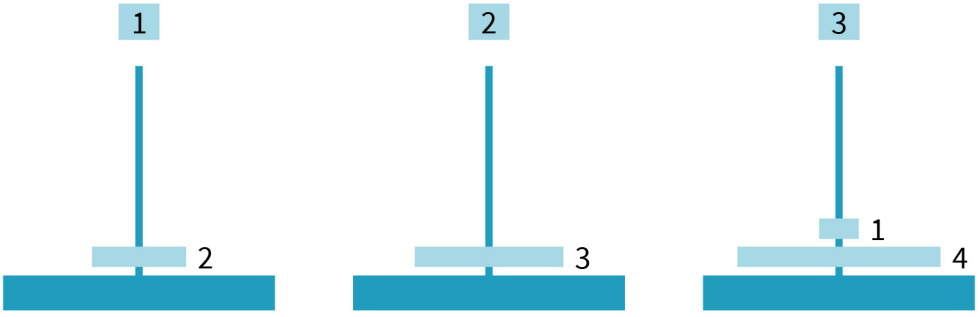
nDisk = 3
```

This problem illustrates how the mathematical representation of a dynamic programming model can assume several forms, and more than one of these can be used in a single implementation. First, take the state space $\boldsymbol{\mathcal{X}}$. A state $x \in \boldsymbol{\mathcal{X}}$ represents a complete description of the position of each disk.

At first glance, this would seem to include, for each disk, both the peg label and the relative position on that peg. However, because of the rules of the game, the disks will always be arranged in increasing order on each peg, so that only the peg label needs to be specified. A state then consists of a specification of the current peg label for each disk.

Figure below illustrates a particular state for the four disk problem. We may denote this state $x = (x_1, x_2, x_3, x_4) = (3, 1, 2, 3)$. Thus, x is a N-dimensional vector for the N disk game, where element x_i is the current peg label of disk i. The size of the state space for an N disk is therefore, by the rule of product, $\left|\boldsymbol{\mathcal{X}}\right| = 3^N$.

Figure 27: Towers of Hanoi game platform with four disks. The state $x = (3, 1, 2, 3)$ is illustrated.



The state x = (3, 1, 2, 3) is illustrated

B.3.1. Representation of the State Space

Then, how do we represent $x \in \boldsymbol{\mathcal{X}}$ in a computer program? We need to visit each state, so we need to be able to create a constructive enumeration of $\boldsymbol{\mathcal{X}}$. However, we also need to be able to access data associated with any specific state. For example, when we apply Bellman's equation, we will need to efficiently access the value function $V(x)$ for any state x.

To create an efficient constructive enumeration of $\boldsymbol{\mathcal{X}}$ in vector form $x = (x_1, ..., x_N)$, we can mimic the behavior of an automobile's odometer, which consists of an ordered sequence of digits which begins counting kilometers with the setting 000000. The only difference is that the available digits for the Towers of Hanoi model are limited to $\{1, 2, 3\}$. In `IUBH-RL-Unit02-Example02.py` the function `incState(stateArray)` accepts a state $x = (x_1, ..., x_N)$ and returns a new state incremented by one count (similar to adding one kilometer to an odometer, except the counters are incremented from left to right). See the following examples for the four disk problem. Note that the final state of this enumeration is $(3, 3, 3, 3)$.

Accordingly, the `incState(stateArray)` "turns over" to state $(1, 1, 1, 1)$ when the final state is reached.

Code

```
print(incState([1,1,1,1]))
[2, 1, 1, 1]
print(incState([3,2,1,1]))
[1, 3, 1, 1]
print(incState([3,3,3,1]))
[1, 1, 1, 2]
print(incState([3,3,3,3]))
[1, 1, 1, 1]
```

When incorporated into a programming loop of length 3^N, each state can be systematically visited. The advantage of this scheme is that it permits an alternative representation of the state space. The value function $V(x)$ and policy function $\phi(x)$ are functions of state $x \in \boldsymbol{\mathcal{X}}$, so we need a way to efficiently access these values for any given x. If x is given in vector form $x = (x_1, ..., x_N)$ it is, in principle, possible to represent $V(x)$ and $\phi(x)$ as N-dimensional arrays, but it may be better programming practice to represent them as single dimensional arrays, referenced by a state index, or a unique integer assigned to each state. If we enumerate the states, by success increments, starting at initial state $x = (1, ..., 1)$, we generate a canonical ordering which can be used for such an index assignment scheme.

As an example, the following code generates the first five states of this ordering for the four disk problem.

Code

```
stateArray = [1,1,1,1]
for i in range(5):
     print('State Index = ',i,',
     State Vector = ',stateArray)
     stateArray = incState(stateArray)

...

State Index =  0 , State Vector =  [1, 1, 1, 1]
State Index =  1 , State Vector =  [2, 1, 1, 1]
State Index =  2 , State Vector =  [3, 1, 1, 1]
State Index =  3 , State Vector =  [1, 2, 1, 1]
State Index =  4 , State Vector =  [2, 2, 1, 1]
```

Thus, the state $x = (1, 2, 1, 1)$ (disks 1,3,4 on peg 1; disk 2 on peg 2), can be alternative represented by index $i = 3$. The value function $V(x)$ and policy function $\phi(x)$ would then be one-dimensional arrays of length $3^4 = 81$, indexed by integers $i = 0, 1, ..., 80$.

We will also need a method of converting a state in its vector representation to its corresponding state index, so that $V(x)$ and $\phi(x)$ can be accessed as needed. This is done using function `pHash(stateArray)`, demonstrated below.

Code

```
print(pHash([1,1,1,1]))
0
print(pHash([2,1,1,1]))
1
print(pHash([3,1,1,1]))
2
print(pHash([2,3,3,3]))
79
print(pHash([3,3,3,3]))
80
```

B.3.2. Representation of the Action Space

How, then, do we represent the action space? Clearly, the set of all actions is very large. However, fewer than six of these are available at each state.

The potential moves can be reduced to one of six types, which we label $a = 0, 1, ..., 5$, given in the following table.

Table 4

$a =$	**Potential Move**
0	Smallest disk on Peg 1 to Peg 2
1	Smallest disk on Peg 1 to Peg 3
2	Smallest disk on Peg 2 to Peg 1
3	Smallest disk on Peg 2to Peg 3
4	Smallest disk on Peg 3 to Peg 1
5	Smallest disk on Peg 3 to Peg 2

Of course, not all actions will be admissible at each state. For example, from the state $x = (3, 2, 1, 3)$ illustrated in figure above, actions $a = 0, 4, 5$ will be admissible, but not actions $a = 1, 2, 3$, since we cannot move, for example, disk 3 to any other peg. In addition, a move is inadmissible if the first peg contains no disk. In our program, the action space is therefore represented by the labels $\mathcal{A} = \{0, 1, ..., 5\}$, which can be interpreted accordingly for each state. As usual, we can denote the admissible actions for state x as, for example, $\mathcal{A}_x = \{0, 4, 5\}$.

B.3.3. Representation of the Dynamic Programming Solution

In `IUBH-RL-Unit02-Example02.py` the value function $V(x)$ is represented by a one-dimensional array `valueFunction` of length 3^N:

Code

```
# Create the value function.

valueFunction = array('i')
valueFunction = [0]*3**nDisk
```

Note that `nDisk` is already set. Because the algorithm is iterative, we create a second object identical to `valueFunction`.

Code

```
# Create temporary value function.

valueFunctionOld = array('i')
valueFunctionOld = [0]*3**nDisk
```

Then `valueFunctionOld` stores the current solution while the next solution is being updated and stored in `valueFunction`. The policy function $\phi(x)$ is represented by a one-dimensional array policyFunction of length 3^N:

Code

```
# Create policy function.

policyFunction = array('i')
policyFunction = [0]*3**nDisk
```

B.3.4. Solution Update

We are using value iteration to solve a deterministic shortest path SDP. A single iteration requires evaluation of

$$V_{n+1}(x) = \min_{a \in \mathcal{A}_x} R(a,x) + V_n(q(x,a))$$

for each $x \in \boldsymbol{\mathcal{X}}$. Note that in our program, V_n is stored in `valueFunctionOld`, while the updated solution V_{n+1} is stored in `valueFunction`. Then consider a fixed state in vector form $x = (x_1, ..., x_N)$. We examine each of the six potential actions and calculate the Q-value

$$V(x,a) = R(a,x) + V_n(q(x,a))\,.$$

We note, however, that it is not necessary to keep in memory $V(x,a)$ for each state-action pair $(x,a) \in \mathcal{K}$. We may therefore represent $V(x,a)$ as a single one-dimensional vector of length six, reusing this memory for each new state we consider. This is represented by object `valueActionFunction`:

Code

```
# Create value-action function V(x,a).
# We only required a fixed amount of memory.

valueActionFunction = array('i')
valueActionFunction = [0]*6
```

When we examine each action, we need to determine whether or not it is admissible. One way to do this efficiently is to identify for a given state $x = (x_1, ..., x_N)$ the smallest disk on each peg. In `IUBH-RL-Unit02-Example02.py` this is done using function topDisks(state-Array), which inputs a state in vector form, and outputs a vector identifying the smallest disk on each in matching order. If there is no disk currently on the peg, the entry will be $N+1$. For example:

Code

```
print(topDisks([1,1,1,1]))
[1, 5, 5]
print(topDisks([3,1,2,3]))
[2, 3, 1]
```

Regarding the state $x = (3, 2, 1, 3)$ illustrated in figure above, the function `topDisks(stateArray)` identifies the smallest disks as being 2,3 and 1, respectively. Once we have determined which actions are admissible, it will be straightforward to identify the subsequent state $q(x, a)$ by suitably altering the state vector (we need only change the peg label of the disk being moved).

The solution update method requires that we calculate $V(x, a)$ for each admissible action a, then set $V_{n+1}(x)$ to be the minimum of those values.

However, there are two ways to distinguish, as is necessary, admissible from inadmissible actions. The first is to ensure that the minimization operation

$$V_{n+1}(x) = \min_{a \in \mathcal{A}_x} R(x, a) + V_n(q(x, a)), \tag{2.3}$$

is restricted to admissible solutions. However, it may be more efficient from a programming point of view to use the "big M" method. If a is admissible, we set $R(x, a) = 1$, representing the cost of a single move, unless $x = x_\Delta = (3, ..., 3)$ is the terminal state, in which case $R(x_\Delta, a) = 0$ for any a. If a is inadmissible we set $V(x, a) = M$, where M is a

value which will be larger than any value of $V(x)$. Some programming environments will support a value $M = \infty$, which would be entirely appropriate. Otherwise, we note that an optimal path cannot be longer than the total number of states, which here is 3^N. We use that value in `IUBH-RL-Unit02-Example02.py`.

Code

```
# We need a large number that will exceed
# any admissible policy cost.
# This can be the total number of states.

bigM = 3**nDisk
```

Thus, we set $V(x, a) = R(a, x) + V_n(q(x, a))$ if a is admissible for x, and $V(x, a) = M$ otherwise. If we then perform the update

$$V_{n+1}(x) = \min_{a \in \{0, 1, ..., 5\}} V(x, a), \tag{2.4}$$

then an inadmissible action will never be selected. We can store the current policy by setting $\phi(x)$ (stored in `policyFunction`) equal to the subsequent state $q(x, a^*)$ given the minimizing action a^* (for convenience this is stored as a state index).

B.3.5. Value Iteration

The value iteration algorithm consists of the iterations

$$V_{n+1} = TV_n, \ n = 1, 2, ...$$

where T is the dynamic programming operator, and V_1 is any suitable initial solution. In `IUBH-RL-Unit02-Example02.py` each iteration is evaluated by applying Eqn. (B.4) to each state $x \in \mathcal{X}$. We take as the initial state $V_1(x) = 0$ for all $x \in \mathcal{X}$. We also apply the constraint $V_n(x_\Delta) = 0$ for terminal state $x_\Delta = (3, ..., 3)$ for all $n \geq 1$.

The number of iterations required for convergence is generally not known in advance. However, because the value function is integer valued, we can test for convergence using the condition $V_{n+1}(x) = V_n(x)$ for all $x \in \mathcal{X}$.

When this occurs, the current policy function ϕ will be the optimal policy, and $V_{n+1}(x)$ will be the optimal value function.

The value iteration loop is controlled by a flag variable `flagContinue`.

Code

```
# Iterations continue while flagContinue equals True.

flagContinue = True
while flagContinue:
```

Within this loop, `flagContinue` is set to `False` when the convergence condition is met. First, `flagContinue` is set to `False` at the beginning of the iteration:

Code

```
# We will terminate iterations unless the
# value function changes.

flagContinue = False
```

Following the update of Eqn. (B.4) for each state x, if $V_{n+1}(x) \neq V_n(x)$ then `flagContinue` is set to `True`, since this means the convergence condition has not been met. This test is implemented in the following code.

Code

```
# If the value function has changed,
# set flagContinue = True so that
# the iterations will continue.

if (valueFunction[j] != valueFunctionOld[j]):
    flagContinue = True
```

Within an iteration, an inner loop enumerates all states by setting initial state $x = (1, ..., 1)$, then incrementing x with function `incState(stateArray)` near the end of the code block. We know in advance that the number of states is 3^N. However, $V(x_\Delta) = 0$ for terminal state $x_\Delta = (3, ..., 3)$, so the loop only performs $3^N - 1$ iterations:

Code

```
# Loop through all states, except for the terminal state,
# for which the  value function is always zero.

for j in range(3**nDisk - 1):
```

At the beginning of the code block, for the currently visited state x, the "smallest disks" vector is calculated using the `topDisks(stateArray)` function. The Q-value vector `valueActionFunction` is initialized to the "big M" value, and a vector `action` is initialized to store the subsequent state $q(x, a)$ for each admissible action a (the value associated with inadmissible states is not relevant).

Code

```
# Extract the top disks for the current array.

topDiskArray = topDisks(stateArray)

# Initialize value-action function and action array.

valueActionFunction = [bigM]*6
action = [0]*6
```

Then for each potential state there is a block of code which determines admissibility, then calculates the Q-values and subsequent state $q(c, a)$ if a is admissible These are stored in `valueActionFunction` and `action`, respectively. If a is not admissible, the code block is not executed. In this case, because all Q-values are initialized to M, this will remain the value of $V(x, a)$ for that action, which will therefore not be the minimizing action in the update (B.4). The code for action label $a = 0$ is shown below.

Code

```
# We next consider the six possible moves.
# The IF statement checks for admissibility.
# If the action is not admissible,
# then V(x,a) = bigM, and that action will not be selected.

# Peg 1 to Peg 2.

if (topDiskArray[0] < topDiskArray[1]):
           for iii in range(nDisk):
              stateArrayTemp[iii] = stateArray[iii]
           stateArrayTemp[topDiskArray[0]-1] = 2
           valueActionFunction[0] = 1 +
             valueFunctionOld[pHash(stateArrayTemp)]
           action[0] = pHash(stateArrayTemp)
```

The conditional `if` statement tests for admissibility. The elements `topDiskArray[0]`, `topDiskArray[1]` store the labels of the smallest disks on pegs 1 and 2, respectively. If peg 1 is empty, then `topDiskArray[0]` will contain the value $N + 1$, so that the admissibility condition will not be met.

B.3.6. Output

The optimal path is not explicitly created by the value iteration algorithm, but is implicitly stored in the policy function $\phi(x)$, which returns the next state from x within an optimal path. In `IUBH-RL-Unit02-Example02.py` this is stored in the array `policyFunction`. The array is referenced by the state index, and returns the next state also in indexed form.

To usefully display a solution we will build a map from the state index to the state vector. This takes the form of a $3^N \times N$ matrix named `stateMap`.

The row labels $i = 0, ..., 3^N - 1$ correspond to the state indices, and the state in vector form is stored in the corresponding row. The code used to do this is as follows:

Code

```
# In order to display the solution, create a map
# from the state index to the state array,
# which is more easily interpreted.
# This is stored in the matrix stateMap.

stateMap = numpy.zeros(shape=(3**nDisk,nDisk),dtype='i')

# Re-initialize the state array.

stateArray = array('i')
stateArray = [1]*nDisk

# Create the map by looping through each state.

for j in range(3**nDisk):
    stateMap[j] = stateArray
    stateArray = incState(stateArray)
```

Then, if x_1 is the initial state $(1, ..., 1)$, then the solution can be recovered by the iterations:

$$x_1 = (1, ..., 1),$$
$$x_{n+1} = \phi(x_n),$$

which continue until the terminal state $x_{\Delta} = (3, ..., 3)$ is reached. The optimal path is then $x_1, x_2, ..., x_{\Delta}$. Alternatively, we know that the length of the optimal path from initial state x_1 is equal to $V(x_1)$, so the iterations can be controlled by a fixed length loop. We use `stateMap` to create the `optimalpath`, stored in optimalPath, in state vector form. The code is as follows:

Code

```
# Capture the optimal path from the policy function
# The number of transitions of the optimal path is
# given by valueFunction[0].

optimalPath = numpy.zeros(
    shape=(valueFunction[0]+1,nDisk),dtype='i'
    )
a = 0
for j in range(valueFunction[0]+1):
    optimalPath[j] = stateMap[a]
    a = policyFunction[a]
```

Finally, we have a function `makeTowerPic(stateArray)` which creates a simple text-based graphic similar to figure above for display. The following code prints the solution.

Code

```
# Print the optimal path

print('The optimal number of transitions is ',
    valueFunction[0],'.')
print('The optimal sequence of states is:')
print()

for i in range(valueFunction[0]+1):
   currentTowerPic = makeTowerPic(optimalPath[i])
   print('--------------------------------   Stage ',i+1)
   for j in range(3):
       print(currentTowerPic[j])

print('--------------------------------   STOP')

# PROGRAM ENDS
```

```
The optimal number of transitions is  7 .
The optimal sequence of states is:

--------------------------------   Stage  1
|321
|
```

```
|
---------------------------------   Stage  2
|32
|
|1
---------------------------------   Stage  3
|3
|2
|1
---------------------------------   Stage  4
|3
|21
|
---------------------------------   Stage  5
|
|21
|3
---------------------------------   Stage  6
|1
|2
|3
---------------------------------   Stage  7
|1
|
|32
---------------------------------   Stage  8
|
|
|321
---------------------------------   STOP
```

C - Application of MDPs: An Example from Queueing Theory

C - Introduction/Case study

The implementation is in file `IUBH-RL-Unit02-Example03.py`

3.1 C Introduction

Queuing systems form a class of stochastic process of considerable importance in operations research, and present a rich set of applications for MDP models. A queuing system is easy to describe. It contains a queue into which an arrival process of customers enter. It

also contains m servers, who service customers. The time of service has a specified distribution. A customer in the queuing system is either being served, or is in the queue waiting to be served. A server is either busy serving a customer, or is free.

Upon service completion the customer exits the system and the server immediately begins service of some customer from the queue if it is not empty.

Accordingly, a customer entering the system begins service immediately if there is a free server, or enters the queue otherwise to wait for service.

Queuing models are commonly classified using PlusKendall's notation which originally took form $A/B/m$, as originally proposed by Kendall in (Kendall, 1953). It is assumed that the customer arrival process is a renewal process with a renewal distribution described by A. Then B refers in the same way to the service time (which are assumed to be independent), and m is the number of servers. The convention has since been extended, most commonly to $A/B/m/K/M$, in which K is the system capacity, and M is the number of customers in a finite population. The last two parameters are often omitted when they equal ∞. The symbols for A, B are standard, with M denoting the exponential distribution (M for 'memoryless'), D a deterministic, or constant distribution, and G denoting a general distribution (the assumption of independence is sometimes indicated by the symbol GI).

The analysis is especially tractable for $M/M/m/K$ queues, since they are completely memoryless.

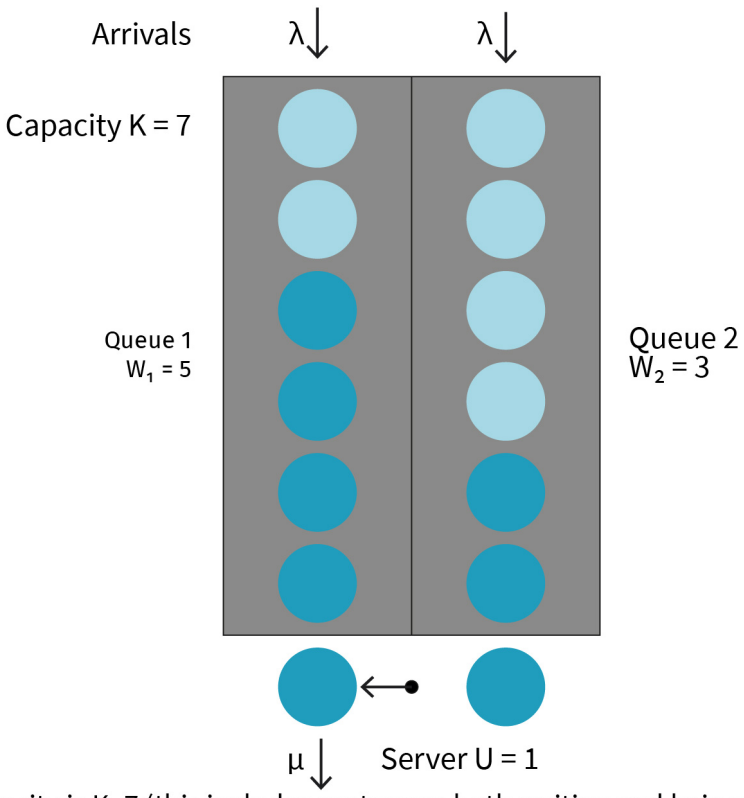
An excellent introduction to queuing theory can be found in (S. Ross, 2014), while a more advanced treatment can be found in the well-known textbook (Kleinrock, 1975).

3.2 C The Switchover Queue Control Problem

We will consider a control problem associated with a modification of the standard queue model. There are two separate queues, each with a separate arrival process with rate λ. Each has a finite capacity K. A customer arriving at a full queue is lost to the system. A single server services both queues. A decision epoch occurs when a customer enters or leaves the system, and defines the beginning of a new stage. At each decision epoch the server can decide to `Stay` at the current queue or `Switch` to the other queue and begin serving customers there. Therefore, the state of the system will be the triple $x = (W_1, W_2, U)$, where W_i is the number of customers in queue $i = 1, 2$ (including any customer in service), and U is the queue being served. See Figure below.

Figure 28: Schematic representation of the switchover queue model. Capacity is $K = 7$ (this includes customers both waiting and being served). The state illustrated is $x = (W_1, W_2, U) = (5, 3, 1)$. Queue 1 has 1 customer in service, and 4 more waiting in

queue, so $W_1 = 5$. and $U = 1$. Queue 2 has 3 customers waiting in queue and none being served, so $W_2 = 3$. The arrival rates λ and serve rate μ are indicated. These are in units of customer per time unit.



Capacity is K=7 (this includes customers both waiting and being served).
The state illustrated is x = (W_1, W_2, U) = (5, 3, 1). Queue 1 has 1 customer in service and 4 more waiting in the queue, so W_1 = 5 and U = 1.
Queue 2 has 3 customerswaiting in the queue and none being served, so W_2 = 3. The arrival rates λand serve rate μ are indicated.
These are in units of customer per time unit.

The action space contains two decisions, $\mathcal{A} = \{Stay, Switch\}$. As is common for this type of problem, there are more than one "sensible" representations of the various components of the model. For example, we could also represent the action space by the queue index, $\mathcal{A} = \{1, 2\}$, which represents the subsequent queue position of the server. So if the action $a = 1$ is taken from state $x = (3, 1, 2)$, the server is choosing to `Switch` from queue 2 to queue 1. In this application, we will use whichever representation is most convenient for the task.

Finally, there will be two sources of cost. There will be a cost of B units accrued whenever the server switches queues. In addition, when there are W_1, W_2 customers in the respective queues, the system assumes a cost of $g(W_1, W_2)$ per unit time. The cost function is therefore

$$R(x,a) = B \times I\{a = \boldsymbol{Switch}\} + \tau(x,a) \times g(W_1, W_2), \tag{3.1}$$

where $\tau(x,a)$ is the expected time length of the stage given that action a is selected from state x.

The final step is to calculate the probability transition kernel $P(y \mid x,a)$ and the stage time length $\tau(x,a)$. This can be done by recognizing the model as a continuous-time Markov process, in which transitions between discrete states occur in real time $\mathcal{T} = [0,\ \infty\ )$. Furthermore, the process is memoryless, in the sense that the future distributional properties of the process at time $t$depend only on the current state x_t. The theory of the continuous-time Markov process is considerably simplified by understanding the properties of the exponential distribution.

C.2.1. State Transition Diagrams

To define the MDP model, we will make use of the following theorem.



THEOREM

The following properties hold for the exponential distribution:

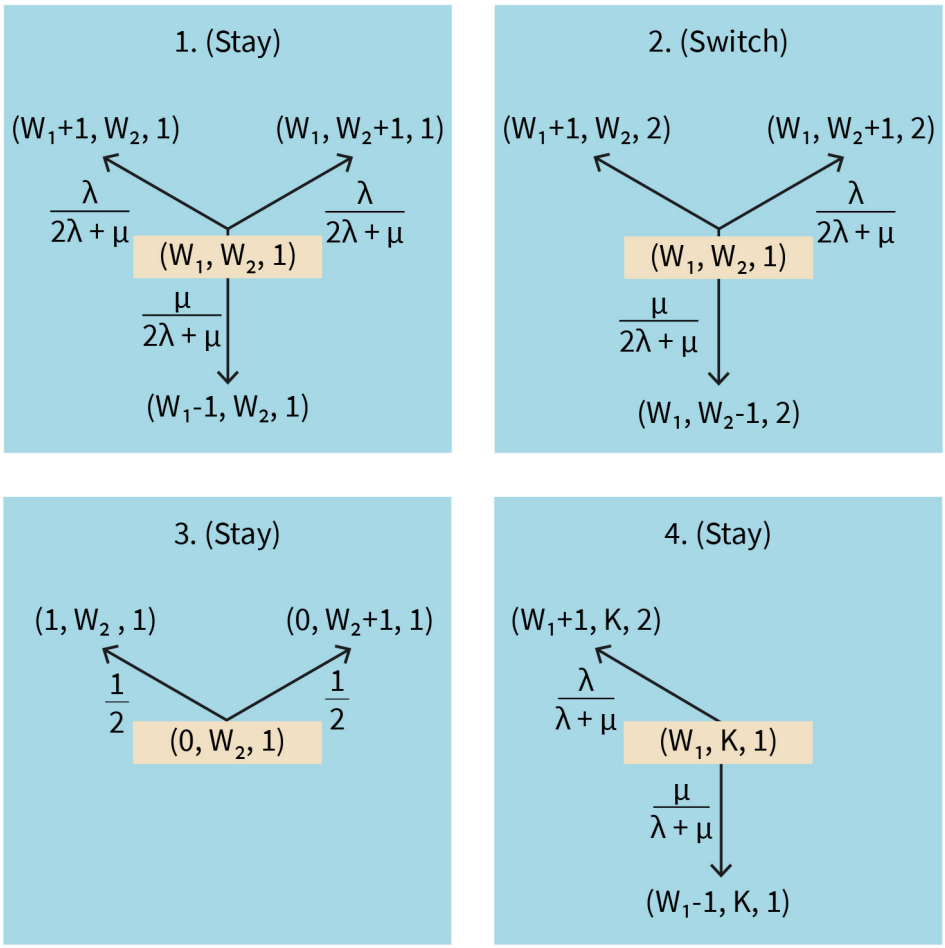
i) The exponentially distributed random variable is the unique memoryless waiting time with support $[0,\ \infty\ )$.

ii) If $X_1, ..., X_n$ are independent exponentially distributed random variables with rates $\lambda_1, ..., \lambda_n$ then $Y = \min_i X_i$ is exponentially distributed with rate $\sum_i \lambda_i$.

iii) In addition, $P\big(Y = X_j\big) = \lambda_j / \sum_i \lambda_i$.

To see how this theorem may be used, we will construct a few examples of state transition diagrams. First, consider Diagram 1 of figure below.

Figure 29: Examples of state transition diagrams for the switchover queue model. Unless otherwise specified, W_1 and W_2 are assumed to be strictly between 0 and K.



The system is currently in state $x = (W_1, W_2, 1)$. Assume, for now, that $0 < W_i < K$. The action is $a = \mathbf{Stay}$, so the server remains in queue 1. This means that the stage ends as soon as one of the three events occurs:

1. The server finishes serving the current customer in queue 1. This occurs after an exponentially distributed time Y with rate μ.
2. A new customer enters queue 1. This occurs after an exponentially distributed time X_1 with rate λ.
3. A new customer enters queue 2. This occurs after an exponentially distributed time X_2 with rate λ.

Note that any of these events can occur if $0 < W_i < K$. This formulation is possible because of the memoryless property of the exponential distribution, so the time measures Y, X_1, X_2 begin at the beginning of a stage (so that, for example, Y need not equal the total service time of a customer).

The trick is to recognize that the event which ends the stage, and which therefore defines the state transition, is the one with the smallest waiting time (that is, it is the first event to occur). Using the preceding theorem, we then have

$$\begin{aligned} P(\text{Server finishes service first}) &= \frac{\mu}{2\lambda + \mu} \\ P(\text{Customer enters queue 1 first}) &= \frac{\lambda}{2\lambda + \mu} \\ P(\text{Customer enters queue 2 first}) &= \frac{\lambda}{2\lambda + \mu}\,. \end{aligned}$$

Diagram 1 of figure above is simply a summary of this calculation. The current state is $x = (W_1, W_2, 1)$, and action `Stay` is selected. If the next event is the arrival of a new customer in queue 1, then the process transitions to state $(W_1 + 1, W_2, 1)$, and this occurs with probability $\lambda/(2\lambda + \mu)$. This is represented in the diagram by a directed edge labeled by the transition probability. The remaining two possible transitions are similarly represented.

We also need the expected stage length. This is also obtained from the preceding theorem by noting that the observed stage length is

$$Z = \min\{Y, X_1, X_2\},$$

which is exponentially distributed with rate $2\lambda + \mu$. Therefore, the expected stage length is

$$\tau(x, \mathbf{Stay}) = (2\lambda + \mu)^{-1}.$$

Diagram 2 of figure above is similar. The current state is also $x = (W_1, W_2, 1)$, but the action selected is **Switch**. The probability calculations are essentially the same, but the state transitions themselves differ. Since the server was originally at queue 1, but switched to queue 2, if the next event is a service completion, the new state becomes $x' = (W_1, W_2 - 1, 2)$, rather than $x' = (W_1 - 1, W_2, 1)$ shown in Diagram 1. Otherwise, the transition is to $(W_1 + 1, W_2 - 1, 2)$ or $(W_1, W_2 + 1, 2)$. We also have expected stage length $\tau(x, \mathbf{Switch}) = (2\lambda + \mu)^{-1}$.

If either state variable W_1 or W_2 is a boundary value 0 or K, the possible state transitions are limited to a strict subset of the original three possibilities.However, the preceding theorem can still be used in essentially the same way. The state $(0, W_2, 1)$ and action **Stay** are represented in Diagram 3. After the action is taken, the server is in queue 1, which is empty, so that no customer is being served. This means the state transition can only occur when a new customer arrives at either queue. These two transitions occur with equal probably, as indicated in the state transition diagram.

The state $(W_1, K, 1)$ and action **Stay** are represented in Diagram 4. Queue 2 is at full capacity, so no customers can arrive there. Only two types of state transitions can take place: either a customer arrives at Queue 1, or a customer completes service. These transitions are indicated in the state transition diagram.

3.3 C Program Implementation

The python code in file `IUBH-RL-Unit02-Example03.py` implements the value iteration algorithm for the switchover queue control model. This is a MDP for which cost function $R(x, a)$, stochastic transition kernel $P(y \mid x, a)$ and discount factor β must be represented.

Each queue contains W_i customers, $i = 1, 2$ where $0 \leq W_i \leq K$. In addition, the queue $U \in \{1, 2\}$ is being serviced. The state is defined by the triple $x = (U, W_1, W_2)$. By the rule of product, the state space therefore has size

$$|\mathcal{X}| = 2(K+1)^2.$$

The value function $V(U, W_1, W_2)$ is stored in **valueFunction** as a threedimensional array, the dimensions corresponding to U, W_1 and W_2. Because the algorithm is iterative, we create a second object identical to **valueFunction**, named **valueFunctionOld**. Then **valueFunctionOld** stores the current solution while the next solution is being updated and stored in **valueFunction**. The queue capacity is first defined (here, $K = 10$ is stored in variable **qCapacity**).

Code

```
# Define dynamic programming solution components.

qCapacity = 10
valueFunction = numpy.zeros(
    shape=(2,qCapacity+1,qCapacity+1)
    )
valueFunctionOld = numpy.zeros(
    shape=(2,qCapacity+1,qCapacity+1)
    )
policyFunction = numpy.zeros(
    shape=(2,qCapacity+1,qCapacity+1)
    )
```

Model parameters (arrival rate λ, service rate μ and discount factor β) are defined in one place:

Code

```
# Define model parameters: service rate (murate);
# arrival rate (lambdaRate); discount factor (beta).

muRate = 1.0
lambdaRate = 0.9
beta = 0.99
```

The number of iterations is fixed by variable **nTrace**. One element of the value function $V(0,0,0)$ will be stored in array **vTrace** after each iteration, so that progress towards the limit can be observed.

Code

```
# The number of iterations is fixed by nTrace.
# One element of the value function V(0,0,0) will be
# stored after each iteration in array vTrace, so that
# progress towards the limit can be observed.

nTrace = 1000
vTrace = array('l')
vTrace = [0]*nTrace
```

The value iteration algorithm is controlled by a fixed length outer loop.

The first step in the code block is to store current value function $V_n(x)$ into **valueFunctionOld**. The updated solution $V_{n+1}(x)$ will be stored in **valueFunction**.

Code

```
# Begin value function iterations.

for iii in range(nTrace):

        # Store current value function in
        # valueFunctionOld.

        for i in range(qCapacity+1):
            for j in range(qCapacity+1):
                for k in range(2):
                    valueFunctionOld[k][i][j] = \
                        valueFunction[k][i][j]
```

We then loop through the entire state space $\mathcal{X}$. The state vector **stateArray** is created from state variables **u**, **w1**, **w2**, representing queue being serviced, and the two queue lengths. A subsequent state vector **stateArrayNext** is also created. The Q-values $V(x,a)$ are also intialized here. We note, however, that it is not necessary to keep in memory $V(x,a)$ for

each state-action pair $(x, a) \in \mathcal{K}$. We may therefore represent $V(x, a)$ as a single one-dimensional vector of length two, reusing this memory for each new state we consider. This is represented by object **valueActionFunction**:

Code

```
# Loop through entire stateArray space.

        for u0 in range(2):
            for w1 in range(qCapacity+1):
                for w2 in range(qCapacity+1):

                    # Current queue is either u = 1 or 2.
                    # Inititalize a current state vector
                    # stateArray = [w1,w2,u], and a
                    # subsequent state vector
                    # stateArrayNext.

                    u = u0+1
                    stateArray = array('i')
                    stateArray = [w1,w2,u]
                    stateArrayNext = array('i')
                    stateArrayNext = [0,0,0]

                    # Inititalize Q-values V(x,a).

                    valueActionFunction = [0]*2
```

The value $V(x, a)$ is then evaluated for each action.

The components needed to do this are given by function `makeTransitionList(stateArray,a0+1,muRate lambdaRate,qCapacity)`, which, for each state-action pair (x, a) outputs all the data needed to analyze the state transition diagrams of the type shown in Fig. C.2. The variable **cost** contains $R(x, a)$.

The list of all states to which the process can transition from x is given in **transitionList**, while the transition probabilities are given in **pTransitions**. The expected stage length $\tau(x, a)$ is given in **stageLength**.

These components are then used to evaluate $V(x, a)$.

Code

```
# Evaluate V(x,a) for each action.

for a0 in range(2):

     # Collect components to evaluate V(x,a)
```

```
     nTransitions, stageLength, transitionList, \
     pTransitions, cost = \
     makeTransitionList(
        stateArray,a0+1,muRate,
        lambdaRate,qCapacity
        )
     valueActionFunction[a0] = cost

    # Perform the evaluation.

     for i in range(nTransitions):
          for ii in range(3):
            stateArrayNext[ii] = transitionList[i][ii]
          valueActionFunction[a0] = \
            valueActionFunction[a0] +
            beta*valueFunctionOld[stateArrayNext[2]-1]
                [stateArrayNext[0]]
                [stateArrayNext[1]]*pTransitions[i]

     # Store minimum V(x,a) into value function,
     # then store minimizing action.

    # put array indices in helper variables
    idx1 = stateArray[2]-1
    idx2 = stateArray[0]
    idx3 = stateArray[1]
     valueFunction[idx1][idx2][idx3]
          = min(valueActionFunction)

     policyFunction[idx1][idx2][idx3]
          = valueActionFunction.index(
                min(valueActionFunction)
                )+1
```

Note that the cost function defined in Eqn. C.1 used in this example is

$$R\big(x,a\big) = I\Big\{a = \mathbf{Switch}\Big\} + \tau\big(x,a\big) \times 3\big(W_1^2 + W_2^2\big),$$

which is implemented in the following code:

Code

```
# Calculate cost.

cost = 0
if (action!=stateArray[2]): cost = 1
```

```
cost = cost + 3*stageLength*(
    (stateArray[0]**2)+(stateArray[1]**2)
    )
```

After the value function is completely updated, the value $V_{n+1}(1,0,0)$ is stored in the array **vTrace**.

Code

```
# Store current value of V(0,0,0).

vTrace[iii] = valueFunction[0][0][0]
```

C.3.1. Output

The optimal policy function $\phi(x)$ is stored in the three-dimensional array **policyFunction**. Given **python** conventions, **policyFunction[0]** is a $(K+1) \times (K+1)$ matrix with rows labeled $W_1 = 0, 1, ..., K$, and columns labeled $W_2 = 0, 1, ..., K$, which gives the optimal action from state (W_1, W_2, U), with $U = 1$. Then `policyFunction[1]` gives the corresponding matrix when $U = 2$. We can output the optimal policy with the following code:

Code

```
# Output optimal policy.

print("Optimal Policy: Row = W1, Column = W2, U = 1")
print(policyFunction[0])
print()
print("Optimal Policy: Row = W1, Column = W2, U = 2")
print(policyFunction[1])
print()

The optimal policy is then:

Optimal Policy: Row = W1, Column = W2, U = 1
[[1. 2. 2. 2. 2. 2. 2. 2. 2. 2. 2.]
 [1. 1. 2. 2. 2. 2. 2. 2. 1. 1. 1.]
 [1. 1. 1. 2. 2. 1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 2. 2. 1. 1. 1. 1. 1. 1.]
 [1. 1. 2. 2. 2. 2. 1. 1. 1. 1. 1.]
 [1. 1. 2. 2. 2. 2. 2. 1. 1. 1. 1.]
 [1. 2. 2. 2. 2. 2. 2. 2. 1. 1. 1.]
 [1. 2. 2. 2. 2. 2. 2. 2. 2. 1. 1.]
 [1. 2. 2. 2. 2. 2. 2. 2. 2. 2. 1.]]

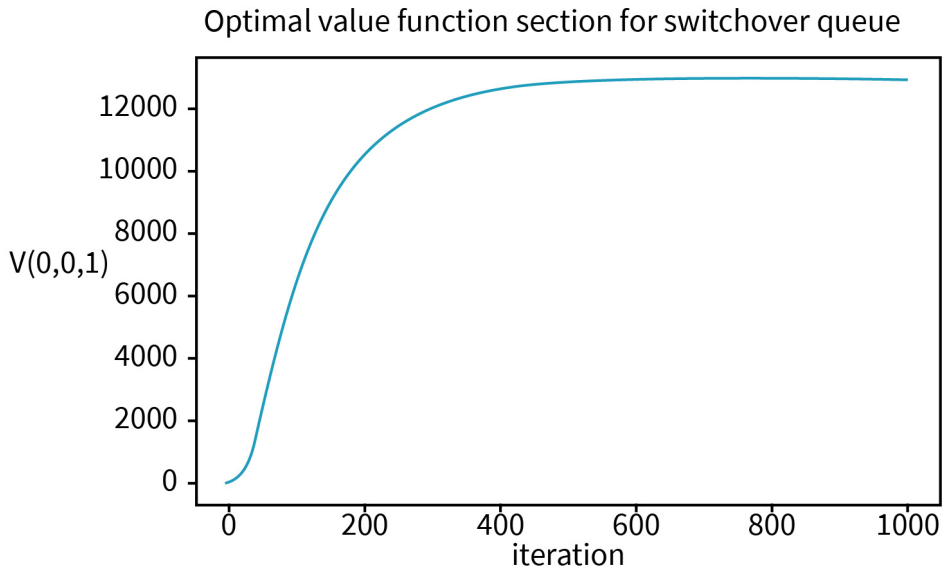
Optimal Policy: Row = W1, Column = W2, U = 2
[[2. 2. 2. 2. 2. 2. 2. 2. 2. 2. 2.]
```

```
[1. 2. 2. 2. 2. 2. 2. 2. 1. 1. 1.]
[1. 1. 2. 2. 2. 2. 1. 1. 1. 1. 1.]
[1. 1. 1. 2. 2. 1. 1. 1. 1. 1. 1.]
[1. 1. 1. 2. 2. 1. 1. 1. 1. 1. 1.]
[1. 1. 2. 2. 2. 2. 1. 1. 1. 1. 1.]
[1. 1. 2. 2. 2. 2. 2. 1. 1. 1. 1.]
[1. 1. 2. 2. 2. 2. 2. 2. 1. 1. 1.]
[1. 2. 2. 2. 2. 2. 2. 2. 2. 1. 1.]
[1. 2. 2. 2. 2. 2. 2. 2. 2. 2. 1.]
[1. 2. 2. 2. 2. 2. 2. 2. 2. 2. 2.]]
```

The matrix entry gives the subsequent queue for the server. In other words, for the first matrix, $U = 1$, so the server begins the stage at queue 1. If the action is $a = 1$, the server “stays", and if $a = 2$, the server “switches".Conversely, the second matrix represents states for which $U = 2$, the server beginning the stage at queue 2. If the action is $a = 1$, the server “switches", and if $a = 2$, the server “stays.

As an example, consider the second row of the first matrix. Here, $W_1 = 1$, $U = 1$, so that queue 1 contains one customer being served, but no more customers waiting. The entries correspond to columns $W_2 = 0, 1, ..., K$. This means that if $W_2 \geq 2$ (no more than 2 customers in queue 2) the optimal action is $a = 1$, so the server will not switch. If $3 \leq W_2 \leq 4$ (3 or 4 customers in queue 2) the optimal action is $a = 2$, so the server will switch.Interestingly, if $W_2 \leq 5$(at least 5 customers in queue 2) the optimal action returns to $a = 1$, so the server will not switch. This is an example of a non-monotone control policy, in the sense that as W_2 increases from 0, the policy changes from “stay" to “switch", but then returns to “stay" as W_2 increases further. In effect, the policy exploits the fact that when a queue is close to full capacity, the bound K limits further growth in queue size. See, for example, (Rosa-Hatko & Gunn, 1997) for a survey of this control problem.

Figure 30: Trace array of the value iteration algorithm for the switchover queue problem.



Finally, we plot the **arrayvTrace**, which contains the value of $V_n(0,0,1)$ for each iteration of the value iteration algorithm. The resulting plot is shown in figure above. Note that by the $n = 1000th$iteration, $V_n(0,0,1)$ appears to have converged to a specific value, suggesting that the algorithm has come very close to the optimal solution.

Code

```
# Plot value function trace.

plt.plot(range(nTrace),vTrace,linewidth=2)
plt.xlabel('Iteration')
plt.ylabel('V(0,0,1)')
plt.title('Optimal Value Function Section for
    Switchover Queue')
plt.show()
```

D - The Cake-Eating Problem: An Application of Dynamic Programming to Utility Theory

D - Introduction/Case study

An amount of wealth that is to be distributed is sometimes referred to, rather informally, as a “cake". We might be concerned with the problem of “baking a bigger cake", or how the cake should be distributed within a group.

Accordingly, in economic theory, the sequential allocation of wealth finds expression in the “cake-eating problem", an application of dynamic programming that relies on the concept of utility. This problem concerns a single decision maker in possession of a fixed amount of wealth W (this is the “cake"). At discrete time points (or stages), a portion of the cake is consumed, and a corresponding amount of utility is accrued. The object is to maximize the total utility, which is possible discounted. This relatively simple problem leads to a quite rich class of new problems, once various modifications are introduced (Adda et al., 2003).

We will define a deterministic K-stage finite horizon SDP. At stage j an amount of wealth x_j is invested, and a payoff $u\big(x_j\big)$ is accrued, where $u(x)$ is a suitable utility function. The utility is assumed to be strictly increasing and differentiable. Furthermore, the utility is assumed to be concave, or risk-averse. In most situations, the marginal utility of consuming a cake decreases as the amount consumed increases. In other words, there is less utility in consuming the tenth ounce than in consuming the first ounce.

On the other hand, if the utility were convex, or risk-seeking, it would be optimal to consume the entire cake in the first stage. It may be appropriate from an economic point of view to discount future payoffs, and the discount factor β served this purpose. The total discounted payoff accrued is therefore given by

$$R = \sum_{j=1}^{K} \beta^{j-1} u\big(x_j\big) \tag{4.1}$$

for some $\beta \in [0, 1]$. Furthermore, the amount to be invested in each stage is constrained in the following way. The SDP begins with a total wealth of $w_K = W > 0$, where w_k will denote the wealth available for investment when k stages remain. In particular, wealth w_k is available at the beginning of Stage $j = K - k + 1$. When k stages remain, an amount $c_k \leq w_k$ is invested. Then the wealth available at the subsequent stage will be $w_{k-1} = w_k - c_k$. This means the Stage 1 investment is $x_1 = c_K$, the Stage 2 investment is $x_2 = c_{K-1}$, and so on. The problem is to determine the investment decisions that maximize R, the total discounted utility accrued.

4.1 D Dynamic Programming Formulation of the Cake-Eating Problem

This problem bears an interesting resemblance to the Treasure Seeking problem of Sec. 2.3.2. In particular, an optimal decision clearly requires knowledge of the number of remaining stages. So we define the state to be $(w, k) \in \boldsymbol{\mathcal{X}}$, where w is the wealth currently available, and k is the number of stages remaining (including the current one). It is important to note that the state space $\boldsymbol{\mathcal{X}}$ is not discrete, since w can be any real number in the interval $[0, W]$. Clearly, we can define a terminal state $x_\Delta = (0, 0)$, with value function $V(0, 0) = 0$, assuming that once the allotted stages have expired, no more wealth can be invested, and no more payoff accrued.

The action at stage j is then $a_j = c_{K-j+1}$. Thus, the action space is $\boldsymbol{\mathcal{A}} = [0, W]$ and, like the state space $\boldsymbol{\mathcal{X}}$, is not discrete. Note that an action is only admissible if the resulting allocation does not exceed the total wealth remaining at that stage.

The reader will by now have noticed our preference for counting stages backwards, using index k rather than j. This makes more natural the process illustrated in Sec. 2.3.2 of indefinitely extending a finite horizon problem, so that a solution can be conceived without the need to fix the number of stages K.

Bellman's equation is straightforward:

$$V(w, k) = \max_{c \in [0, w]} u(c) + \beta V(w - c, k - 1)\,. \tag{4.2}$$

noting the recursive relationship $w_{k-1} = w_k - c_k$. At this point, we take an approach to the evaluation of this equation that necessarily differs from what we have so far seen.

The state-action space is no longer discrete, so we need to use other optimization methods. Under the conditions imposed on utility function $u(x)$, the maximization operation required of Eqn. (D.2) can be carried out by identifying the stationary point with respect to c. By taking first order derivatives, this leads to:

$$u'(c) = \beta V'(w - c, k - 1)\,. \tag{4.3}$$

At this point we may use backward induction to determine the optimal allocation for stages $k = 1, 2, ..., K$ in sequence. We will use utility function $u(w) = \log(w)$, which is risk-averse. This will be based on the recursive relationship of Eqn. (D.3). Unlike our previous examples, this step makes use of calculus, rather than enumeration over all admissible actions, but it otherwise possesses the same essential structure of any other problem solvable by backward induction. The recursion rule Eqn. (D.3) becomes

$$u'(c_k) = \beta V'(w_k - c_k, k - 1),\ k > 1\,. \tag{4.4}$$

As usual, the final stage, indexed by $k = 1$, is readily solved. The value function is simply

$$V(w,1) = \max_{c \in [0,w]} u(c) + \beta V(0,0) = \max_{c \in [0,w]} u(c) = u(w),$$

since $u(w)$ is increasing (note that stationary conditions are not relevant here).

The second-last stage is indexed by $k = 2$. At the start of the stage, wealth w_2 is available, and the action taken is the investment amount c_2.

The recursive relationship of Eqn. (D.4) gives:

$$u'(c_2) = \beta V'(w_2 - c_2, 1) = \beta u'(w_2 - c_2) = \beta u'(c_1),$$

so that, noting that $u'(w) = 1/w$, we have the identities:

$$\begin{aligned} w_2 &= c_1 + c_2, \\ c_1 &= \beta c_2, \\ c_2 &= \frac{w_2}{1+\beta}. \end{aligned}$$

This gives

$$\begin{aligned} V(w_2,2) &= u(c_2) + \beta u(c_1) \\ V(w_2,2) &= u(c_2) + \beta u(\beta c_2) \\ &= u\Big(\frac{w_2}{1+\beta}\Big) + \beta u\Big(\frac{\beta w_2}{1+\beta}\Big). \end{aligned}$$

So far, we have

$$\begin{aligned} V(w_1,1) &= u(w_1) \\ &= \log(w_1), \\ V(w_2,2) &= u\Big(\frac{w_2}{1+\beta}\Big) + \beta u\Big(\frac{\beta w_2}{1+\beta}\Big) \\ &= (1+\beta)\log(w_2) + \log(\beta) - 2\log(1+\beta) \\ &= \big(1+\beta\big)\log\big(w_2\big) + C_\beta, \end{aligned}$$

where C_β is a constant which does not depend on w_2. Then the third last stage is indexed by $k = 2$, and the recursive relationship of Eqn. (D.4) gives:

$$u'(c_3) = \beta V'(w_3 - c_3, 2) = \frac{\beta(1+\beta)}{w_3 - c_3},$$

which may be rewritten

$$c_3 = \frac{w_3}{1+\beta+\beta^2}.$$

If we look for a pattern in these solutions, we may notice:

$$c_1 = w_1,$$

$$c_2 = \frac{w_2}{1+\beta},$$

$$c_3 = \frac{w_3}{1+\beta+\beta^2}\,.$$

In fact, if we continue indefinitely, we will find that the optimal allocations are

$$c_k = \frac{w_k}{1+\beta+\beta^2+\ldots+\beta^{k-1}}\,.$$

Of course, more sophisticated models can be solved in much the same way.

For example, we could introduce a return on investment t_k, leading to recursive relationship

$$w_{k-1} = w_k - c_k + t_k,$$

allowing the investment process to continue indefinitely, possibly as an infinite horizon SDP. These models are described extensively in, for example (Adda et al., 2003). A detailed discussion of the application of utility theory in general to dynamic programming is given in (Bertsekas, 1995a).

E - Q-learning Implementation of the Optimal Treasure Seeking Problem

E - Introduction/Case study

The implementation is in file `IUBH-RL-Unit04-Example01.py`

Consider the following stochastic dynamic programming problem. Imagine, while vacationing in the Bahamas, you hear about a sunken treasure worth about $13,000. For $700/hr you can hire a boat, equipment and crew, for up to 5 hours a day. You estimate that the time it takes to find the treasure is an exponential random variable with mean $\mu = 13$ hours. You have only three days left on your vacation. How long should you hire the boat each day?

It is useful when solving optimization problems to try to anticipate the general form of the solution. Suppose on a certain day we decide to rent the boat for 5 hours. It is understood that this decision must be taken before the day's search begins. Thus, if after the first hour's search the treasure is found, we must still pay for the four remaining hours that were planned (dynamic programming is often used to solve this type of planning problem).

We might then do a “back of the envelope" calculation. Suppose we simply decided in advance to rent the boat for 13 hours (the expected search time).

The probability of finding the treasure would be $1 - \exp(-13/13) \approx 0.632$.

The total expected reward would therefore be

$$R = \$13,000 \times 0.632 - \$\,700 \times 13 = -\$884\ ,$$

yielding an expected net loss. Clearly, we would be better off planning the search as a SDP.

We then have a finite horizon SDP with $K = 3$ stages (each of the remaining three days will define a stage). We must first decide how to define the state x. We want to formulate a memoryless SDP, and also include all information needed to make a decision. Clearly, we need to know whether or not the treasure has already been found (we would not spend money on the boat if it has). So, set $s = 1$ if the treasure has been found, and $s = 0$ otherwise. However, our discussion suggests that the number of search days remaining would also be important information. Denote this quantity w. So, on the morning of our third and final day the state would be $x_3 = (w, s) = (1, 0)$ if the treasure has not yet been found, and $x_3 = (1, 1)$ if it has. The action space is $\boldsymbol{\mathcal{A}} = \{0, 1, 2, 3, 4, 5\}$ (the possible number of hours in the boat), and the stage reward is

$$R\big(w, 0, a\big) = 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a,\ \ a \in \big\{0, 1, 2, 3, 4, 5\big\},$$

with $R(w, 1, a) = 0$. Bellman's equation then becomes:

$$\begin{aligned} V^*(w, 1) &= 0,\ \ w \geq 0, \\ V^*(w, 0, a) &= R\big(w, 0, a\big) + V^*\big(w - 1, 0\big) \times e^{-a/13}, \\ V^*(w, 0) &= \max_{a\,\in\,\mathcal{A}} V^*(w, 0, a) \end{aligned} \tag{5.1}$$

with terminal value function $V^*(0, s) = 0$ for $s = \quad 0, 1$. This means that if we know the value of $V^*(w - 1, 0)$, we can substitute it into the expression for $V^*(w, 0, a)$ in Eqn. E.1, then maximize $V^*(w, 0, a)$ over $a \in \boldsymbol{\mathcal{A}}$. This gives $V^*(w, 0)$, and the maximizing action a^* determines the optimal policy $\phi^*(w, 0) = a^*$, while $\phi^*(w, 1) = 0$, since there would be no benefit to renting the boat if we have already found the treasure.

Starting with $V^*(0, 0) = 0$ at the final Stage 3, we make the following calculation:

Stage 3: Substitute $V^*(0, 0) = 0$ into Eqn. E.1.

$$\begin{aligned} V^*(1, 0, a) &= 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a + V^*\big(0, 0\big) \times e^{-a/13} \\ &= 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a, \end{aligned}$$

then maximize over $a \in \boldsymbol{\mathcal{A}}$. It is not hard to verify that the optimal action is $\phi^*(1, 0) = a^* = 5$, yielding $V^*(1, 0) = 650.7$.

Stage 2: Substitute $V^*(1,0) = 650\,.\,7$ into Eqn. E.1.

$$\begin{aligned} V^*(2,0,a) =& 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a + V^*\left(1,0\right) \times e^{-a/13} \\ =& 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a + 650\,.\,7 \times e^{-a/13}, \end{aligned}$$

then maximize over $a \in \mathcal{A}$. Now, the optimal action will be $\phi^*(2,0) = a^* = 4$, yielding $V^*(2,0) = 1121\,.\,5$.

Stage 1: Finally, substitute $V^*(2,0) = 1121\,.\,5$ into Eqn. E.1.

$$\begin{aligned} V^*(3,0,a) =& 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a + V^*\left(2,0\right) \times e^{-a/13} \\ =& 13000 \times \left(1 - e^{-a/13}\right) - 700 \times a + 1121\,.\,5 \times e^{-a/13}\,. \end{aligned}$$

As before, maximizing over $a \in \mathcal{A}$ yields optimal action $\phi^*(3,0) = a^* = 3$ and value function $V^*(3,0) = 1469\,.\,4$.

The optimal strategy is to rent the boat for 3,4 or 5 hours, if needed, on days 1, 2 or 3, respectively. This yields an expected reward of $1469.40. As anticipated, the optimal strategy is to budget search time more sparingly at the beginning of the schedule, when there is greater flexibility in planning.

5.1 E - Reinforcement Learning Implementation - The Agent-Environment Interaction Model

To develop a reinforcement learning algorithm to find the optimal policy, we first refer to the agent-environment interaction model of figure entitled as "Cat and Mouse Game" (Unit 4). Clearly, we need a method of simulating paths (it is unlikely that sufficient real life data would be available). We have a shortest path model that terminates by the third stage. So the algorithm will be based on repeated simulations of this path.

The state x_k consists of the days left available for search, say w, and the indicator variable $s = 1$ if the treasure has been found. The current stage will be the final stage if either $w = 1$, or the treasure is found during the stage. The action taken is $a \in \{0, 1, ..., 5\}$, representing the number of hours scheduled for a day's search. Finding the treasure is a random event. We may simulate an exponentially distributed random variable Z with mean 13 hours. Then we can define the random event,

$$\{\text{Treasure is found}\} = \{Z \leq a\},$$

interpretable as the event that the treasure is found within a hours of continual searching.

The reward is also random. It includes $13,000 if the treasure is found, and always includes a deduction of $-\$\ 700 \times a$ for the cost of renting the boat and crew. Therefore, the reward can be modeled as

$$R = \begin{cases} 13000 - 700 \times a \ ; \ Z \leq a \\ -700 \times a \qquad\quad ; \ Z > a \end{cases} \tag{5.2}$$

The state transitions can also be modeled as random events. If the current state is $x_k = (w, 0)$, the subsequent state will be

$$x_{k+1} = \begin{cases} (w-1, 1) \ ; \ Z \leq a \\ (w-1, 0) \ ; \ Z > a \end{cases} \tag{5.3}$$

Regarding the agent-environment interaction model, the agent is responsible for selecting the actions. Q-learning is an off-policy algorithm, so the behavior can be entirely exploratory. Here, we will simply have the agent select an action a from action space $\{0, 1, ..., 5\}$ randomly with equal probability.

Whenever the environment receives an action $a_k = a$ from the agent it can assume that the treasure has not yet been found, otherwise the path would have been terminated before that point (we are using the notation of Figure 1 of Unit 4). The environment then generates a pseudo-random variable Z as just described, and outputs x_{k+1} and $r_k = R$ as determined by Eqn. (E.2) and (E.2). Then given subsequent state $x_{k+1} = (w', s')$, if $w' = 0$ or $s' = 1$ the current path terminates, and a new path is initiated by resetting k to 1, and $x_1 = (3, 0)$. Of course, the path history is captured and used for the Q-learning updates.

5.2 E Reinforcement Learning Implementation - Q-learning Updates

Recall from Sec. 4.1 the Q-learning update rule. We need only modify the rule by changing the minimization operator to a maximization operator:

$$Q(x_k, a_k) \leftarrow Q(x_k, a_k) + \gamma \left(r_k + \beta \left[\max_{a \in \mathcal{A}_y} Q(x_{k+1}, a) \right] - Q(x_k, a_k) \right). \tag{5.4}$$

In the context of the agent-environment interaction model, the update is performed as soon as the environment outputs x_{k+1} and r_k as already defined. At this point we already know x_k and a_k. We first increase the number of visits to state-action pair (x_k, a_k) by 1 (sufficient computer memory is allocated for storing this). Then the update defined in Eqn. (E.4) can be performed. n our application we use learning rate

$$\gamma = \frac{1}{n(x_k, a_k)},$$

where $n_{(x_k, a_k)}$ is the number of visits to state-action pair (x_k, a_k).

5.3 E Computer Implementation in Python

The following code lists the program implementing the reinforcement learning algorithm in its entirety. Note that 1,000,000 replicated paths are specified.

Code

```
import random
import numpy
import math
from array import array

# Set random seed.

random.seed(13215)

# Define the number of stages

nStages = 3

# Create the Qfactor as an nStages x 6 matrix.
# Assign initial value 2000
# Create a nStages x 6 matrix visitCount to
# store the number of visits
# to each state/action pair.

Qfactor = 2000*numpy.ones(shape=(nStages,6))
visitCount = numpy.zeros(shape=(nStages,6))

# Set the number of replications.

nReplications = 1000000

# Begin main loop over the replications.

for iii in range(nReplications):

    # Within each replication a path is simulated.
    # Variable w represents days left to search.
    # When the path ends, Boolean flag
    # endSearch is set to True.

    w = nStages
```

```
endSearch = False

# The path continues until endSearch = True

while (not endSearch):

    # At each stage an action a is selected at random.
    # The Agent is essentially represented by this
    # single line of code.

    a = random.randint(0,5)

    # Simulate an exponentially distributed random
    # with mean 13 hours, which is stored in Z.
    # If Z <= a, the treasure is found,
    # since a is the number of hours scheduled for
    # the day's search.
    # The variable reward is initially set to zero.
    # If the treasure is found, $13,000 is
    # added to reward.
    # For any outcome, -$700*a is subtracted
    # from reward.
    # In addition, if the treasure is found,
    # set endSearch = True,
    # so that the path will terminate.

    # This next block of code represents the
    # Environment.

    Z = -13*math.log(random.uniform(0,1))
    reward = 0
    if Z <= a:
        reward = 13000
        endSearch = True
    reward = reward - 700*a

    # At this point, the environment has
    # generated the output
    # r[k] and x[k+1].
    # So the Q-learning update follows.

    # Update visitCount.

    visitCount[w-1][a] = visitCount[w-1][a]+1

    # Perform the Q-learning update

    if (w==1) or (endSearch):
```

```
            target = reward - Qfactor[w-1][a]
        else:
            target = reward + max(Qfactor[w-2]) - \
                Qfactor[w-1][a]

        Qfactor[w-1][a] = Qfactor[w-1][a] + \
            target/visitCount[w-1][a]

        # If this is the final day
        # available for searching,
        # set endSearch = True,
        # so that the path will terminate.
        # Otherwise, decrease w by 1.

        if w==1:
            endSearch = True
        else:
            w = w - 1

# Finally, create and output the policy and
# value functions.

policyFunction = array('i')
policyFunction = [0]*nStages
valueFunction = array('d')
valueFunction = [0]*nStages

Qtemp = array('d')
Qtemp = [0]*6
for w in range(nStages):
    for i in range(6): Qtemp[i] = Qfactor[w][i]
    policyFunction[w] = Qtemp.index(max(Qtemp))
    valueFunction[w] = max(Qtemp)

print(policyFunction)
print(valueFunction)
```

The value function and policy are output at the end of the program.

Code

```
    ...: print(policyFunction)
    ...: print(valueFunction)
[5, 4, 3]
[656.1925252455154, 1137.4759825903282, 1511.377028257831]
```

The exact solution was calculated above. The optimal policy is

$$\phi(3) = 5$$
$$\phi(2) = 4$$
$$\phi(1) = 3$$

which is the policy produced by the Q-learning algorithm, represented by vector `[5, 4, 3]` in the program's output. The optimal value function was found in Sec. 4.1 to be

$$V^*(3,0) = 1469.4$$
$$V^*(2,0) = 1121.5$$
$$V^*(1,0) = 650.7.$$

The Q-learning algorithm yields estimates

$$\widehat{V}^*\big(3,0\big) = 1511.4$$
$$\widehat{V}^*\big(2,0\big) = 1137.5$$
$$\widehat{V}^*\big(1,0\big) = 656.2.$$

The values are reasonably close to the true values, but far from exact.

Convergence of value function estimates to the true values is generally quite slow for Q-learning, which which motivates many of the modifications to reinforcement learning

LIST OF REFERENCES

Adda, J., Cooper, R., & Cooper, R. W. (2003). *Dynamic economics: Quantitative methods and applications*. MIT Press.

Aggarwal, C. C. (2018). *Neural networks and deep learning*. Springer.

Almudevar, A. L. (2014). *Approximate iterative algorithms*. CRC Press.

Andre, D., Friedman, N., & Parr, R. (1998). Generalized prioritized sweeping. In *Advances in neural information processing systems* (pp. 1001– 1007).

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., . . . Zaremba, W. (2017). Hindsight experience replay. In *Advances in neural information processing systems* (pp. 5048–5058).

Arrow, K. J., Blackwell, D., & Girshick, M. A. (1949). Bayes and minimax solutions of sequential decision problems. *Econometrica, Journal of the Econometric Society*, 213–244.

Arrow, K. J., Harris, T., & Marschak, J. (1951). Optimal inventory policy. *Econometrica, 19* (3), 250–272. Retrieved from http://www.jstor.org/stable/1906813

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine learning, 47* (2-3), 235–256.

Ball, P., Parker-Holder, J., Pacchiano, A., Choromanski, K., & Roberts, S. (2020). Ready policy one: World building through active learning. *ArXiv, abs/2002.02693* .

Bellman, R. (1957). *Dynamic programming*. Princeton University Press.

Berry, D. A., & Fristedt, B. (1985). Bandit problems: sequential allocation of experiments (monographs on statistics and applied probability). *London: Chapman and Hall, 5* , 71–87.

Bertsekas, D. P. (1995a). *Dynamic programming and optimal control* (2nd ed., Vol. 1). Belmont: Athena Scientific.

Bertsekas, D. P. (1995b). *Dynamic programming and optimal control* (2nd ed., Vol. 2). Belmont: Athena Scientific.

Bertsekas, D. P. (2019). *Reinforcement learning and optimal control*. Athena Scientific.

Bertsekas, D. P., & Tsitsiklis, J. N. (1996). *Neuro-dynamic programming.* Belmont: Athena Scientific.

Brafman, R. I., & Tennenholtz, M. (2002). R-max-a general polynomial time algorithm for near-optimal reinforcement learning. *Journal of Machine Learning Research,* 3 (Oct), 213–231.

Brémaud, P. (1999). *Gibbs fields, monte carlo simulation, and queues*. New York, NY: Springer-Verlag.

Cesa-Bianchi, N., Gentile, C., Lugosi, G., & Neu, G. (2017). Boltzmann exploration done right. *CoRR, abs/1705.10257* . Retrieved from http://arxiv.org/abs/1705.10257

Chambers, J. (1977). *Computational methods for data analysis*. John Wiley & Sons.

Cheng, J., Cheng, H., Meng, M. Q.-H., & Zhang, H. (2018). Autonomous navigation by mobile robots in human environments: A survey. In *2018 ieee international conference on robotics and biomimetics (robio)* (pp. 1981–1986).

Chentanez, N., Barto, A. G., & Singh, S. P. (2005). Intrinsically motivated reinforcement learning. In *Advances in neural information processing systems* (pp. 1281–1288).

Chow, C., & Tsitsiklis, J. N. (1989). The complexity of dynamic programming. *Journal of Complexity, 5* , 466-488.

Chow, C., & Tsitsiklis, J. N. (1991). An optimal one-way multigrid algorithm for discrete-time stochastic control. *IEEE Transactions on Automatic Control, 36* (8), 898-914.

Cichosz, P. (1999). An analysis of experience replay in temporal difference learning. *Cybernetics & Systems, 30* (5), 341–363.

Coulom, R. (2006). Efficient selectivity and backup operators in montecarlo tree search. In *International conference on computers and games* (pp. 72–83).

Esteva, A., Robicquet, A., Ramsundar, B., Kuleshov, V., DePristo, M., Chou, K., . . . Dean, J. (2019). A guide to deep learning in healthcare. *Nature medicine, 25* (1), 24–29.

Ferdowsi, A., Challita, U., Saad, W., & Mandayam, N. B. (2018). Robust deep reinforcement learning for security and safety in autonomous vehicle systems. In *2018 21st international conference on intelligent transportation systems (itsc)* (pp. 307–312).

Freedman, B. (1987). Equipoise and the ethics of clinical research. *New England Journal of Medicine, 317* (3), 141–145. doi: 10.1056/nejm198707163170304

Geron, A. (2019). *Hands-on machine learning with scikit-learn, keras, and tensorflow: Concepts, tools, and techniques to build intelligent systems* (2nd ed.). O'Reilly UK Ltd.

Gittins, J., Glazebrook, K., & Weber, R. (2011). *Multi-armed bandit allocation indices*. John Wiley & Sons.

Gottlieb, M. A. (2017). *Feynman's restaurant problem revealed*. https://www.feynmanlectures.caltech.edu/info/other/Feynmans_Restaurant_Problem_ (Last accessed: 2020-Feb-10)

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actorcritic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *CoRR, abs/1801.01290*. Retrieved from http://arxiv.org/abs/1801.01290

Hasselt, H. V. (2010). Double q-learning. *In Advances in neural information processing systems* (pp. 2613–2621).

Hassoun Mohamad, H. (1995). *Fundamentals of artificial neural networks*. MIT Press, Cambridge, Massa chusetts.

Howard, R. A. (1960). *Dynamic programming and markov processes*. John Wiley.

Isele, D., & Cosgun, A. (2018). Selective experience replay for lifelong learning. In *Thirty-second aaai conference on artificial intelligence*.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning*. Springer.

Janner, M., Fu, J., Zhang, M., & Levine, S. (2019). When to trust your model: Model-based policy optimization. *CoRR, abs/1906.08253*. Retrieved from http://arxiv.org/abs/1906.08253

Kao, H.-C., Tang, K.-F., & Chang, E. Y. (2018). Context-aware symptom checking for disease diagnosis using hierarchical reinforcement learning. In *Thirty-second aaai conference on artificial intelligence*.

Kearns, M., Mansour, Y., & Ng, A. (2002). A sparse sampling algorithm for near-optimal planning in large markov decision processes. *Machine Learning, 49* (2-3), 193 - 208.

Kearns, M., & Singh, S. (2002). Near-optimal reinforcement learning in polynomial time. *Machine learning, 49* (2-3), 209–232.

Kendall, D. G. (1953). Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded markov chain. *Ann. Math. Statist., 24* (3), 338–354.

Kleinrock, L. (1975). *Queueing systems (volume 1: Theory)*. New York, NY: John Wiley and Sons.

Kober, J., Bagnell, J. A., & Peters, J. (2013). Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research, 32* (11), 1238–1274.

Kulkarni, T. D., Narasimhan, K., Saeedi, A., & Tenenbaum, J. (2016). Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *Advances in neural information processing systems* (pp. 3675–3683).

Lai, T., & Yakowitz, S. (1995). Machine learning and nonparametric bandit theory. *IEEE Transactions on Automatic Control, 30*, 1199–1209.

Lai, T. L., & Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics, 6* (1), 4–22.

Li, J., Cheng, H., Guo, H., & Qiu, S. (2018). Survey on artificial intelligence for vehicles. *Automotive Innovation, 1* (1), 2–14.

Lin, L.-J. (1993). *Reinforcement learning for robots using neural networks* (Tech. Rep.). Pittsburgh PA School of Computer Science: Carnegie- Mellon University.

Loquercio, A., & Scaramuzza, D. (2018). Learning to control drones in natural environments: A Survey. In *eicra workshop on perception, inference, and learning for joint semantic, geometric, and physical understanding, brisbane, 2018*.

McClelland, J. L., McNaughton, B. L., & O'Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological review, 102* (3), 419.

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics, 5* (4), 115–133.

Melo, F. S. (2001). Convergence of q-learning: A simple proof. *Institute Of Systems and Robotics, Tech. Rep*, 1–4.

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., . . . Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. *CoRR, abs/1602.01783*. Retrieved from http://arxiv.org/abs/1602.01783

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. A. (2013). Playing atari with deep reinforcement learning. *CoRR, abs/1312.5602*. Retrieved from http://arxiv.org/abs/1312.5602

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., . . . others (2015a). Human-level control through deep reinforcement learning. *Nature, 518* (7540), 529.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., . . . Hassabis, D. (2015b, February). Human-level control through deep reinforcement learning. *Nature, 518* (7540), 529–533. doi: 10.1038/nature14236

Moerland, T. M., Broekens, J., & Jonker, C. M. (2018). Emotion in reinforcement learning agents and robots: A survey. *Machine Learning, 107* (2), 443–480.

Moore, A. W., & Atkeson, C. G. (1993a). Prioritized sweeping: reinforcement learning with less data and less time. *Machine Learning, 13*, 103-130.

Moore, A. W., & Atkeson, C. G. (1993b). Prioritized sweeping: Reinforcement learning with less data and less time. *Machine learning, 13* (1), 103–130.

Muñoz, G., Barrado, C., Çetin, E., & Salami, E. (2019). Deep reinforcement learning for drone delivery. *Drones, 3* (3), 72.

OpenAI. (2017). Openai baselines: Acktr & a2c. https://openai.com/blog/baselines-acktr-a2c/. (Last accessed: 2020-Feb-10)

Osband, I., Blundell, C., Pritzel, A., & Roy, B. V. (2016). Deep exploration via bootstrapped DQN. *CoRR, abs/1602.04621* . Retrieved from http://arxiv.org/abs/1602.04621

Osband, I., Russo, D., & Van Roy, B. (2013). (more) efficient reinforcement learning via posterior sampling. In *Advances in neural information processing systems* (pp. 3003–3011).

O'Neill, J., Pleydell-Bouverie, B., Dupret, D., & Csicsvari, J. (2010). Play it again: reactivation of waking experience and memory. *Trends in neurosciences, 33* (5), 220–229.

Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiositydriven exploration by self-supervised prediction. In *Proceedings of the ieee conference on computer vision and pattern recognition workshops* (pp. 16–17).

Powell, W. B. (2011). *Approximate dynamic programming: Solving the curses of dimensionality* (Second ed.). Hoboken, NJ: John Wiley and Sons, Inc.

Puterman, M. L. (1994). *Markov decision processes: Discrete stochastic dynamic programming*. New York, NY: John Wiley and Sons, Inc.

Rosa-Hatko, W., & Gunn, E. A. (1997, Jan 01). Queues with switchover - a review and critique. *Annals of Operations Research, 69* (0), 299– 322. Retrieved from https://doi.org/10.1023/A:1018941214239 doi: 10.1023/A:1018941214239

Ross, S. (2014). *Introduction to probability models*. Elsevier Science. Retrieved from https://books.google.com/books?id=A3YpAgAAQBAJ

Ross, S. M. (1996). *Stochastic processes* (second ed.). New York, NY: John Wiley and Sons.

Rummery, G. A., & Niranjan, M. (1994). *On-line q-learning using connectionist systems* (Vol. 37). University of Cambridge, Department of Engineering Cambridge, England.

Russo, D., Roy, B. V., Kazerouni, A., & Osband, I. (2017). A tutorial on thompson sampling. *CoRR, abs/1707.02038* . Retrieved from http://arxiv.org/abs/1707.02038

Rust, J. P. (1989). A dynamic programming model of retirement behavior. In T*he economics of aging* (pp. 359–404). University of Chicago Press.

Scarf, H. E., & Hansen, T. (1973). *The computation of economic equilibria*. Yale University Press.

Schaul, T., Horgan, D., Gregor, K., & Silver, D. (2015). Universal value function approximators. In *International conference on machine learning* (pp. 1312–1320).

Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2015). Prioritized experience replay. *arXiv preprint arXiv:1511.05952*.

Schmidhuber, J. (1991). Curious model-building control systems. In *Proc. international joint conference on neural networks* (pp. 1458–1463).

Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR, abs/1707.06347*. Retrieved from http://arxiv.org/abs/1707.06347

Shotwell, P. (1994). The game of go: speculations on its origins and symbolism in ancient china. *Changes, 2008*.

Shubik, M. (1959). *Strategy and market structure*. Wiley, New York.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., . . . others (2016). Mastering the game of go with deep neural networks and tree search. *nature, 529* (7587), 484.

Singh, S., & Dayan, P. (1998). Analytical mean squared error curves for temporal difference learning. *Machine Learning, 32* (1), 5–40.

Stadie, B. C., Levine, S., & Abbeel, P. (2015). Incentivizing exploration in reinforcement learning with deep predictive models. *CoRR, abs/1507.00814*. Retrieved from http://arxiv.org/abs/1507.00814

Sutton, R. S. (1990). Dyna, an integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Machine learning proceedings 1990* (pp. 216–224). Elsevier.

Sutton, R. S., & Barto, A. G. (2018a). *Reinforcement learning: An introduction* (2nd ed.). MIT press.

Sutton, R. S., & Barto, A. G. (2018b). *Reinforcement learning: An introduction* (2nd ed.). MIT press.

Sutton, R. S., Modayil, J., Degris, M. D. T., Pilarski, P. M., & White, A. (2011). Horde: A scalable real-time architecture for learning knowledge from unsupervised sensorimotor interaction. *Proceedings of the 10th International Conference on Autonomous Agents and Multi-agent Systems (AAMAS 2011)*.

Tesauro, G. (1994). TD-Gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation, 6* (2), 215–219.

Tesauro, G. (1995). Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38 (3), 58–68.

Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika, 25* (3/4), 285–294.

Thompson, W. R. (1935). On the theory of apportionment. *American Journal of Mathematics, 57* (2), 450–456.

Tromp, J., & Farnebäck, G. (2006). Combinatorics of go. In *International conference on computers and games* (pp. 84–99).

Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double q-learning. In *Thirtieth aaai conference on artificial intelligence*.

Van Loan, C. F., & Golub, G. H. (1983). *Matrix computations*. Johns Hopkins University Press.

Von Neumann, J., Morgenstern, O., & Kuhn, H. W. (1944). *Theory of games and economic behavior*. Princeton University Press.

Wang, T., Bao, X., Clavera, I., Hoang, J., Wen, Y., Langlois, E., . . . Ba, J. (2019). Benchmarking model-based reinforcement learning. *CoRR, abs/1907.02057* . Retrieved from http://arxiv.org/abs/1907.02057

Wang, Z., Bapst, V., Heess, N., Mnih, V., Munos, R., Kavukcuoglu, K., & de Freitas, N. (2016). Sample efficient actor-critic with experience replay. *arXiv preprint arXiv:1611.01224*.

Watkins, C. J., & Dayan, P. (1992). Q-learning. *Machine learning, 8* (3-4), 279–292.

Watkins, C. J. C. H. (1989). *Learning from delayed rewards* (Unpublished doctoral dissertation). King's College, Cambridge.

Wesselmann, P., Wu, Y.-C., & Gašić, M. (2019). Curiosity-driven reinforcement learning for dialogue management. In *Icassp 2019-2019 ieee international conference on acoustics, speech and signal processing (icassp)* (pp. 7210–7214).

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning, 8* (3-4), 229–256.

Wolpin, K. I. (1996). Public-policy uses of discrete-choice dynamic programming models. *The American Economic Review, 8*6 (2), 427–432.

Xu, J., Du, T., Foshey, M., Li, B., Zhu, B., Schulz, A., & Matusik, W. (2019). Learning to fly: Computational controller design for hybrid UAVs with reinforcement learning. *ACM Transactions on Graphics (TOG), 38* (4), 42:1-42:12.

Xue, S., Li, Z., & Yang, L. (2019). Training a model-free reinforcement learning controller for a 3-degree-of-freedom helicopter under multiple constraints. *Measurement and Control*, 0020294019847711.

You, C., Lu, J., Filev, D., & Tsiotras, P. (2018). Highway traffic modeling and decision making for autonomous vehicle using reinforcement learning. In *2018 ieee intelligent vehicles symposium (iv)* (pp. 1227– 1232).

LIST OF TABLES AND FIGURES

Figure 1: Agent-environment interaction model of reinforcement learning, adapted from figure 3.1 of (Sutton & Barto, 2018a). 29

Figure 2: ANN architecture for automated poker application of Sec. u1.box.poker. 48

Figure 3: Stagecoach problem map. 56

Figure 4: Stagecoach problem. Map of Fig. 2.1 is shown partitioned into Stages $k = 1, ..., 6$. 57

Figure 5: State transition diagram for infection model. 77

Figure 6: Stagecoach problem. Section of the map of Fig. 2.1 representing the transition from Stage 5 to Stage 6. 84

Figure 7: Stagecoach problem. Section of the map of Fig. 2.1 representing the transition from Stage 1 to Stage 2. 86

Figure 8: Output from sample python program for the Treasure Seeking problem Sec. u2.sec.treasure.example. 91

Figure 9: Utility functions for thirsty customer and retailer for the soft drink example of Sec. 2.4.1. The gray lines represent constant marginal utility of $1.00 and $0.25 per bottle. 102

Figure 10: Examples of risk-averse, risk-neutral and risk-seeking utility functions. 104

Figure 11: Stagecoach problem with minimax cost criterion. Section of the map of Fig. 2.1 representing the calculations for Stages 4 and 5. 106

Figure 12: Maximum reward selection policy discussed in Sec. 3.2 for a three-armed bandit process. 120

Figure 13: Softmax policy for a three-armed bandit process, for temperature parameter $\tau = 1$. 124

Figure 14: Softmax policy for a three-armed bandit process, using temperature parameter $\tau = 100$. 125

Figure 15: UCB1 policy, discussed in Sec. 3.2.2. 132

Figure 16: Cat and Mouse Game 136

Figure 17: Elements of the reinforcement environment 137

Figure 18: Videogame on a gaming console 150

Figure 19: Atari video game: Space Invaders 151

Figure 20: Schematic setup of Deep Q-Networks: The network learns the mapping of state-action pairs (s, a) to a Q-value (left) or computes a Q-value for all possible actions (right). 152

Figure 21: Blind Cliff Walk 157

Figure 22: Layout of a complex games with multiple sub-goals 157

Figure 23: Actor-Critic Setup 163

Figure 24: Model-based reinforcement learning 173

Figure 25: Game Tree in Tic-Tac-Toe 175

Figure 26: Towers of Hanoi game platform 188

Figure 27: Towers of Hanoi game platform with four disks. The state $x = (3, 1, 2, 3)$ is illustrated. 191

Figure 28: Schematic representation of the switchover queue model. Capacity is $K = 7$ (this includes customers both waiting and being served). The state illustrated is $x = (W_1, W_2, U) = (5, 3, 1)$. Queue 1 has 1 customer in service, and 4 more waiting in queue, so $W_1 = 5$. and $U = 1$. Queue 2 has 3 customers waiting in queue and none being served, so $W_2 = 3$. The arrival rates λ and serve rate μ are indicated. These are in units of customer per time unit. 202

Figure 29: Examples of state transition diagrams for the switchover queue model. Unless otherwise specified, W_1 and W_2 are assumed to be strictly between 0 and K. 205

Figure 30: Trace array of the value iteration algorithm for the switchover queue problem. 213

IU Internationale Hochschule GmbH
IU International University of Applied Sciences
Juri-Gagarin-Ring 152
D-99084 Erfurt

Mailing Address
Albert-Proeller-Straße 15-19
D-86675 Buchdorf

media@iu.org
www.iu.org

Help & Contacts (FAQ)
On myCampus you can always find answers to questions concerning your studies.